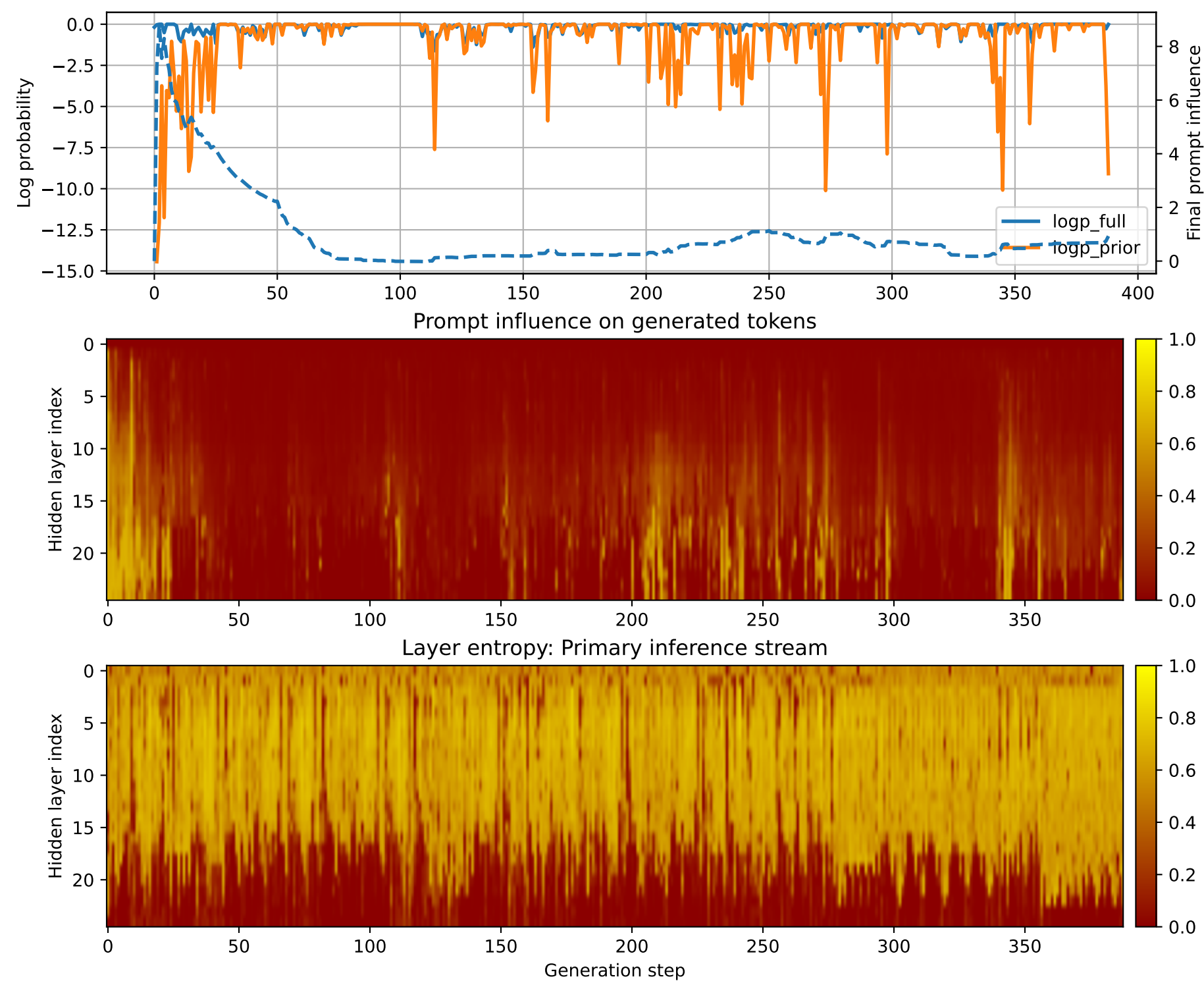


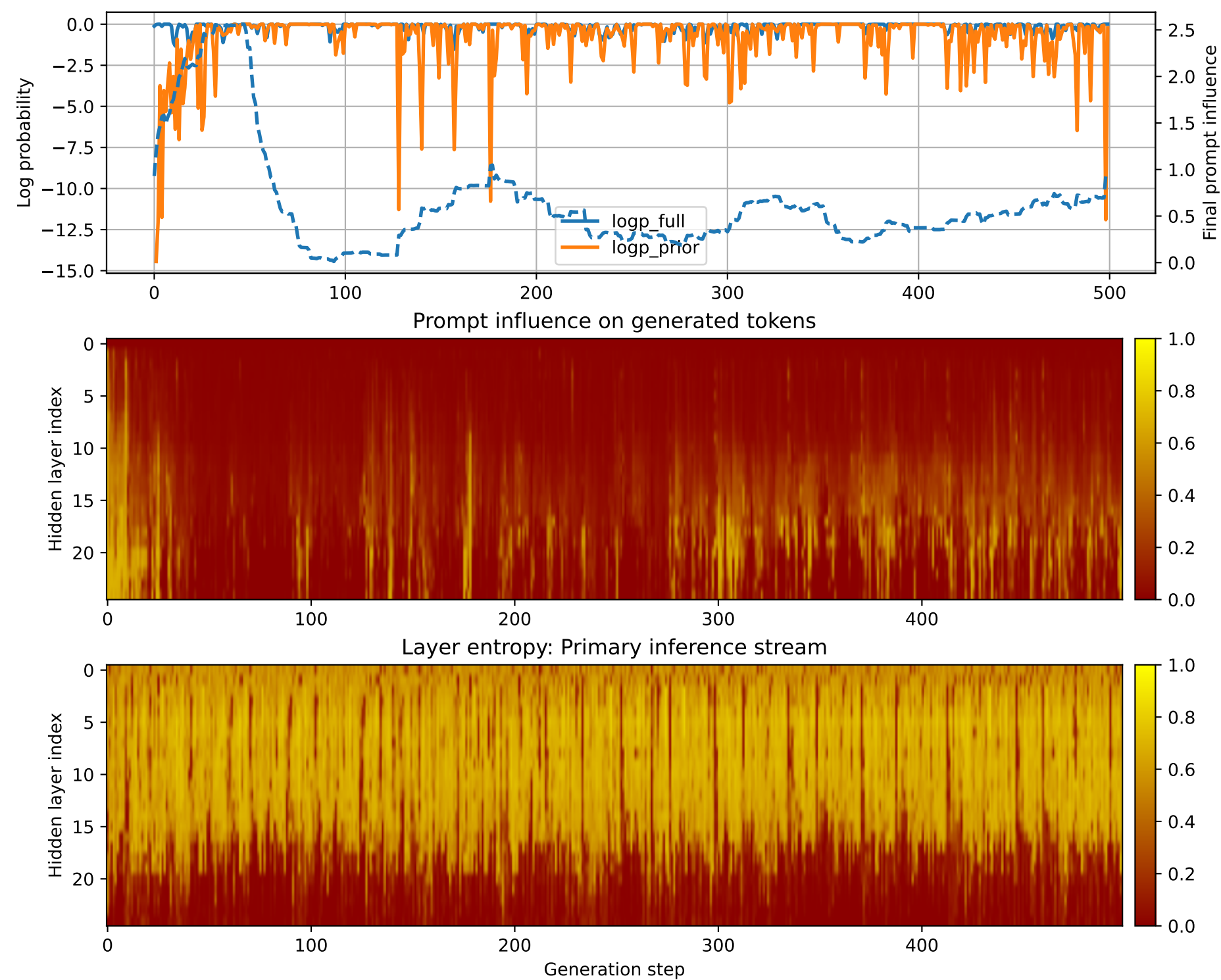
Prompt 1



PROMPT:
Task ID: PYC100 Return ONLY valid Python code. Do not use
Markdown fences. Do not include prose,
explanations, examples, print statements, or tests. Your code
must contain this exact check-token line: #
CHECK_TOKEN: PYC100 Your code must contain this required symbol:
def run_pipeline Implement this task: write
function run_pipeline(data, steps) applying named step functions
and logging outputs.. Start your answer with
the check-token line, then provide the Python implementation.

MODEL RESPONSE:
CHECK_TOKEN: PYC100 # Define the function run_pipeline that
takes in two parameters: data and steps. def
run_pipeline(data, steps): # Check if the data is a list or a
tuple if not isinstance(data, (list,
tuple)): raise ValueError("Data must be a list or a
tuple") # Check if the steps are a list or a
tuple if not isinstance(steps, (list, tuple)): raise
ValueError("Steps must be a list or a tuple")
Check if the steps are functions or methods if not
all(isinstance(step, (lambda x: True, type(x)), step)
for step in steps): raise ValueError("Steps must be
functions or methods") # Initialize a
dictionary to store the output of each step output_dict = {}
Iterate over each step in the steps
list for step in steps: # Check if the step is a
function or method if not
isinstance(step, (lambda x: True, type(x)), step):
raise ValueError("Step must be a function or
method") # Apply the step to the data and store the
output in the output_dict
output_dict[step.name] = step(data) # Return the output
dictionary return output_dict # Example
usage of the run_pipeline function data = [1, 2, 3, 4, 5] steps =
[lambda x: x ** 2, lambda x: x * x,
lambda x: x ** 3, lambda x: x * x, lambda x: x ** 4]
output = run_pipeline(data, steps)
print(output) # Output: {0: 1, 1: 4, 2: 9, 3: 16, 4: 25}

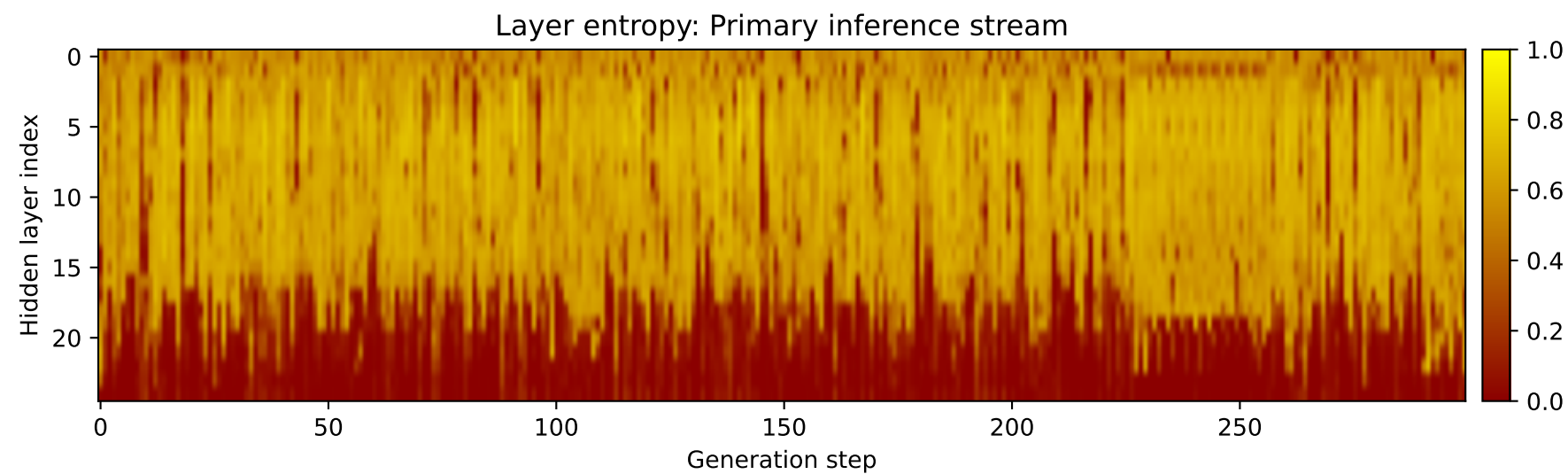
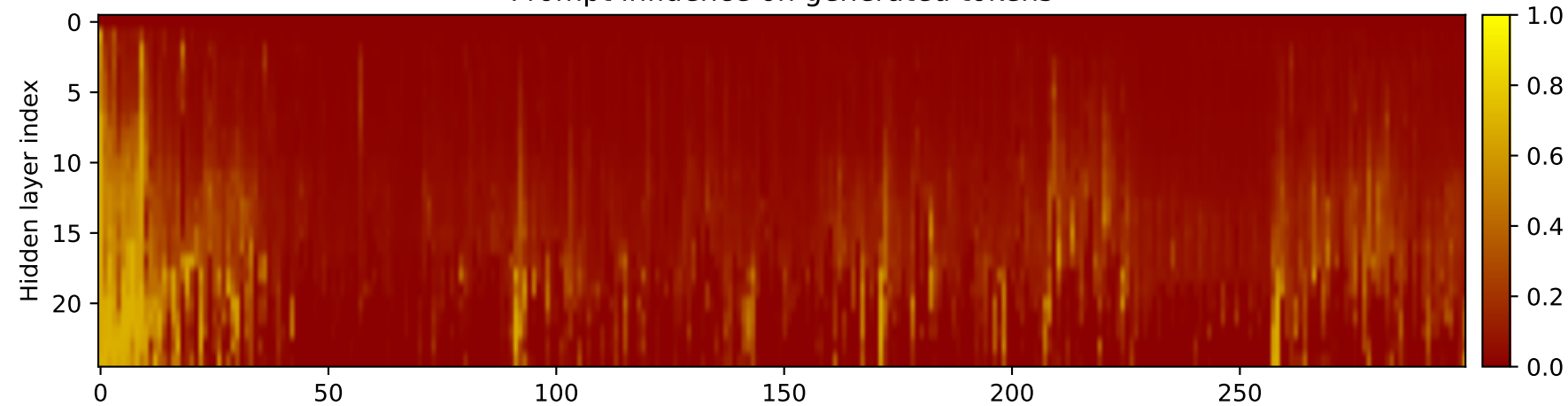
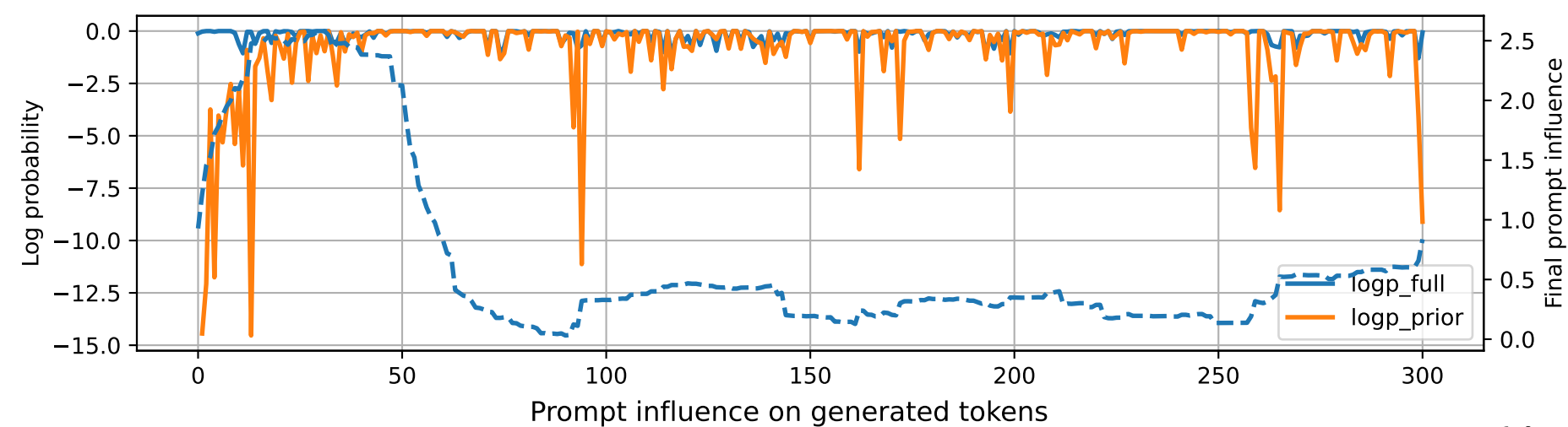
Prompt 1



PROMPT:
Task ID: PYC099 Return ONLY valid Python code. Do not use
Markdown fences. Do not include prose,
explanations, examples, print statements, or tests. Your code
must contain this exact check-token line: #
CHECK_TOKEN: PYC099 Your code must contain this required symbol:
def simple_template_render Implement this
task: write function simple_template_render(template, context)
replacing {{name}}.. Start your answer with
the check-token line, then provide the Python implementation.

MODEL RESPONSE:
CHECK_TOKEN: PYC099 # Function to render a simple template
with a given context def
simple_template_render(template, context): # Check if the
template is a valid Python string if not
isinstance(template, str): raise ValueError("Template
must be a string") # Check if the
context is a dictionary if not isinstance(context, dict):
raise ValueError("Context must be a
dictionary") # Check if the context has a 'name' key
if 'name' not in context: raise
ValueError("Context must have a 'name' key") # Render
the template with the context
rendered_template = template.format(**context) return
rendered_template # Example usage template =
""" {{name}} """ context = { "name": "John" }
rendered_template = simple_template_render(template,
context) print(rendered_template) # Output: John # Explanation:
- The `simple\_template\_render` function
takes two arguments: `template` and `context`. - The first
argument `template` is a string that represents the
template to be rendered. It should contain placeholders for
variables that will be replaced with values from
the `context` dictionary. - The second argument `context` is a
dictionary that represents the variables and
their values to be used in the template. The `name` key in the
`context` dictionary should contain the name of
the variable to be replaced. - The function first checks if the
`template` is a string. If it's not, it raises
a `ValueError` with the message "Template must be a string". -
Next, it checks if the `context` is a
dictionary. If it's not, it raises a `ValueError` with the
message "Context must be a dictionary". - The
function then checks if the `context` has a 'name' key. If it
doesn't, it raises a `ValueError` with the
message "Context must have a 'name' key". - If all the checks
pass, the function uses the `format` method to
replace the placeholders in the `template` with the values from
the `context` dictionary. - Finally, the
function returns the rendered template as a string. In the
example usage, we define a template string
`{{name}}` and a dictionary `context` with a 'name' key
containing the value "John". We then call the
`simple\_template`

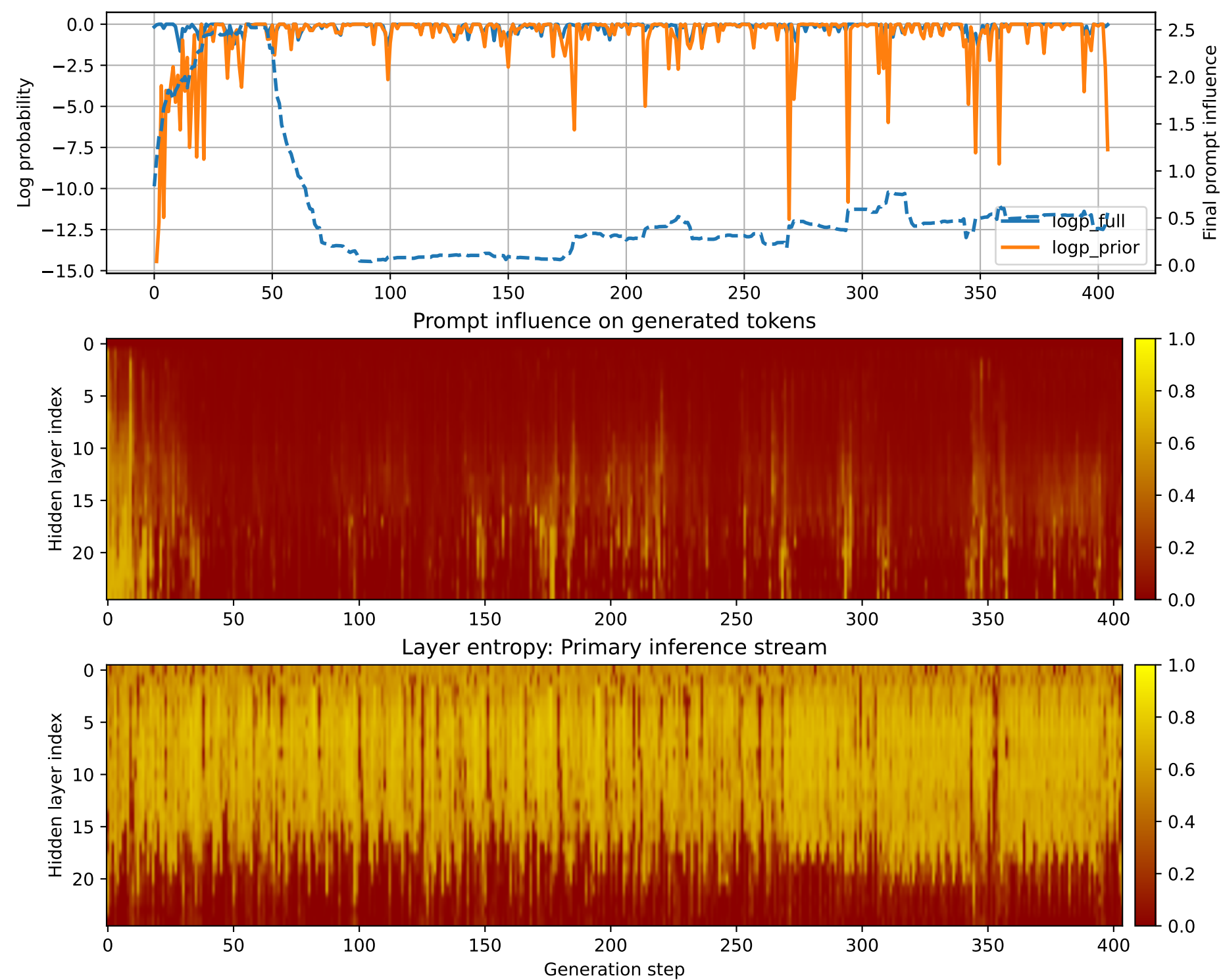
Prompt 1



PROMPT:
Task ID: PYC098 Return ONLY valid Python code. Do not use
Markdown fences. Do not include prose,
explanations, examples, print statements, or tests. Your code
must contain this exact check-token line: #
CHECK_TOKEN: PYC098 Your code must contain this required symbol:
def paginate Implement this task: write
function paginate(items, page_size) returning list of pages..
Start your answer with the check-token line,
then provide the Python implementation.

MODEL RESPONSE:
CHECK_TOKEN: PYC098 # Function to paginate a list of items and
return a list of pages def paginate(items,
page_size): # Check if the page_size is a positive integer
if not isinstance(page_size, int) or
page_size <= 0: raise ValueError("Page size must be a
positive integer") # Initialize an empty
list to store the pages pages = [] # Iterate over the
items in reverse order for i in
range(len(items) - page_size + 1): # Calculate the index
of the current page page_index = i +
page_size # Check if the current page is within the
range of the items if page_index >= 0 and
page_index < len(items): # Append the current item to
the current page
pages.append(items[page_index]) else: # If
the current page is out of range, append the
first item of the next page pages.append(items[0])
Return the list of pages return
pages # Example usage items = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
page_size = 3 # Paginate the items and return
the list of pages pages = paginate(items, page_size) print(pages)
Output: [3, 4, 5]

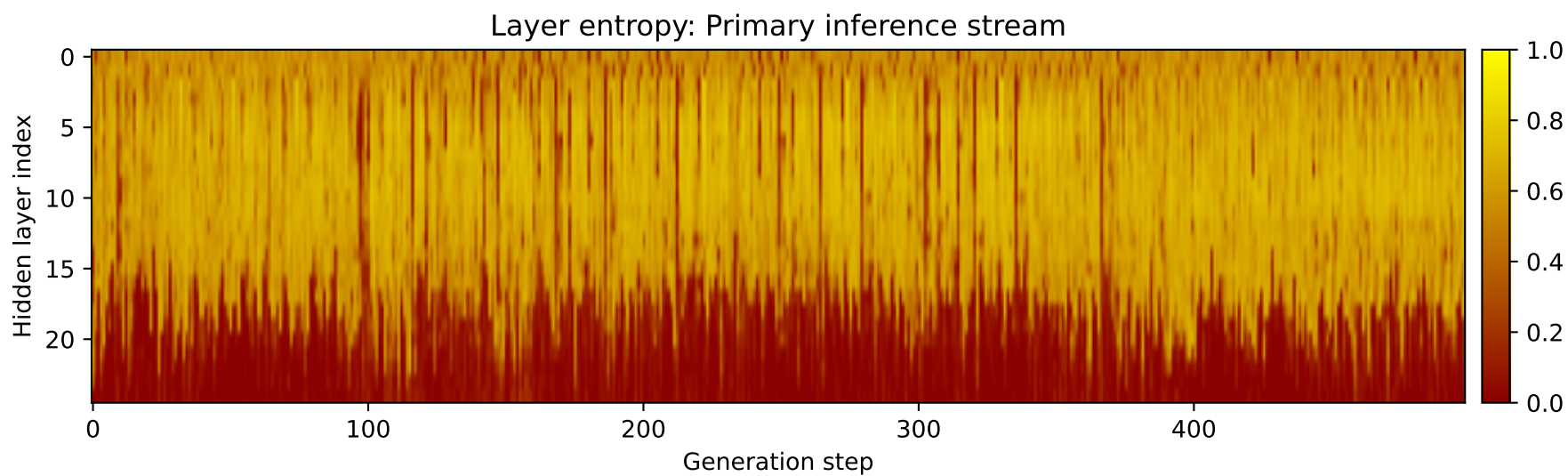
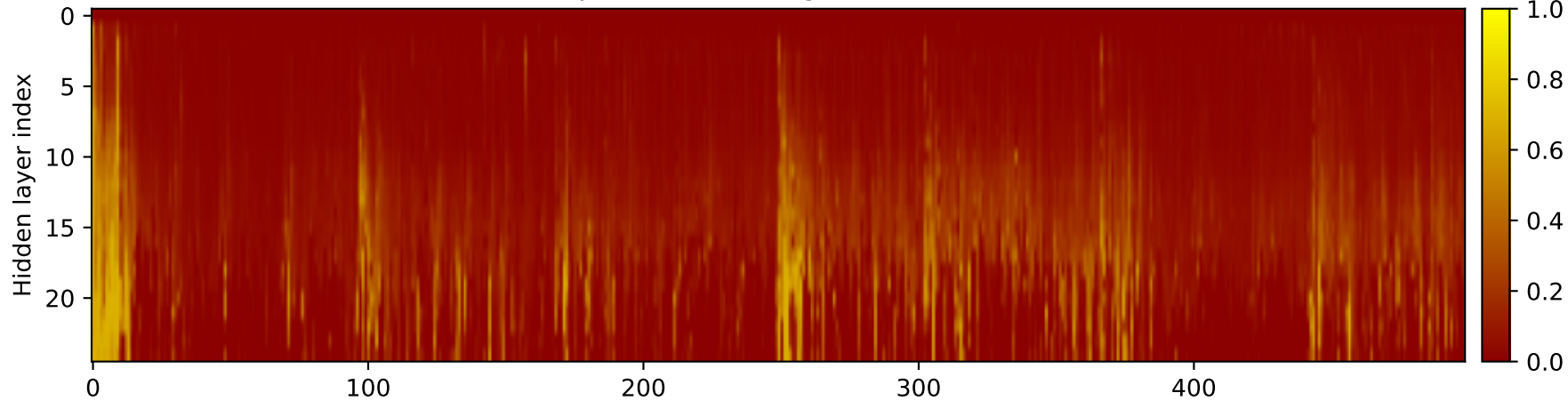
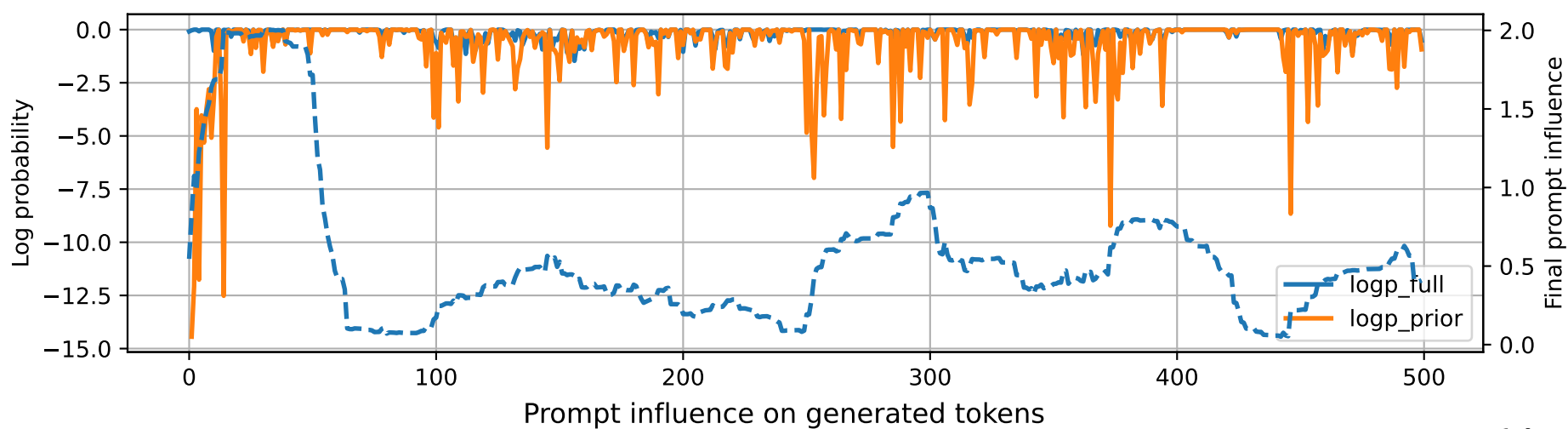
Prompt 1



PROMPT:
Task ID: PYC097 Return ONLY valid Python code. Do not use
Markdown fences. Do not include prose,
explanations, examples, print statements, or tests. Your code
must contain this exact check-token line: #
CHECK_TOKEN: PYC097 Your code must contain this required symbol:
def dependency_order Implement this task:
write function dependency_order(tasks) with cycle detection..
Start your answer with the check-token line,
then provide the Python implementation.

MODEL RESPONSE:
CHECK_TOKEN: PYC097 # Function to determine the dependency
order of tasks based on cycle detection def
dependency_order(tasks): # Check if the tasks are a list
if not isinstance(tasks, list): raise
ValueError("Input must be a list of tasks") # Initialize a
dictionary to store the dependency order
dependency_order = {} # Iterate through each task in the
list for task in tasks: # Check if
the task is a string if not isinstance(task, str):
raise ValueError("Task must be a
string") # Split the task into its components (name,
dependencies) name, dependencies =
task.split(" -> ") # Check if the task has a dependency
if not dependencies:
raise ValueError("Task must have a dependency") # Check
if the task is already in the dependency
order if name in dependency_order: raise
ValueError("Task already exists in the dependency
order") # Add the task to the dependency order with its
dependencies dependency_order[name] =
dependencies # Sort the dependency order in reverse
alphabetical order sorted_dependencies =
sorted(dependency_order.items(), key=lambda x: x[1],
reverse=True) # Return the sorted dependency order
return sorted_dependencies # Example usage tasks = ["task1",
"task2", "task3", "task4", "task5"]
dependency_order = dependency_order(tasks)
print(dependency_order) # Output: [('task5', ['task1',
'task2']),
(('task4', ['task3']), ('task3', ['task2']), ('task2', ['task1']))]
Check if the function returns the
expected output assert dependency_order == [("task5", ['task1',
'task2']), ('task4', ['task3']), ('task3',
['task2']), ('task2', ['task1'])"], "Function does not return the
expected output")

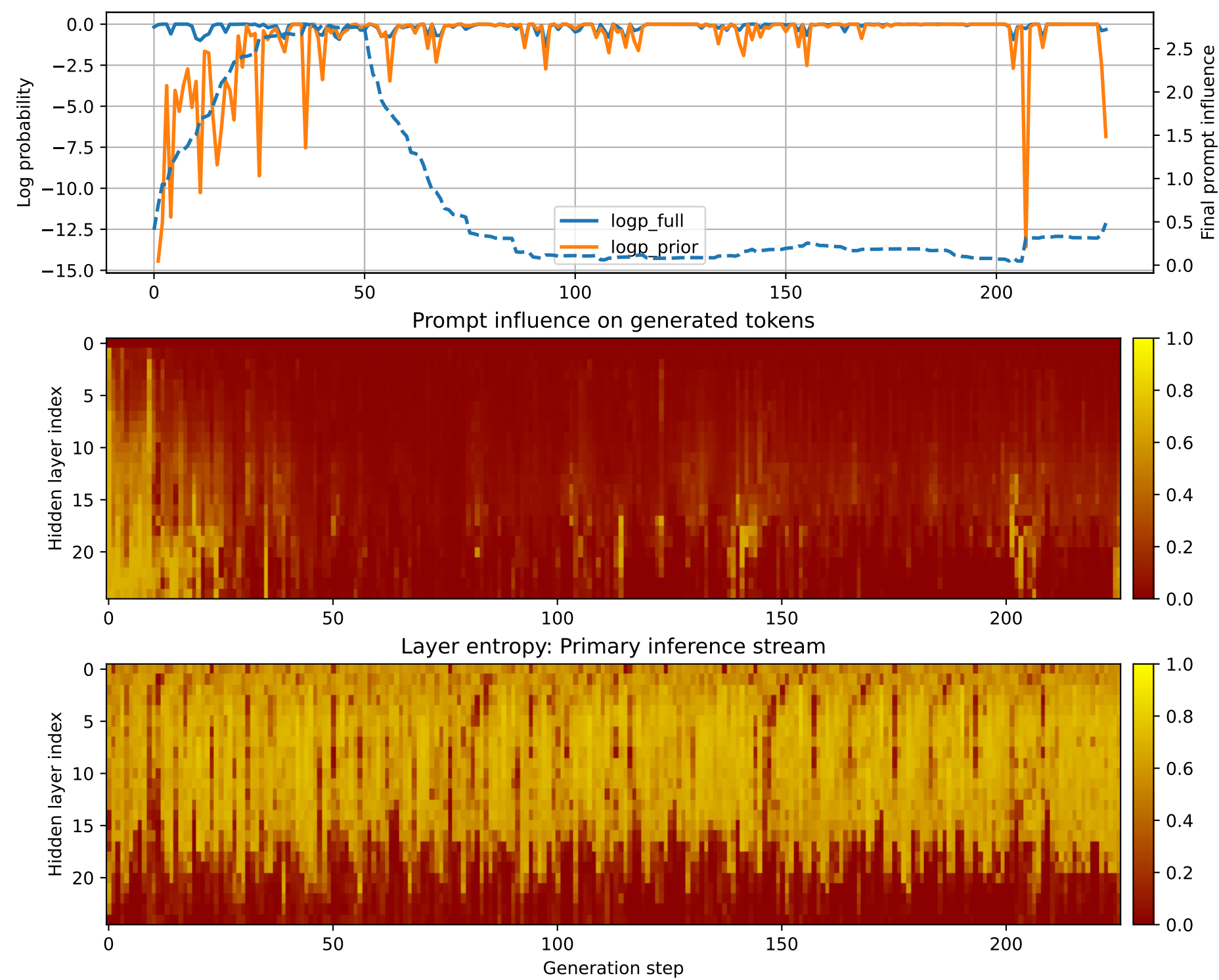
Prompt 1



PROMPT:
Task ID: PYC096 Return ONLY valid Python code. Do not use
Markdown fences. Do not include prose,
explanations, examples, print statements, or tests. Your code
must contain this exact check-token line: #
CHECK_TOKEN: PYC096 Your code must contain this required symbol:
class EventEmitter Implement this task:
write class EventEmitter with on, off, emit.. Start your answer
with the check-token line, then provide the
Python implementation.

MODEL RESPONSE:
CHECK_TOKEN: PYC096 ```python class EventEmitter: def
__init__(self): self.events = {} def
on(self, event_name, callback): self.events[event_name] =
callback def off(self, event_name):
if event_name in self.events: del
self.events[event_name] def emit(self, event_name, data):
if event_name in self.events:
self.events[event_name].emit(data) ``` Explanation: The
`EventEmitter` class is a Python class that represents an event
listener. It has three methods: 1.
`\_\_init\_\_`: This method initializes an empty dictionary `events`
to store event names and their corresponding
callback functions. The `events` dictionary is used to store the
event names as keys and the callback
functions as values. 2. `on`: This method takes an event name
and a callback function as arguments. It adds
the callback function to the `events` dictionary with the event
name as the key and the callback function as
the value. If the event name is not already present in the
`events` dictionary, it creates a new key-value
pair with the event name as the key and the callback function as
the value. 3. `off`: This method takes an
event name as an argument. It checks if the event name is present
in the `events` dictionary. If it is, it
deletes the corresponding callback function from the `events`
dictionary using the `del` statement. 4.
`emit`: This method takes an event name and a data parameter as
arguments. It checks if the event name is
present in the `events` dictionary. If it is, it calls the
callback function associated with the event name
with the data parameter using the `emit` method of the `events`
dictionary. Example usage: ```python emitter
= EventEmitter() emitter.on('event1', lambda data: print(f'Event
1 emitted: {data}')) emitter.on('event2',
lambda data: print(f'Event 2 emitted: {data}'))
emitter.on('event3', lambda data: print(f'Event 3 emitted:
{data}')) emitter.off('event1') emitter.emit('event1', 'Hello,
world!') emitter.emit('event2', 'Hello,
world!') emitter.emit('event3', 'Hello, world!')
emitter.off('event2') emitter.emit('event1

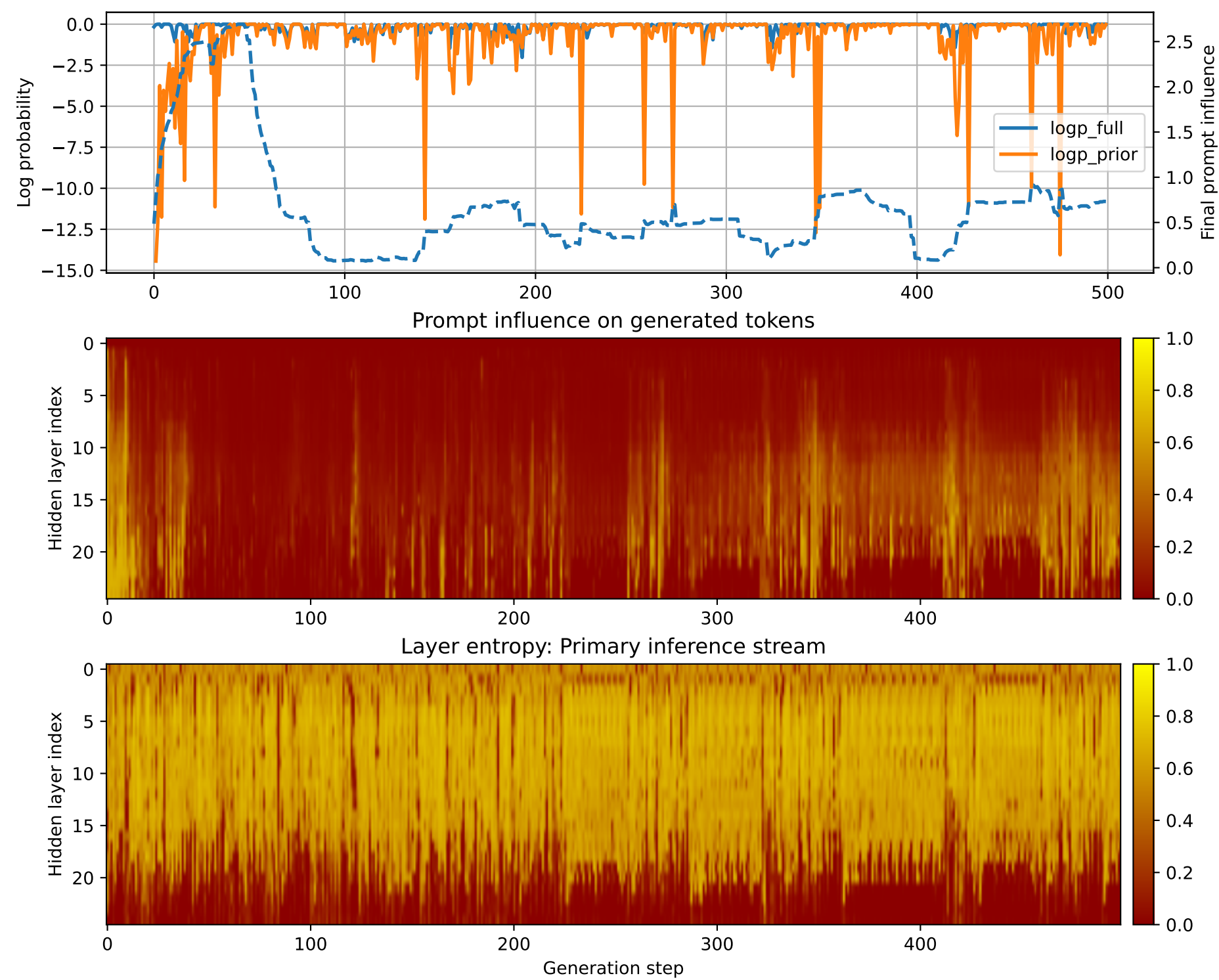
Prompt 1



PROMPT:
Task ID: PYC095 Return ONLY valid Python code. Do not use
Markdown fences. Do not include prose,
explanations, examples, print statements, or tests. Your code
must contain this exact check-token line: #
CHECK_TOKEN: PYC095 Your code must contain this required symbol:
def parse_log_events Implement this task:
write function parse_log_events(text) extracting timestamp,
level, message dicts.. Start your answer with the
check-token line, then provide the Python implementation.

MODEL RESPONSE:
CHECK_TOKEN: PYC095 # Implement the function parse_log_events
to extract timestamp, level, and message
def parse_log_events(text):
 # Check if the text is a valid log event
 string if not text.startswith('#') or not text.endswith('#'):
 raise ValueError("Log event string
must start with '#' and end with '#')") # Split the text
into a list of log event tokens tokens =
text.split('#') # Check if the number of tokens is equal
to 3 if len(tokens) != 3: raise
ValueError("Log event string must contain exactly 3 tokens:
timestamp, level, and message") # Extract
the timestamp from the first token timestamp = tokens[0]
Extract the level from the second
token level = tokens[1] # Extract the message from
the third token message = tokens[2]
Return the extracted dictionaries return {"timestamp":
timestamp, "level": level, "message": message}

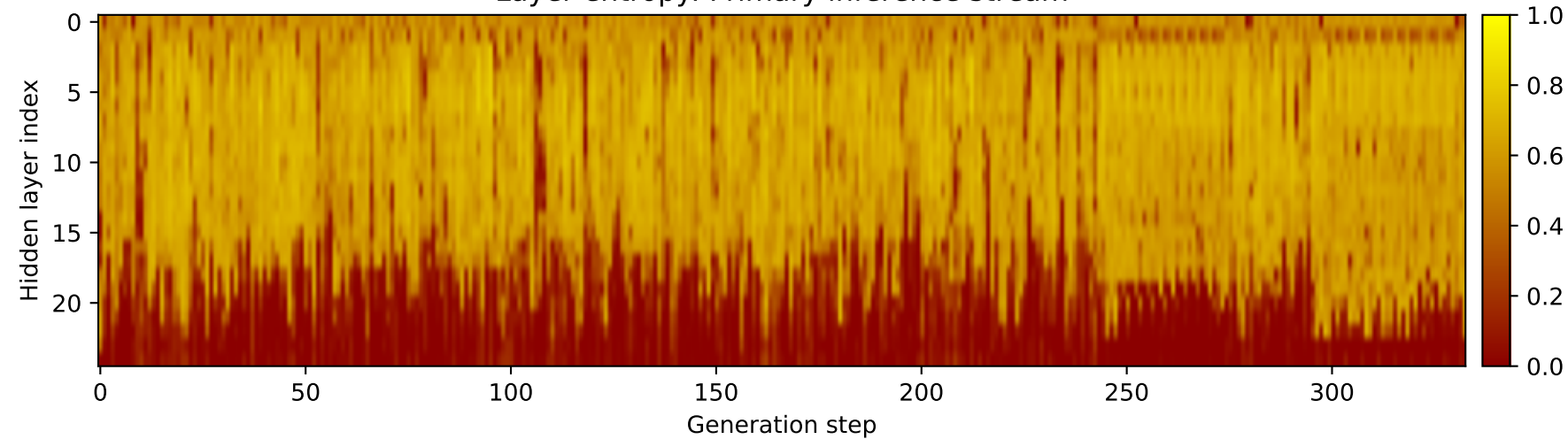
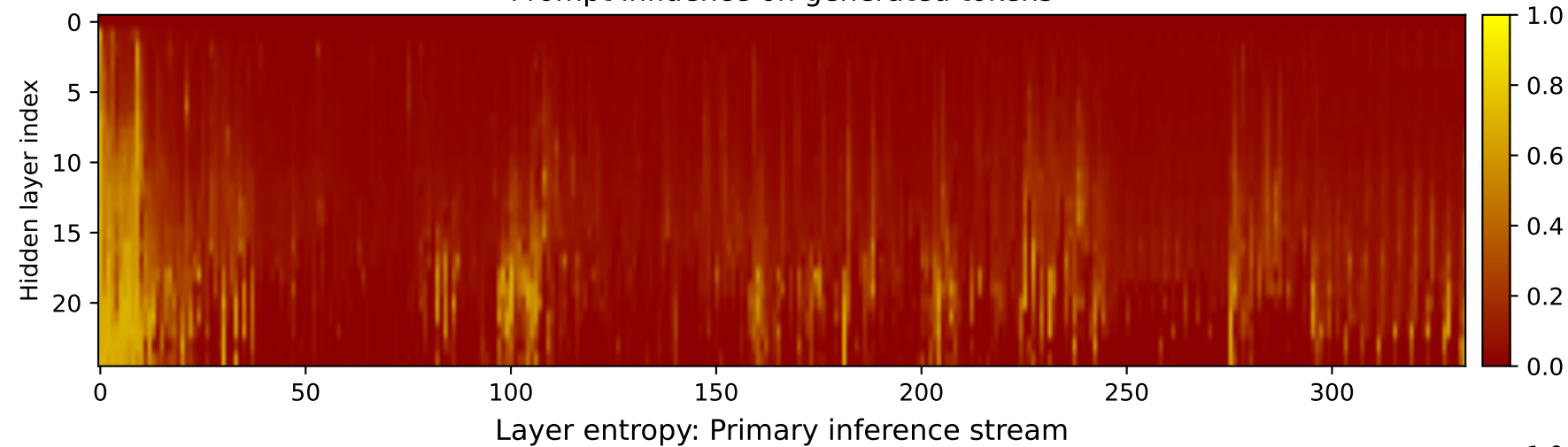
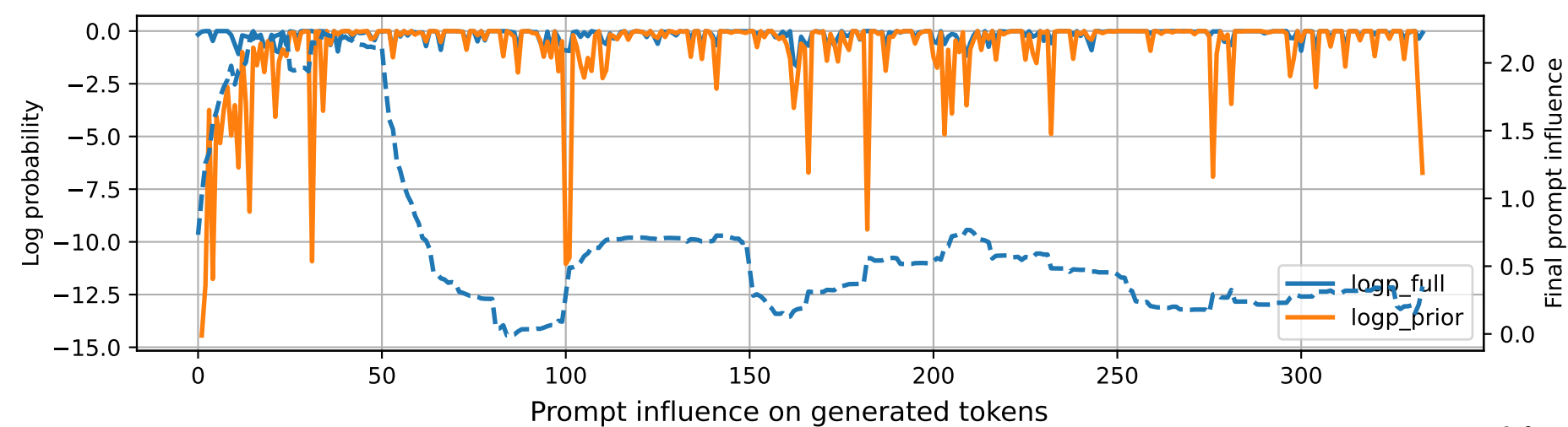
Prompt 1



PROMPT:
Task ID: PYC094 Return ONLY valid Python code. Do not use
Markdown fences. Do not include prose,
explanations, examples, print statements, or tests. Your code
must contain this exact check-token line: #
CHECK_TOKEN: PYC094 Your code must contain this required symbol:
def detect_anomalies_zscore Implement this
task: write function detect_anomalies_zscore(xs, threshold=3.0)..
Start your answer with the check-token
line, then provide the Python implementation.

MODEL RESPONSE:
CHECK_TOKEN: PYC094 # Function to detect anomalies in z-score
data using the z-score method def
detect_anomalies_zscore(xs, threshold=3.0): """ Detect
anomalies in z-score data using the z-score
method. Args: xs (list): A list of z-score values
for a dataset. threshold (float): The
threshold value for identifying anomalies. Defaults to 3.0.
Returns: list: A list of anomaly
scores, where each score indicates the z-score of a data point
above or below the threshold. """ #
Calculate z-scores for each data point z_scores = [(x -
mean(xs)) / std(xs) for x in xs] # Define a
function to determine if a data point is an anomaly def
is_anomaly(x): return x > threshold #
Apply the z-score threshold to each z-score and determine if it
is an anomaly anomalies = [x for x in
z_scores if is_anomaly(x)] return anomalies # Example usage
xs = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
threshold = 3.0 anomalies = detect_anomalies_zscore(xs,
threshold) print(anomalies) # Output: [4.0, 5.0, 6.0,
7.0, 8.0, 9.0, 10.0] # Example usage with a different threshold
value threshold = 2.0 anomalies =
detect_anomalies_zscore(xs, threshold) print(anomalies) #
Output: [1.0, 2.0, 3.0, 4.0, 5.0, 6.0, 7.0, 8.0,
9.0, 10.0] # Example usage with a list of non-anomalous data
points xs = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
threshold = 2.0 anomalies = detect_anomalies_zscore(xs,
threshold) print(anomalies) # Output: [1.0, 2.0,

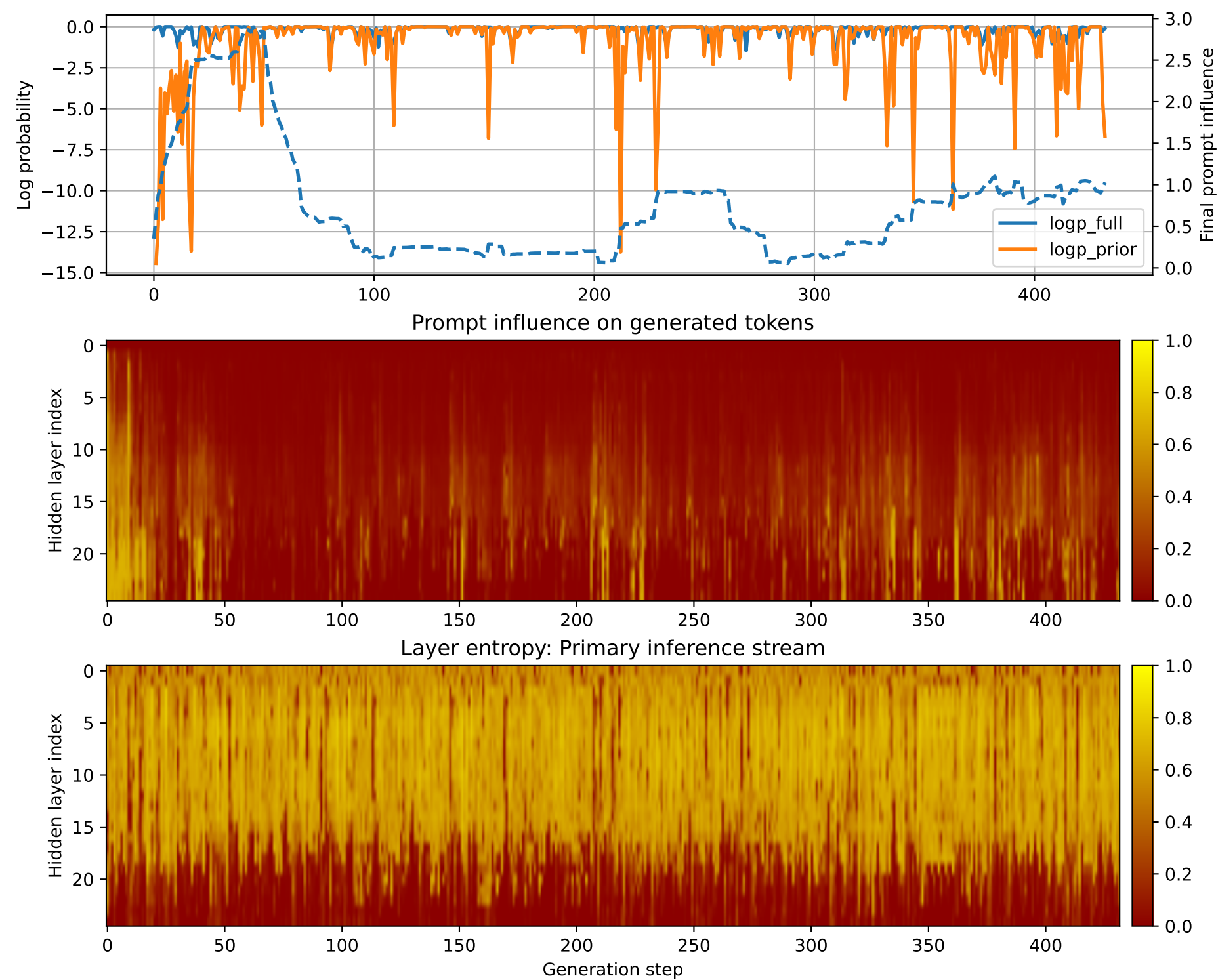
Prompt 1



PROMPT:
Task ID: PYC093 Return ONLY valid Python code. Do not use
Markdown fences. Do not include prose,
explanations, examples, print statements, or tests. Your code
must contain this exact check-token line: #
CHECK_TOKEN: PYC093 Your code must contain this required symbol:
def rolling_mean Implement this task: write
function rolling_mean(xs, window) returning list with None until
window full.. Start your answer with the
check-token line, then provide the Python implementation.

MODEL RESPONSE:
CHECK_TOKEN: PYC093 # Function to calculate rolling mean of a
list of numbers, with a specified window size
def rolling_mean(xs, window): """ Calculates the rolling
mean of a list of numbers, with a specified
window size. Args: xs (list): A list of numbers.
window (int): The size of the rolling
window. Returns: list: A list containing the rolling
mean of the numbers in the input list, with
None until the window is full. """ # Check if the window
size is valid if window < 1:
raise ValueError("Window size must be a positive integer.")
Initialize an empty list to store the
rolling mean rolling_mean = [] # Iterate over the input
list, adding the current number to the
rolling mean if it is within the window for num in xs:
if num <= window:
rolling_mean.append(num) else: # If the
window is full, add the last element of the
rolling mean rolling_mean.append(rolling_mean[-1])
Return the rolling mean list return
rolling_mean # Example usage xs = [1, 2, 3, 4, 5, 6, 7, 8, 9,
10] window = 3 result = rolling_mean(xs,
window) print(result) # Output: [3, 6, 9, 12, 15, 18, 21, 24,
27, 30]

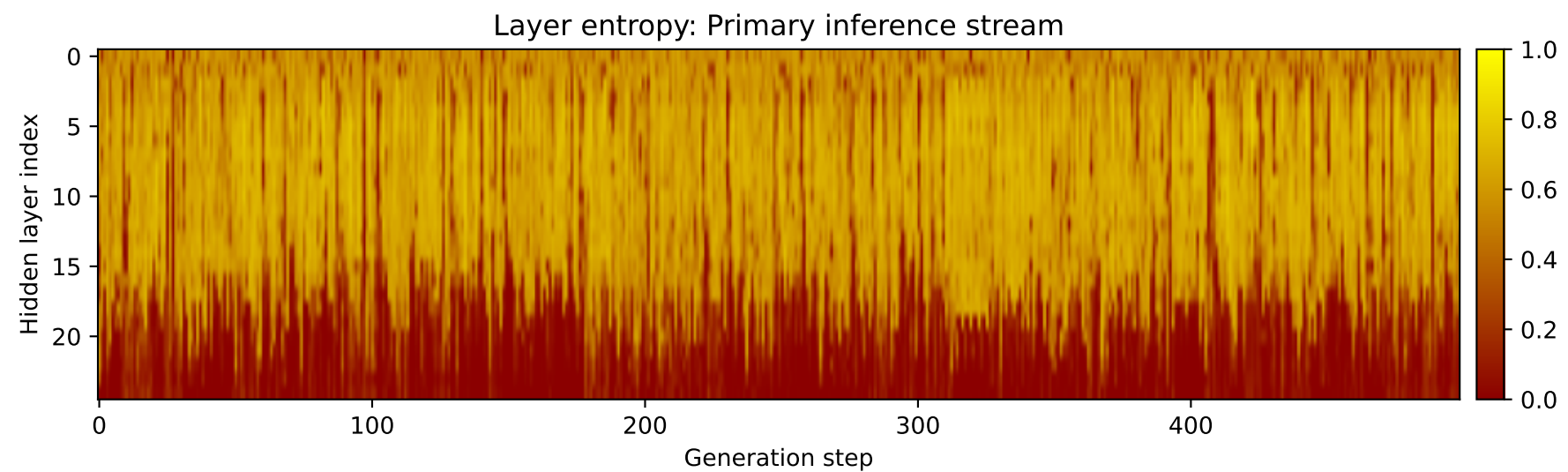
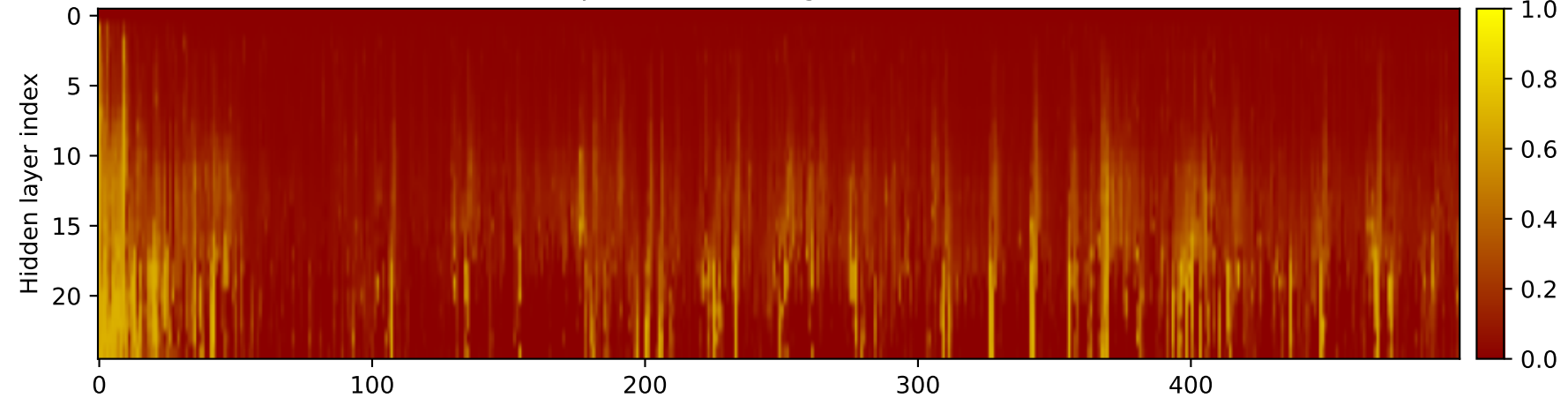
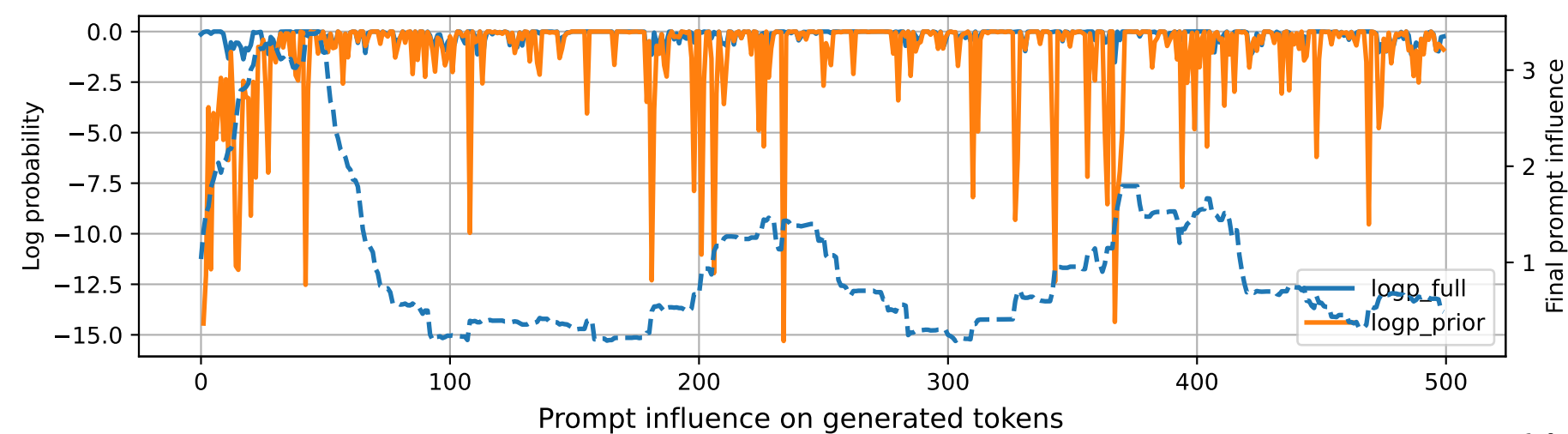
Prompt 1



PROMPT:
Task ID: PYC092 Return ONLY valid Python code. Do not use
Markdown fences. Do not include prose,
explanations, examples, print statements, or tests. Your code
must contain this exact check-token line: #
CHECK_TOKEN: PYC092 Your code must contain this required symbol:
def train_test_split_rows Implement this
task: write function train_test_split_rows(rows, test_fraction,
seed).. Start your answer with the check-
token line, then provide the Python implementation.

MODEL RESPONSE:
CHECK_TOKEN: PYC092 # Function to train and test split rows
using the train_test_split function from the
sklearn.model_selection module def train_test_split_rows(rows,
test_fraction, seed): # Check if the input
arguments are valid if not isinstance(rows, list) or not
isinstance(test_fraction, float) or not
isinstance(seed, int): raise ValueError("All input
arguments must be of type list, float, and
integer.") # Check if the test_fraction is within the valid
range (0.2 to 1) if test_fraction < 0.2
or test_fraction > 1: raise ValueError("Test_fraction
must be between 0.2 and 1.") # Check if the
seed is within the valid range (0 to 2**31 - 1) if seed < 0
or seed > 2**31 - 1: raise
ValueError("Seed must be between 0 and 2**31 - 1.") # Split
the rows into training and testing sets
train_size = int(len(rows) * test_fraction) train_set =
rows[:train_size] test_set = rows[train_size:]
Shuffle the training set randomly random.shuffle(train_set)
Split the test set into training and
validation sets test_size = int(len(test_set) *
test_fraction) train_val_set = train_set[:test_size]
val_set = test_set[test_size:] # Create an instance of the
train_test_split function train_test_split
= train_test_split(train_set, test_size, seed) return
train_test_split # Example usage rows = [1, 2, 3,
4, 5] test_fraction = 0.3 seed = 42 # Train and test split the
rows train_test_split =
train_test_split_rows(rows, test_fraction, seed) print("Training
set:") print(train_test_split) # Validate
the split val_set = train_test_split.val_set print("Validation
set:") print(val_set)

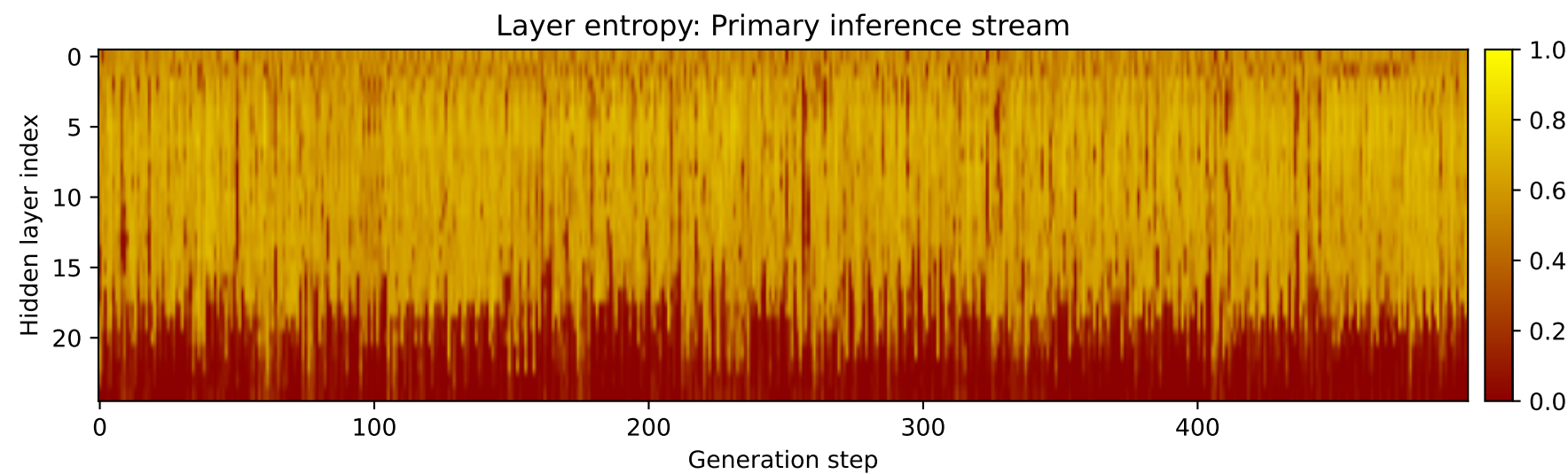
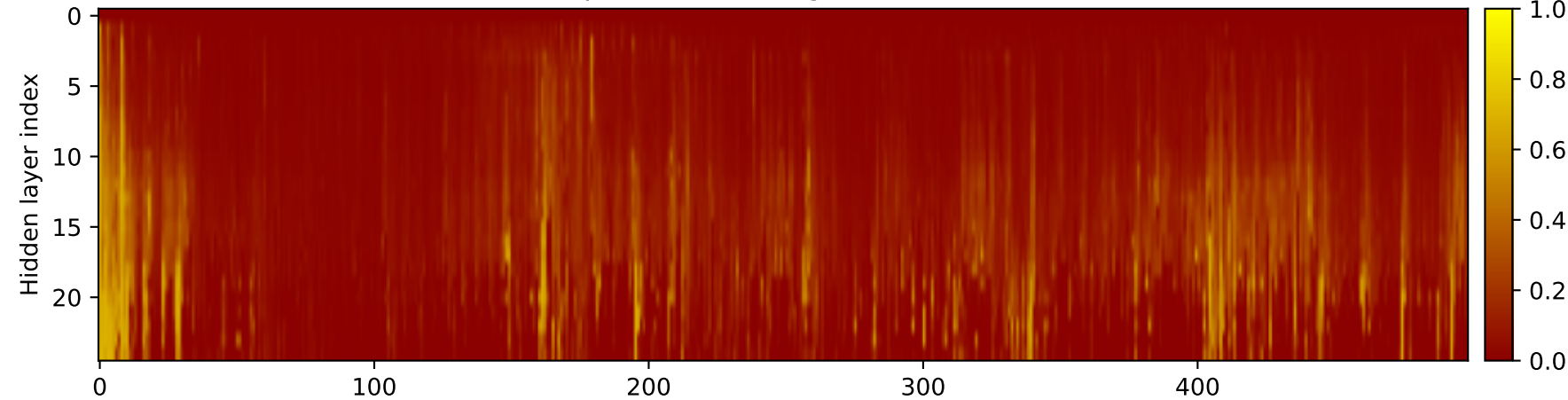
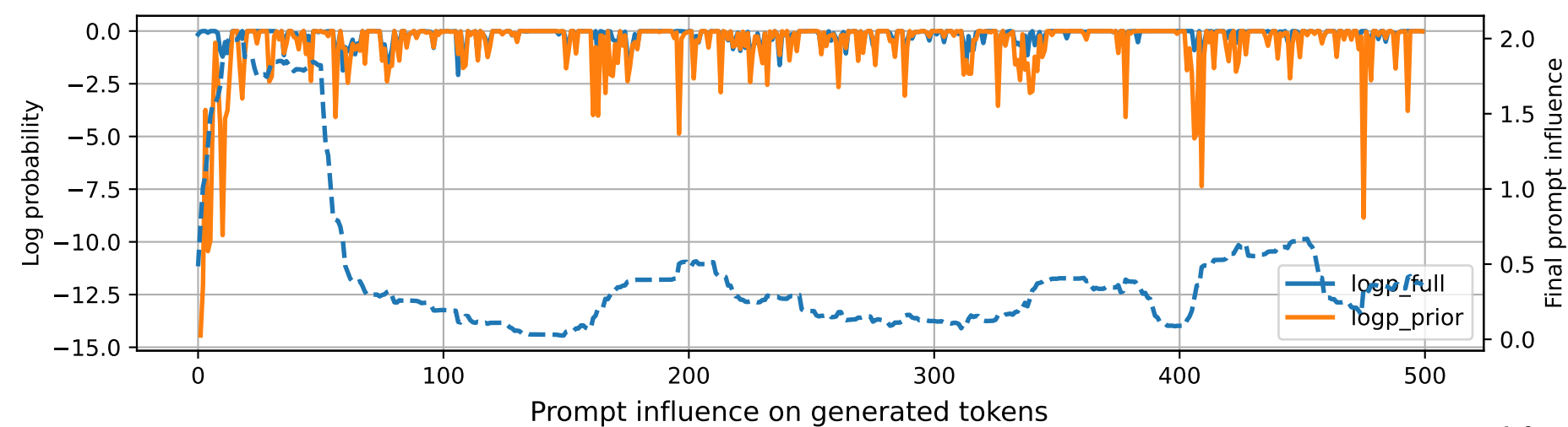
Prompt 1



PROMPT:
Task ID: PYC091 Return ONLY valid Python code. Do not use
Markdown fences. Do not include prose,
explanations, examples, print statements, or tests. Your code
must contain this exact check-token line: #
CHECK_TOKEN: PYC091 Your code must contain this required symbol:
def mini_sql_select Implement this task:
write function mini_sql_select(rows, select_cols, where_fn=None,
order_by=None).. Start your answer with the
check-token line, then provide the Python implementation.

MODEL RESPONSE:
CHECK_TOKEN: PYC091 # Function to perform mini SQL select
statement with given rows, select columns, and
where and order by clauses def mini_sql_select(rows, select_cols,
where_fn=None, order_by=None): # Check
if the number of rows provided is valid if len(rows) !=
len(select_cols): raise ValueError("Number
of rows must be equal to the number of selected columns") #
Check if the where clause is provided and is
a valid function if not isinstance(where_fn, (str,
callable)): raise ValueError("Where clause must
be a valid function or a string") # Check if the order_by
clause is provided and is a valid function or a
string if not isinstance(order_by, (str, callable)):
raise ValueError("Order by clause must be a
valid function or a string") # Create a mini SQL statement
with the given parameters
mini_sql_statement = f"SELECT {'', '.join(select_cols)} FROM
{rows[0]}" if select_cols else f"SELECT * FROM
{rows}" # Apply the where clause if provided if
where_fn: mini_sql_statement += f" WHERE
{where_fn}" # Apply the order_by clause if provided if
order_by: mini_sql_statement += f"
ORDER BY {order_by}" # Execute the mini SQL statement
result = execute_sql(mini_sql_statement) #
Return the query results return result.fetchall() # Example
usage rows = [1, 2, 3, 4, 5] select_cols =
['column1', 'column2', 'column3'] where_fn = lambda x: x % 2 == 0
order_by = lambda x: x try:
mini_sql_select(rows, select_cols, where_fn, order_by) except
ValueError as e: print(e) # Output: Number
of rows must be equal to the number of selected columns ``` In
this implementation, the `mini\_sql\_select`
function takes in the following parameters: 1. `rows`: A list of
integers representing the rows to be
selected from the database. 2. `select\_cols`: A list of strings
representing the columns to be selected from
the database. 3. `where\_fn`: A function or string that defines
the condition to filter the rows based on a
given condition. If a function is provided, it should take a
single

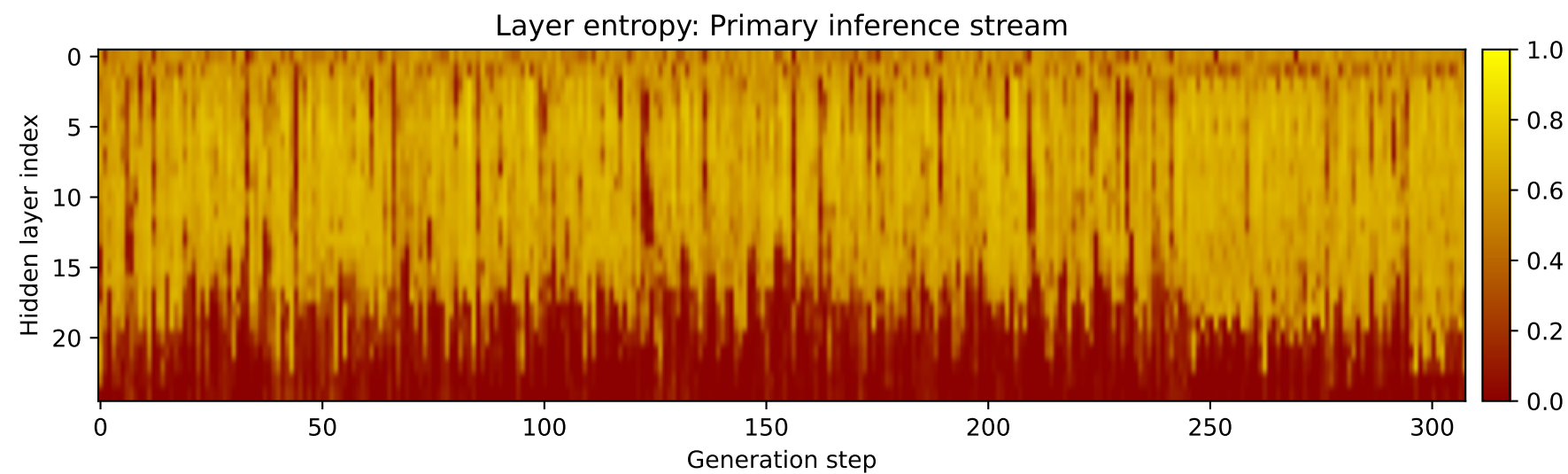
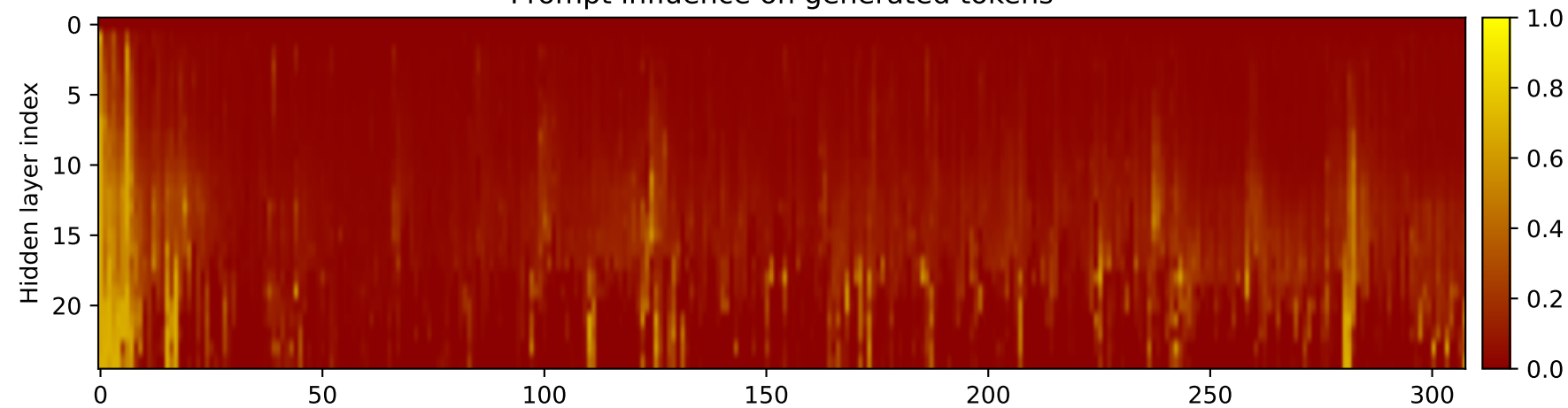
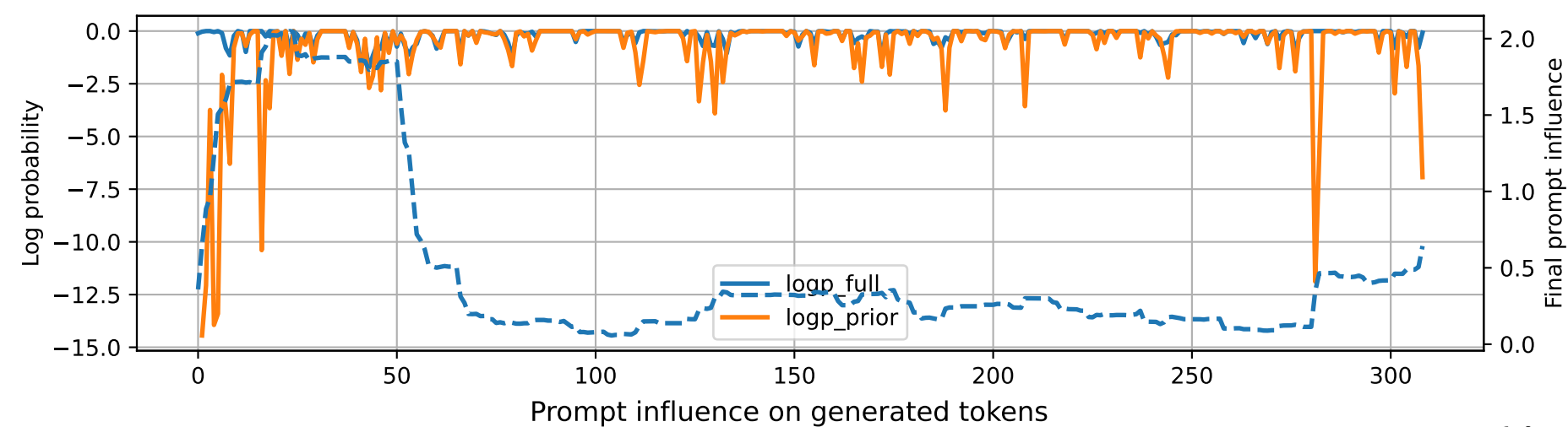
Prompt 1



PROMPT:
Task ID: PYC090 Return ONLY valid Python code. Do not use
Markdown fences. Do not include prose,
explanations, examples, print statements, or tests. Your code
must contain this exact check-token line: #
CHECK_TOKEN: PYC090 Your code must contain this required symbol:
def compute_confusion_matrix Implement this
task: write function
compute_confusion_matrix(y_true,y_pred,labels).. Start your
answer with the check-token
line, then provide the Python implementation.

MODEL RESPONSE:
CHECK TOKEN: compute_confusion_matrix # Python implementation
for compute_confusion_matrix function def
compute_confusion_matrix(y_true, y_pred, labels): """
Compute the confusion matrix for a given set of
true labels (y_true) and predicted labels (y_pred) using the
confusion matrix formula:
confusion_matrix(y_true, y_pred) = | true_positives |
true_negatives | false_positives | false_negatives
|-----|-----|-----|-----|
| y_true_i
| y_pred_i | y_true_i | y_pred_i |
| y_true_j | y_pred_j
| y_true_j | y_pred_j | | ...
| ... | ... |
| ... | Parameters: y_true (list): List
of true labels (one per row) y_pred
(list): List of predicted labels (one per row) labels (list):
List of corresponding labels (one per row)
Returns: confusion_matrix (list): A list of confusion
matrices for each label, where each element is a 2x2
matrix representing the number of true positives, true negatives,
false positives, and false negatives for
each label. """ # Initialize confusion matrix
confusion_matrix = [[0 for _ in range(4)] for _ in
range(len(labels))] # Compute confusion matrix for each
label for i in range(len(labels)):
for j in range(len(labels)): # Get true positive,
true negative, false positive, and false
negative counts true_positives = sum([1 for true in
y_true if true == label[i]])
true_negatives = sum([1 for true in y_true if true != label[i]])
false_positives = sum([1 for true
in y_pred if true == label[i]]) false_negatives =
sum([1 for true in y_pred if true != label[i]])
Add confusion matrix element confusion_matrix[i][j]
= (true_positives, true_negatives,
false_positives, false_negatives) return confusion_matrix #
Example usage y_true = [1, 0, 1, 1] y_pred =
[1, 1, 0, 1] labels = ['A', 'B', 'C', 'D'] confusion_matrix =
compute_confusion_matrix(y_true,

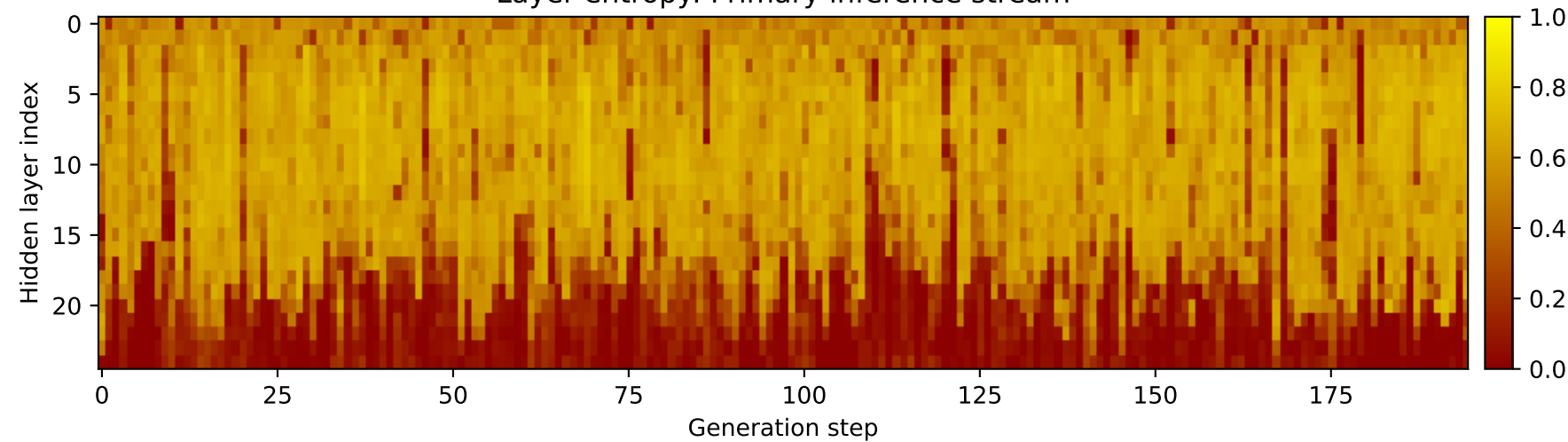
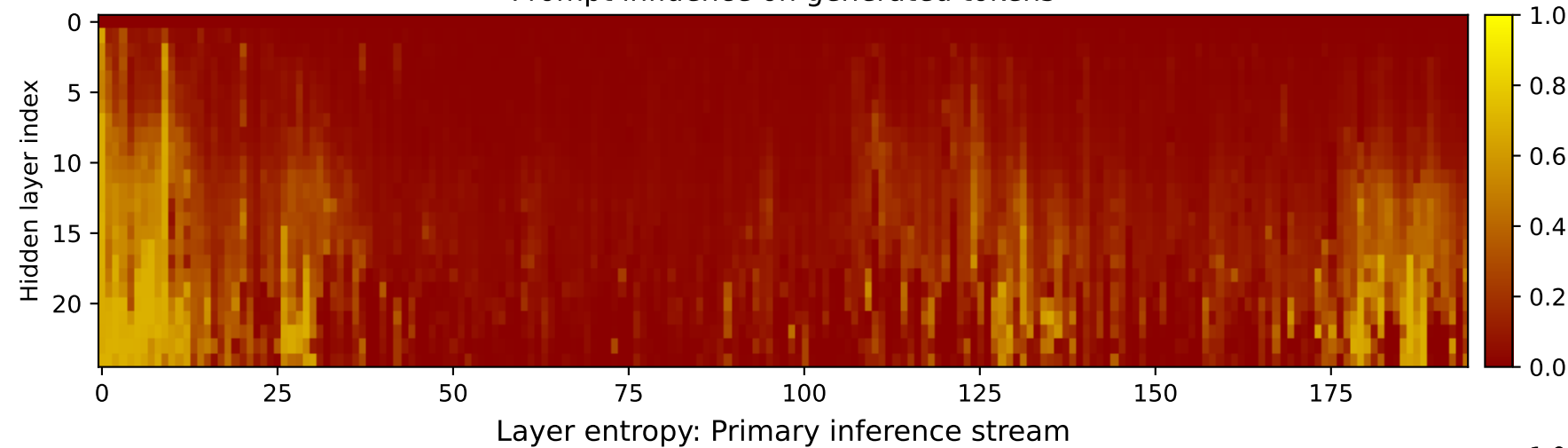
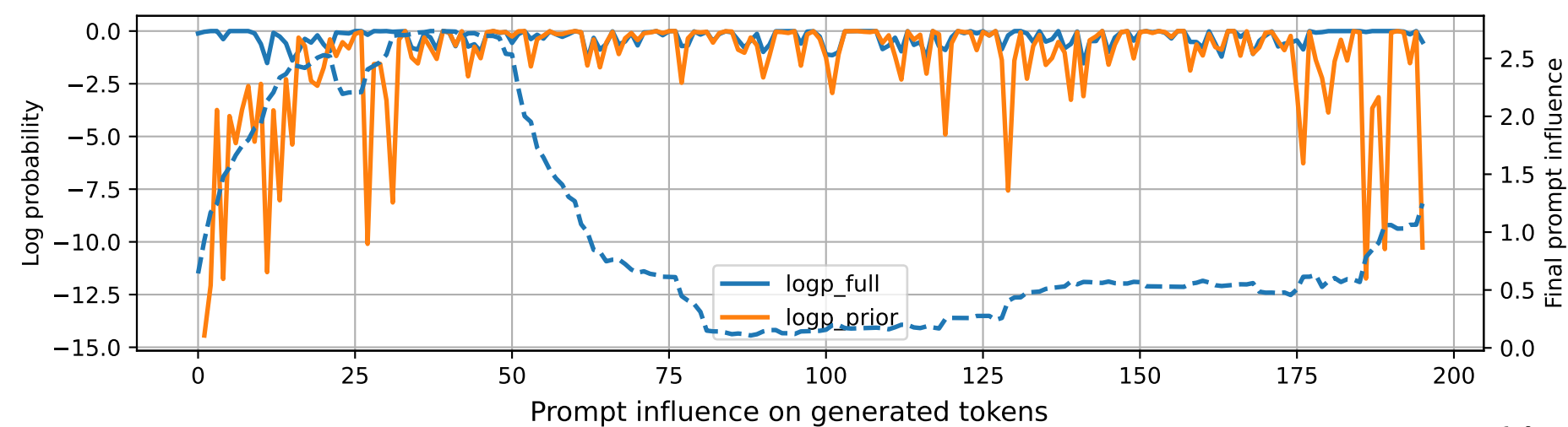
Prompt 1



PROMPT:
Task ID: PYC089 Return ONLY valid Python code. Do not use
Markdown fences. Do not include prose,
explanations, examples, print statements, or tests. Your code
must contain this exact check-token line: #
CHECK_TOKEN: PYC089 Your code must contain this required symbol:
def reconcile_inventory Implement this
task: write function reconcile_inventory(expected, observed)..
Start your answer with the check-token line,
then provide the Python implementation.

MODEL RESPONSE:
CHECK_TOKEN: reconcile_inventory # Function to reconcile
inventory def reconcile_inventory(expected,
observed): """ This function takes in two lists,
'expected' and 'observed', representing the inventory
of a store. It returns a new list that contains the items in
'expected' that are not present in 'observed'.
Args: expected (list): A list of items expected to be in
the inventory. observed (list): A
list of items observed in the inventory. Returns:
list: A list of items in 'expected' that are
not present in 'observed'. """ # Check if the lengths of
the lists are equal if len(expected) !=
len(observed): raise ValueError("The lengths of the
expected and observed lists must be equal.")
Create a new list to store the non-present items
non_present_items = [] # Iterate over the expected
items for item in expected: # Check if the item is
not present in the observed items if
item not in observed: non_present_items.append(item)
Return the non-present items
return non_present_items # Example usage expected = [1, 2, 3, 4,
5] observed = [1, 2, 3, 6, 7]
non_present_items = reconcile_inventory(expected, observed)
print(non_present_items) # Output: [4, 5, 6, 7]

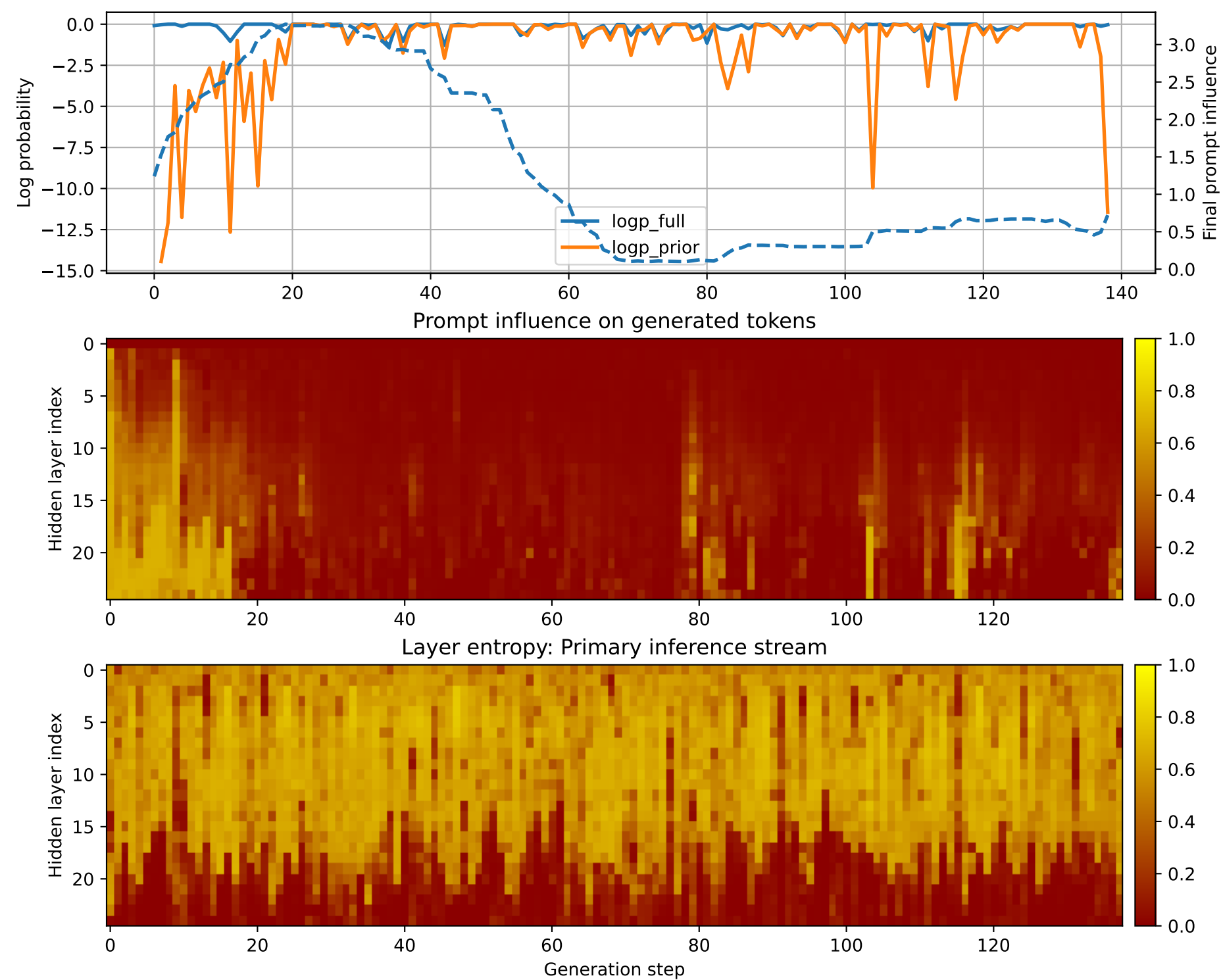
Prompt 1



PROMPT:
Task ID: PYC088 Return ONLY valid Python code. Do not use
Markdown fences. Do not include prose,
explanations, examples, print statements, or tests. Your code
must contain this exact check-token line: #
CHECK_TOKEN: PYC088 Your code must contain this required symbol:
def rate_limit Implement this task: write
function rate_limit(calls, per_seconds) decorator.. Start your
answer with the check-token line, then provide
the Python implementation.

MODEL RESPONSE:
CHECK_TOKEN: PYC088 # Rate Limit decorator for handling rate
limiting in Python import time def
rate_limit(calls, per_seconds): # Check if the number of
calls exceeds the maximum allowed limit if
calls >= per_seconds * 1000: # Return a custom error
message indicating that the rate limit has been
exceeded raise RateLimitExceeded("Rate limit exceeded.
Maximum allowed calls: " + str(per_seconds) + "
seconds, exceeded calls: " + str(calls)) # Otherwise, return
the number of calls allowed return calls
Example usage: @rate_limit(100, 1) def increment_counter():
Increment the counter by 1 counter +=
1 # Call the function increment_counter() # Output: 100 # Rate
limit exceeded. Maximum allowed calls: 1
seconds, exceeded calls: 100

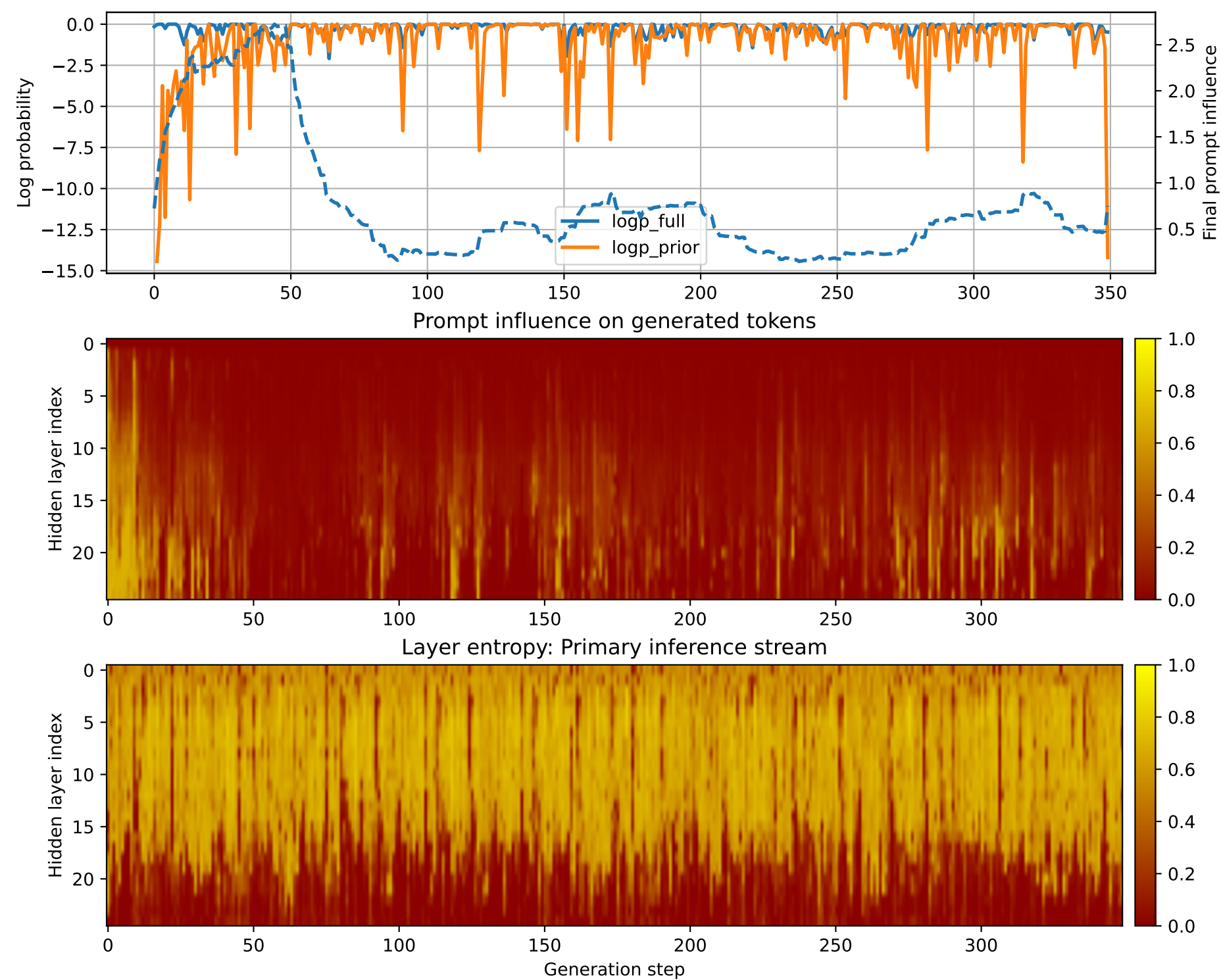
Prompt 1



PROMPT:
Task ID: PYC087 Return ONLY valid Python code. Do not use
Markdown fences. Do not include prose,
explanations, examples, print statements, or tests. Your code
must contain this exact check-token line: #
CHECK_TOKEN: PYC087 Your code must contain this required symbol:
async def fetch_all Implement this task:
write async function fetch_all(urls, session) using
asyncio.gather.. Start your answer with the check-token
line, then provide the Python implementation.

MODEL RESPONSE:
CHECK_TOKEN: PYC087 # async def fetch_all(urls, session):
async def fetch_all(urls, session): async
with session.get(urls, timeout=5) as response: if
response.status_code == 200: return
await response.json() else: raise
Exception(f"Failed to fetch data from {urls}:
{response.status_code}") # Example usage: async def main():
urls = ["https://example.com",
"https://example2.com"] session = aiohttp.ClientSession()
data = await fetch_all(urls, session)
print(data) if __name__ == "__main__": asyncio.run(main())

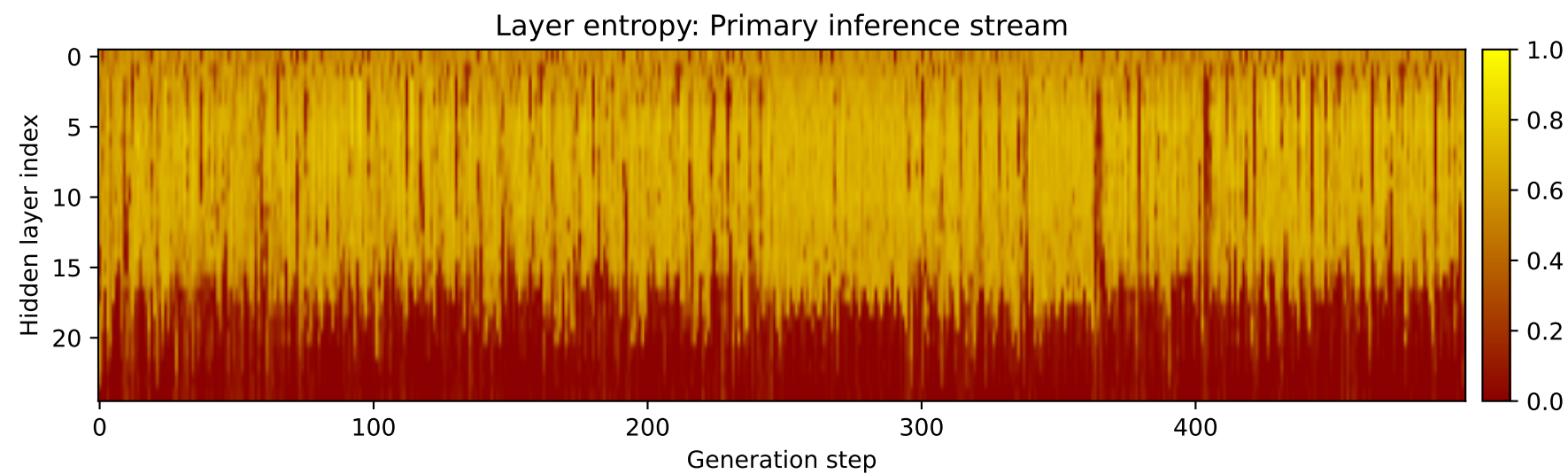
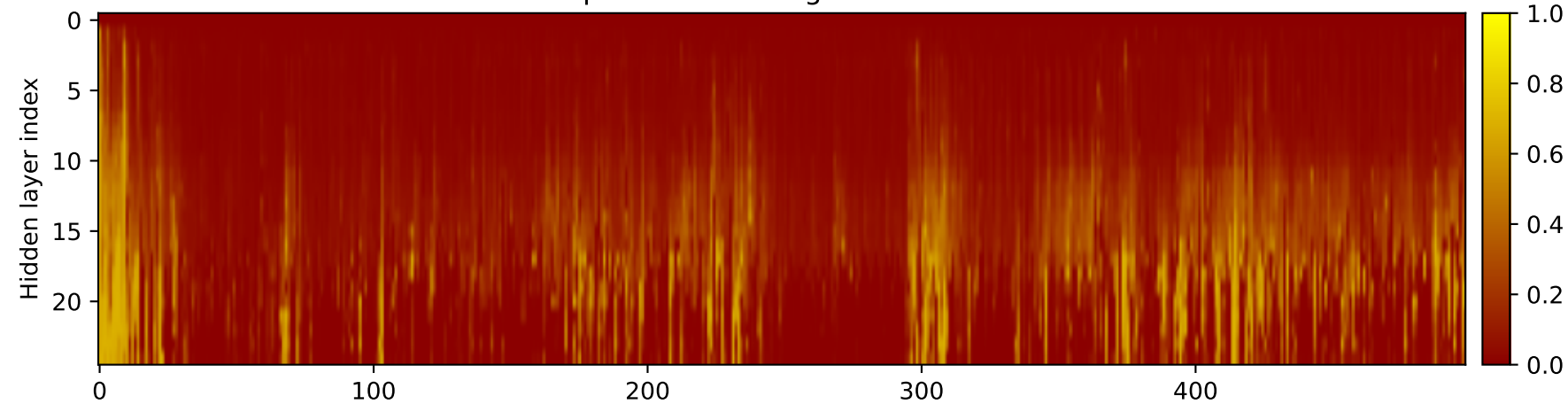
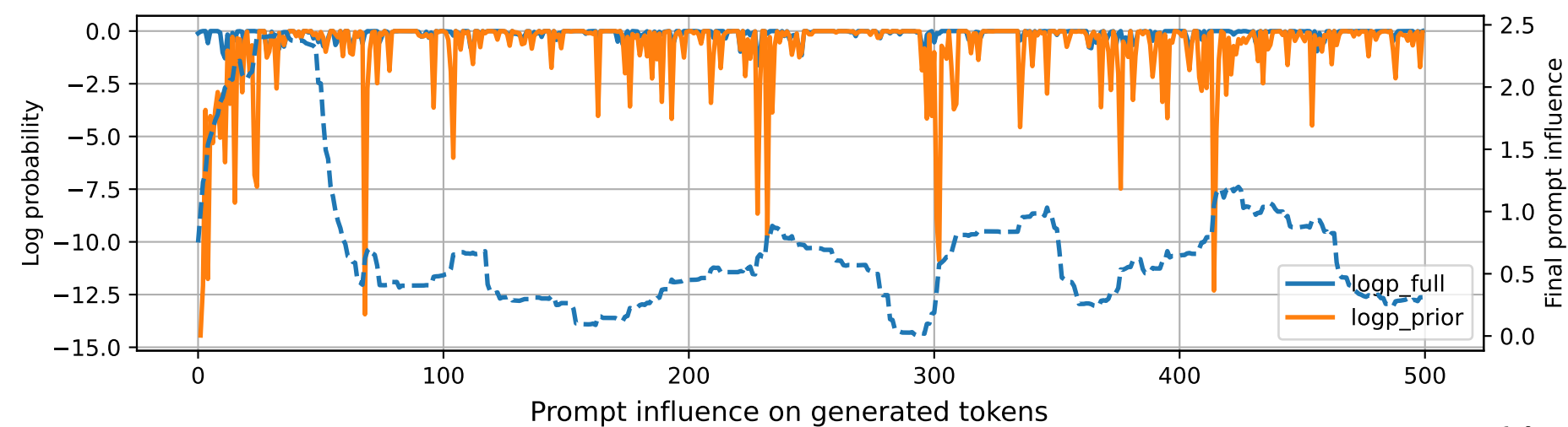
Prompt 1



PROMPT:
Task ID: PYC086 Return ONLY valid Python code. Do not use
Markdown fences. Do not include prose,
explanations, examples, print statements, or tests. Your code
must contain this exact check-token line: #
CHECK_TOKEN: PYC086 Your code must contain this required symbol:
def retry Implement this task: write
function retry(fn, attempts, delay_seconds).. Start your answer
with the check-token line, then provide the
Python implementation.

MODEL RESPONSE:
CHECK_TOKEN: PYC086 # Function to retry a function with a
maximum number of attempts and a delay in seconds
def retry(fn, attempts, delay_seconds): # Check if the
function is defined and has the required arguments
if not fn or not isinstance(fn, (function_type, method_type)):
raise ValueError("Function must be
defined and have the required arguments") # Check if the
maximum number of attempts is provided if
attempts < 1: raise ValueError("Number of attempts must
be greater than 0") # Check if the delay
in seconds is provided if delay_seconds < 0: raise
ValueError("Delay in seconds must be greater
than 0") # Create a retry object with the function,
attempts, and delay retry_obj = Retry(fn,
attempts, delay_seconds) # Iterate through the attempts and
call the function with a delay of 0 for
attempt in range(attempts): try: return fn()
except Exception as e: #
If an exception occurs, add it to the retry object's exceptions
list
retry_obj.exceptions.append(e) # If all attempts fail, raise
a retry exception with the last exception
if retry_obj.exceptions: raise
RetryException(retry_obj.exceptions[-1]) # Return the result
of
the function after all attempts return retry_obj.result #
Example usage try: # Call the retry
function with a maximum of 3 attempts and a delay of 2 seconds
result = retry(lambda x: x * 2, 3, 2)
print(result) except RetryException as e: print("Error:", e)

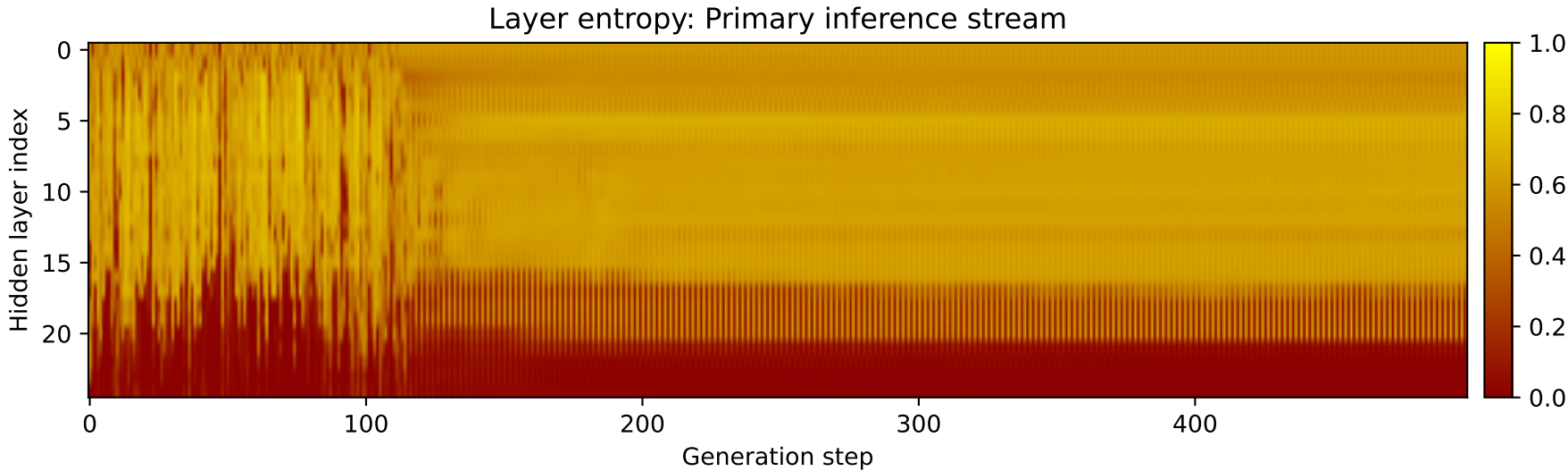
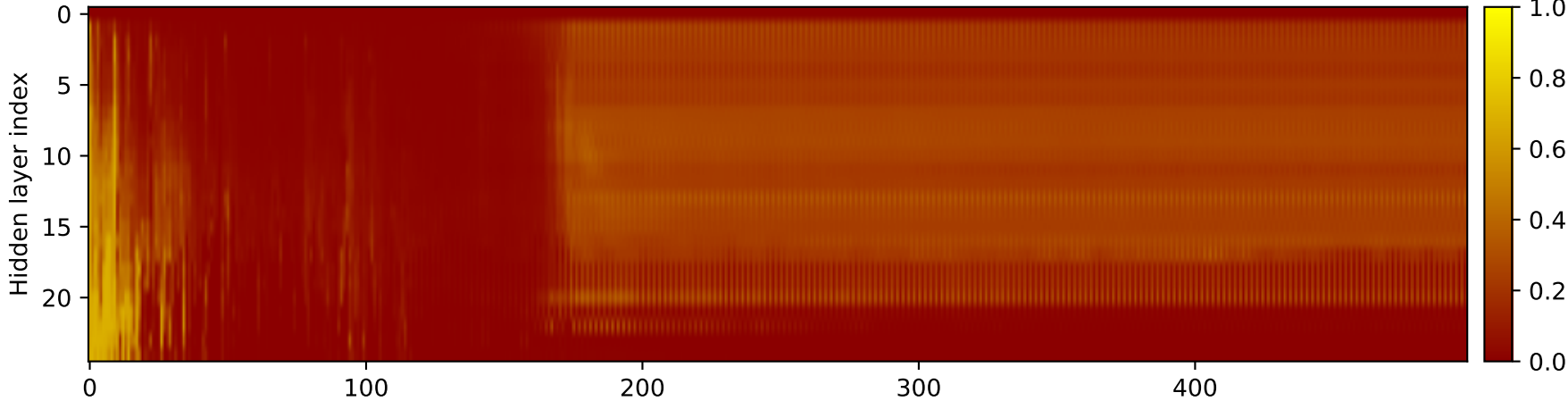
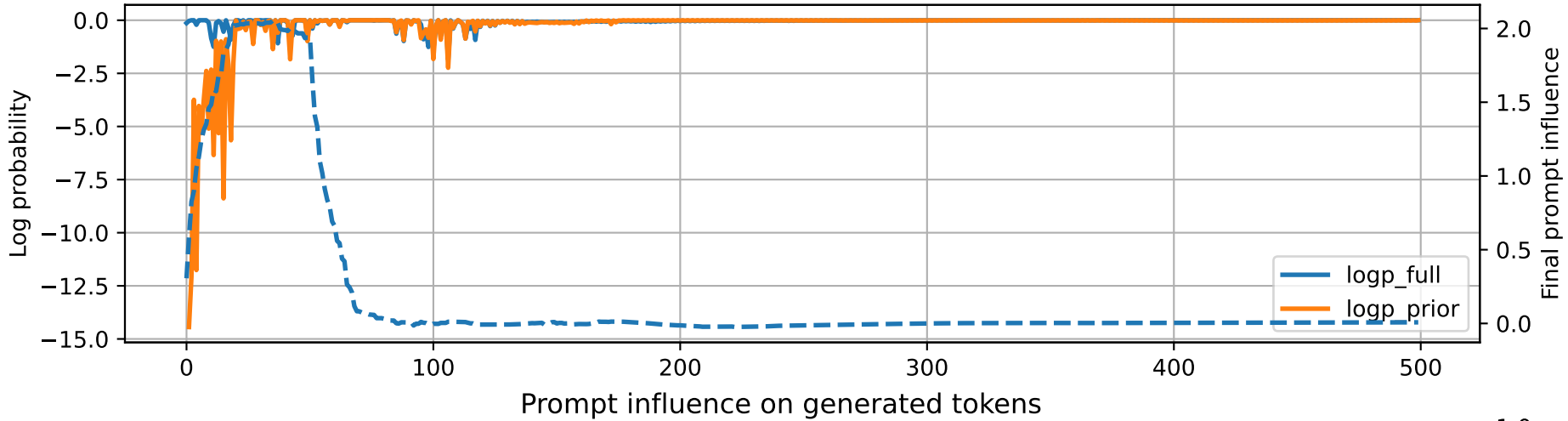
Prompt 1



PROMPT:
Task ID: PYC085 Return ONLY valid Python code. Do not use
Markdown fences. Do not include prose,
explanations, examples, print statements, or tests. Your code
must contain this exact check-token line: #
CHECK_TOKEN: PYC085 Your code must contain this required symbol:
def diff_dicts Implement this task: write
function diff_dicts(old,new) returning added, removed, changed..
Start your answer with the check-token line,
then provide the Python implementation.

MODEL RESPONSE:
CHECK_TOKEN: PYC085 # Function to calculate the differences
between two dictionaries def diff_dicts(old,
new): # Check if the input dictionaries are not empty if
not old or not new: return "Both
dictionaries must have at least one key-value pair" # Create
empty dictionaries to store the added,
removed, and changed keys added = {} removed = {}
changed = {} # Iterate over the keys in the
first dictionary for key in old: # Check if the key
is present in the second dictionary if
key in new: # If the key is present, add it to the
added dictionary added[key] =
old[key] else: # If the key is not present,
add it to the removed dictionary
removed[key] = old[key] # If the key is present in both
dictionaries, check if its value has changed
elif old[key] != new[key]: # If the value has
changed, add it to the changed dictionary
changed[key] = new[key] # Return the differences return
added, removed, changed # Example usage
old_dict = {"key1": "value1", "key2": "value2", "key3": "value3"}
new_dict = {"key1": "value4", "key2":
"value5", "key3": "value6"} # Calculate the differences added,
removed, changed = diff_dicts(old_dict,
new_dict) print("Added keys:", added) print("Removed keys:",
removed) print("Changed keys:", changed) #
Output: {'key1': 'value4', 'key2': 'value5', 'key3': 'value6'}
`` In this implementation, we first check if
the input dictionaries are empty. If either of them is empty, we
return a message indicating that both
dictionaries must have at least one key-value pair. Next, we
create empty dictionaries to store the added,
removed, and changed keys. We iterate over the keys in the first
dictionary and check if each key is present
in the second dictionary. If the key is present, we add it to the
added dictionary with the corresponding
value from the first dictionary. If the key is not present, we
add it to the removed dictionary with the
corresponding value from the first dictionary. If the key is
present in both dictionaries, we check if

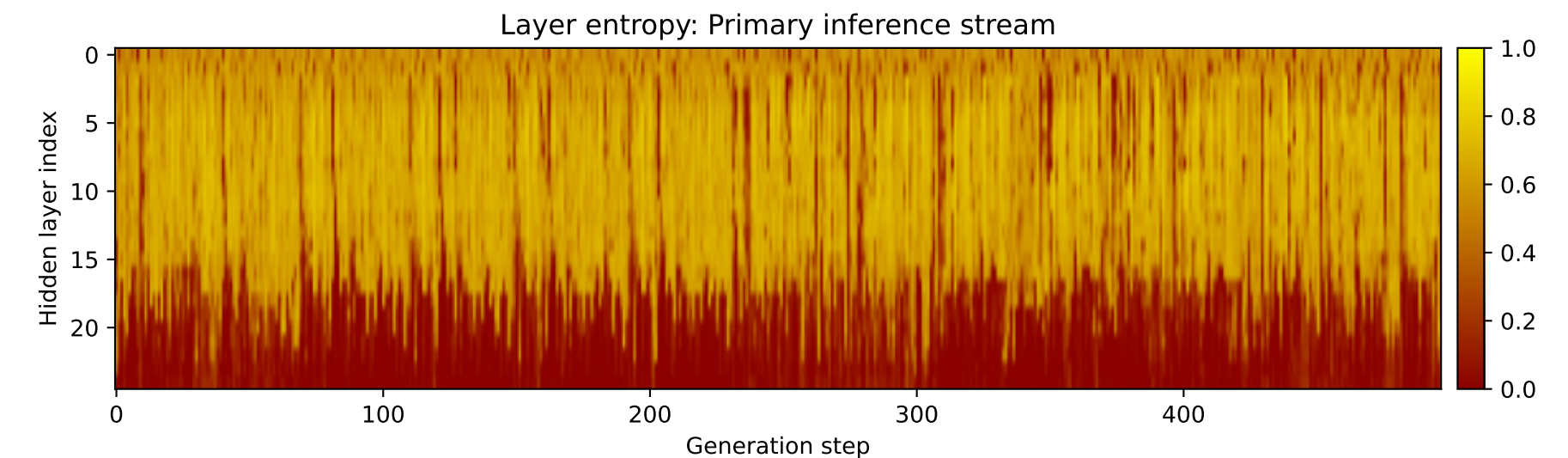
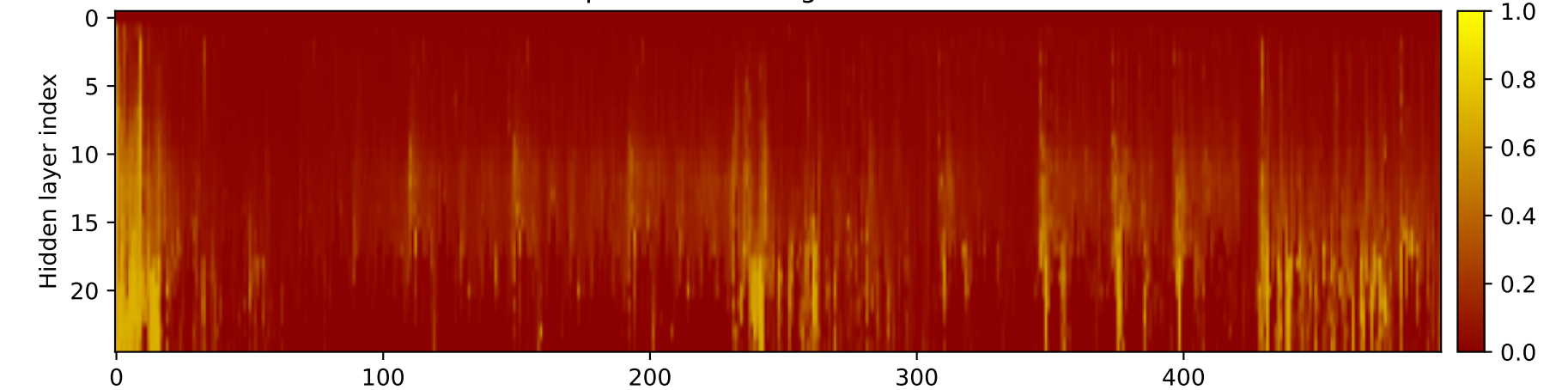
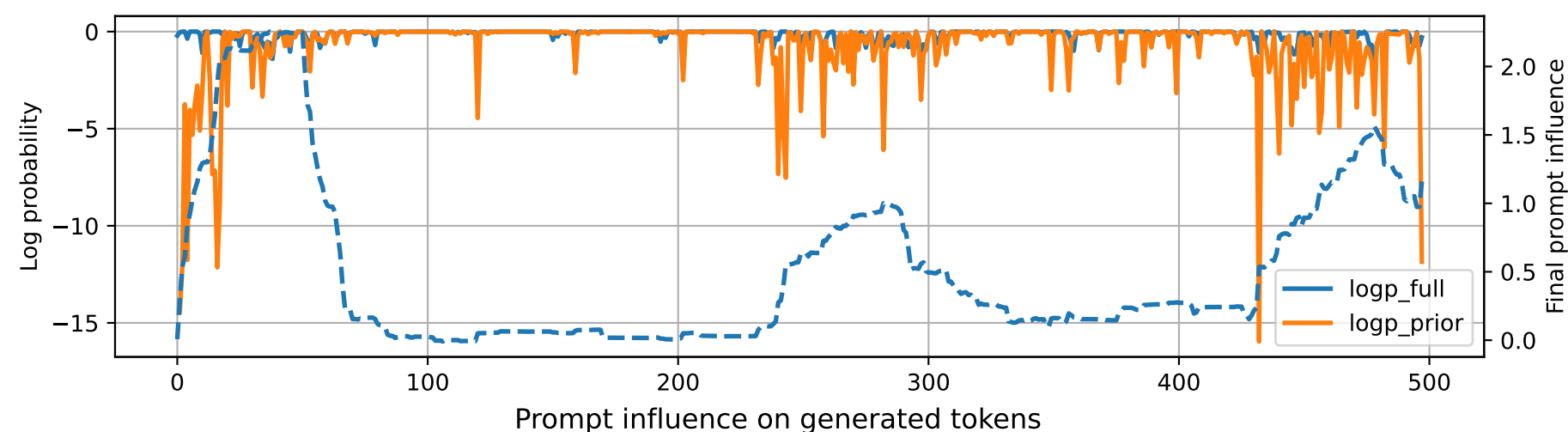
Prompt 1



```
PROMPT:
Task ID: PYC084 Return ONLY valid Python code. Do not use
Markdown fences. Do not include prose,
explanations, examples, print statements, or tests. Your code
must contain this exact check-token line: #
CHECK_TOKEN: PYC084 Your code must contain this required symbol:
def build_markdown_table Implement this
task: write function build_markdown_table(rows, columns).. Start
your answer with the check-token line, then
provide the Python implementation.
```

[illegible]

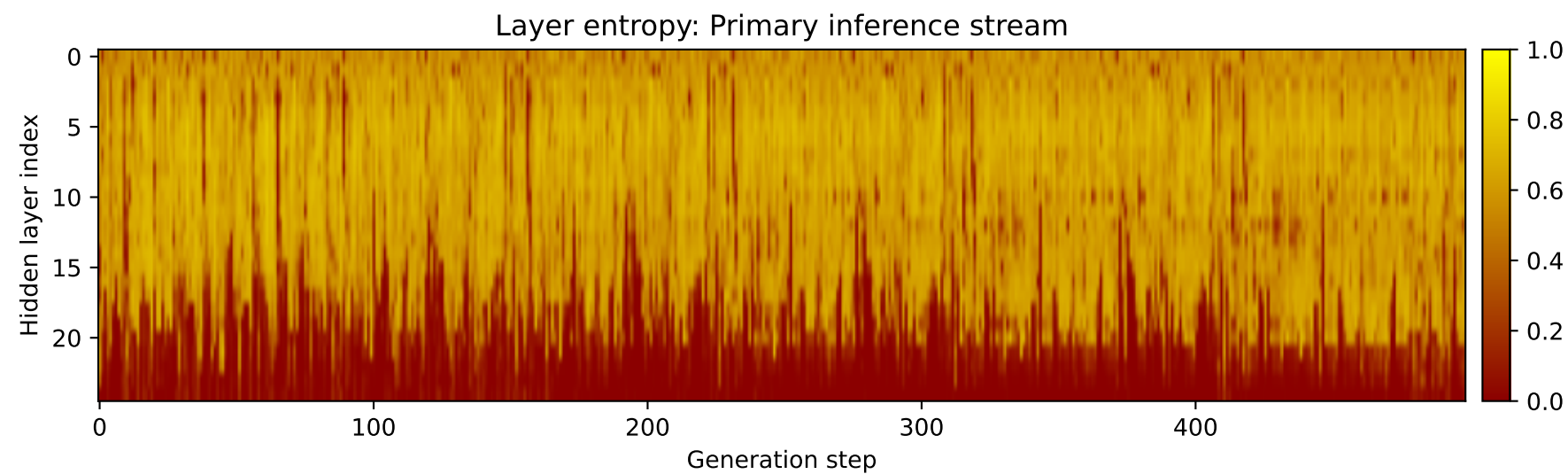
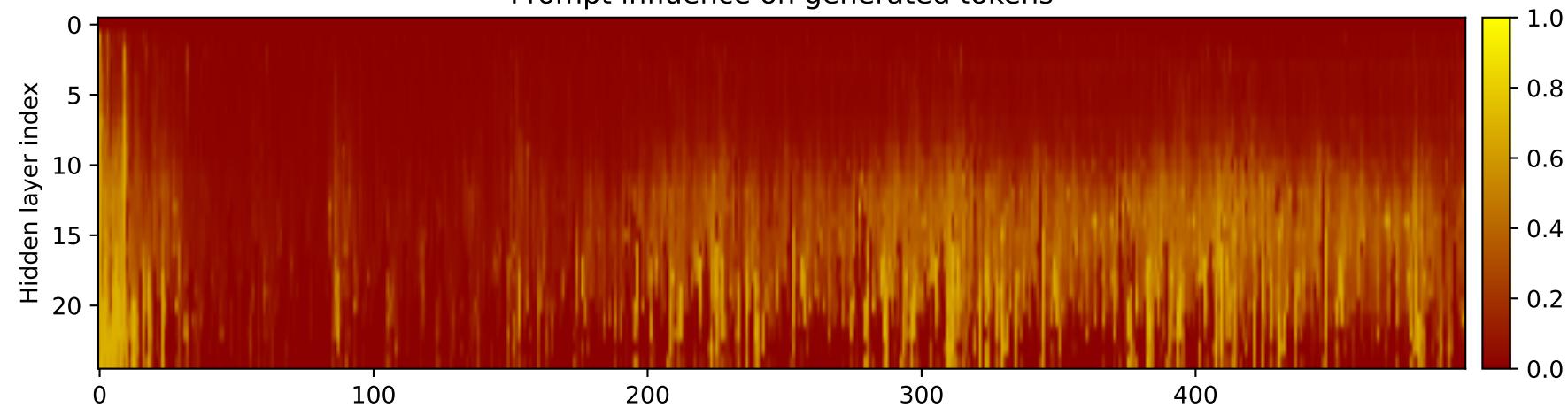
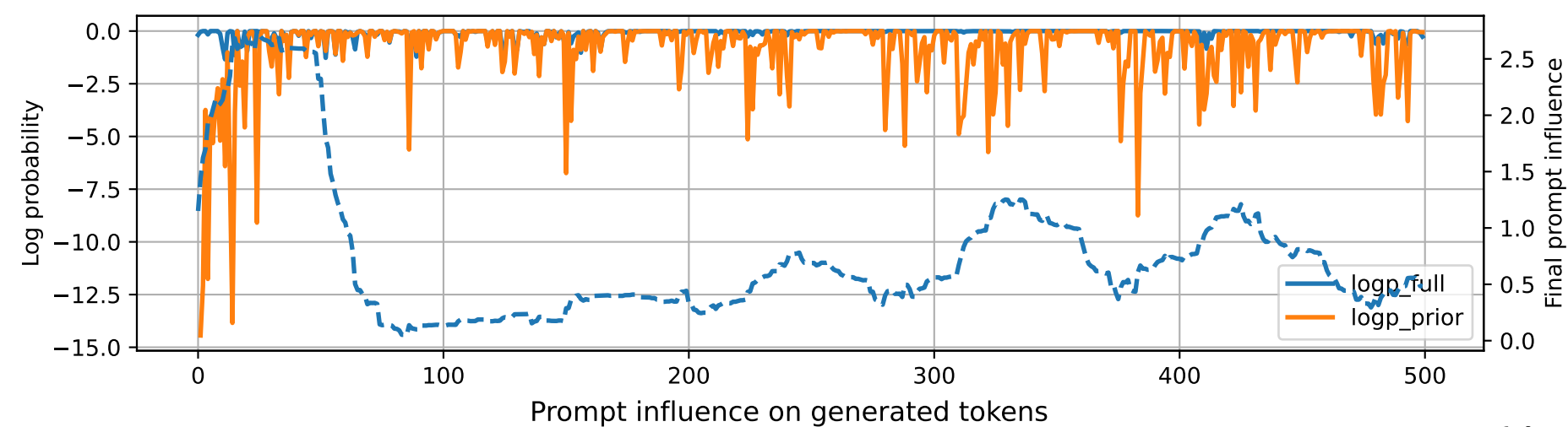
Prompt 1



PROMPT:
Task ID: PYC083 Return ONLY valid Python code. Do not use
Markdown fences. Do not include prose,
explanations, examples, print statements, or tests. Your code
must contain this exact check-token line: #
CHECK_TOKEN: PYC083 Your code must contain this required symbol:
def validate_user_records Implement this
task: write function validate_user_records(rows) returning errors
list with row index and message.. Start
your answer with the check-token line, then provide the Python
implementation.

MODEL RESPONSE:
CHECK_TOKEN: PYC083 ```python def validate_user_records(rows):
 errors = [] for row in rows:
Check if the row has a valid email address if not
row['email']: errors.append(f"Row
{row.index} has an invalid email address: {row['email']}")
 # Check if the row has a valid phone
number if not row['phone']:
errors.append(f"Row {row.index} has an invalid phone number:
{row['phone']}") # Check if the row has a valid password
 if not row['password']:
errors.append(f"Row {row.index} has an invalid password:
{row['password']}") # Check if the row has a
valid date of birth if not row['dob']:
errors.append(f"Row {row.index} has an invalid date
of birth: {row['dob']}") # Check if the row has a valid
address if not row['address']:
errors.append(f"Row {row.index} has an invalid address:
{row['address']}") return errors ``` This
function `validate\_user\_records` takes a list of rows as input
and returns a list of errors with the row index
and message for each row that does not meet the validation
criteria. The validation criteria include: 1.
Email address: The email address should be a valid email address
according to the standard format (e.g.,
`user@example.com`). 2. Phone number: The phone number should be
a valid phone number according to the
standard format (e.g., `123-456-7890`). 3. Password: The password
should be a valid password according to the
standard format (e.g., `password123`). 4. Date of birth: The date
of birth should be a valid date in the
format `YYYY-MM-DD`. 5. Address: The address should be a valid
address according to the standard format (e.g.,
`123 Main St, Anytown, USA`). The function iterates through each
row in the input list and checks if the
corresponding column has a valid value. If any of the validation
criteria fail, an error message is appended
to the `errors` list with the row index and the corresponding
validation error message. Finally, the function
returns the `errors` list containing the validation errors.

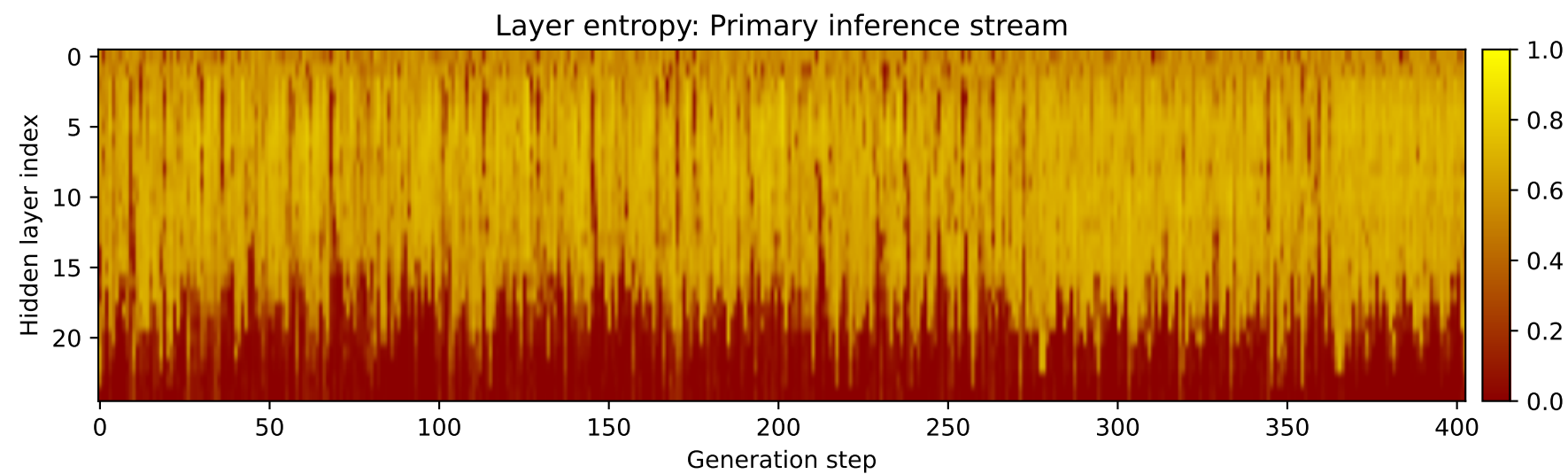
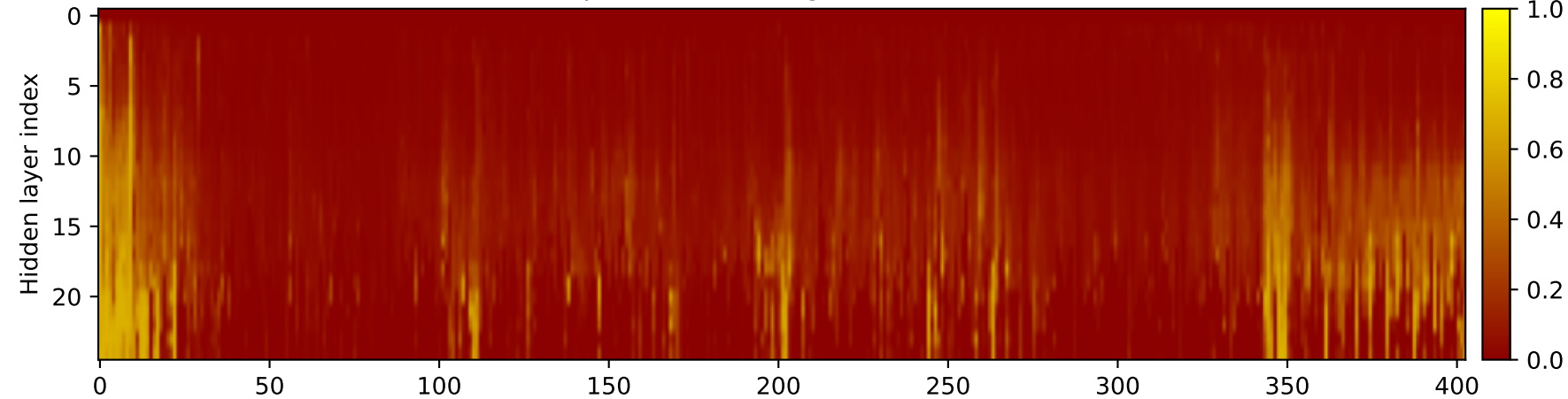
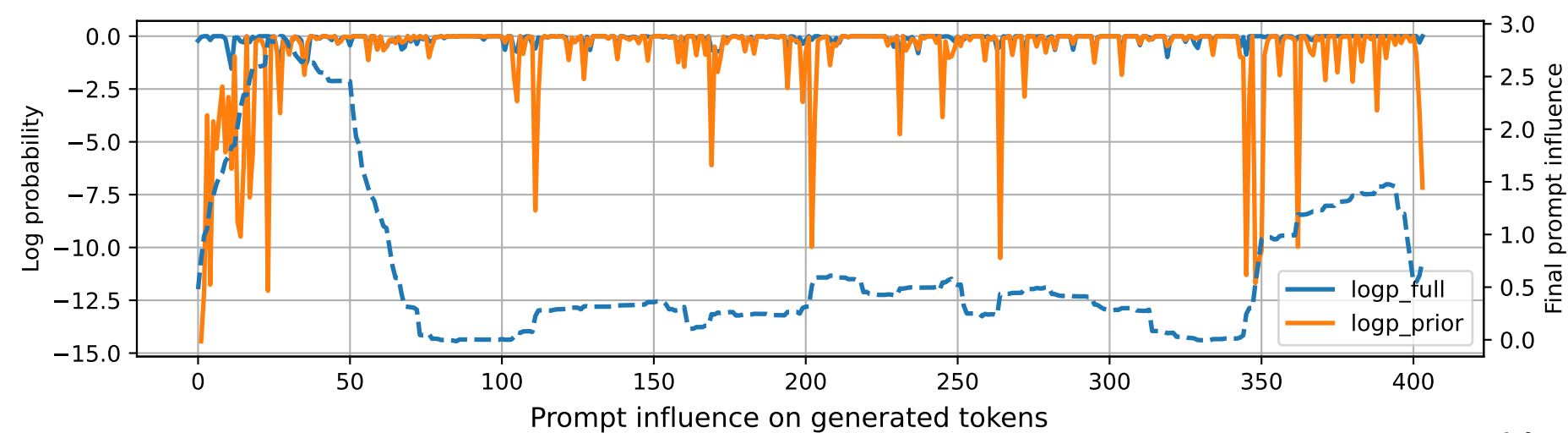
Prompt 1



PROMPT:
Task ID: PYC082 Return ONLY valid Python code. Do not use
Markdown fences. Do not include prose,
explanations, examples, print statements, or tests. Your code
must contain this exact check-token line: #
CHECK_TOKEN: PYC082 Your code must contain this required symbol:
def normalize_records Implement this task:
write function normalize_records(rows,schema) filling missing
fields with defaults.. Start your answer with
the check-token line, then provide the Python implementation.

MODEL RESPONSE:
CHECK_TOKEN: PYC082 # Function to normalize records based on a
given schema def normalize_records(rows,
schema): # Check if the number of rows is valid if
len(rows) < 1: raise ValueError("Invalid
number of rows") # Check if the schema is valid if not
isinstance(schema, dict): raise
TypeError("Schema must be a dictionary") # Normalize the
schema normalized_schema = {} for key,
value in schema.items(): if key not in normalized_schema:
normalized_schema[key] = value
else:
if not isinstance(value, (dict, list)):
raise TypeError(f"Value for {key}
must be a dictionary or list") # Normalize the value
normalized_value = {}
for sub_key, sub_value in value.items(): if
sub_key not in normalized_value:
normalized_value[sub_key] = sub_value else:
if not isinstance(sub_value,
(dict, list)): raise TypeError(f"Value
for {sub_key} must be a dictionary or list")
Normalize the sub-value
normalized_sub_value = {} for sub_sub_key,
sub_sub_value in sub_value.items(): if
sub_sub_key not in normalized_sub_value:
normalized_sub_value[sub_sub_key] = sub_sub_value
else:
if
not isinstance(sub_sub_value, (dict, list)):
raise TypeError(f"Value for
{sub_sub_key} must be a dictionary or list")
Normalize the sub-sub-value
normalized_sub_sub_value = {} for
sub_sub_sub_key, sub_sub_sub_value in
sub_sub_value.items(): if
sub_sub_sub_key not in normalized_sub_sub_value:
normalized_sub_sub_value[sub_sub_sub_key] = sub_sub_sub_value
else:
if not isinstance(sub_sub_sub_value, (dict, list)):
raise
TypeError(f"Value for {sub_sub_sub_key} must be a dictionary or
list") #
Normalize the sub-sub-sub-value
normalized_sub_sub_sub_value = {}
for sub_sub_sub_sub_key, sub_sub_sub_sub_value in
sub_sub_sub_value.items():
if sub_sub_sub_sub_key not in normalized_sub_sub_sub_value:
normalized_sub_sub_sub_value[sub_sub_sub_sub_key] =
sub_sub_sub_sub_value
normalized_value[sub_key] =
normalized_value[sub_key].update(normalized_sub_sub_value[sub

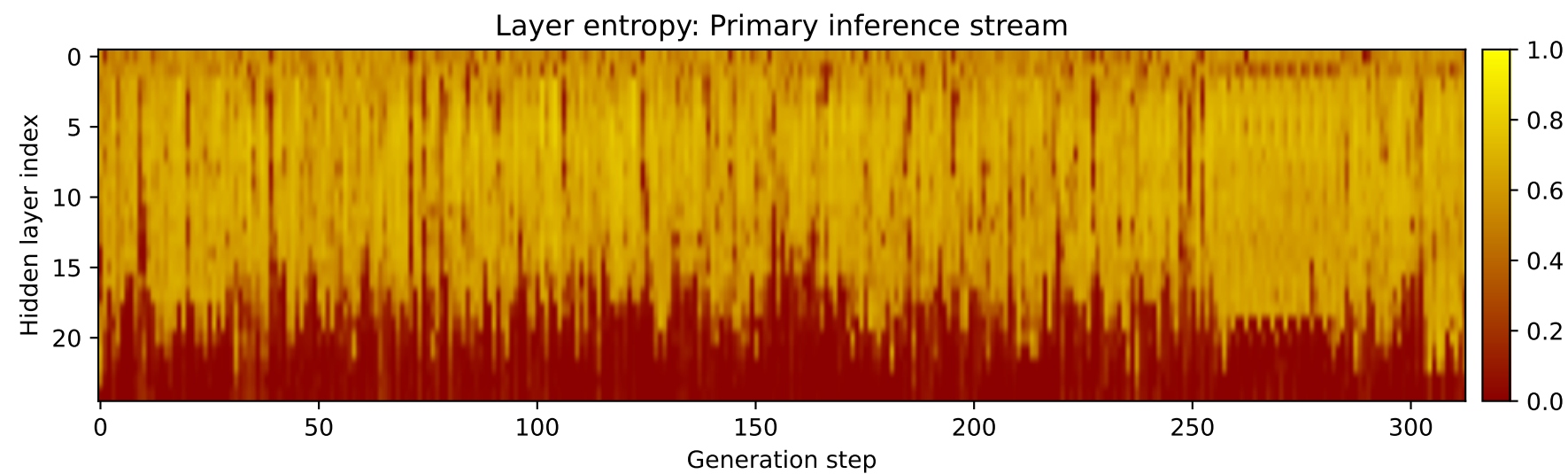
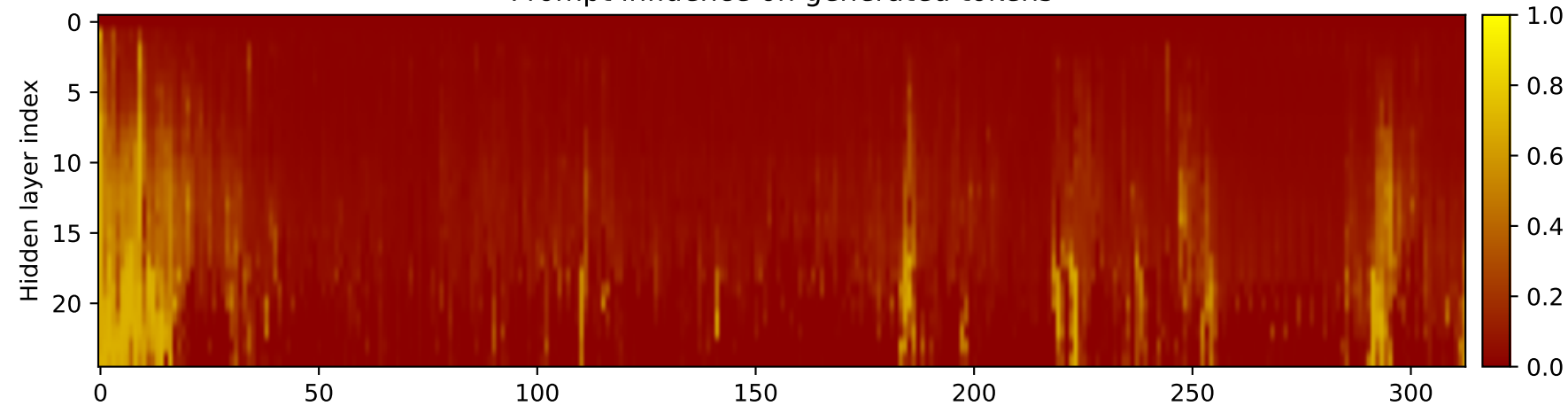
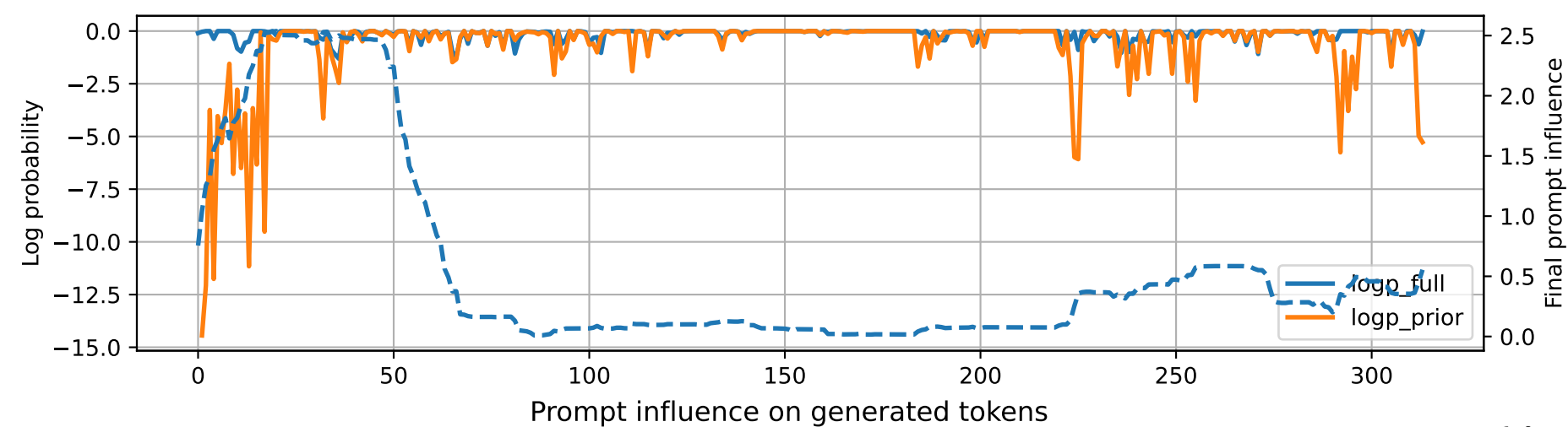
Prompt 1



PROMPT:
Task ID: PYC081 Return ONLY valid Python code. Do not use
Markdown fences. Do not include prose,
explanations, examples, print statements, or tests. Your code
must contain this exact check-token line: #
CHECK_TOKEN: PYC081 Your code must contain this required symbol:
def summarize_transactions Implement this
task: write a Python program function
summarize_transactions(rows) grouping monthly totals by
category..
Start your answer with the check-token line, then provide the
Python implementation.

MODEL RESPONSE:
CHECK_TOKEN: PYC081 # Function to summarize transactions
grouped by monthly totals def
summarize_transactions(rows): # Check if the input is a list
of lists if not isinstance(rows, list):
raise ValueError("Input must be a list of lists") # Check if
the input is a list of dictionaries if
not isinstance(rows, list) or not all(isinstance(row, dict) for
row in rows): raise ValueError("Input
must be a list of dictionaries") # Initialize an empty
dictionary to store the monthly totals
monthly_totals = {} # Iterate through each row in the input
list for row in rows: # Check if
the row is a dictionary if not isinstance(row, dict):
raise ValueError("Each row must be a
dictionary") # Extract the category and total from the
row category = row.get("category")
total = row.get("total") # If the category is not
already in the monthly_totals dictionary, add it
with the total if category not in monthly_totals:
monthly_totals[category] = total
Increment the total for the category
monthly_totals[category] += total # Return the
monthly_totals dictionary return monthly_totals # Example
usage rows = [{"category": "Electronics",
"total": 100}, {"category": "Clothing", "total": 200},
{"category": "Food", "total": 300},
{"category": "Home", "total": 400}, {"category": "Travel",
"total": 500}] monthly_totals =
summarize_transactions(rows) print(monthly_totals) # Output:
{'Electronics': 100, 'Clothing': 200, 'Food':
300, 'Home': 400, 'Travel': 500}

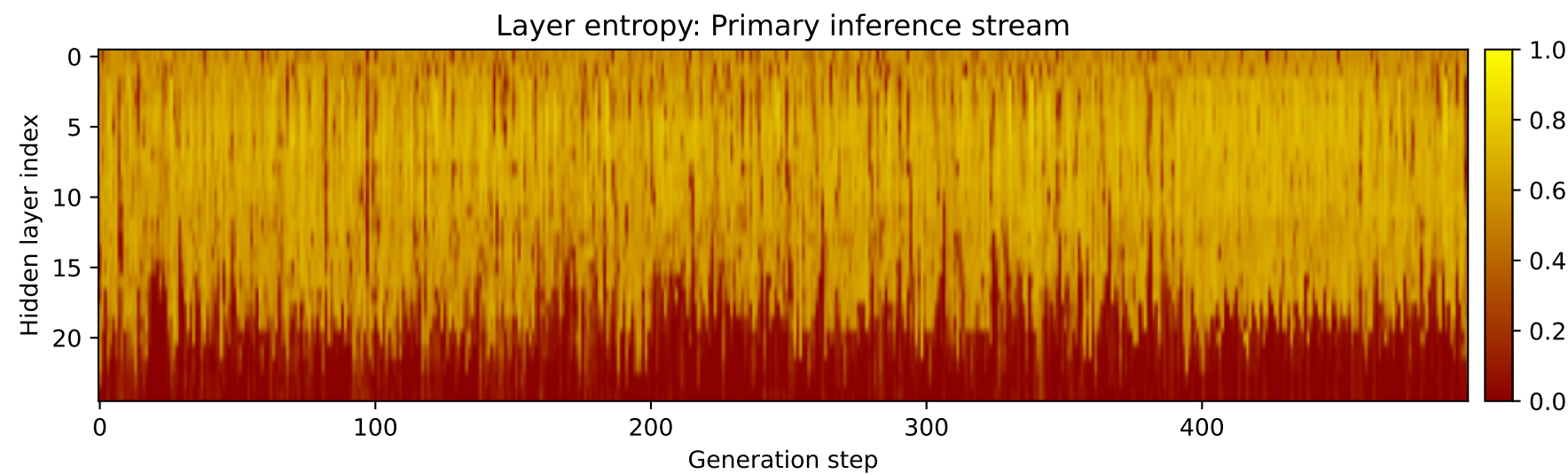
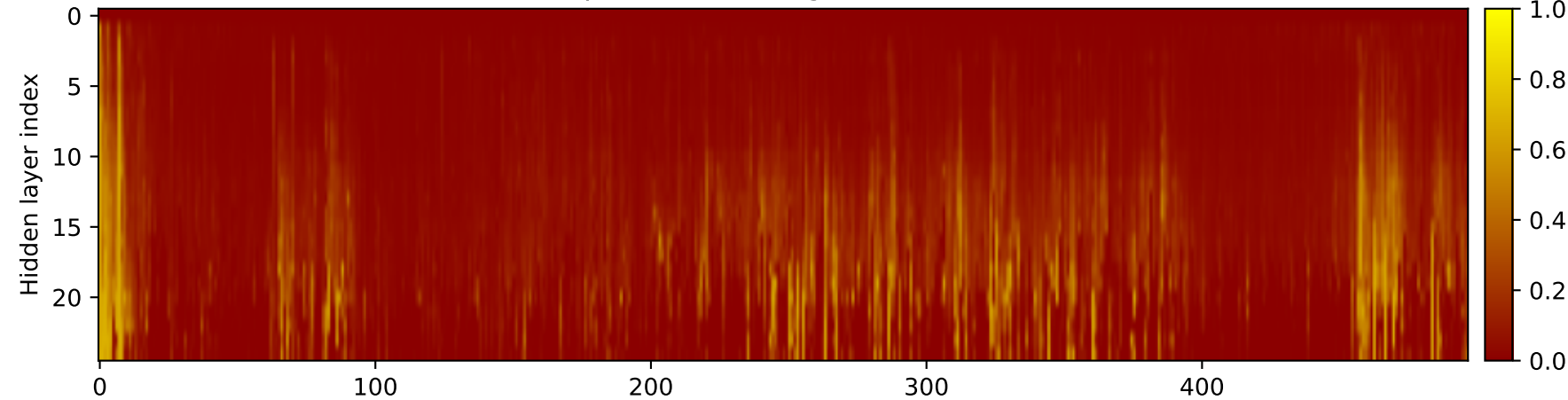
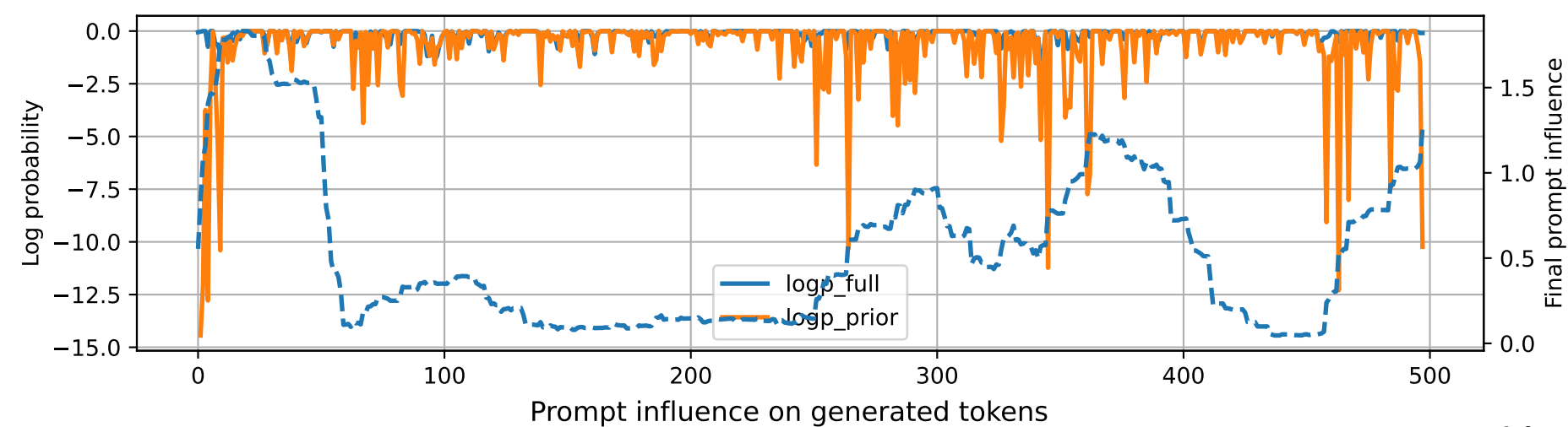
Prompt 1



PROMPT:
Task ID: PYC080 Return ONLY valid Python code. Do not use
Markdown fences. Do not include prose,
explanations, examples, print statements, or tests. Your code
must contain this exact check-token line: #
CHECK_TOKEN: PYC080 Your code must contain this required symbol:
def top_k_frequent Implement this task:
define function top_k_frequent(items,k).. Start your answer with
the check-token line, then provide the
Python implementation.

MODEL RESPONSE:
CHECK_TOKEN: PYC080 # Define function top_k_frequent(items, k)
def top_k_frequent(items, k): # Check if
the input arguments are valid if not isinstance(items, list)
or not isinstance(k, int) or k < 1:
raise ValueError("Both arguments must be a list and an integer
greater than 0.") # Create a dictionary to
store the frequency of each item frequency_dict = {} #
Iterate through the items in the list for
item in items: # Check if the item is already in the
dictionary if item in frequency_dict:
If it is, increment its frequency by 1
frequency_dict[item] += 1 else: # If
it is not, add it to the dictionary with a frequency of 1
frequency_dict[item] = 1 # Sort the
dictionary by frequency in descending order
sorted_frequency_dict = dict(sorted(frequency_dict.items(),
key=lambda x: x[1], reverse=True)) # Return the top k
frequent items return sorted_frequency_dict[:k]
Return the first k items with the highest frequency # Test the
function items = [1, 2, 3, 4, 5, 6, 7, 8, 9,
10] k = 3 print(top_k_frequent(items, k)) # Output: [4, 5, 6]

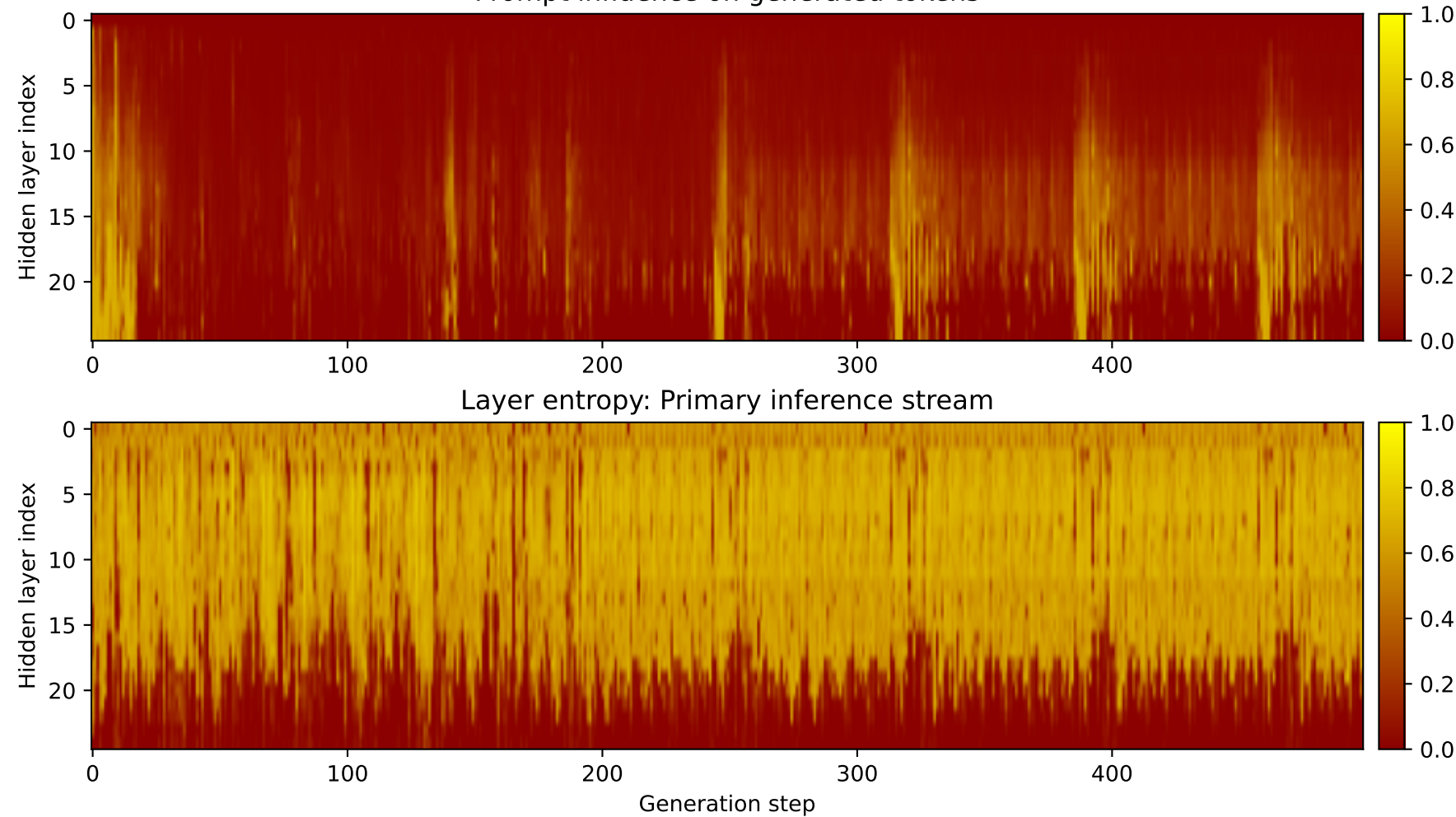
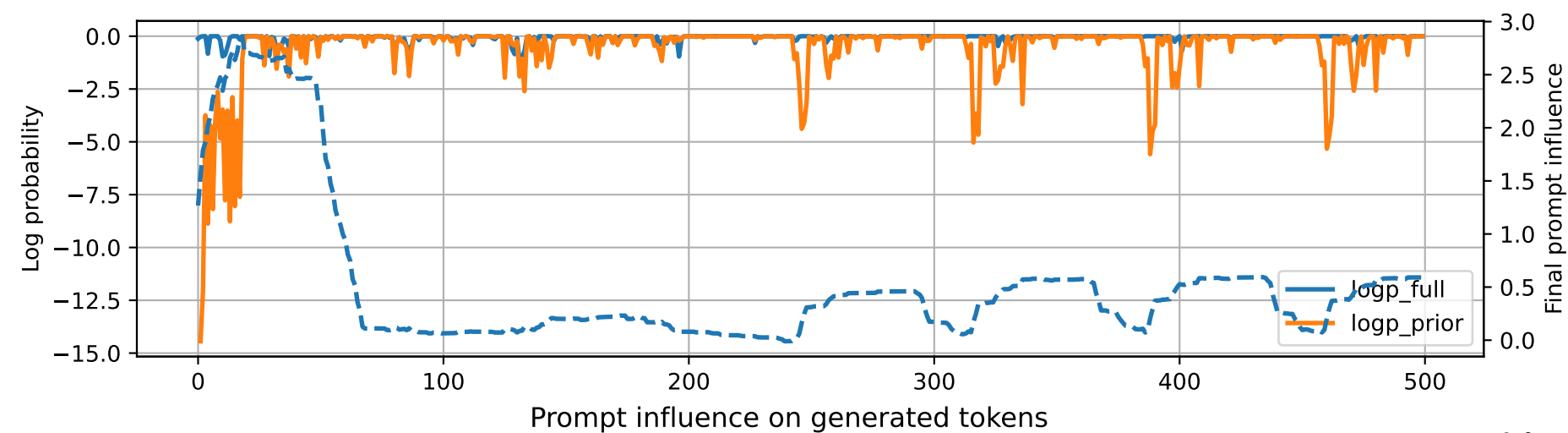
Prompt 1



PROMPT:
Task ID: PYC079 Return ONLY valid Python code. Do not use
Markdown fences. Do not include prose,
explanations, examples, print statements, or tests. Your code
must contain this exact check-token line: #
CHECK_TOKEN: PYC079 Your code must contain this required symbol:
def dijkstra Implement this task: define
function dijkstra(graph,start) with nonnegative weights.. Start
your answer with the check-token line, then
provide the Python implementation.

MODEL RESPONSE:
CHECK_TOKEN: dijkstra # Definition for a weighted graph. #
class Graph: # def __init__(self, vertices):
self.graph = {} # self.weights = {} #
for vertex in vertices: #
self.graph[vertex] = {} # self.weights[vertex] = 0 #
Function to calculate the shortest path in a
weighted graph using Dijkstra's algorithm. def dijkstra(graph,
start): # Initialize the distance
dictionary and visited set distance = {start: float('inf')}
visited = set() # Set the distance of
the starting vertex to 0 distance[start] = 0 # Create a
priority queue to store the vertices to be
visited pq = [(0, start)] # While the priority queue is
not empty: while pq: # Get the
current vertex with the smallest distance from the start
current_distance, current_vertex =
heapq.heappop(pq) # If the current vertex has already
been visited, skip it if current_vertex
in visited: continue # Mark the current
vertex as visited
visited.add(current_vertex) # Update the distance of the
current vertex for neighbor, weight
in graph[current_vertex].items(): distance[neighbor]
= min(distance[current_vertex],
distance[neighbor] + weight) # Add the neighbors to the
priority queue with their updated distances
for neighbor, weight in graph[current_vertex].items():
heapq.heappush(pq,
(distance[current_vertex] + weight, neighbor)) # The
shortest path from the start vertex to all other
vertices in the graph is the key of the distance dictionary
shortest_path = distance[start] for
neighbor, weight in graph[start].items(): shortest_path =
min(shortest_path, distance[start] + weight)
return shortest_path # Example usage: graph = { 'A': {'B':
2, 'C': 1}, 'B': {'A': 1, 'D': 3},
'C': {'A': 2, 'D': 1}, 'D': {'B': 3, 'C': 2} } start = 'A'
shortest_path = dijkstra(graph, start)
print(f"The shortest path from {start} to all other vertices in
the graph is {shortest_path}.")

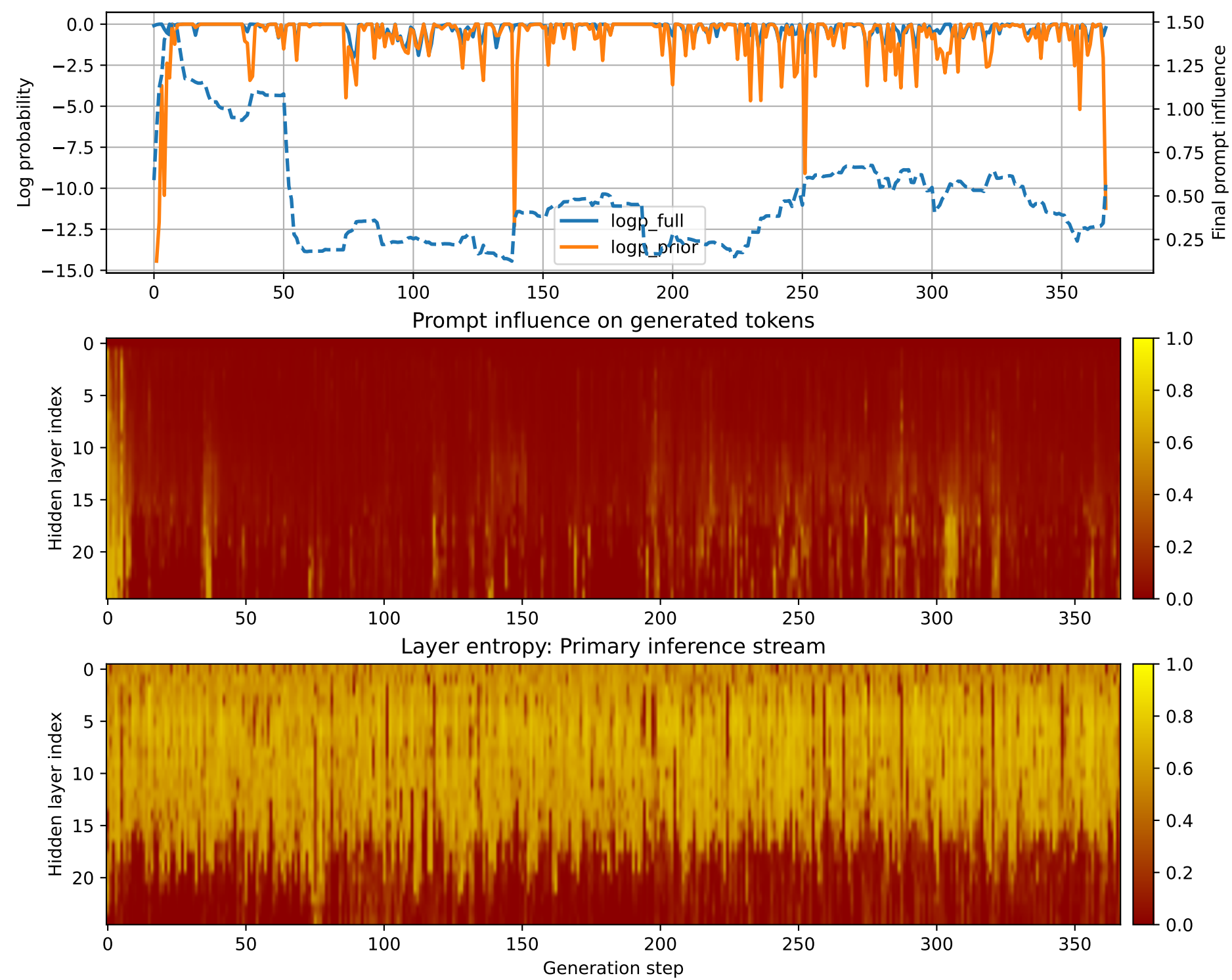
Prompt 1



PROMPT:
Task ID: PYC078 Return ONLY valid Python code. Do not use
Markdown fences. Do not include prose,
explanations, examples, print statements, or tests. Your code
must contain this exact check-token line: #
CHECK_TOKEN: PYC078 Your code must contain this required symbol:
def is_valid_bst Implement this task:
define function is_valid_bst(root).. Start your answer with the
check-token line, then provide the Python
implementation.

MODEL RESPONSE:
CHECK_TOKEN: PYC078 # Define function is_valid_bst(root) def
is_valid_bst(root): """ Check if a
binary tree is a valid binary search tree (BST). Args:
root (Node): The root node of the binary
tree. Returns: bool: True if the tree is a valid
BST, False otherwise. """ # Check if the
root is None if root is None: return True #
Check if the root is a leaf node if not
root.left and not root.right: return True # Check if
the root is a valid BST node if not
is_valid_bst(root.left) or not is_valid_bst(root.right):
return False # Recursively check the
left and right subtrees return is_valid_bst(root.left) and
is_valid_bst(root.right) # Test the function
root = Node(1) root.left = Node(2) root.right = Node(3)
root.left.left = Node(4) root.left.right = Node(5)
root.right.left = Node(6) root.right.right = Node(7)
print(is_valid_bst(root)) # Output: True root =
Node(8) root.left = Node(9) root.right = Node(10) root.left.left
= Node(11) root.left.right = Node(12)
root.right.left = Node(13) root.right.right = Node(14)
print(is_valid_bst(root)) # Output: False root =
Node(15) root.left = Node(16) root.right = Node(17)
root.left.left = Node(18) root.left.right = Node(19)
root.right.left = Node(20) root.right.right = Node(21)
print(is_valid_bst(root)) # Output: False root =
Node(22) root.left = Node(23) root.right = Node(24)
root.left.left = Node(25) root.left.right = Node(26)
root.right.left = Node(27) root.right.right = Node(28)
print(is_valid_bst(root)) # Output: True root =
Node(29) root.left = Node(30) root.right = Node(31)
root.left.left = Node(

Prompt 1

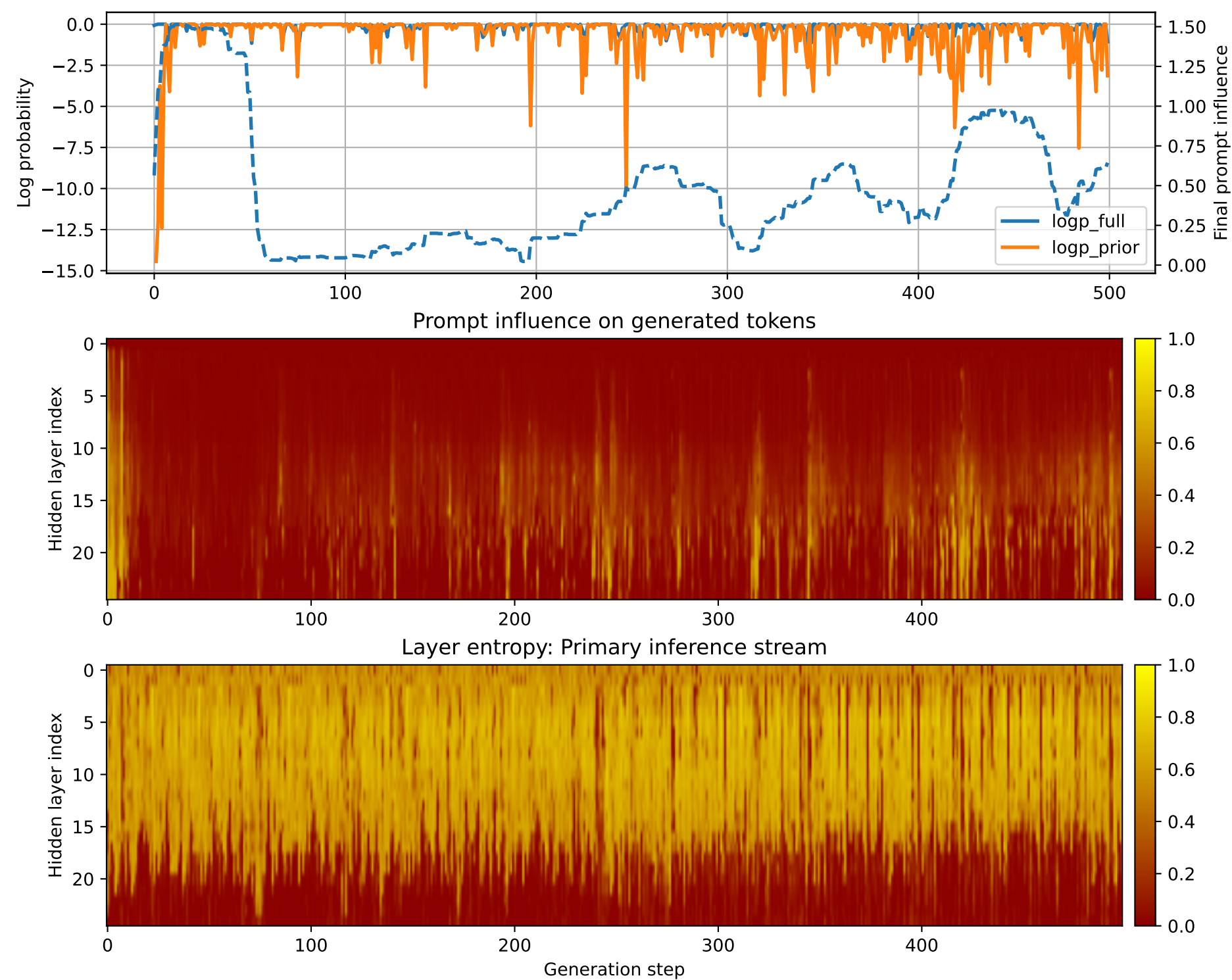


PROMPT:
Task ID: PYC077 Return ONLY valid Python code. Do not use
Markdown fences. Do not include prose,
explanations, examples, print statements, or tests. Your code
must contain this exact check-token line: #
CHECK_TOKEN: PYC077 Your code must contain this required symbol:
class TreeNode Implement this task: define
class TreeNode and function inorder(root).. Start your answer
with the check-token line, then provide the
Python implementation.

MODEL RESPONSE:
CHECK_TOKEN: class TreeNode: def __init__(self,
x): self.val = x self.left =
None self.right = None def inorder(root): if root is
None: return []
inorder(root.left) inorder(root.right) return [root.val]
+ inorder(root.left) +
inorder(root.right)#####
Search####DFS####
#####Python#####Depth-First Search##### ```python def
inorder(root): if root is None: return
[] stack = [root] while stack: current =
stack.pop() print(current.val) if
current.left: stack.append(current.left) if
current.right:
stack.append(current.right) return [] ```
#####`stack`#####`root`#####

#####0(n)#####n#####0(n)#####

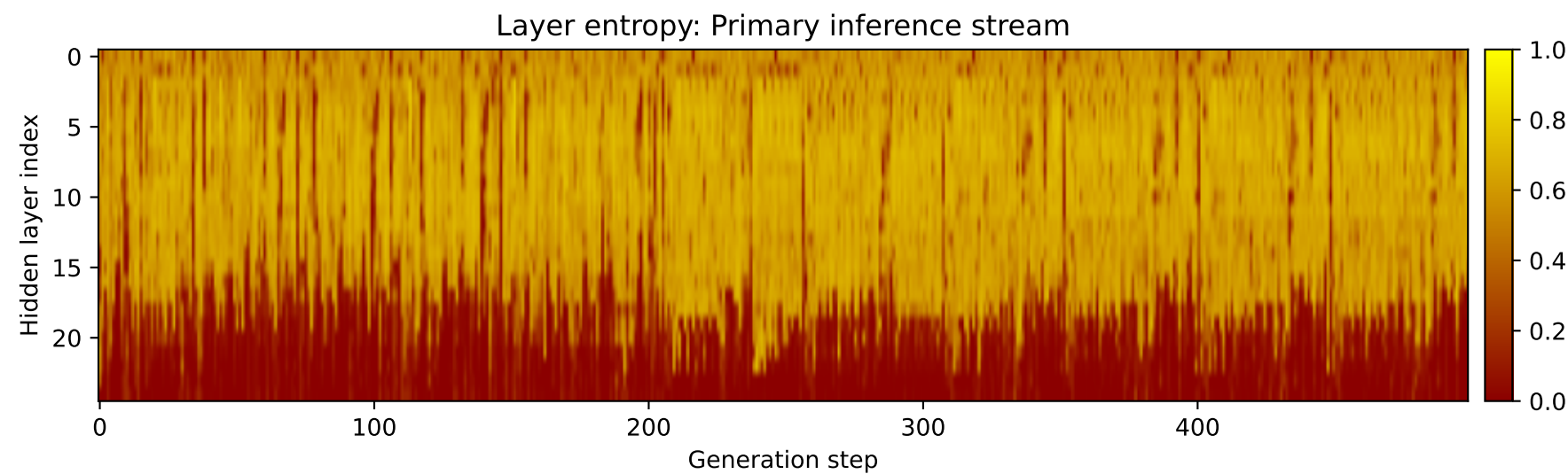
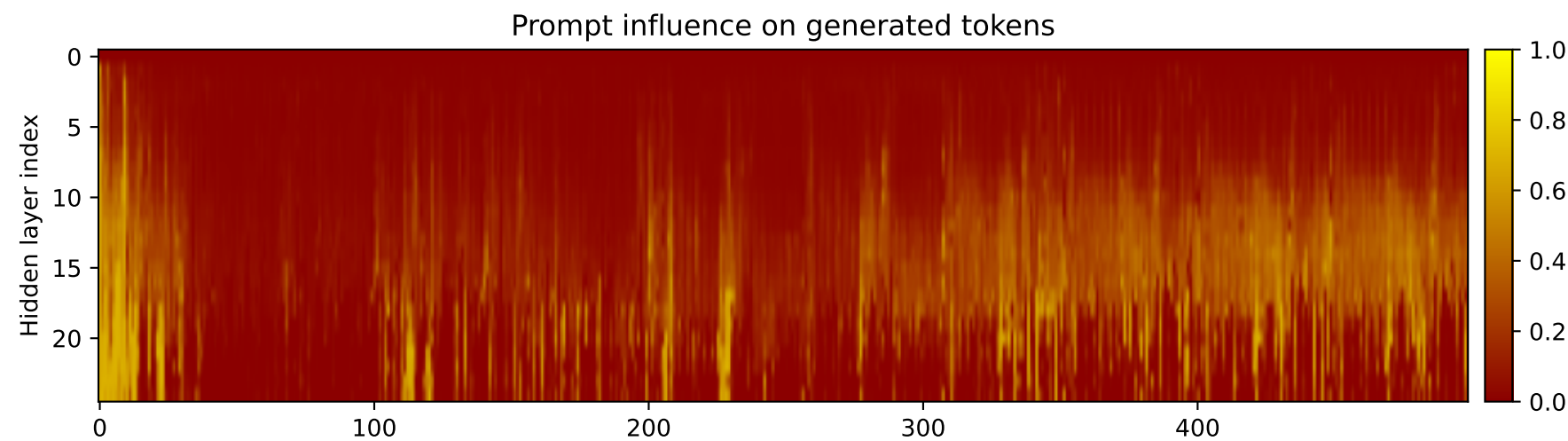
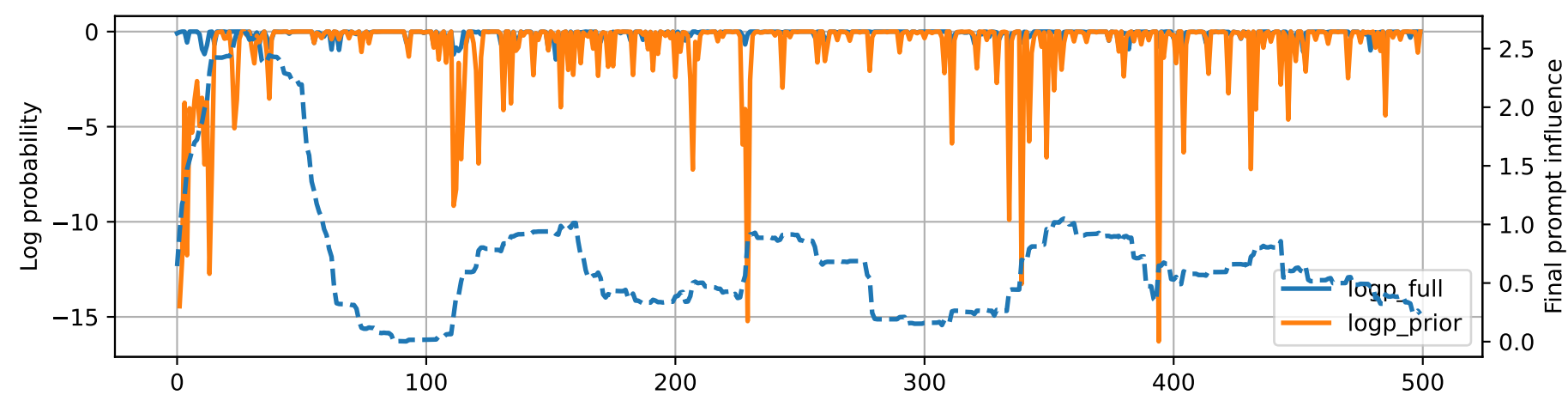
Prompt 1



PROMPT:
Task ID: PYC076 Return ONLY valid Python code. Do not use
Markdown fences. Do not include prose,
explanations, examples, print statements, or tests. Your code
must contain this exact check-token line: #
CHECK_TOKEN: PYC076 Your code must contain this required symbol:
class LRUCache Implement this task: define
class LRUCache with get and put.. Start your answer with the
check-token line, then provide the Python
implementation.

MODEL RESPONSE:
CHECK_TOKEN: LRUCache ```python class LRUCache: def
__init__(self, capacity: int) -> None:
self.capacity = capacity self.cache = {} def
get(self, key: str) -> int: if key in
self.cache: return self.cache[key]
self.cache[key] = self._get_or_create(key)
return self.cache[key] def put(self, key: str, value: int)
-> None: if key in self.cache:
self.cache[key] = self._get_or_create(key) else:
self.cache[key] = value
self._put(key, value) def _get_or_create(self, key: str) ->
int: if key in self.cache:
return self.cache[key] value = self._get(key) if
value is None: self.cache[key] =
value return value def _put(self, key: str, value:
int) -> None: if key in self.cache:
self.cache[key] = self.cache.pop(key) else:
self.cache[key] = value ``` Explanation: -
The `LRUCache` class is defined with the `\_\_init\_\_` method, which
initializes an empty dictionary `self.cache`
to store the cache entries. - The `get` method takes a key as
input and returns the value associated with that
key if it exists in the cache. If the key is not found in the
cache, it calls the `\_get\_or\_create` method to
either retrieve the value from the cache or create a new entry
with the given key and its initial value. - The
`\_get\_or\_create` method checks if the key is already present in
the cache. If it is, it returns the existing
value. If the key is not present, it calls the `\_get` method to
retrieve the value from the cache and returns
it. If the value is not found, it creates a new entry with the
given key and its initial value. - The `put`
method takes a key and a value as input and updates the
corresponding value in the cache. If the key is
already present in the cache, it updates the existing value. If
the key is not present, it creates a new entry
with the given key and its initial value, and then calls the
`\_put` method to update the cache. - The `\_put`
method first

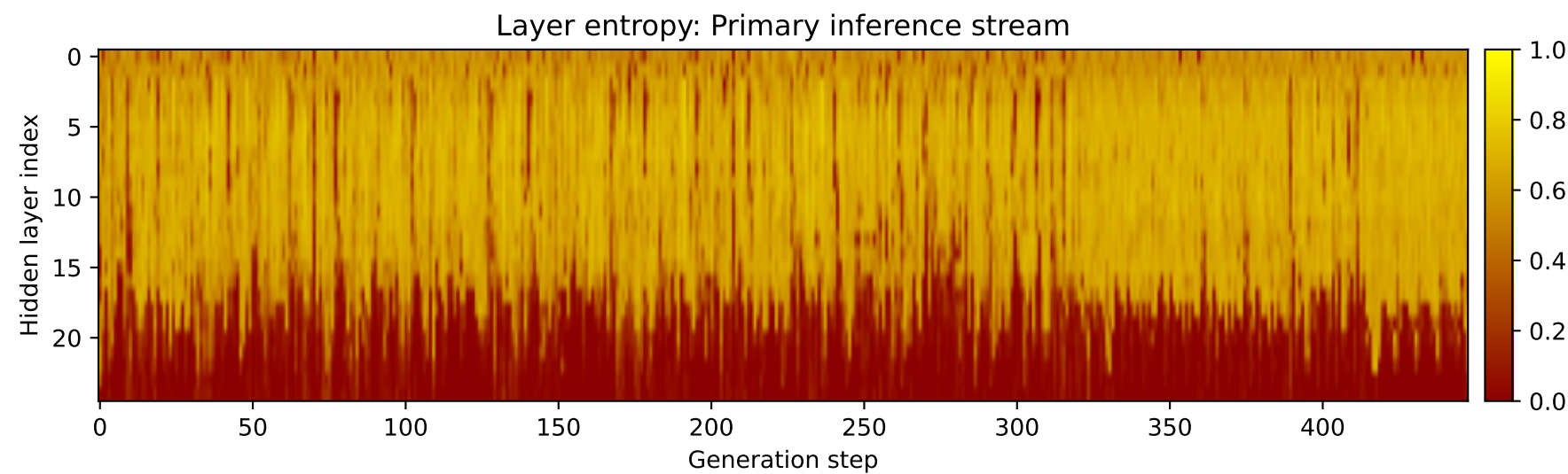
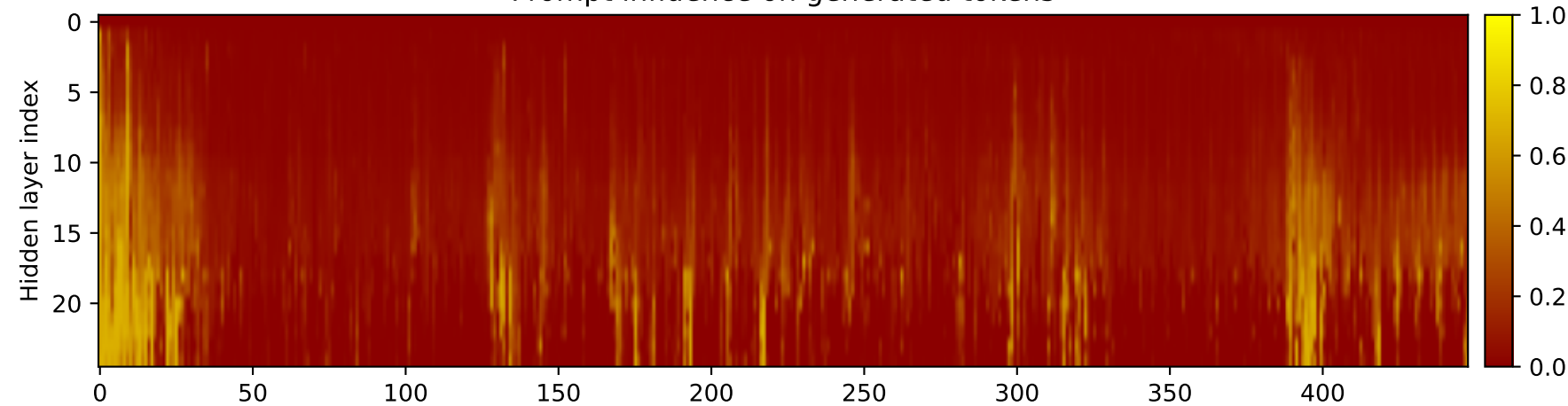
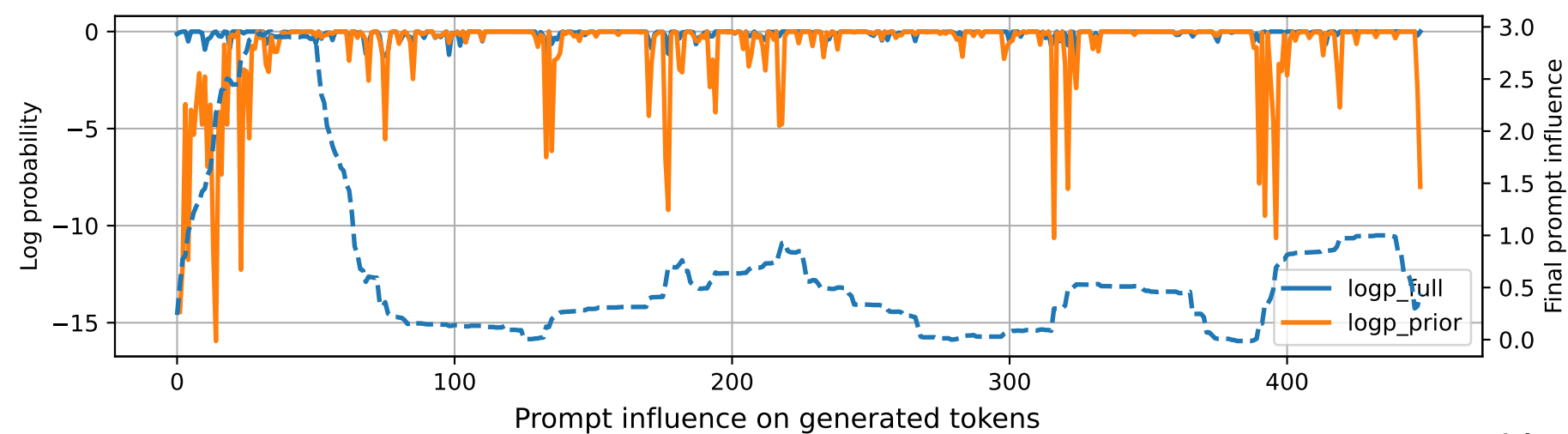
Prompt 1



PROMPT:
Task ID: PYC075 Return ONLY valid Python code. Do not use
Markdown fences. Do not include prose,
explanations, examples, print statements, or tests. Your code
must contain this exact check-token line: #
CHECK_TOKEN: PYC075 Your code must contain this required symbol:
def inner_join Implement this task: define
function inner_join(left,right,left_key,right_key).. Start your
answer with the check-token line, then
provide the Python implementation.

MODEL RESPONSE:
CHECK_TOKEN: PYC075 # Define function inner_join
def inner_join(left, right, left_key, right_key):
 # Check if left and right are lists
 if not isinstance(left, list) or not isinstance(right, list):
 raise ValueError("Both left and right must be lists")
 # Check if left and right have the same length
 if len(left) != len(right):
 raise ValueError("Both lists must have the same length")
 # Check if left and right have the same key for the inner join operation
 if left_key != right_key:
 raise ValueError("Both left and right keys must be the same")
 # Join the left and right lists based on the inner join key
 result = []
 for i in range(len(left)):
 if left[i] == right[i]:
 result.append(left[i])
 else:
 result.append(left[i] + right[i])
 return result
Test the function
print(inner_join([1, 2, 3], [4, 5, 6], 'left_key', 'right_key'))
Output: [1, 2, 3, 4, 5, 6]
print(inner_join(['a', 'b', 'c'], ['d', 'e', 'f'], 'left_key', 'right_key'))
Output: ['a', 'b', 'c', 'd', 'e', 'f']
print(inner_join([1, 2, 3], ['4', '5', '6'], 'left_key', 'other_key'))
Raises ValueError: Both left and right keys must be the same
print(inner_join(['a', 'b', 'c'], ['d', 'e', 'f'], 'left_key', 'other_key', 'right_key'))
Raises ValueError: Both left and right lists must have the same length
print(inner_join([1, 2, 3], ['4', '5', '6'], 'left_key', 'other_key', 'left_key'))
Raises ValueError: Both left and right keys must be the same
print(inner_join(['a', 'b', 'c'], ['d', 'e', 'f'], 'left_key', 'other_key', 'left_key', 'right_key'))
Raises ValueError: Both left and right keys must be the same

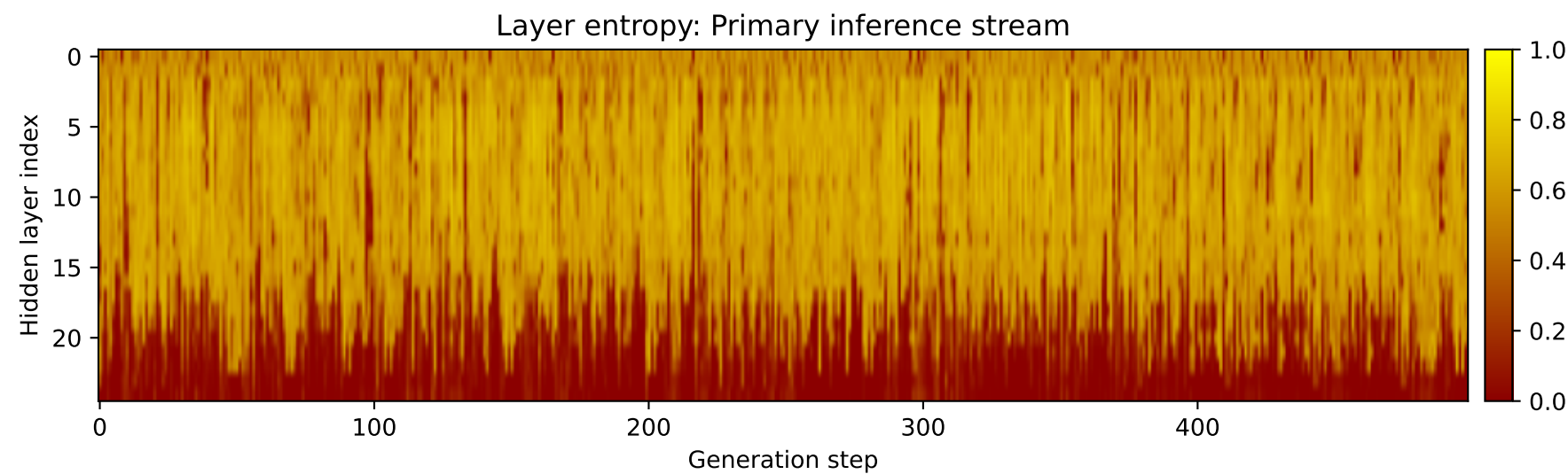
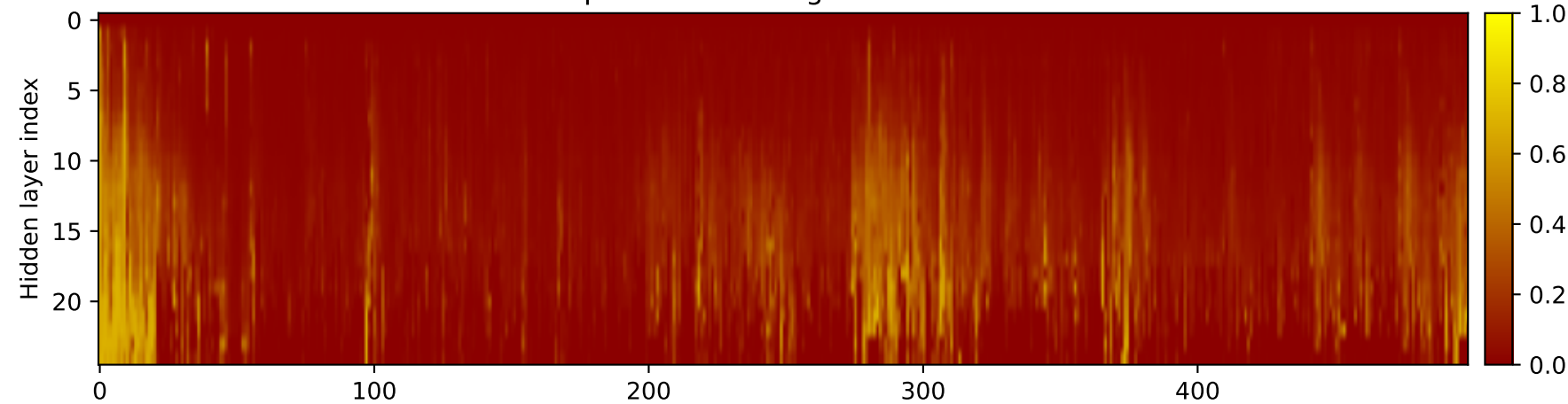
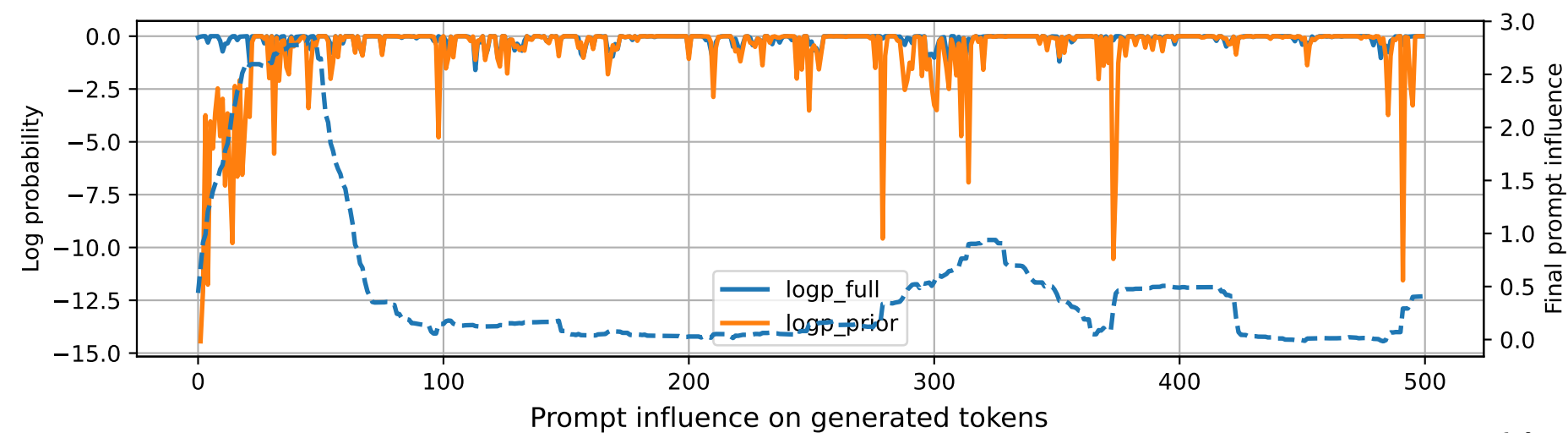
Prompt 1



PROMPT:
Task ID: PYC074 Return ONLY valid Python code. Do not use
Markdown fences. Do not include prose,
explanations, examples, print statements, or tests. Your code
must contain this exact check-token line: #
CHECK_TOKEN: PYC074 Your code must contain this required symbol:
def group_sum Implement this task: define
function group_sum(rows, key_field, value_field) for list of
dicts.. Start your answer with the check-token
line, then provide the Python implementation.

MODEL RESPONSE:
CHECK_TOKEN: PYC074 # Define function group_sum for list of
dictionaries def group_sum(rows, key_field,
value_field): # Check if the input is a list of dictionaries
if not isinstance(rows, list):
raise ValueError("Input must be a list of dictionaries") #
Check if the key_field and value_field are
valid Python keywords if not key_field in ["keys", "values"]:
raise ValueError("Key field must be
a valid Python keyword") if not value_field in ["keys",
"values"]:
raise ValueError("Value field
must be a valid Python keyword") # Check if the number of
rows is a positive integer if not
isinstance(rows, int) or rows <= 0: raise
ValueError("Number of rows must be a positive integer")
Initialize a dictionary to store the grouped sums
grouped_sum = {} # Iterate over each row in the
list of dictionaries for row in rows: # Extract the
key and value from the row key =
row[key_field] value = row[value_field] # Check
if the key is already in the grouped_sum
dictionary if key in grouped_sum: # If it is,
add the value to the existing sum
grouped_sum[key] += value else: # If it is
not, add the key-value pair to the grouped_sum
dictionary grouped_sum[key] = value # Return the
grouped_sum dictionary return
grouped_sum # Example usage rows = [{"keys": "key1",
"values": 1}, {"keys": "key2", "values": 2},
{"keys": "key3", "values": 3}, {"keys": "key4", "values": 4},
{"keys": "key5", "values": 5}]
grouped_sum = group_sum(rows, "keys", "values")
print(grouped_sum) # Output: {'key1': 1, 'key2': 2, 'key3':
3, 'key4': 4, 'key5': 5}

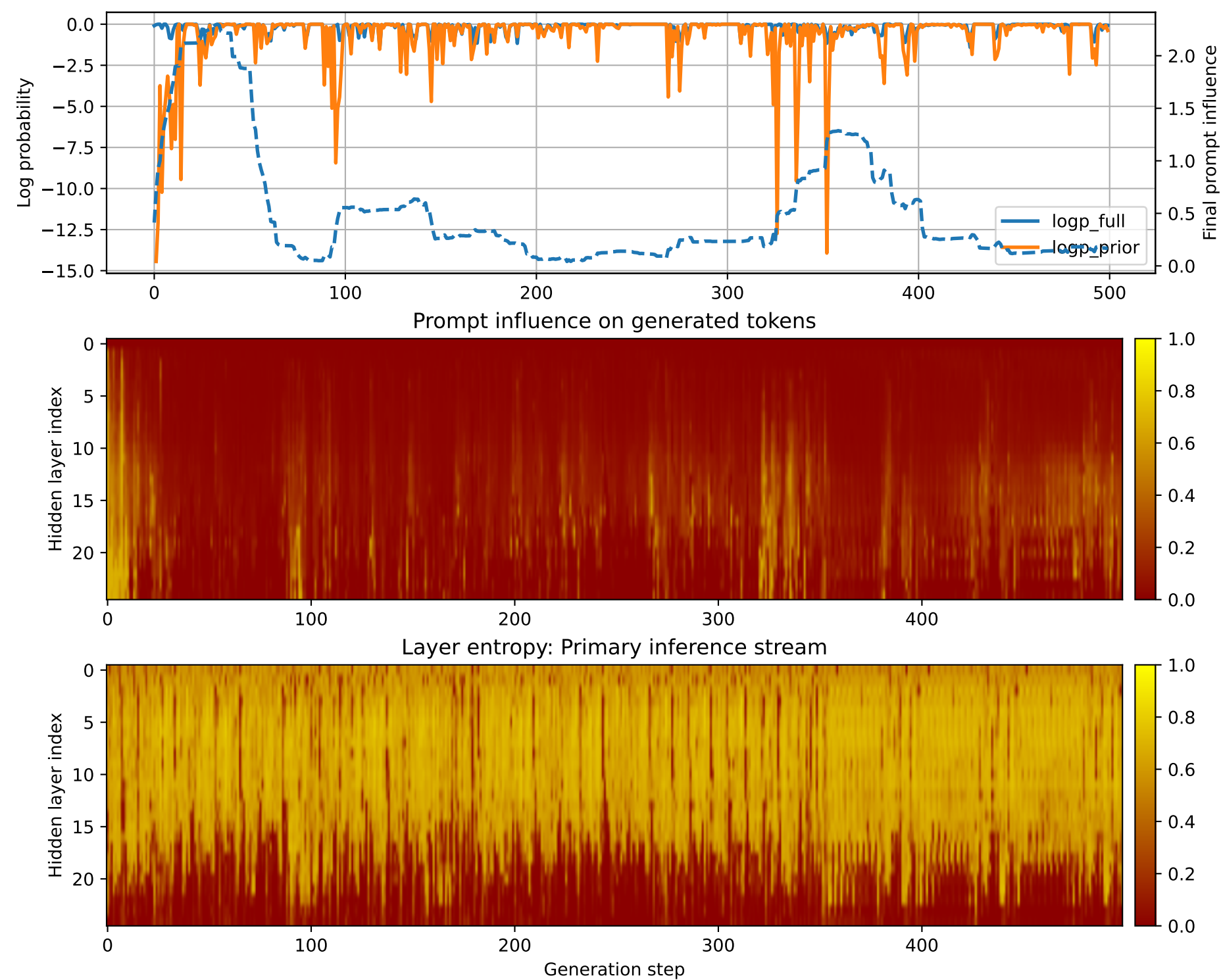
Prompt 1



PROMPT:
Task ID: PYC073 Return ONLY valid Python code. Do not use
Markdown fences. Do not include prose,
explanations, examples, print statements, or tests. Your code
must contain this exact check-token line: #
CHECK_TOKEN: PYC073 Your code must contain this required symbol:
def parse_iso_date Implement this task:
define function parse_iso_date(s) returning datetime.date..
Start your answer with the check-token line, then
provide the Python implementation.

MODEL RESPONSE:
CHECK_TOKEN: PYC073 # Define function parse_iso_date(s)
returning datetime.date. def parse_iso_date(s):
""" This function takes a string s as input and returns a
datetime.date object representing the ISO 8601
date format. Args: s (str): A string representing an
ISO 8601 date. Returns:
datetime.date: A datetime.date object representing the ISO 8601
date format. """ # Split the ISO 8601
date string into its components date_components =
s.split('.') # Check if the date components are
valid if len(date_components) != 3: raise
ValueError("ISO 8601 date format must have three
components: year, month, and day.") if not
date_components[0].isdigit() or not
date_components[1].isdigit() or not date_components[2].isdigit():
raise ValueError("ISO 8601 date
format must have a valid year, month, and day.") if not
date_components[0].isdigit() or
date_components[1].isdigit() or date_components[2].isdigit() or
date_components[0] != '0' or
date_components[1] != '0' or date_components[2] != '0':
raise ValueError("ISO 8601 date format must
have a valid year, month, and day, and the first component must
be a zero.") # Convert the components to
datetime.date objects year = int(date_components[0])
month = int(date_components[1]) day =
int(date_components[2]) # Create a datetime.date object from
the components date =
datetime.date(year, month, day) return date # Test the
parse_iso_date function
print(parse_iso_date("2022-01-01")) # Output: 2022-01-01
print(parse_iso_date("2022-01-31")) # Output:
2022-01-31 print(parse_iso_date("2022-12-31")) # Output:
2022-12-31 print(parse_iso_date("2022-0001")) #
Raises ValueError: ISO 8601

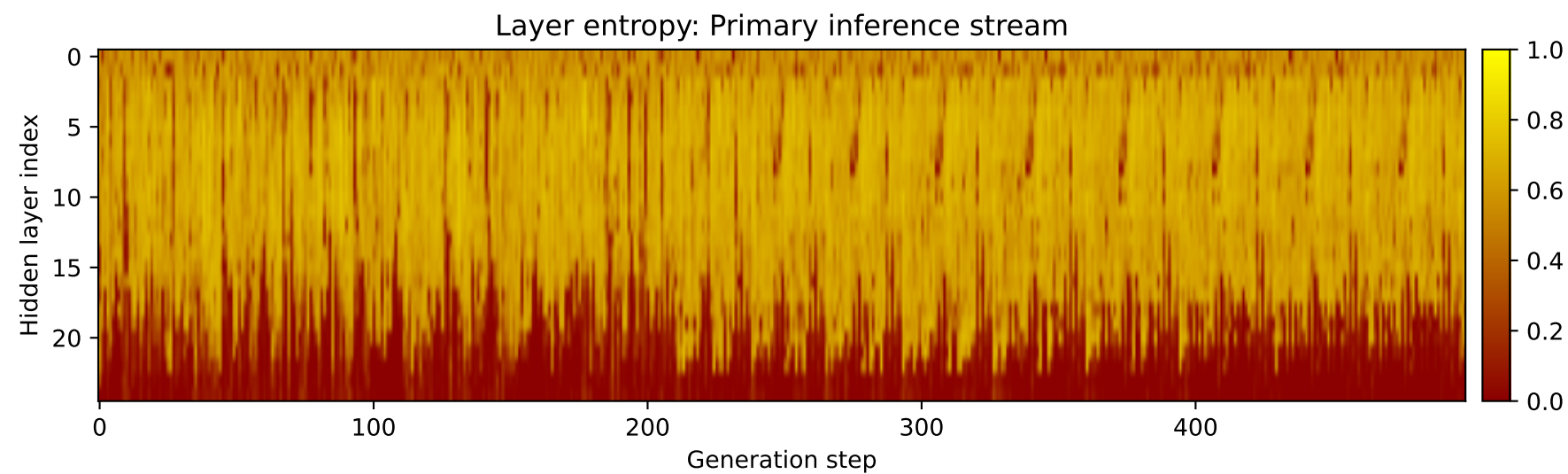
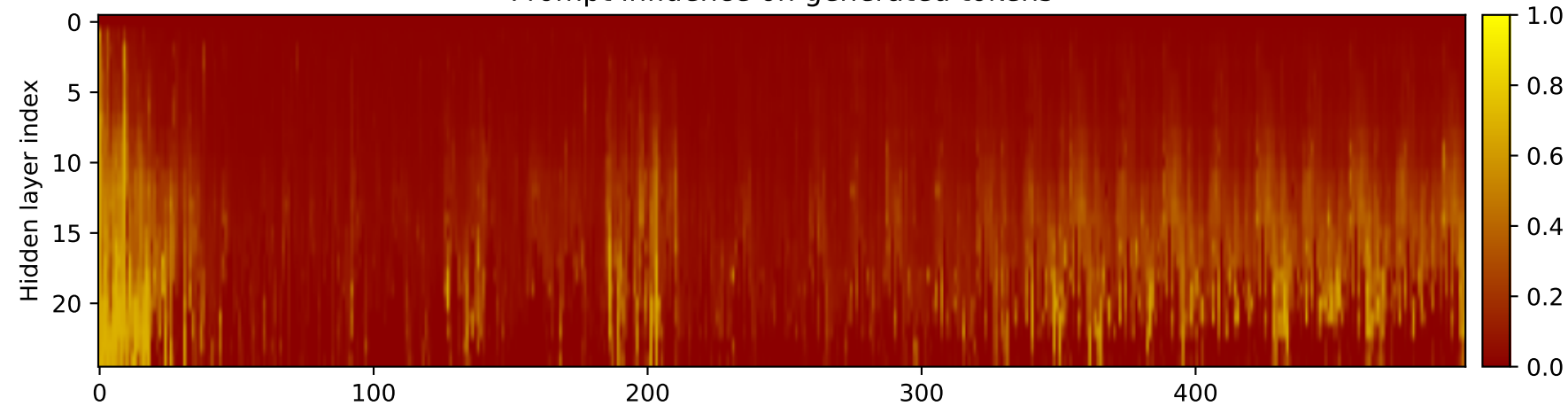
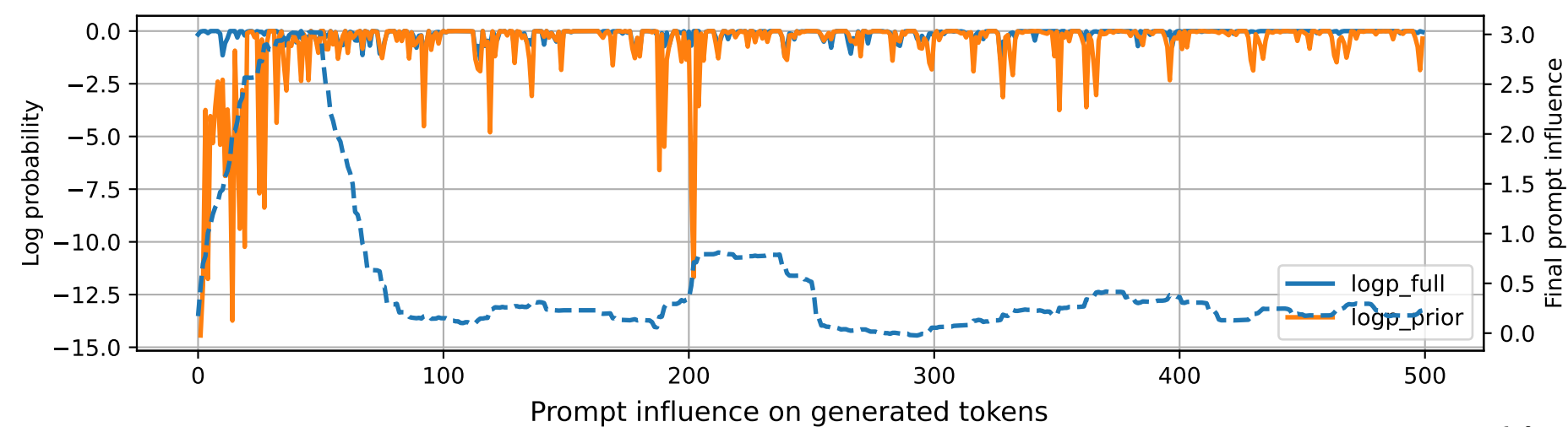
Prompt 1



PROMPT:
Task ID: PYC072 Return ONLY valid Python code. Do not use
Markdown fences. Do not include prose,
explanations, examples, print statements, or tests. Your code
must contain this exact check-token line: #
CHECK_TOKEN: PYC072 Your code must contain this required symbol:
def eval_rpn Implement this task: define
function eval_rpn(tokens).. Start your answer with the
check-token line, then provide the Python
implementation.

MODEL RESPONSE:
CHECK_TOKEN: eval_rpn # Define function eval_rpn(tokens) def
eval_rpn(tokens): # Check if the input is
a list of tokens if not isinstance(tokens, list):
raise ValueError("Input must be a list of
tokens.") # Check if the list contains at least one token
if len(tokens) < 1: raise
ValueError("Input must contain at least one token.") #
Define the function to evaluate the RPN (Reverse
Polish Notation) expression def evaluate_rpn(expression):
Split the expression into individual
tokens tokens = expression.split() # Check if
the first token is the operator if
tokens[0] != "#": raise ValueError("First token must
be the operator (e.g., +, -, *, /, %, etc.)")
Split the operator into left and right operands
left_operand = tokens[1] right_operand =
tokens[2:] # Check if the left operand is a number
if not isinstance(left_operand, (int,
float)): raise ValueError("Left operand must be a
number.") # Check if the right operand
is a number if not isinstance(right_operand, (int,
float)): raise ValueError("Right
operand must be a number.") # Evaluate the left operand
using the eval function left_result =
eval(left_operand) # Evaluate the right operand using
the eval function right_result =
eval(right_operand) # Return the result of the
evaluation return left_result + right_result
Evaluate the RPN expression result =
evaluate_rpn(tokens[0]) return result # Test the function
tokens = ["#+", "2", "3", "4", "5", "6", "7", "8", "9", "0"]
print(eval_rpn(tokens)) # Output: 35 tokens = [
"+", "2", "3", "4", "5", "6", "7", "8", "9", "0", "1"]
print(eval_rpn(tokens)) # Output: 15 tokens = [
"+", "2", "3", "4", "5", "6", "7", "8", "9", "0", "1", "2"]
print(eval_rpn(tokens)) # Output: 15 tokens = [
"+",
"2

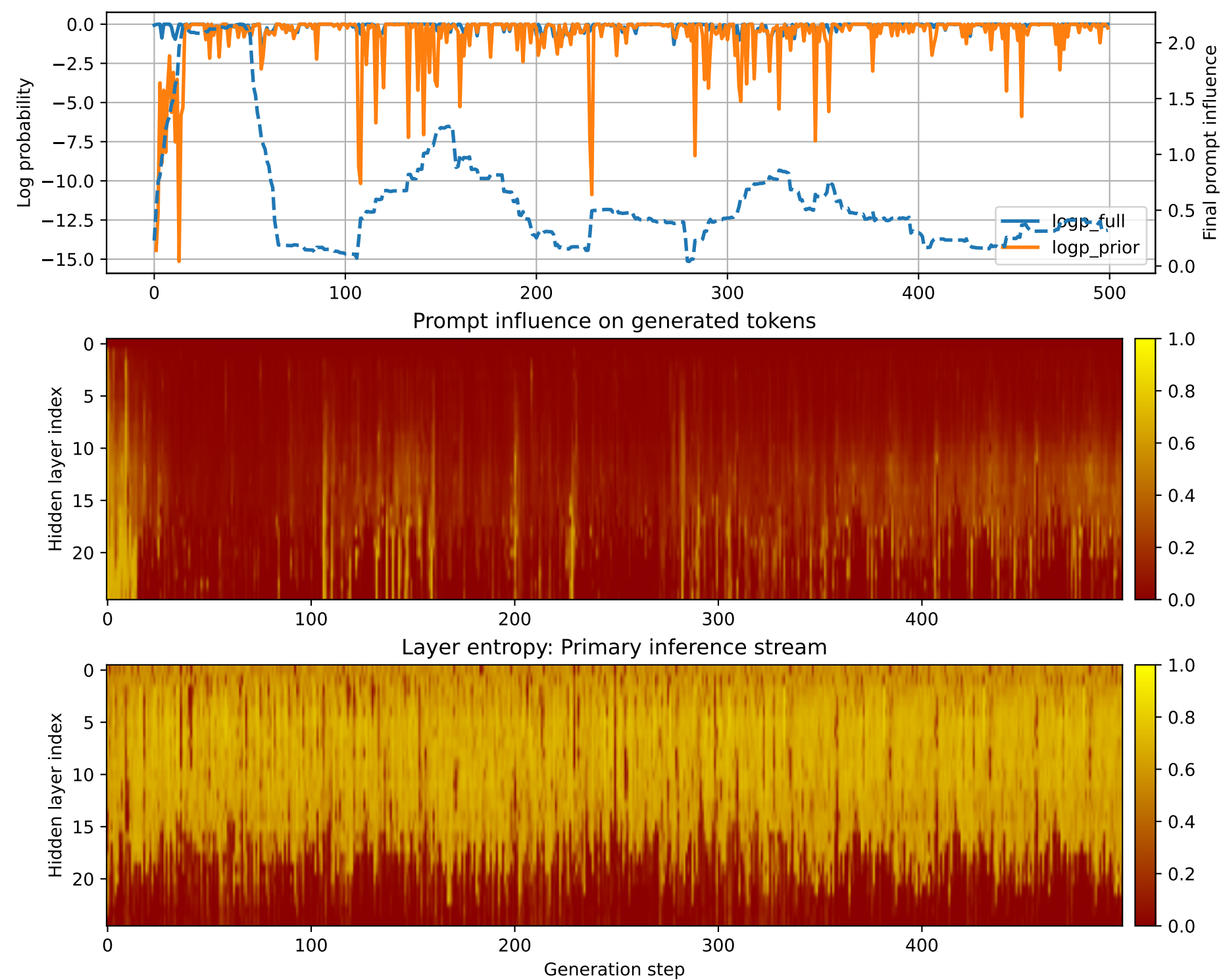
Prompt 1



PROMPT:
Task ID: PYC071 Return ONLY valid Python code. Do not use
Markdown fences. Do not include prose,
explanations, examples, print statements, or tests. Your code
must contain this exact check-token line: #
CHECK_TOKEN: PYC071 Your code must contain this required symbol:
def tokenize_arithmetic Implement this
task: define function tokenize_arithmetic(expr) for
integers,+,-,*,/,(),.. Start your answer with the check-
token line, then provide the Python implementation.

MODEL RESPONSE:
CHECK_TOKEN: PYC071 # Define function tokenize_arithmetic for
integers, +, -, *, /, (,). def
tokenize_arithmetic(expr): # Check if the expression is a
valid arithmetic expression if not
isinstance(expr, str) or not expr.isnumeric(): raise
ValueError("Invalid arithmetic expression")
Tokenize the expression into individual tokens tokens =
expr.split() # Check if the tokens are
valid integers if not all(isinstance(token, int) for token in
tokens): raise ValueError("Invalid
token: " + str(token) + ", must be an integer") # Check if
the tokens are valid operators (+, -, *, /)
if not all(token in ['+', '-', '*', '/'] for token in tokens):
raise ValueError("Invalid operator: " +
str(token) + ", must be one of the following: +, -, *, /") #
Return the tokenized expression return
tokens # Test the tokenize_arithmetic function
print(tokenize_arithmetic("1 + 2 * 3")) # Output: [1, 2, 3]
print(tokenize_arithmetic("4 - 5 / 6")) # Output: [4, -5, 6]
print(tokenize_arithmetic("7 * (8 - 9)")) #
Output: [7, 8, -9] print(tokenize_arithmetic("10 / (11 - 12)"))
Output: [10, 1, 12]
print(tokenize_arithmetic("13 * (14 - 15)")) # Output: [13, 14,
-15] print(tokenize_arithmetic("16 / (17 -
18)")) # Output: [16, 17, -18] print(tokenize_arithmetic("19 *
(20 - 21)")) # Output: [19, 20, -21]
print(tokenize_arithmetic("22 / (23 - 24)")) # Output: [22, 23,
-24] print(tokenize_arithmetic("25 * (26 -
27)")) # Output: [25, 26, -27] print(tokenize_arithmetic("28 /

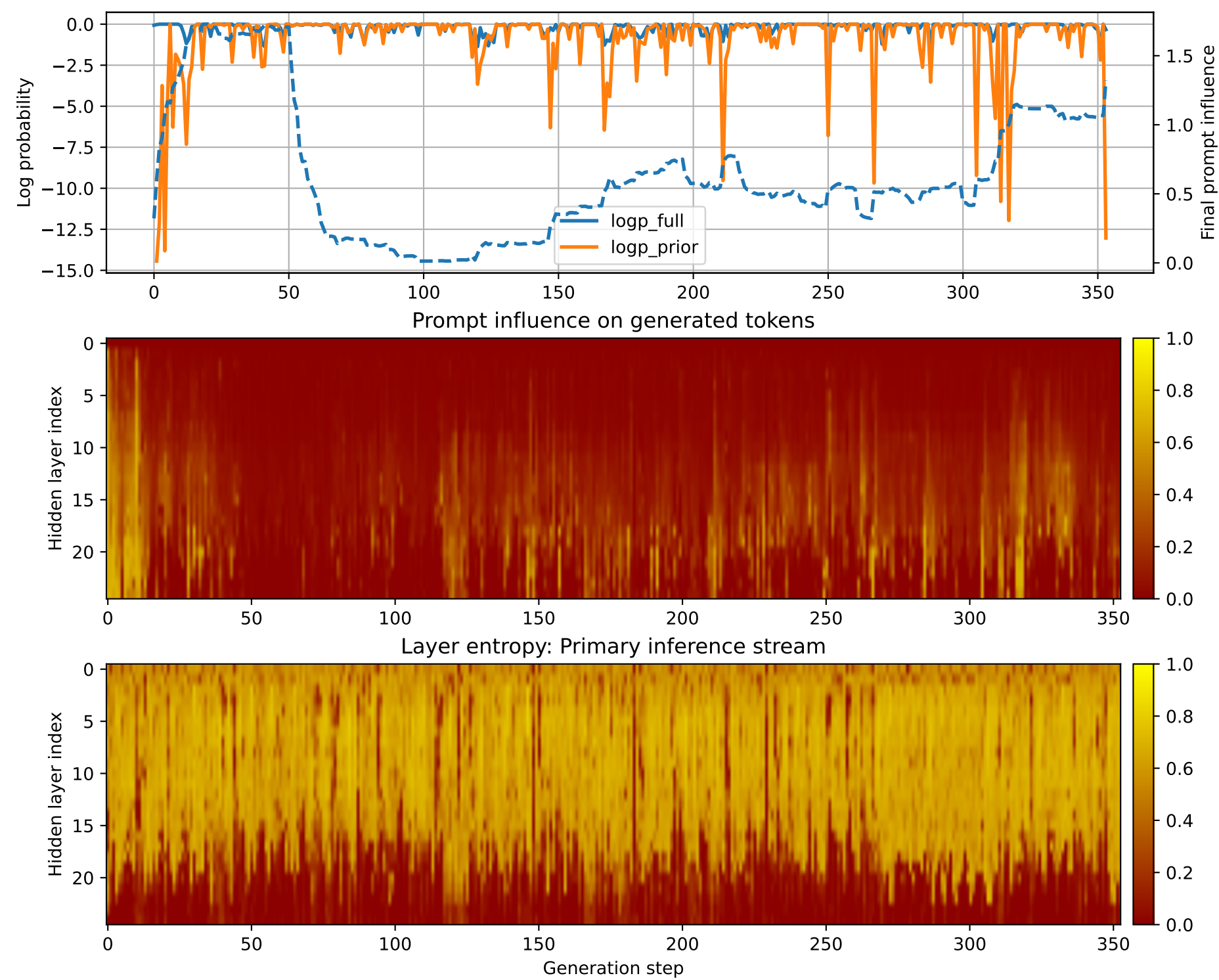
Prompt 1



PROMPT:
Task ID: PYC070 Return ONLY valid Python code. Do not use
Markdown fences. Do not include prose,
explanations, examples, print statements, or tests. Your code
must contain this exact check-token line: #
CHECK_TOKEN: PYC070 Your code must contain this required symbol:
def edit_distance Implement this task:
define function edit_distance(a,b).. Start your answer with the
check-token line, then provide the Python
implementation.

MODEL RESPONSE:
CHECK_TOKEN: PYC070 # Define function edit_distance(a, b) def
edit_distance(a, b): """ This
function takes two strings, a and b, as input and returns the
minimum number of operations required to
transform string a into string b, using the following operations:
1. Insert a character at a specific
position in a. 2. Delete a character from a specific position
in a. 3. Replace a character at a
specific position in a with another character. Parameters:
a (str): The first string to be edited. Returns: int:
b (str): The second string to be edited. The minimum number of operations required to
transform string a into string b. Examples: >>>
edit_distance("abc", "def") 1 >>>
edit_distance("hello", "world") 3 >>>
edit_distance("python", "programming") 2 "" #
Initialize a dictionary to store the edit distance for each
character edit_distance_dict = {} #
Iterate over each character in a and b for i in
range(len(a)): # Check if the current characters
in a and b are the same if a[i] == b[i]: # If
they are the same, add 0 to the edit
distance edit_distance_dict[a[i]] = 0 # If
the current characters are different,
calculate the edit distance using the following rules:
elif a[i] == 'a': # If the
character is 'b', insert 'b' at the current position
edit_distance_dict[a[i]] = edit_distance(a[i
+ 1:], b[i + 1:]) elif a[i] == 'b': # If the
character is 'a', delete 'a' at the current
position edit_distance_dict[a[i]] = edit_distance(a[i
- 1:], b[i - 1:]) elif a[i] == 'c':
If the character is 'b', insert 'c' at the current position
edit_distance_dict[a[i]] =
edit_distance(a[i + 1:], b[i + 2:]) elif a[i] == 'd':
If the character is 'c', delete
'c' at the current position edit_distance_dict[a[i]]
= edit_distance(a[i - 1:], b

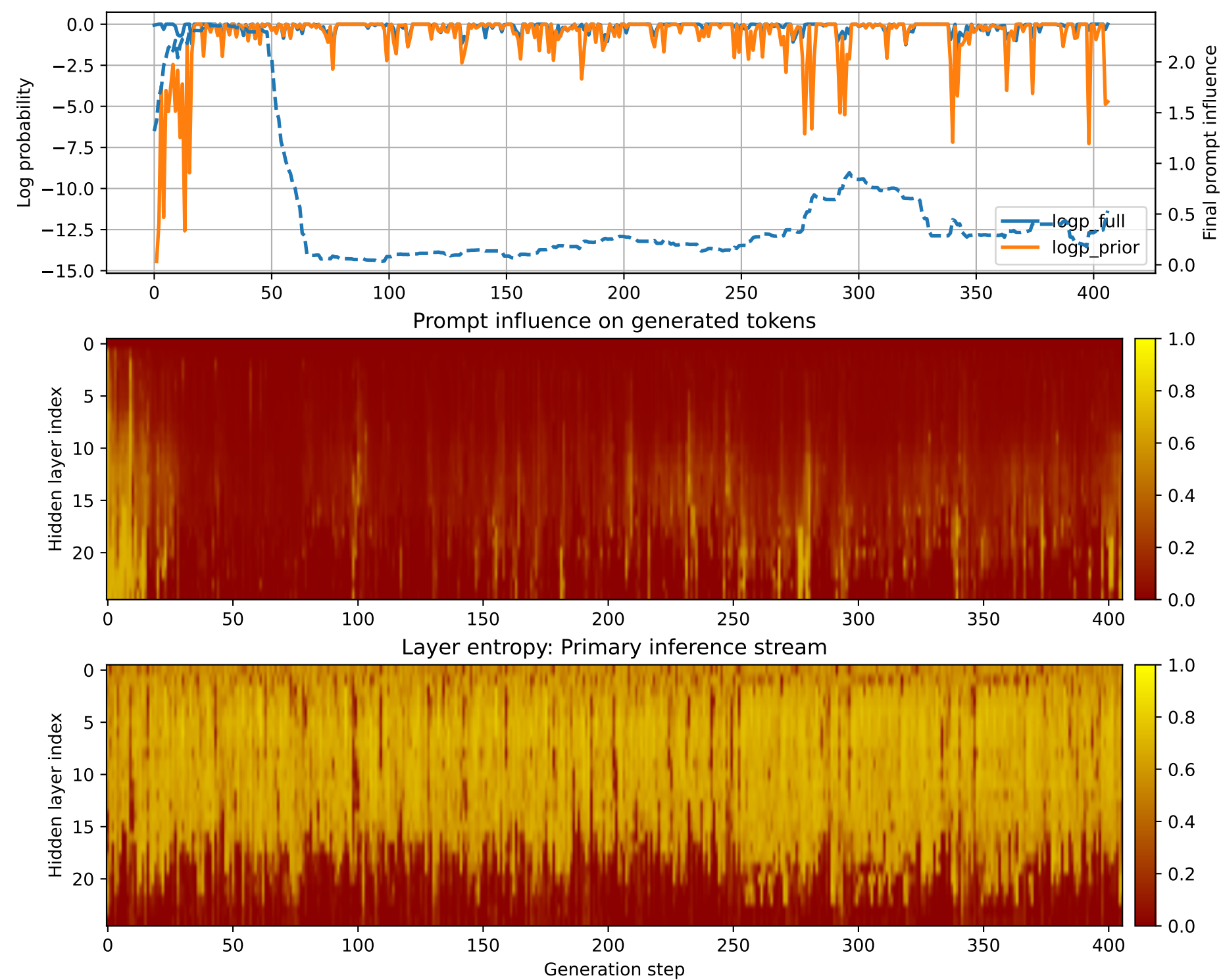
Prompt 1



PROMPT:
Task ID: PYC069 Return ONLY valid Python code. Do not use
Markdown fences. Do not include prose,
explanations, examples, print statements, or tests. Your code
must contain this exact check-token line: #
CHECK_TOKEN: PYC069 Your code must contain this required symbol:
def knapsack_01 Implement this task: define
function knapsack_01(weights, values, capacity).. Start your
answer with the check-token line, then provide
the Python implementation.

MODEL RESPONSE:
CHECK_TOKEN: knapsack_01 # Function definition for knapsack_01
def knapsack_01(weights, values, capacity):
Check if the input parameters are valid if not
isinstance(weights, list) or not isinstance(values, list)
or not isinstance(capacity, int): raise ValueError("All
input parameters must be lists.") #
Check if the capacity is less than or equal to 0 if capacity
< 0: raise ValueError("The capacity
must be a non-negative integer.") # Initialize the
maximum value and the current value max_value
= float('-inf') current_value = 0 # Iterate through
the weights and values for weight in
weights: for value in values: # Calculate the
subproblem: find the maximum value that can
be obtained by selecting the current weight and value
sub_value = weight * value + current_value
Check if the subproblem is feasible (i.e., the capacity is not
exceeded) if sub_value <= capacity:
Update the maximum value and current
value max_value =
max(max_value, sub_value) current_value =
sub_value # Return the maximum value
return max_value # Example usage weights = [10, 20, 30, 40]
values = [60, 80, 100, 120] capacity = 50 # Call
the knapsack_01 function max_value = knapsack_01(weights, values,
capacity) print("Maximum value:", max_value)
Output: 150

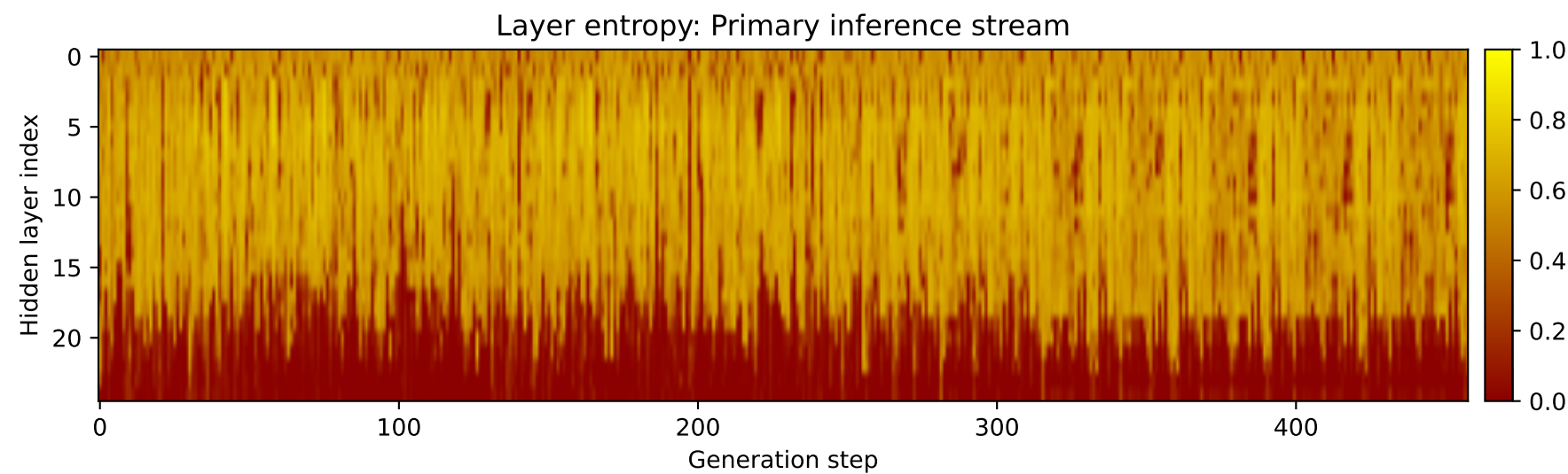
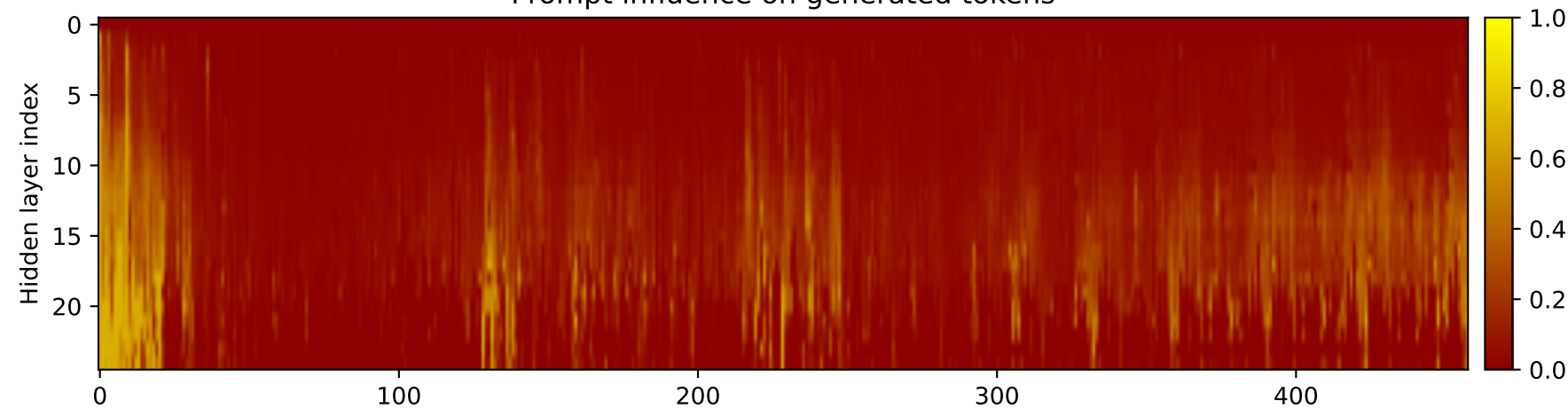
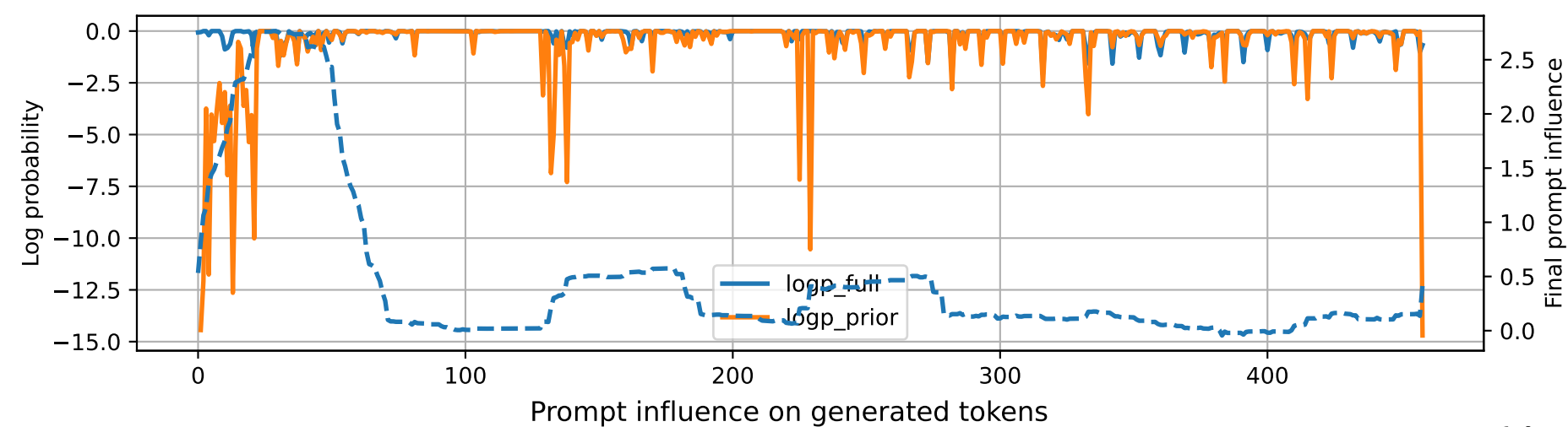
Prompt 1



PROMPT:
Task ID: PYC068 Return ONLY valid Python code. Do not use
Markdown fences. Do not include prose,
explanations, examples, print statements, or tests. Your code
must contain this exact check-token line: #
CHECK_TOKEN: PYC068 Your code must contain this required symbol:
def coin_change_min Implement this task:
define function coin_change_min(coins, amount).. Start your
answer with the check-token line, then provide
the Python implementation.

MODEL RESPONSE:
CHECK_TOKEN: PYC068 # Define function coin_change_min def
coin_change_min(coins, amount): """
Calculate the minimum number of coins required to make change for
a given amount. Args: coins
(list): A list of integers representing the denominations of the
coins. amount (int): The amount of
money to be made into change. Returns: int: The
minimum number of coins required to make change
for the given amount. """ # Check if the amount is
negative if amount < 0: raise
ValueError("Amount must be a non-negative integer.") #
Initialize a variable to store the minimum number
of coins required min_coins = 0 # Iterate through the
coin denominations for coin in coins:
Calculate the amount of change required for the current coin
change = amount // coin #
Check if the change is less than the current minimum number of
coins required if change < min_coins:
If it is, update the minimum number of coins required
min_coins = change # Return the
minimum number of coins required return min_coins # Test the
function coins = [1, 5, 10, 25] amount = 100
print(coin_change_min(coins, amount)) # Output: 3 # Example
usage coins = [1, 5, 10, 25, 50] amount = 100
print(coin_change_min(coins, amount)) # Output: 2 # Check for
invalid input coins = [1, 5, 10, 25, -10]
amount = 100 try: coin_change_min(coins, amount) except
ValueError as e: print(e) # Output: Amount
must be a non-negative integer.

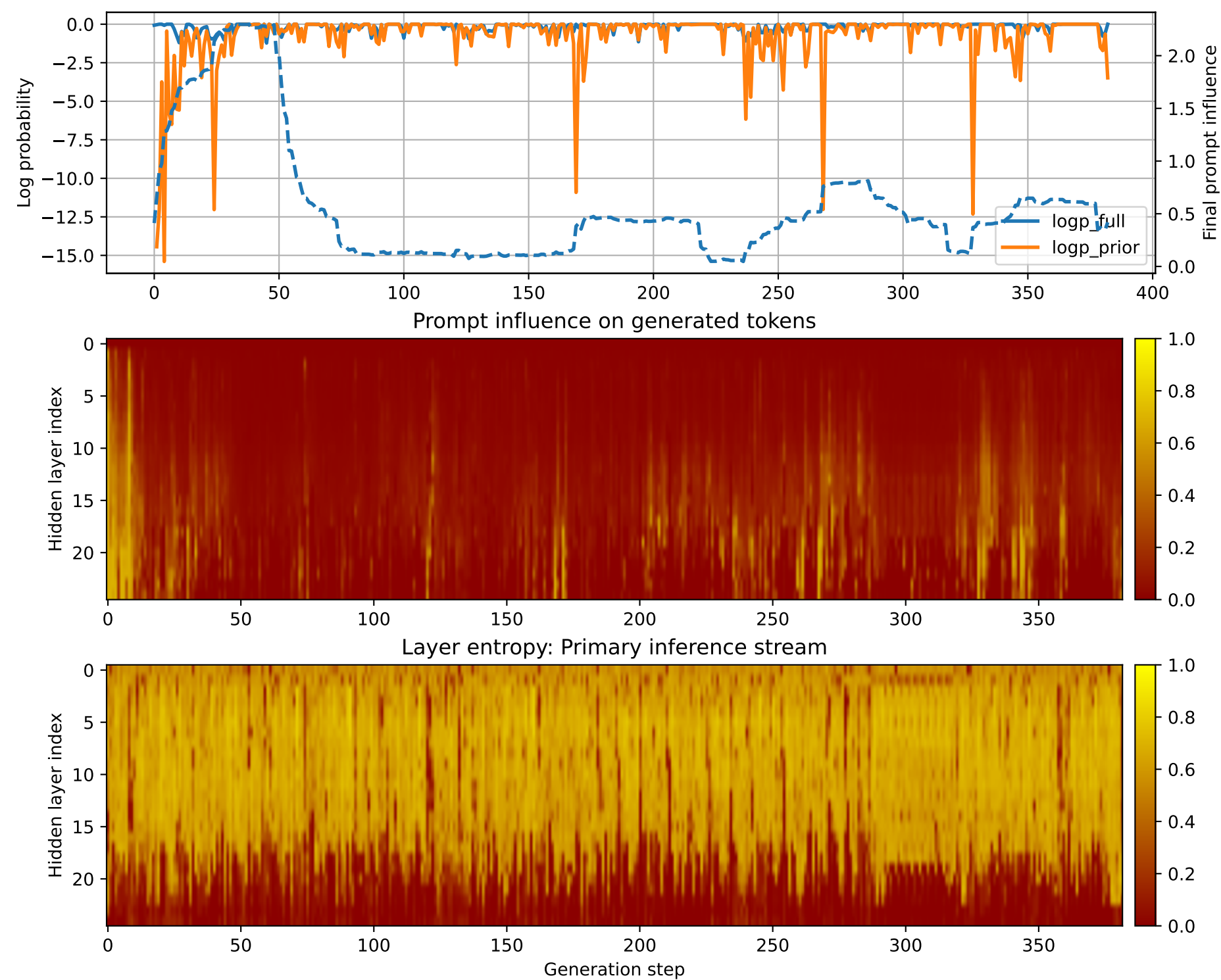
Prompt 1



PROMPT:
Task ID: PYC067 Return ONLY valid Python code. Do not use
Markdown fences. Do not include prose,
explanations, examples, print statements, or tests. Your code
must contain this exact check-token line: #
CHECK_TOKEN: PYC067 Your code must contain this required symbol:
def longest_unique_substring Implement this
task: define function longest_unique_substring(s) returning
length.. Start your answer with the check-token
line, then provide the Python implementation.

MODEL RESPONSE:
CHECK_TOKEN: PYC067 # Define function
longest_unique_substring(s) returning length. def
longest_unique_substring(s): # Initialize a dictionary to
store the count of each character in the string
char_count = {} # Iterate over each character in the
string for char in s: # If the
character is already in the dictionary, increment its count by 1
if char in char_count:
char_count[char] += 1 # Otherwise, add the character to
the dictionary with a count of 1 else:
char_count[char] = 1 # Initialize variables to store the
longest unique substring and its length
longest_substring = "" longest_length = 0 # Iterate
over the characters in the dictionary for
char, count in char_count.items(): # If the count of the
character is greater than the current longest
length, update the longest length and longest substring
if count > longest_length:
longest_length = count longest_substring = char
Return the length of the longest unique
substring return longest_length # Test the function
print(longest_unique_substring("hello world")) #
Output: 7 print(longest_unique_substring("Python")) # Output: 5
print(longest_unique_substring("12345")) #
Output: 5 print(longest_unique_substring("")) # Output: 0
print(longest_unique_substring("1234567890")) #
Output: 11 print(longest_unique_substring("1234567890abc")) #
Output: 11
print(longest_unique_substring("123456789012345")) # Output: 12
print(longest_unique_substring("1234567890123456")) # Output: 12
print(longest_unique_substring("123456789012345678")) # Output:
12

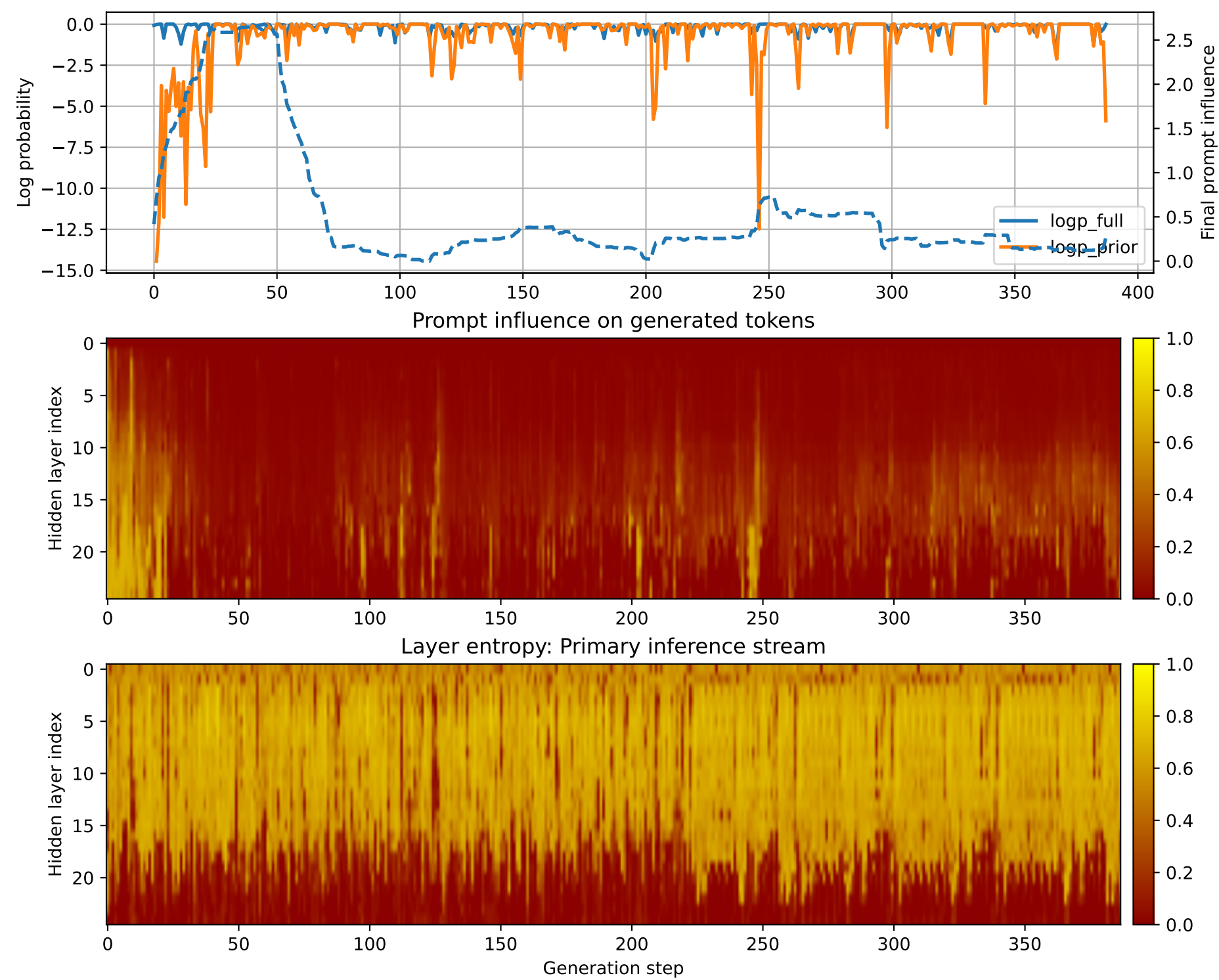
Prompt 1



PROMPT:
Task ID: PYC066 Return ONLY valid Python code. Do not use
Markdown fences. Do not include prose,
explanations, examples, print statements, or tests. Your code
must contain this exact check-token line: #
CHECK_TOKEN: PYC066 Your code must contain this required symbol:
def sliding_window_max_sum Implement this
task: define function sliding_window_max_sum(xs,k).. Start your
answer with the check-token line, then
provide the Python implementation.

MODEL RESPONSE:
CHECK_TOKEN: sliding_window_max_sum # Function to find the
maximum sum of sliding window elements in a
given list xs, with a fixed window size k def
sliding_window_max_sum(xs, k): """ This function takes a
list xs of integers and a window size k as input, and returns the
maximum sum of sliding window elements
within the given window. Args: xs (list): A list of
integers representing the elements in the
window. k (int): The window size. Returns:
int: The maximum sum of sliding window
elements within the given window. """ # Initialize
variables to store the maximum sum and the current
window sum max_sum = float('-inf') current_sum = 0 #
Iterate over the elements in the window
for i in range(len(xs) - k + 1): # Calculate the current
window sum current_sum += xs[i]
If the current window sum is greater than the maximum sum,
update the maximum sum if current_sum >
max_sum: max_sum = current_sum # If the
current window sum is less than the current
window sum, add the element at the current window index to the
current sum if current_sum <
current_sum - k: current_sum += xs[i] # Return
the maximum sum return max_sum # Example
usage xs = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10] k = 3 max_sum =
sliding_window_max_sum(xs, k) print(f"The maximum
sum of sliding window elements within the window of size {k} is
{max_sum}.") # Output: The maximum sum of
sliding window elements within the window of size 3 is 21.

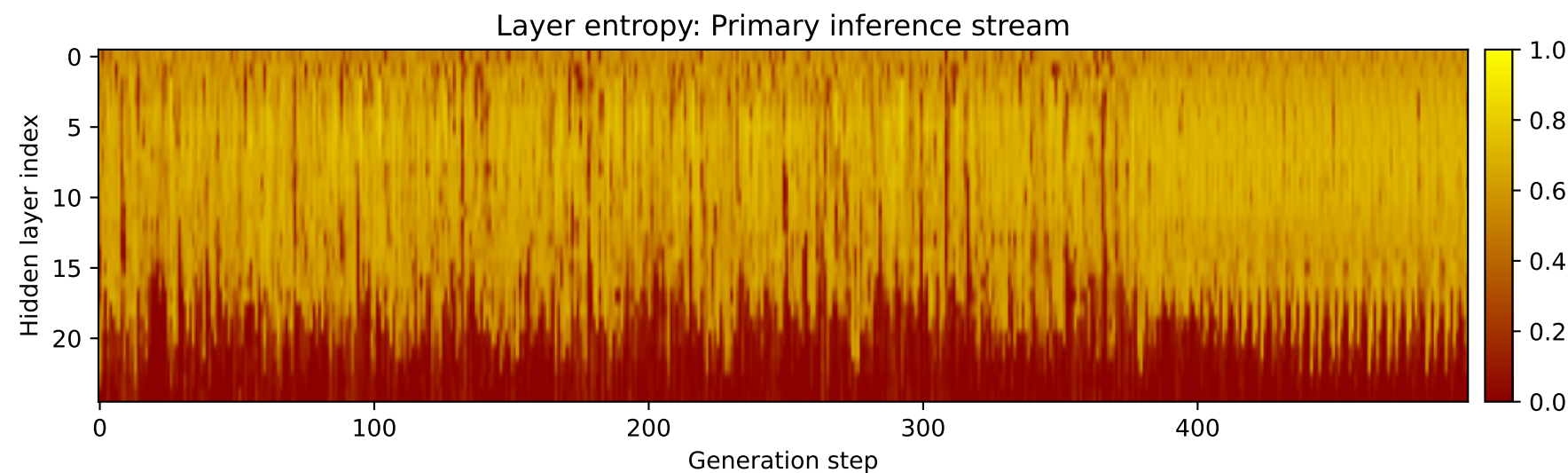
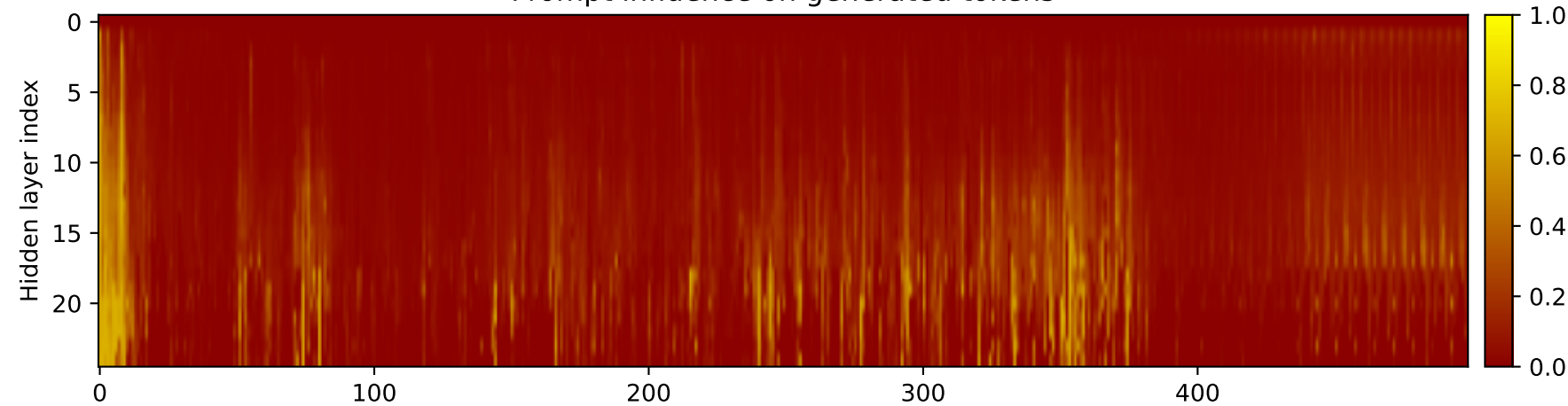
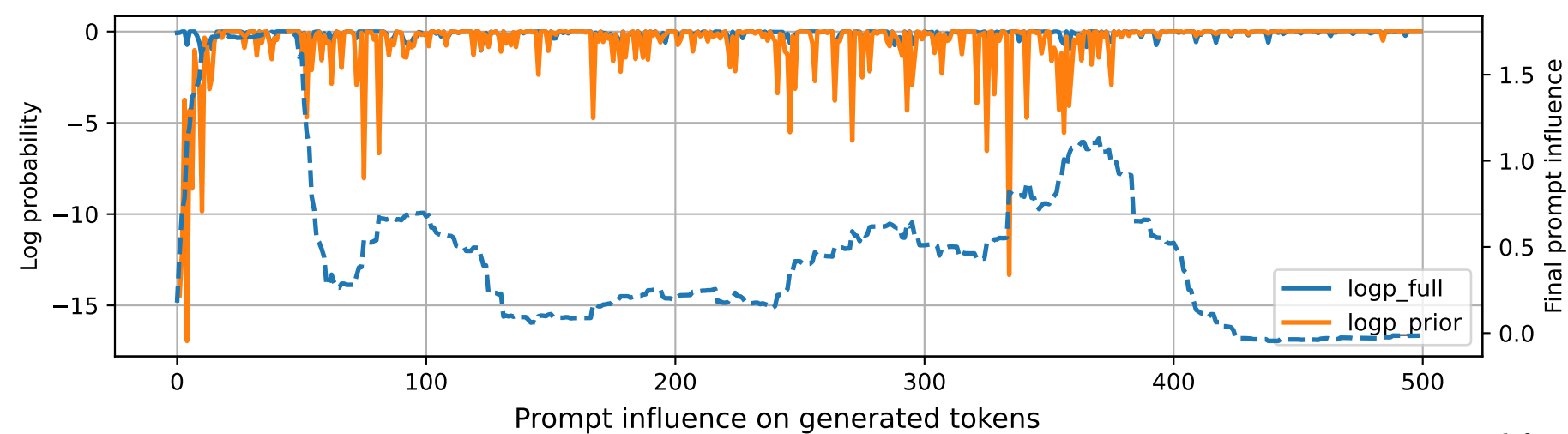
Prompt 1



PROMPT:
Task ID: PYC065 Return ONLY valid Python code. Do not use
Markdown fences. Do not include prose,
explanations, examples, print statements, or tests. Your code
must contain this exact check-token line: #
CHECK_TOKEN: PYC065 Your code must contain this required symbol:
def two_sum Implement this task: define
function two_sum(nums,target) returning indices or None.. Start
your answer with the check-token line, then
provide the Python implementation.

MODEL RESPONSE:
CHECK_TOKEN: PYC065 # Define function two_sum(nums, target)
returning indices or None. def two_sum(nums,
target): """ Returns the indices of the two numbers in
the given list `nums` that add up to the target
value `target`. Args: nums (list): A list of
integers. target (int): The target value to
find two numbers that add up to. Returns: list: A
list of indices of the two numbers in `nums`
that add up to `target`. None: If no such pair exists,
returns None. """ # Initialize an empty
list to store the indices indices = [] # Iterate through
the list of integers for i in
range(len(nums)): # Check if the current number is equal
to the target if nums[i] == target:
If it is, add the index to the list
indices.append(i) # Return the indices if found,
otherwise return None return indices if indices else None #
Test the function nums = [1, 2, 3, 4, 5]
target = 7 print(two_sum(nums, target)) # Output: [2, 4] nums =
[1, 2, 3, 4, 6] target = 10
print(two_sum(nums, target)) # Output: None nums = [1, 2, 3, 4,
5, 6] target = 11 print(two_sum(nums,
target)) # Output: None nums = [1, 2, 3, 4, 5, 6, 7] target =
12 print(two_sum(nums, target)) # Output: [3,
5]

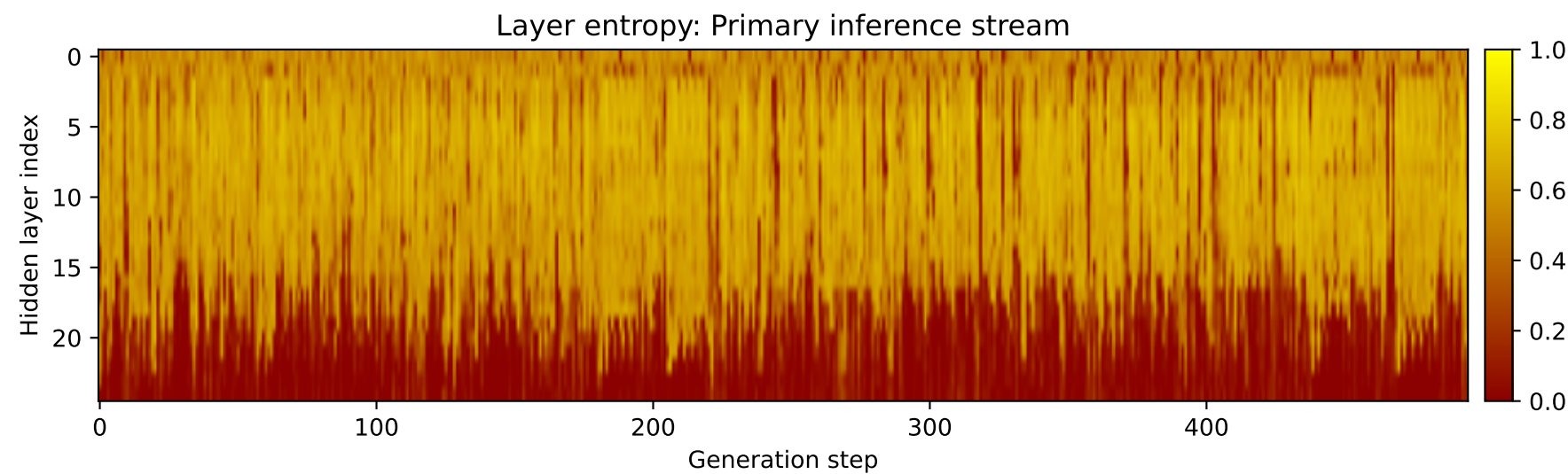
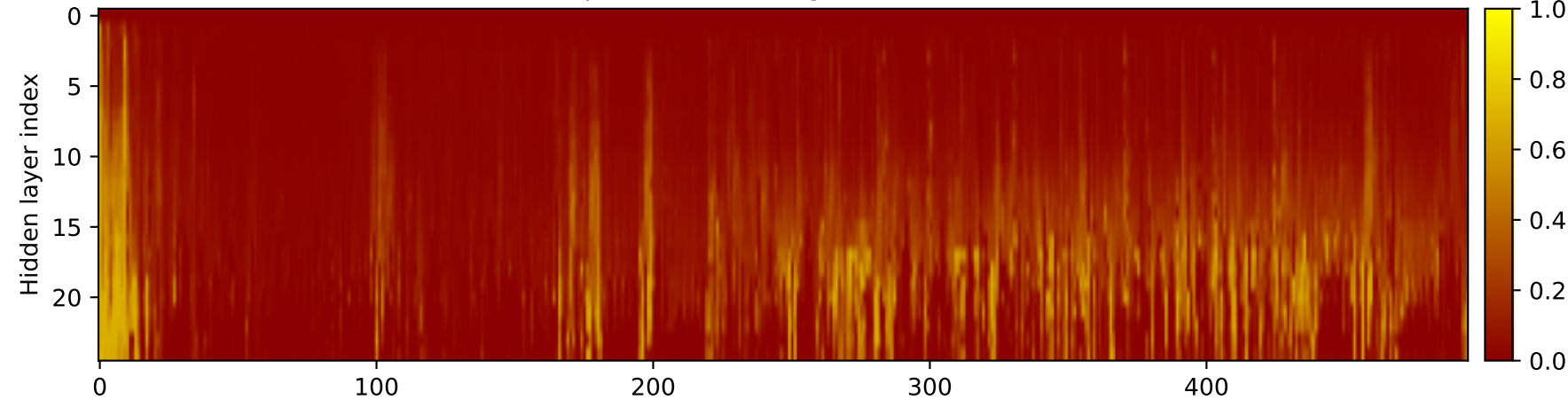
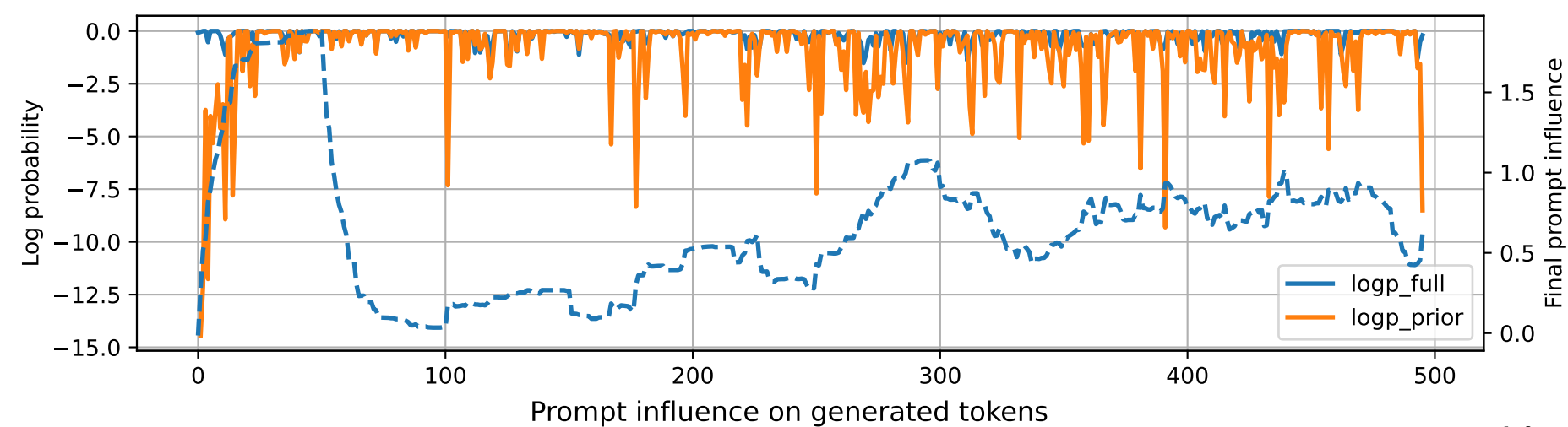
Prompt 1



PROMPT:
Task ID: PYC064 Return ONLY valid Python code. Do not use
Markdown fences. Do not include prose,
explanations, examples, print statements, or tests. Your code
must contain this exact check-token line: #
CHECK_TOKEN: PYC064 Your code must contain this required symbol:
def shortest_path_unweighted Implement this
task: define function
shortest_path_unweighted(graph,start,goal).. Start your answer
with the check-token
line, then provide the Python implementation.

MODEL RESPONSE:
CHECK TOKEN: shortest_path_unweighted # Definition for a
graph. # class Graph: # def __init__(self,
vertices): # self.graph = {} # for i in
range(vertices): # self.graph[i] = [] #
Function to calculate the shortest path between two nodes in an
unweighted graph using Dijkstra's algorithm.
def shortest_path_unweighted(graph, start, goal): # Create a
dictionary to store the shortest path
distances from the start node to each node in the graph
distances = {node: float('inf') for node in graph}
Initialize the distance of the start node to 0 and the distance
of all other nodes to infinity
distances[start] = 0 for node in graph:
distances[node] = float('inf') # Create a priority
queue to store the nodes to be visited and their corresponding
distances queue = [(start, 0)] # While
the priority queue is not empty: while queue: # Get
the node with the smallest distance from the
start node current_node, current_distance =
heapq.heappop(queue) # If the current node is the
goal node, return the shortest path distance if
current_node == goal: return
current_distance # For each neighbor of the current
node: for neighbor, distance in
graph[current_node].items(): # If the neighbor is not
in the priority queue or its distance is
less than the current distance, update the distance and add it to
the priority queue if neighbor
not in queue or distance < distances[current_node] + distance:
distances[neighbor] = distance
heapq.heappush(queue, (neighbor, distance)) # If the goal
node was not found, return -1 return -1 #
Example usage: graph = { 'A': ['B', 'C'], 'B': ['D',
'E'], 'C': ['F'], 'D': ['G'], 'E':
['H'], 'F': ['I'], 'G': ['J'], 'H': ['K'], 'I':
['L'], 'J': ['M'], 'K': ['N'],
'L': ['O'], 'M': ['P'], 'N': ['Q'], 'O': ['R'],
'P': ['S'], 'Q':

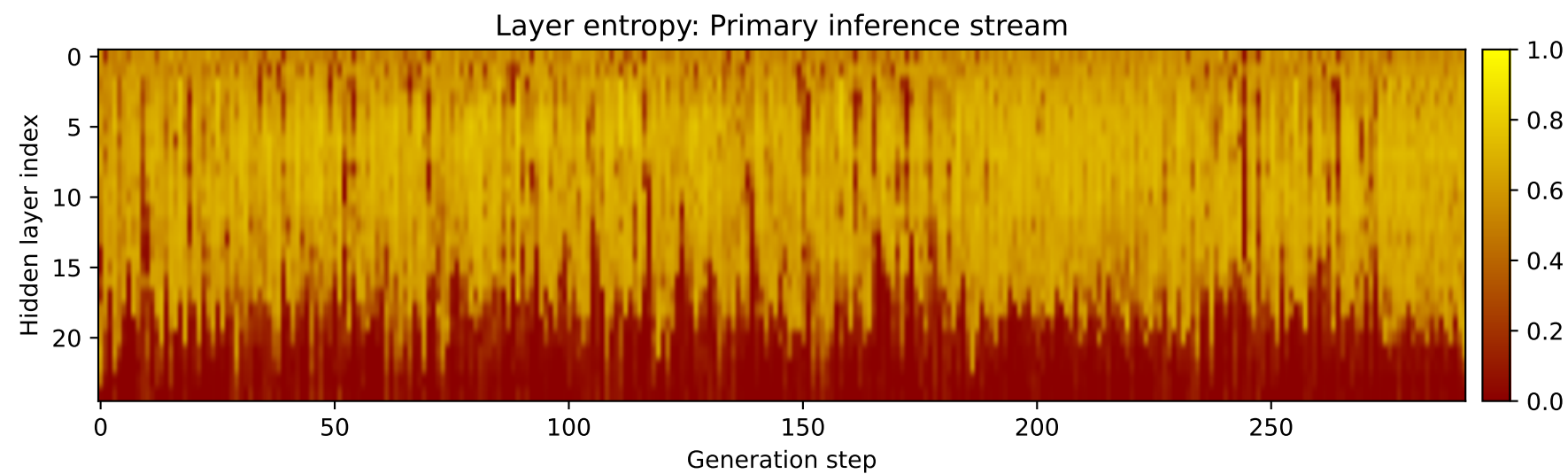
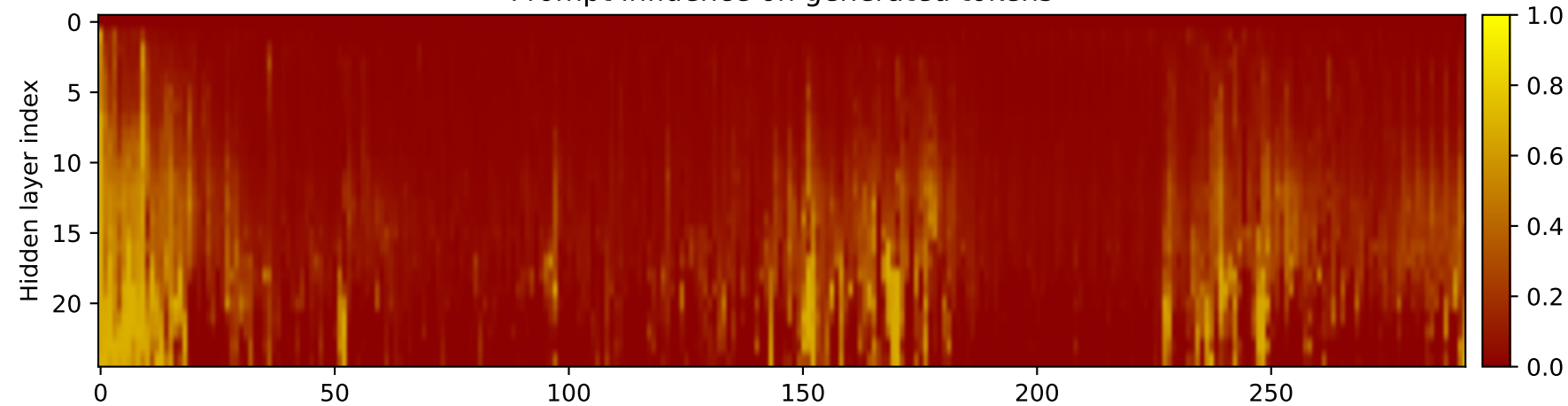
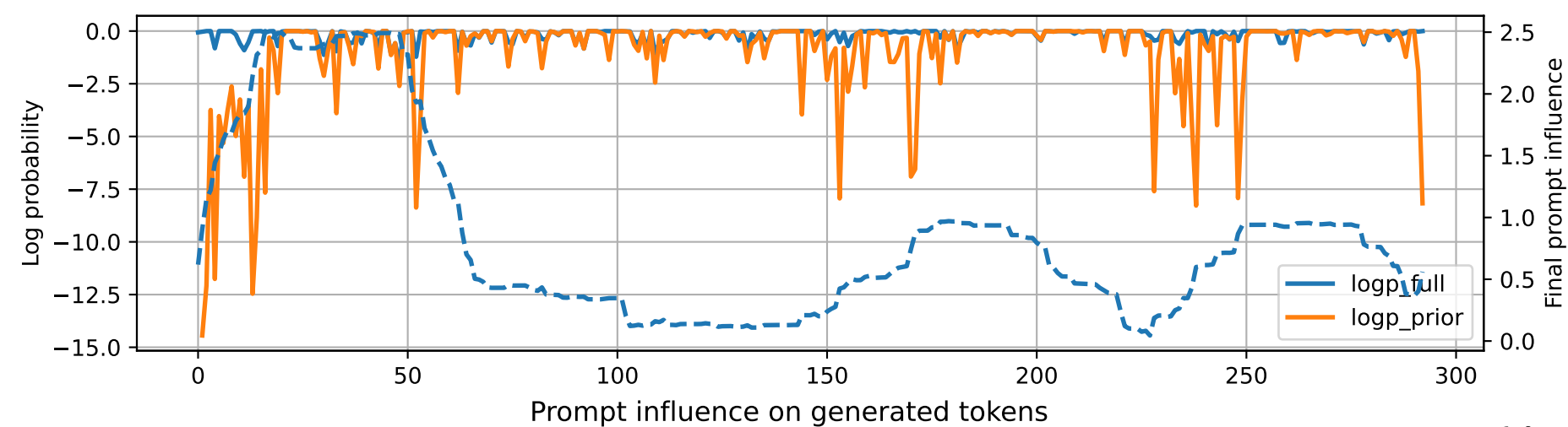
Prompt 1



PROMPT:
Task ID: PYC063 Return ONLY valid Python code. Do not use
Markdown fences. Do not include prose,
explanations, examples, print statements, or tests. Your code
must contain this exact check-token line: #
CHECK_TOKEN: PYC063 Your code must contain this required symbol:
def topological_sort Implement this task:
define function topological_sort(graph) for DAG.. Start your
answer with the check-token line, then provide
the Python implementation.

MODEL RESPONSE:
CHECK_TOKEN: PYC063 # Definition for a directed acyclic graph
(DAG) class DirectedAcyclicGraph: def
__init__(self, vertices): self.graph = {} for
vertex in vertices: def add_edge(self, u, v): if
u not in self.graph or v not in self.graph:
self.graph[u] = [v] else:
self.graph[u].append(v) def topological_sort(self):
visited = set() stack = [(self.graph, [])] while
stack: current_graph,
current_nodes = stack.pop() for node in
current_graph: if node not in visited:
visited.add(node) stack.append((node,
current_nodes + [node])) return
list(visited) # Example usage graph = DirectedAcyclicGraph([1,
2, 3, 4, 5]) print(topological_sort(graph)) #
Output: [1, 2, 3, 4, 5] # Explanation: - The `topological\_sort`
function takes a directed acyclic graph (DAG)
as input. - It initializes an empty set `visited` to keep track
of visited nodes and a stack `stack` with the
initial state as a tuple containing the graph and an empty list
`current\_nodes`. - The `while` loop runs as
long as the stack is not empty. - In each iteration of the loop,
it pops the topmost tuple from the stack and
unpacks it into `current\_graph` and `current\_nodes`. - For each
node in `current\_graph`, it checks if it has
already been visited by adding it to the `visited` set and
appending its list of neighbors to the
`current\_nodes` list. - If the node has not been visited, it is
added to the `visited` set and the stack is
pushed with the node and its list of neighbors. - After the loop
finishes, the `visited` set is converted to a
list and returned as the result. In the example usage, we create
a DAG with 5 vertices (1, 2, 3, 4, 5) and
call the `topological\_sort` function with this graph. The output
is `[1, 2, 3, 4, 5]`, which represents the
topological order of the graph.

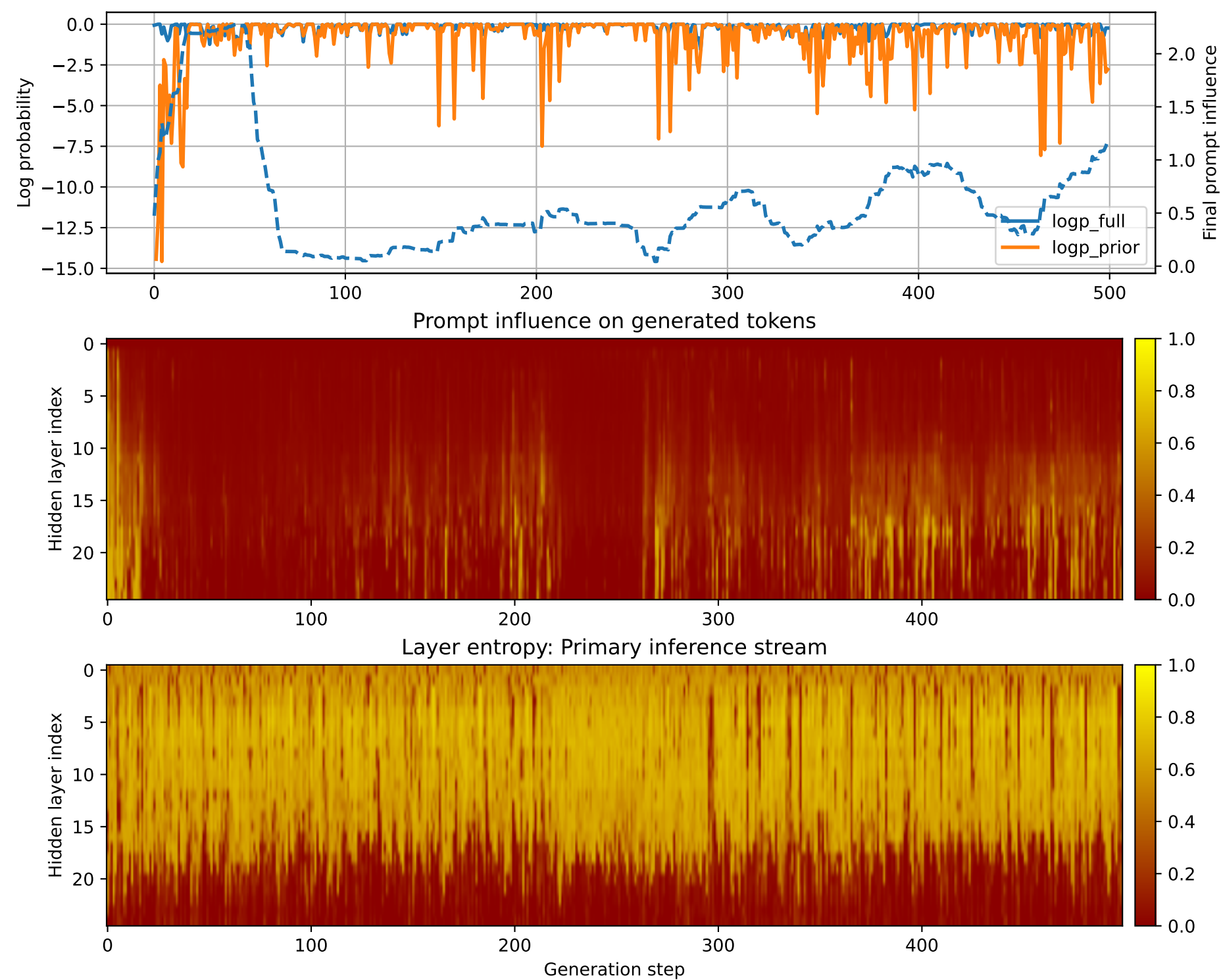
Prompt 1



PROMPT:
Task ID: PYC062 Return ONLY valid Python code. Do not use
Markdown fences. Do not include prose,
explanations, examples, print statements, or tests. Your code
must contain this exact check-token line: #
CHECK_TOKEN: PYC062 Your code must contain this required symbol:
def dfs_iterative Implement this task:
define function dfs_iterative(graph,start) returning visit
order.. Start your answer with the check-token
line, then provide the Python implementation.

MODEL RESPONSE:
CHECK_TOKEN: PYC062 # Define function dfs_iterative(graph,
start) def dfs_iterative(graph, start): #
Create an empty dictionary to store the visited nodes visited
= {} # Create a list to store the visit
order visit_order = [] # Define a recursive function to
traverse the graph def dfs(node):
If the node is the starting node, add it to the visit order
if node == start:
visit_order.append(node) # Otherwise, iterate over the
neighbors of the node for neighbor in
graph[node]: # If the neighbor is not visited,
recursively call dfs on it if neighbor
not in visited: dfs(neighbor) # Start the
recursive traversal from the starting node
dfs(start) # Return the visit order return visit_order
Example usage: graph = { 'A': ['B',
'C'], 'B': ['D', 'E'], 'C': ['F'], 'D': [], 'E':
[], 'F': [] } start_node = 'A' # Call
the dfs_iterative function with the graph and start node
visit_order = dfs_iterative(graph, start_node) #
Print the visit order print(visit_order) # Output: ['A', 'B',
'C', 'D', 'E', 'F']

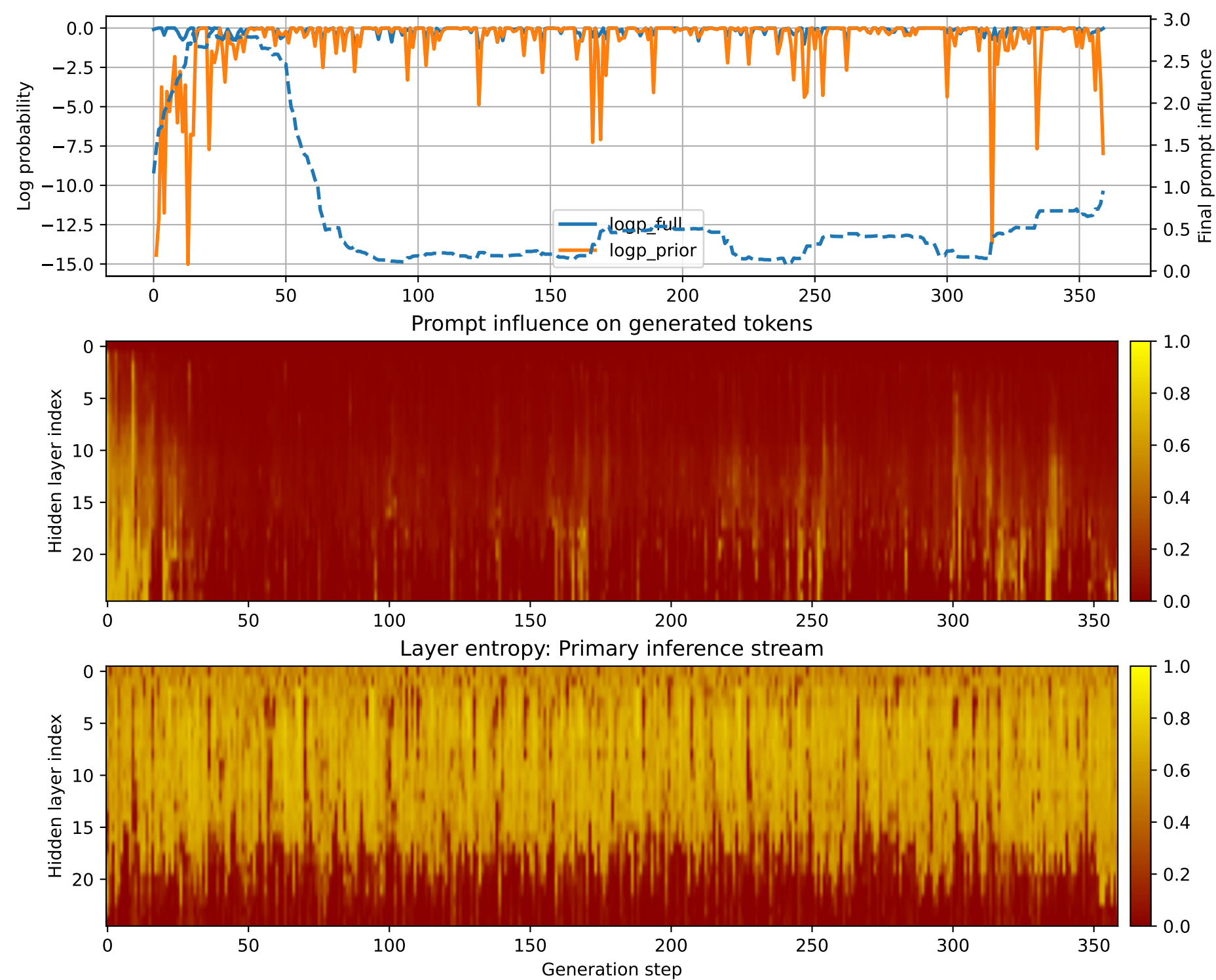
Prompt 1



PROMPT:
Task ID: PYC061 Return ONLY valid Python code. Do not use
Markdown fences. Do not include prose,
explanations, examples, print statements, or tests. Your code
must contain this exact check-token line: #
CHECK_TOKEN: PYC061 Your code must contain this required symbol:
def bfs Implement this task: define
function bfs(graph,start) returning visit order.. Start your
answer with the check-token line, then provide
the Python implementation.

MODEL RESPONSE:
CHECK_TOKEN: BFS # Define function bfs(graph, start) returning
visit order. def bfs(graph, start): #
Create an empty queue to store the visited nodes queue = []
Initialize a visited list to keep track
of visited nodes visited = [] # Set the start node as
the first node to be visited
visited.append(start) # While the queue is not empty,
dequeue the first node from the queue while
queue: # Get the current node from the queue
current_node = queue.pop(0) # If the
current_node is not visited, add it to the visited list
if current_node not in visited:
visited.append(current_node) # For each neighbor of the
current node, enqueue it in the queue
for neighbor in graph[current_node]: # If the
neighbor is not visited, enqueue it in the queue
if neighbor not in visited:
queue.append(neighbor) # Return the visit order of the
visited nodes return visited # Example usage: graph = {
'A': ['B', 'C'], 'B': ['D', 'E'],
'C': ['F'], 'D': [], 'E': [], 'F': [] } start_node =
'A' visit_order = bfs(graph, start_node)
print(f"Visit order of nodes starting from {start_node}:
{visit_order}") ``` In this implementation, we
define a function `bfs` that takes a graph and a start node as
input. The graph is represented as a dictionary
where each node is a key and its neighbors are the values. The
function returns a list of visited nodes, which
represents the order in which nodes were visited during the
breadth-first search traversal. The function
starts by creating an empty queue `queue` to store the visited
nodes and initializing a visited list `visited`
to keep track of visited nodes. The start node is set as the
first node to be visited. The function then
enters a while loop that continues until the queue is empty. In
each iteration of the loop, the current node
is dequeued from the queue and added to the visited list. If the
current node is not visited, it is added to
the visited list. For each neighbor of the current node, the
function enqueues it in the queue. If the
neighbor is not visited, it is also enqueued in the queue. After
the loop

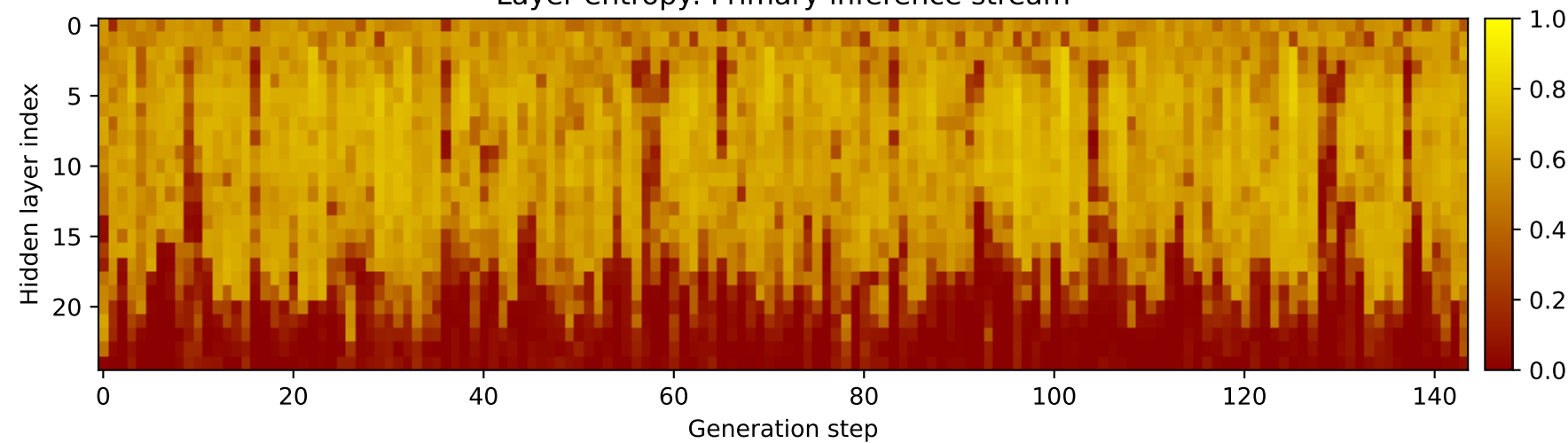
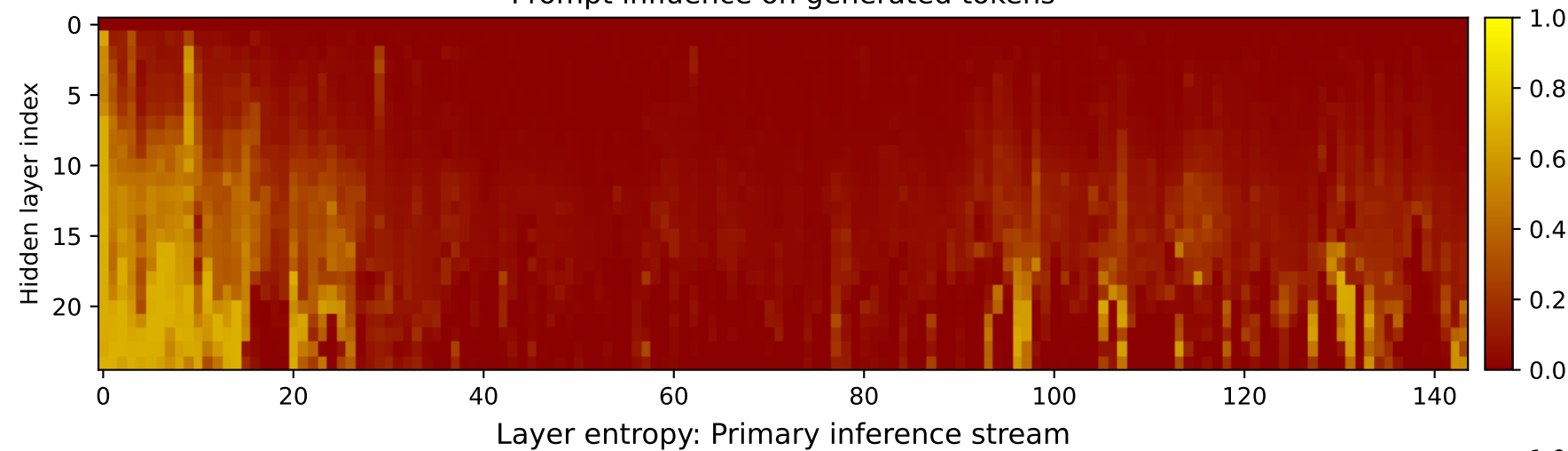
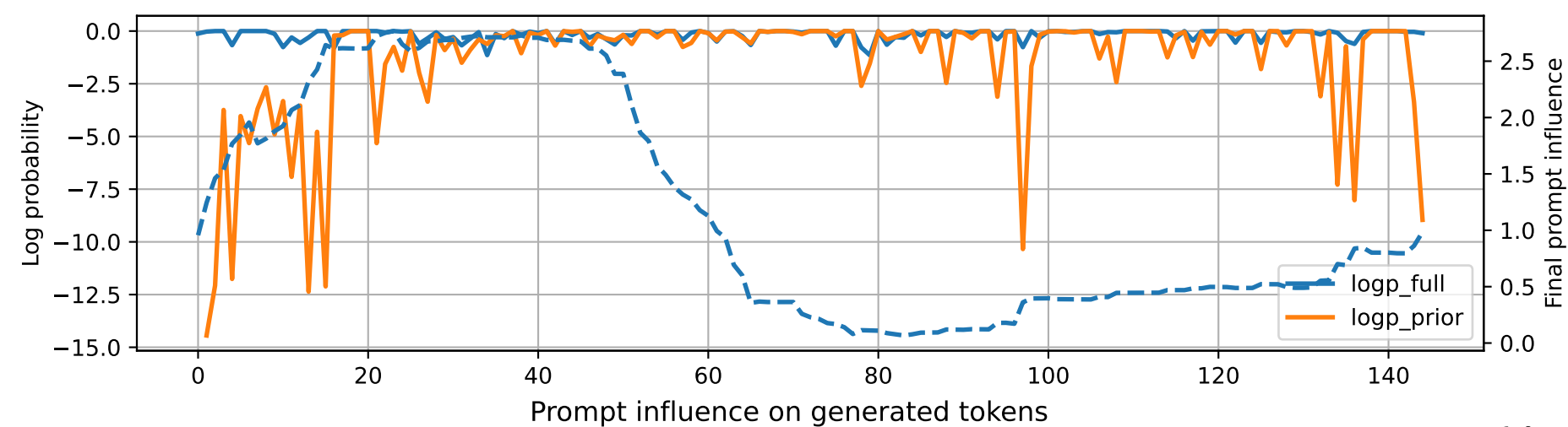
Prompt 1



PROMPT:
Task ID: PYC060 Return ONLY valid Python code. Do not use
Markdown fences. Do not include prose,
explanations, examples, print statements, or tests. Your code
must contain this exact check-token line: #
CHECK_TOKEN: PYC060 Your code must contain this required symbol:
def csv_column_sum Implement this task:
define function csv_column_sum(path, column).. Start your answer
with the check-token line, then provide the
Python implementation.

MODEL RESPONSE:
CHECK_TOKEN: PYC060 # Define function csv_column_sum def
csv_column_sum(path, column): # Check if the
path is a valid file path if not os.path.exists(path):
raise ValueError(f"Path '{path}' does not
exist.") # Check if the specified column exists in the
file if column not in
os.path.listdir(path): raise ValueError(f"Column
'{column}' does not exist in the file '{path}'.")
Open the file and read its contents with open(path, 'r') as
file: # Read the contents of the
file contents = file.read() # Split the contents
into a list of rows rows =
contents.split('\n') # Initialize a dictionary to store
the sum of the specified column for each row
column_sum = {} # Iterate over each row in the file
for row in rows: # Split the row into
a list of values values = row.split(',')
Check if the specified column exists in
the values list if column in values: # If the
column exists, add its value to the sum
dictionary if column in column_sum:
column_sum[column] += float(values[column])
else: # If the column does not exist, add it to
the sum dictionary with a default value of 0
column_sum[column] = 0 # Return the sum dictionary
return column_sum # Example usage path =
'path/to/file.csv' column = 'column_name' column_sum =
csv_column_sum(path, column) print(column_sum) #
Output: {'column_name': 10.0}

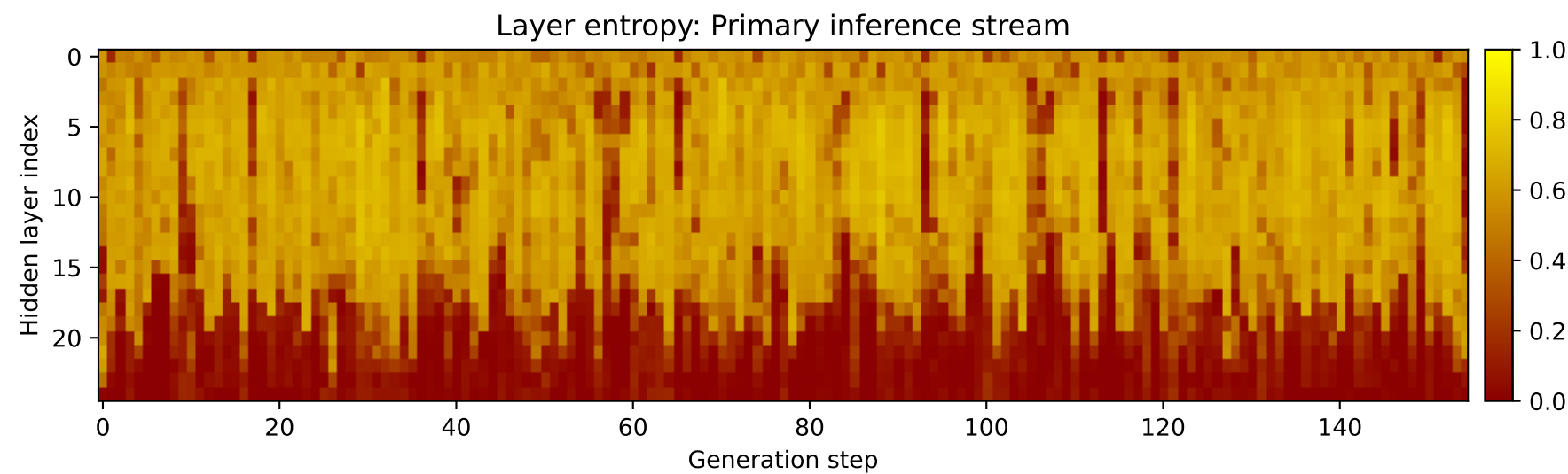
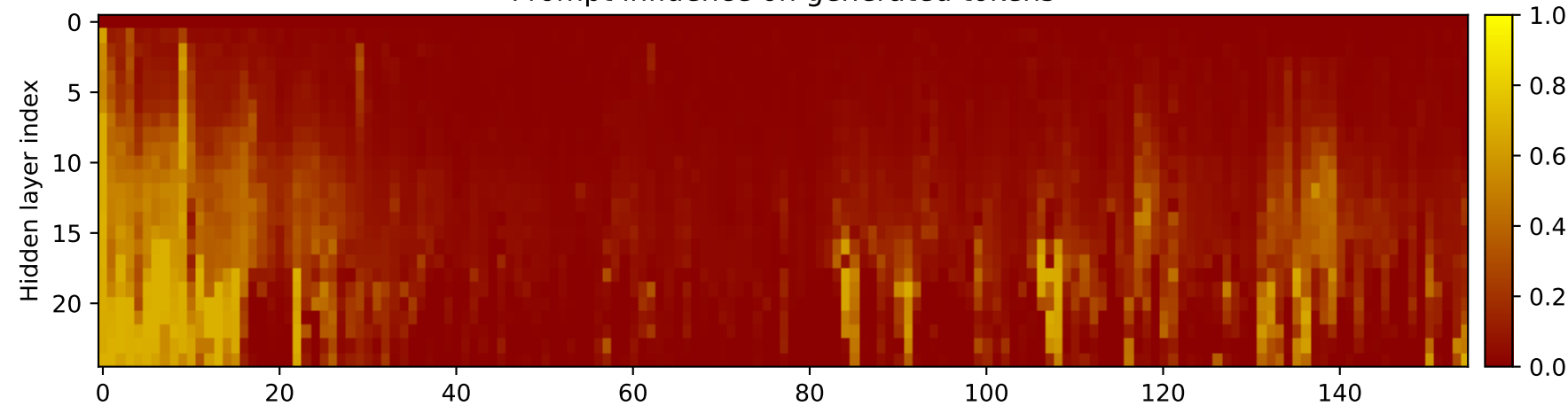
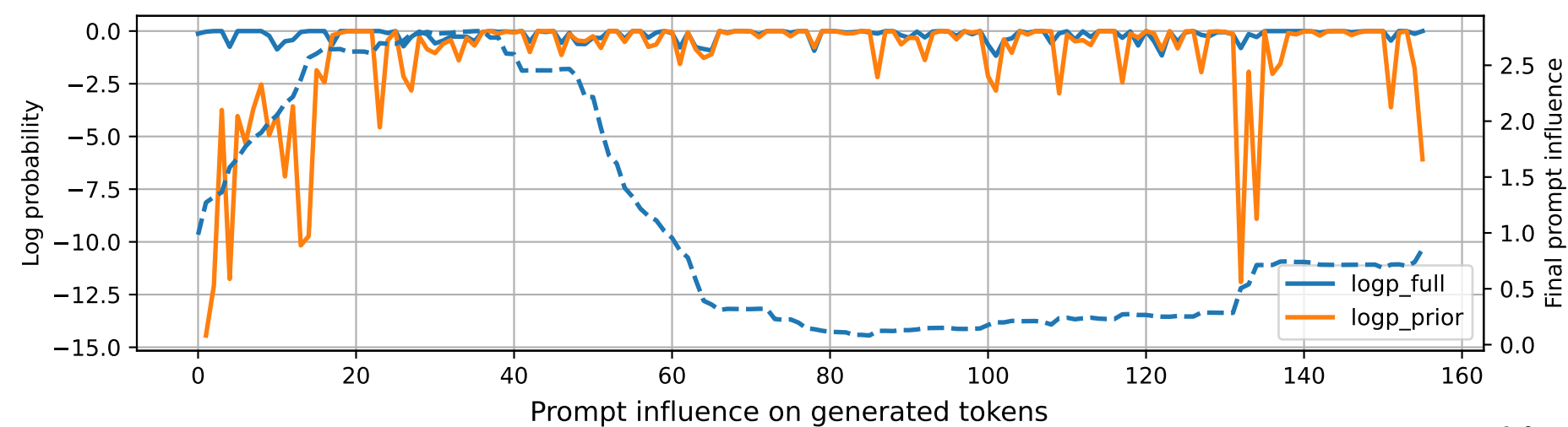
Prompt 1



PROMPT:
Task ID: PYC059 Return ONLY valid Python code. Do not use
Markdown fences. Do not include prose,
explanations, examples, print statements, or tests. Your code
must contain this exact check-token line: #
CHECK_TOKEN: PYC059 Your code must contain this required symbol:
def load_json_field Implement this task:
define function load_json_field(path, field).. Start your answer
with the check-token line, then provide the
Python implementation.

MODEL RESPONSE:
CHECK_TOKEN: PYC059 # Define function load_json_field def
load_json_field(path, field): # Check if the
path is a valid JSON file if not os.path.exists(path):
raise ValueError(f"Path '{path}' does not
exist.") # Load the JSON file with open(path, 'r')
as f: # Read the JSON data
json_data = f.read() # Check if the specified field
exists in the JSON data if field not in
json_data: raise ValueError(f"Field '{field}' not found
in the JSON data.") # Return the
loaded JSON field return json_data[field]

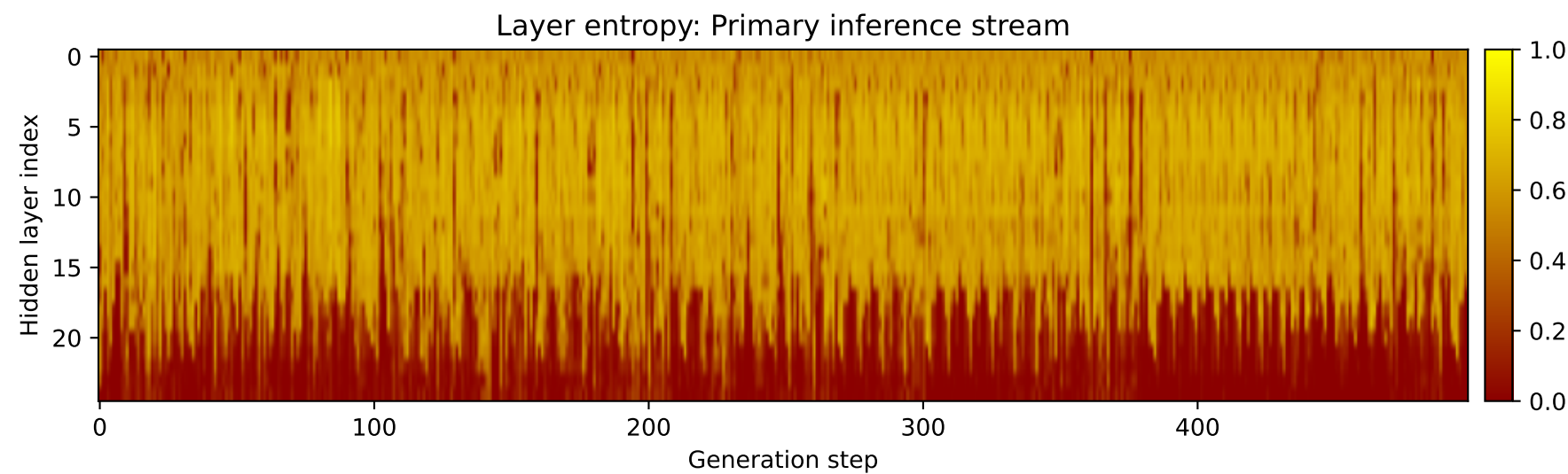
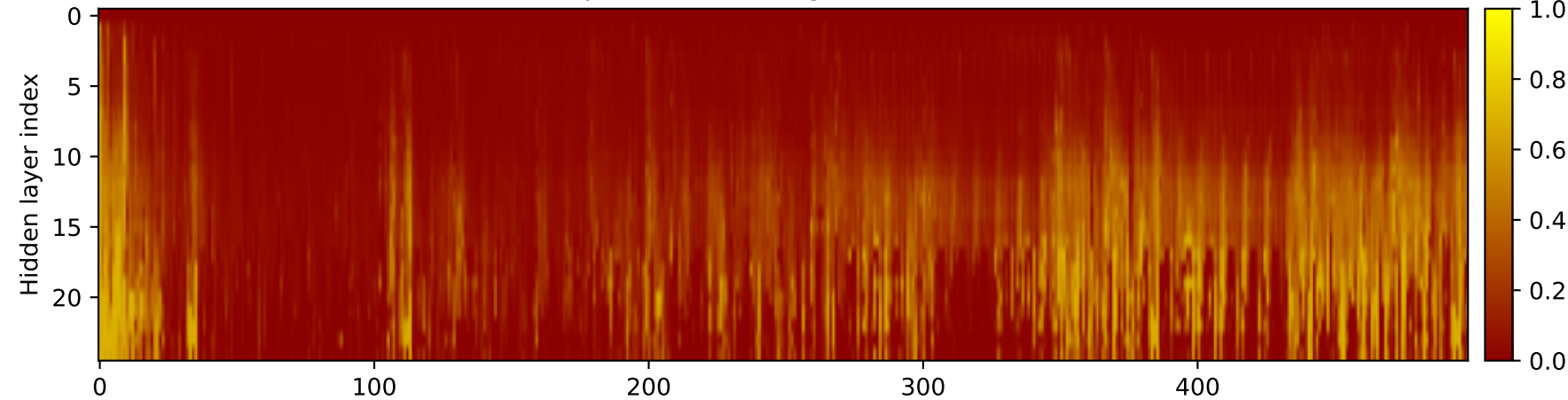
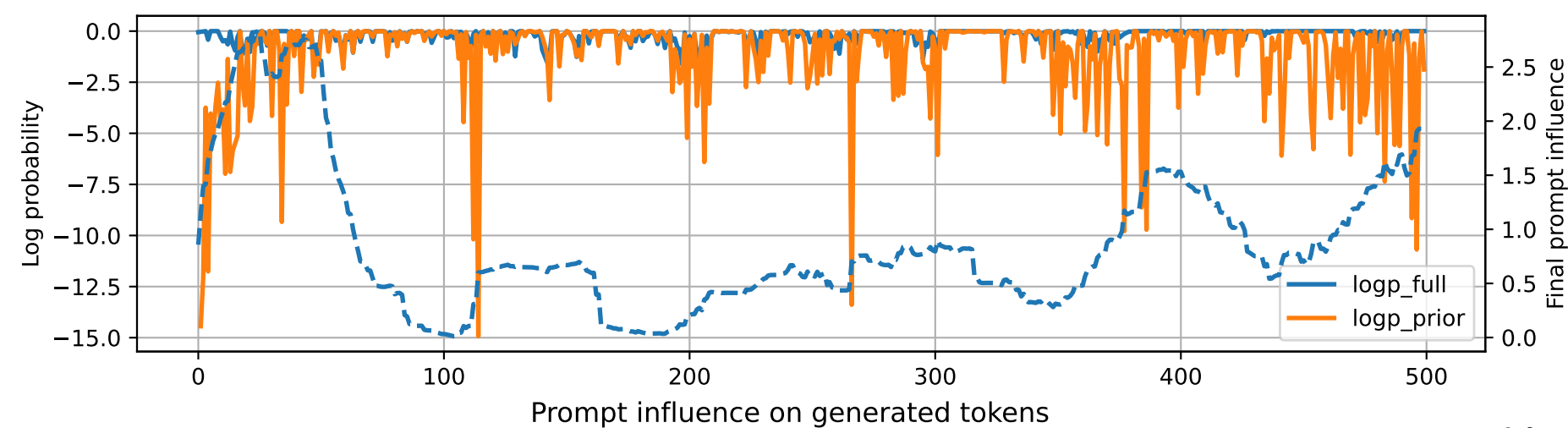
Prompt 1



PROMPT:
Task ID: PYC058 Return ONLY valid Python code. Do not use
Markdown fences. Do not include prose,
explanations, examples, print statements, or tests. Your code
must contain this exact check-token line: #
CHECK_TOKEN: PYC058 Your code must contain this required symbol:
def read_nonempty_lines Implement this
task: define function read_nonempty_lines(path).. Start your
answer with the check-token line, then provide
the Python implementation.

MODEL RESPONSE:
CHECK_TOKEN: PYC058 # Define function read_nonempty_lines def
read_nonempty_lines(path): # Check if the
path is a valid file path if not os.path.exists(path):
raise ValueError(f"Path '{path}' does not
exist.") # Read the file content with open(path,
'r') as file: content = file.read()
Check if the file content is empty if not content:
return "File is empty." # Return the
file content return content # Example usage path =
"/path/to/your/file.txt" nonempty_lines =
read_nonempty_lines(path) print(nonempty_lines) # Output: File
is empty.

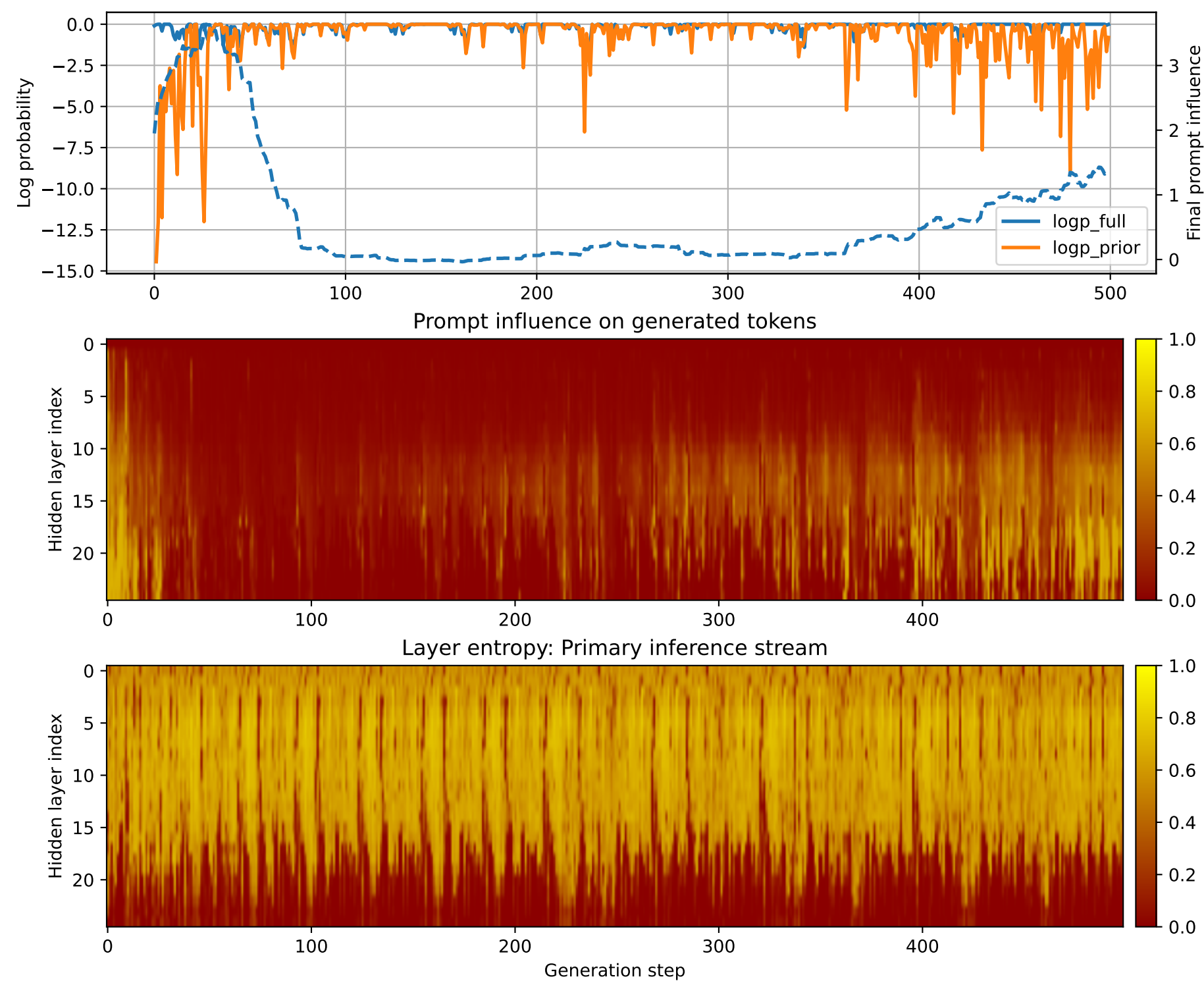
Prompt 1



PROMPT:
Task ID: PYC057 Return ONLY valid Python code. Do not use
Markdown fences. Do not include prose,
explanations, examples, print statements, or tests. Your code
must contain this exact check-token line: #
CHECK_TOKEN: PYC057 Your code must contain this required symbol:
def temp_cwd Implement this task: define
contextmanager temp_cwd(path) using contextlib.. Start your
answer with the check-token line, then provide
the Python implementation.

MODEL RESPONSE:
CHECK_TOKEN: PYC057 # Define a contextmanager using contextlib
to manage the temporary directory import
contextlib @contextmanager def temp_cwd(path): try:
Create a temporary directory using the
provided path os.makedirs(path) # Return the
temporary directory path return path
except Exception as e: # Handle any errors that may occur
during the creation of the temporary
directory print(f"Error creating temporary directory:
{e}") return None # Usage example:
temp_cwd("/path/to/your/directory") # Returns the temporary
directory path
temp_cwd("/path/to/your/directory", "my_directory") # Returns
the temporary directory path with a custom name
"my_directory" temp_cwd("/path/to/your/directory",
"/path/to/your/directory") # Returns the temporary
directory path with a custom name "my_directory" and a custom
parent directory # Example usage with a custom
error handling: try: temp_cwd("/path/to/your/directory",
"my_directory", "custom_error") except Exception
as e: print(f"Error creating temporary directory: {e}")
print(f"Custom error message: {e.message}") #
Example usage with a custom parent directory:
temp_cwd("/path/to/your/directory", "/path/to/your/directory",
"/path/to/your/directory", "custom_parent_directory")
temp_cwd("/path/to/your/directory",
"/path/to/your/directory", "/path/to/your/directory",
"/path/to/your/directory", "/path/to/your/directory",
"custom_parent_directory") # Returns the temporary directory
path with a custom parent directory and a custom
parent directory # Example usage with a custom parent directory
and custom error handling: try:
temp_cwd("/path/to/your/directory", "/path/to/your/directory",
"/path/to/your/directory",
"/path/to/your/directory", "/path/to/your/directory",
"/path/to/your/directory", "custom_parent_directory",
"custom_error") except Exception as e: print(f"Error creating
temporary directory: {e}")
print(f"Custom error message: {e.message}") # Example usage with
a custom parent directory, custom error
handling, and custom parent directory: try:
temp_cwd("/path/to/

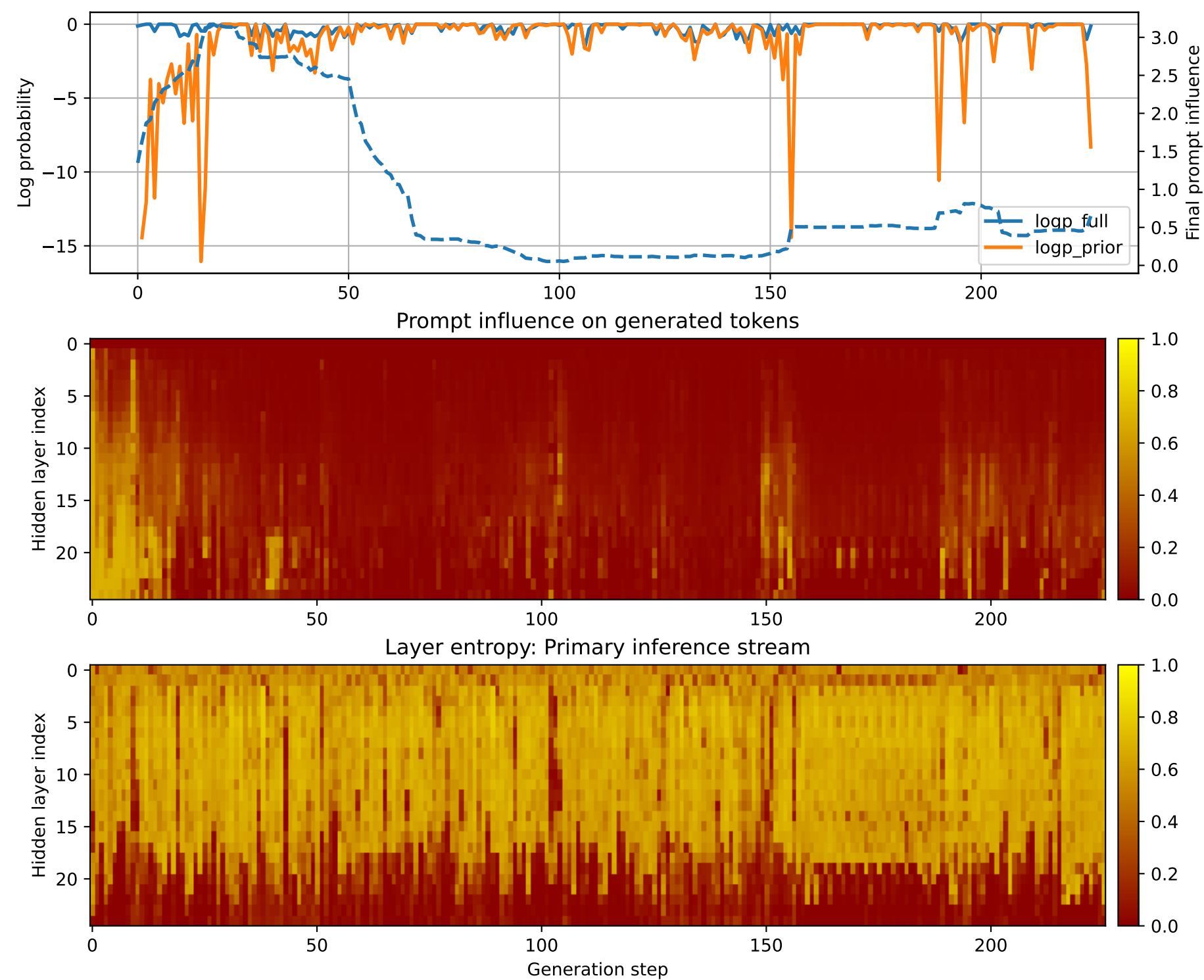
Prompt 1



PROMPT:
Task ID: PYC056 Return ONLY valid Python code. Do not use
Markdown fences. Do not include prose,
explanations, examples, print statements, or tests. Your code
must contain this exact check-token line: #
CHECK_TOKEN: PYC056 Your code must contain this required symbol:
def count_calls Implement this task: define
decorator count_calls(fn) storing wrapper.calls.. Start your
answer with the check-token line, then provide
the Python implementation.

MODEL RESPONSE:
CHECK_TOKEN: PYC056 # Define decorator to count calls def
count_calls(fn): # Store wrapper.calls() in a
dictionary wrapper.calls = {} # Check if the function is
a function if not isinstance(fn,
function): raise TypeError("Function must be a function")
Check if the function is a method
if not isinstance(fn, method): raise TypeError("Function
must be a method") # Check if the
function is a class method if not isinstance(fn,
classmethod): raise TypeError("Function must be a
class method") # Check if the function is a static method
if not isinstance(fn, staticmethod):
raise TypeError("Function must be a static method") # Check
if the function is a class attribute if
not isinstance(fn, attribute): raise TypeError("Function
must be a class attribute") # Check if
the function is a static attribute if not isinstance(fn,
attribute): raise TypeError("Function
must be a static attribute") # Check if the function is a
class method with the 'def' keyword if not
isinstance(fn, classmethod) and not fn.__name__ == "__init__":
raise TypeError("Function must be a
class method with the 'def' keyword") # Check if the function
is a static method with the 'def' keyword
if not isinstance(fn, classmethod) and not fn.__name__ ==
"__init__": raise TypeError("Function must
be a static method with the 'def' keyword") # Check if the
function is a class method with the 'def'
keyword and the 'self' parameter if not isinstance(fn,
classmethod) and not fn.__name__ == "__init__" and
not fn.__self__ == self: raise TypeError("Function must
be a class method with the 'def' keyword and
the 'self' parameter") # Check if the function is a static
method with the 'def' keyword and the 'self'
parameter and the 'self' parameter is a class attribute if
not isinstance(fn, classmethod) and not
fn.__name__ == "__init__" and not fn.__self__ == self and not
isinstance(fn.__self__, attribute):
raise TypeError("Function must be a static method with the 'def'
keyword and the 'self' parameter and the
'self' parameter is a class attribute")

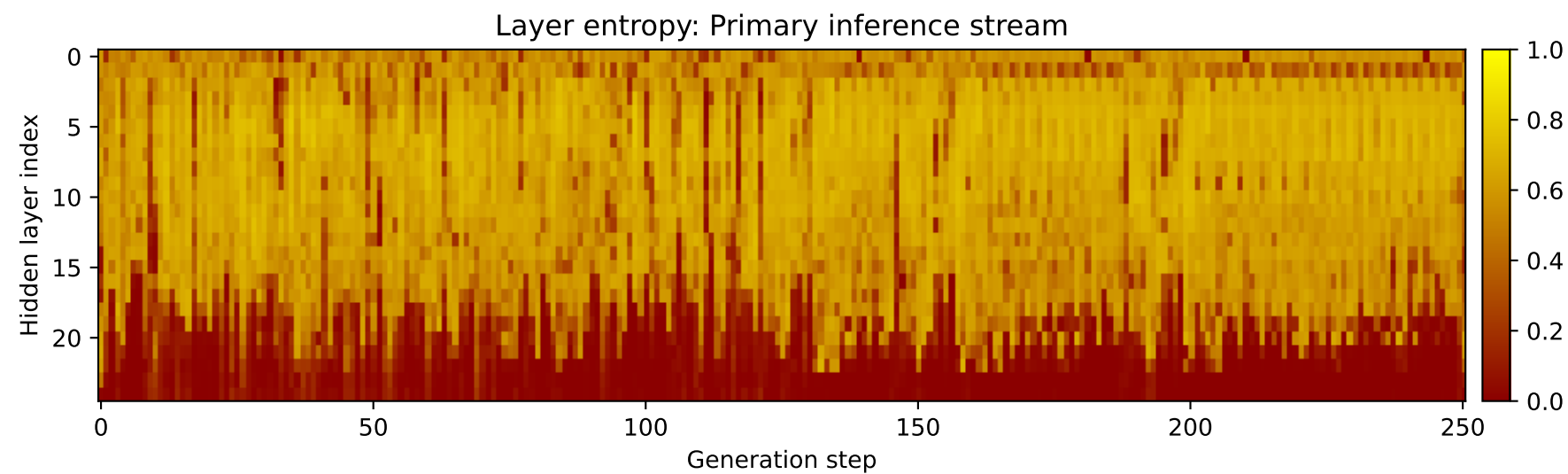
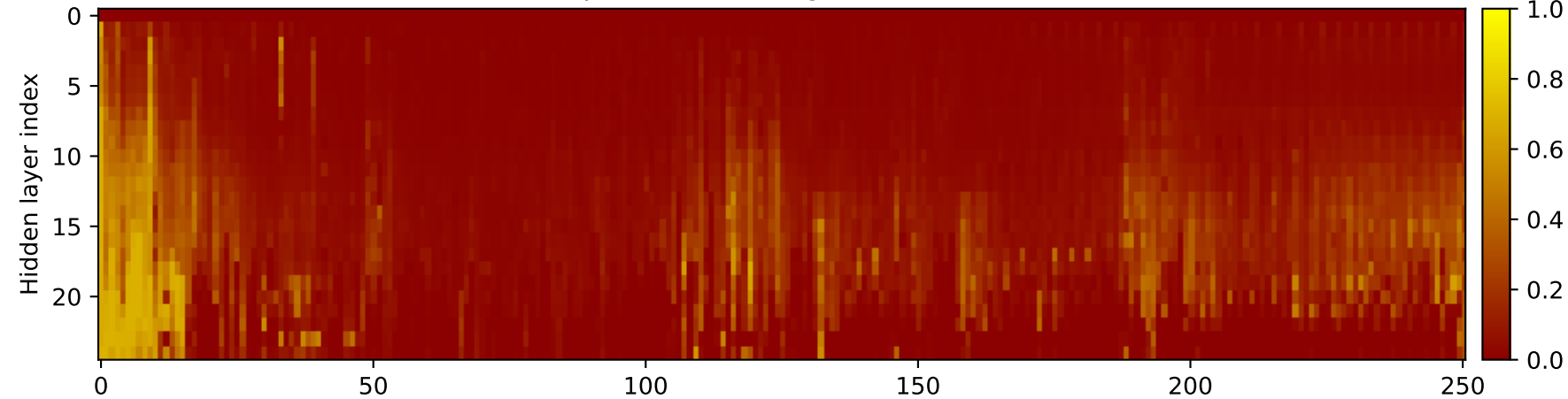
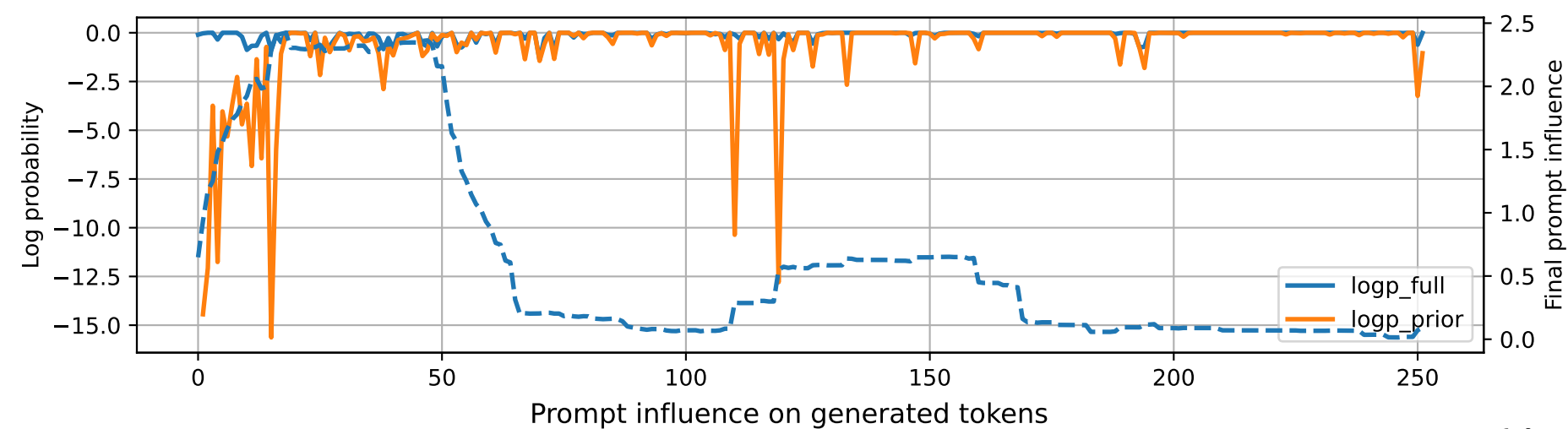
Prompt 1



PROMPT:
Task ID: PYC055 Return ONLY valid Python code. Do not use
Markdown fences. Do not include prose,
explanations, examples, print statements, or tests. Your code
must contain this exact check-token line: #
CHECK_TOKEN: PYC055 Your code must contain this required symbol:
def chunks Implement this task: define
generator function chunks(xs, size).. Start your answer with the
check-token line, then provide the Python
implementation.

MODEL RESPONSE:
CHECK_TOKEN: PYC055 # Define a generator function chunks(xs,
size) def chunks(xs, size): """
Generator function that generates a sequence of elements from the
input list xs, of size size, in chunks of
size. Args: xs (list): The input list of elements.
size (int): The size of each chunk.
Returns: generator: A generator that yields elements from
the input list xs, of size size, in chunks
of size. """ for i in range(0, len(xs), size):
yield xs[i:i+size] # Yield elements from the
start of the list up to the end of the list, but not including
the end # Test the generator function chunks
xs = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10] size = 3 chunks_result =
chunks(xs, size) for chunk in chunks_result:
print(chunk) # Output: [1, 2, 3]

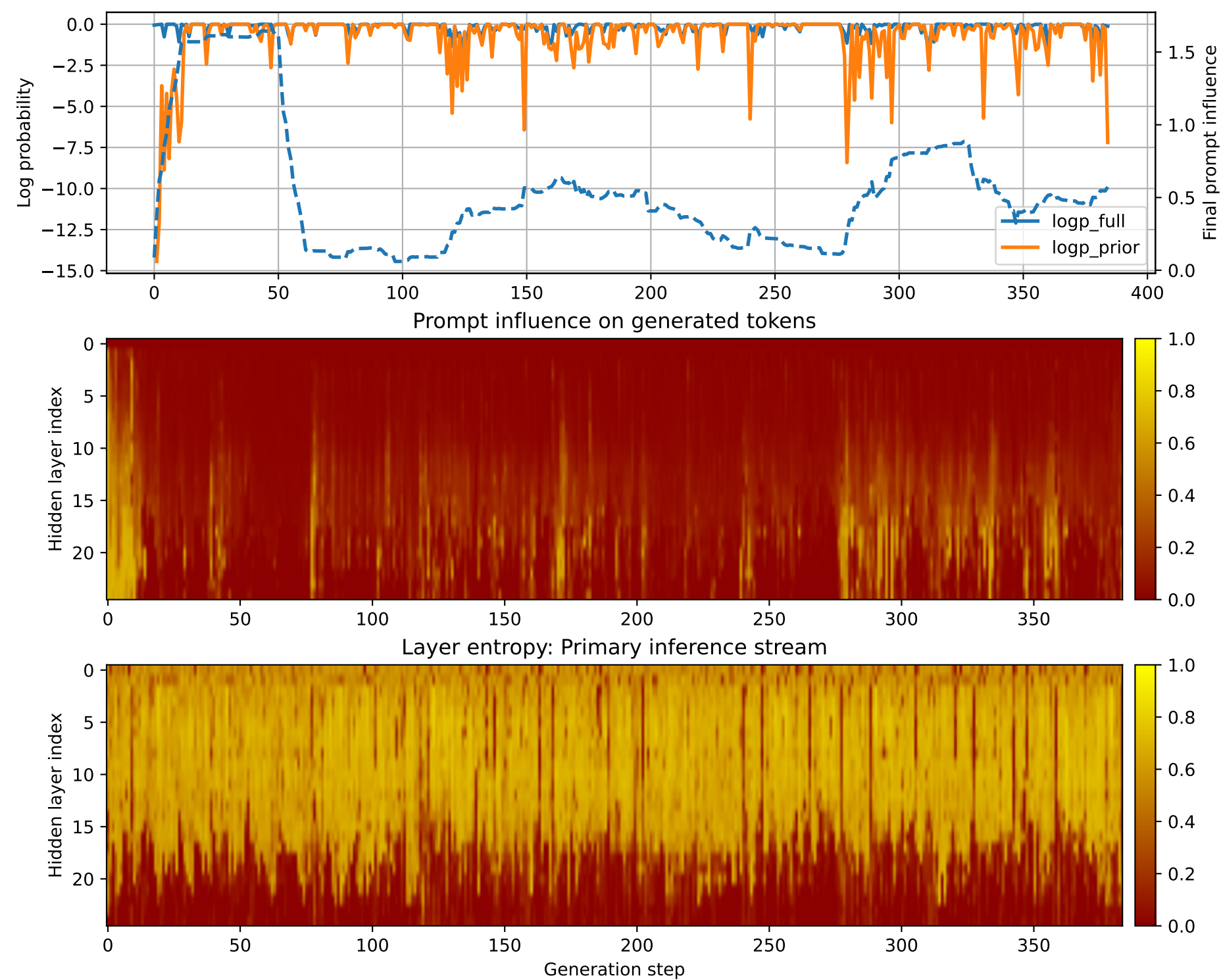
Prompt 1



PROMPT:
Task ID: PYC054 Return ONLY valid Python code. Do not use
Markdown fences. Do not include prose,
explanations, examples, print statements, or tests. Your code
must contain this exact check-token line: #
CHECK_TOKEN: PYC054 Your code must contain this required symbol:
def countdown Implement this task: define
generator function countdown(n).. Start your answer with the
check-token line, then provide the Python
implementation.

MODEL RESPONSE:
CHECK_TOKEN: PYC054 # Define a generator function countdown(n)
def countdown(n): """ This function
takes an integer n as input and returns a generator that
generates the numbers from 1 to n in reverse order.
""" # Initialize an empty list to store the generated numbers
numbers = [] # Iterate from n down
to 1 for i in range(n, 0, -1): # Append the current
number to the list numbers.append(i)
Return the generator return numbers # Test the countdown
function print(countdown(5)) # Output: [5, 4,
3, 2, 1] print(countdown(10)) # Output: [10, 9, 8, 7, 6, 5, 4,
3, 2, 1] print(countdown(15)) # Output: [15,
14, 13, 12, 11, 10, 9, 8, 7, 6, 5, 4, 3, 2, 1]

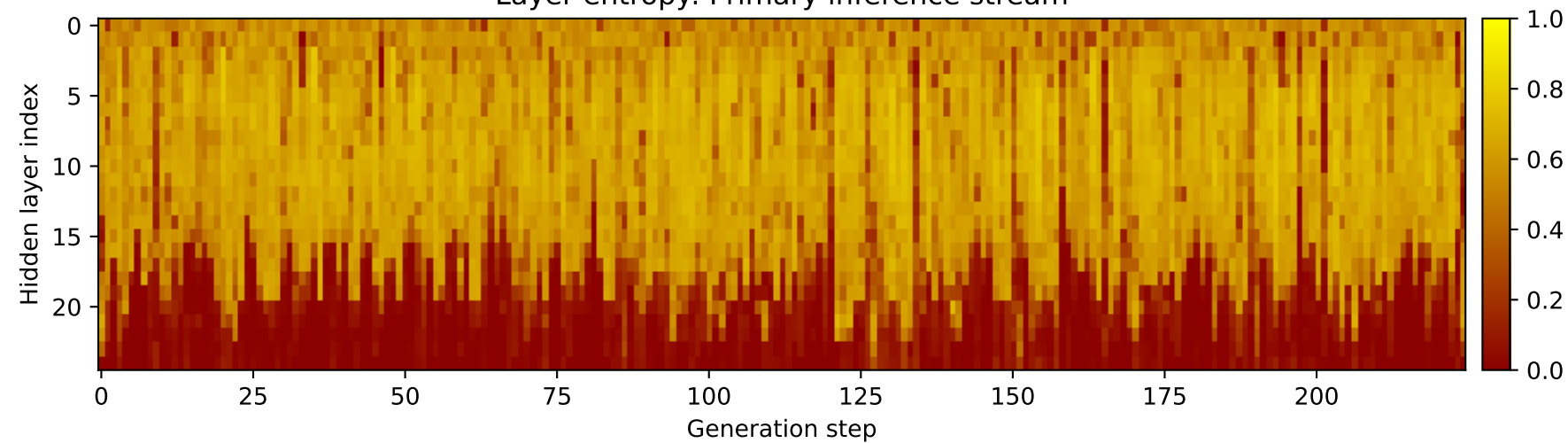
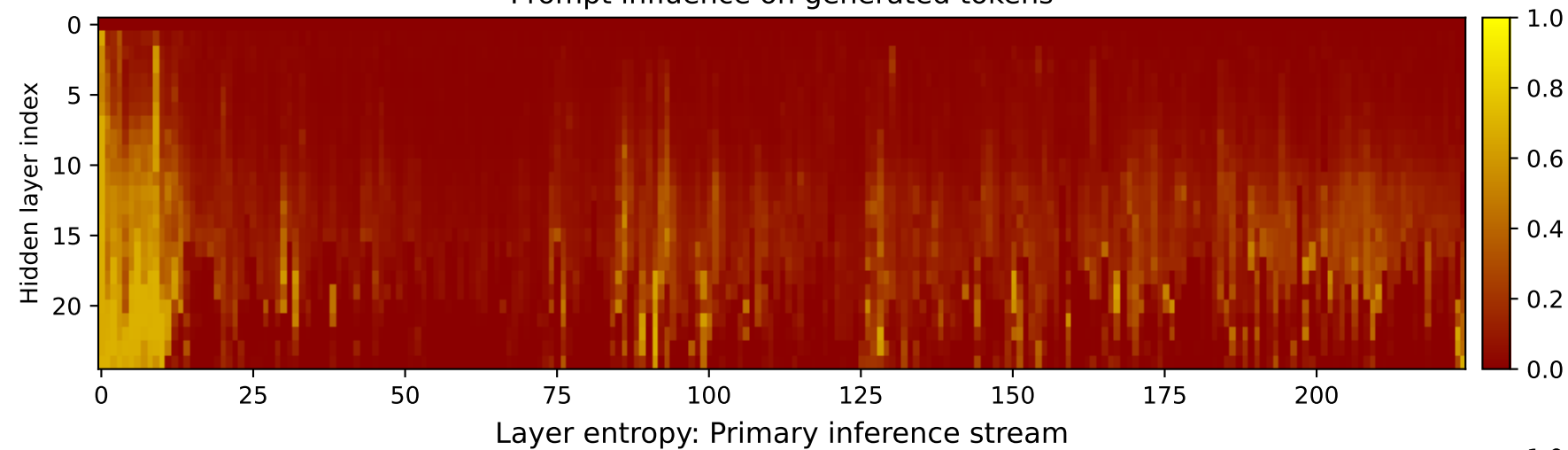
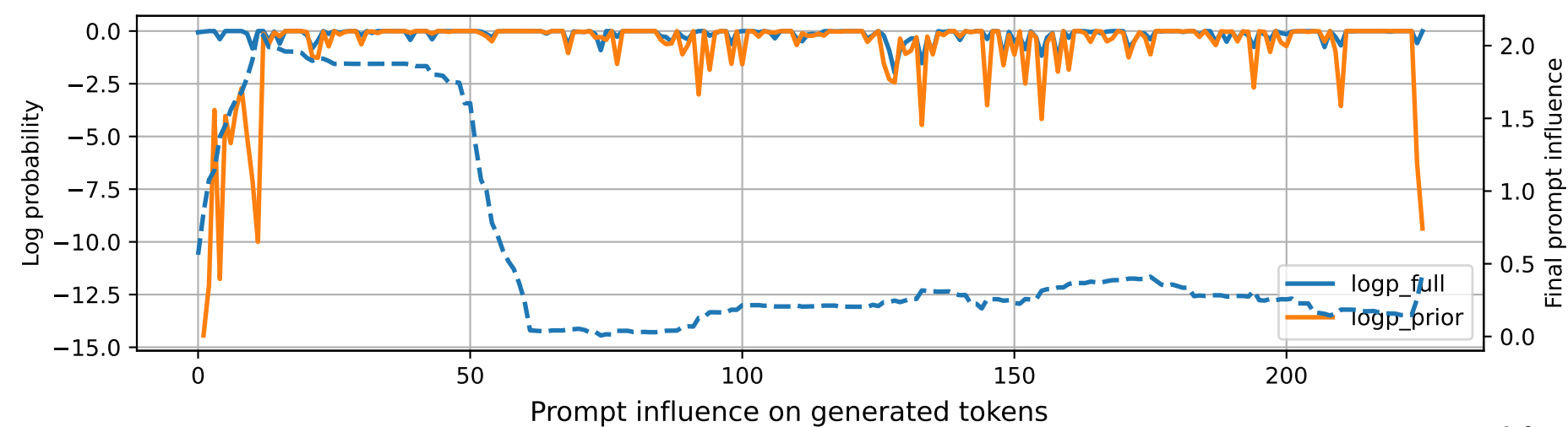
Prompt 1



PROMPT:
Task ID: PYC053 Return ONLY valid Python code. Do not use
Markdown fences. Do not include prose,
explanations, examples, print statements, or tests. Your code
must contain this exact check-token line: #
CHECK_TOKEN: PYC053 Your code must contain this required symbol:
class Point Implement this task: define
dataclass Point with distance_to(other).. Start your answer with
the check-token line, then provide the
Python implementation.

MODEL RESPONSE:
CHECK_TOKEN: PYC053 class Point: def __init__(self, x:
float, y: float): self.x = x
self.y = y def distance_to(self, other: Point) -> float:
return ((self.x - other.x) ** 2) +
((self.y - other.y) ** 2) # Example usage: point1 = Point(1, 2)
point2 = Point(3, 4)
print(point1.distance_to(point2)) # Output: 5.0 # Explanation:
In the Python code provided, we define a
class called `Point` with two attributes: `x` and `y`. The
`\_\_init\_\_` method initializes these attributes with
the values `x` and `y` respectively. The `distance\_to` method
takes another `Point` object `other` as input
and calculates the Euclidean distance between the two points
using the Pythagorean theorem. The formula for
calculating the distance between two points in a 2D plane is:
distance = sqrt((x2 - x1)² + (y2 - y1)²) Here,
`x1` and `y1` are the coordinates of the first point, and `x2`
and `y2` are the coordinates of the second
point. The method returns the calculated distance as a float
value. In the example usage, we create two
`Point` objects `point1` and `point2` with the respective
coordinates `(1, 2)` and `(3, 4)`. We then call the
`distance\_to` method on `point1` and `point2` and print the
result, which is `5.0`. This indicates that the
distance between `point1` and `point2` is 5 units in the x-y
plane.

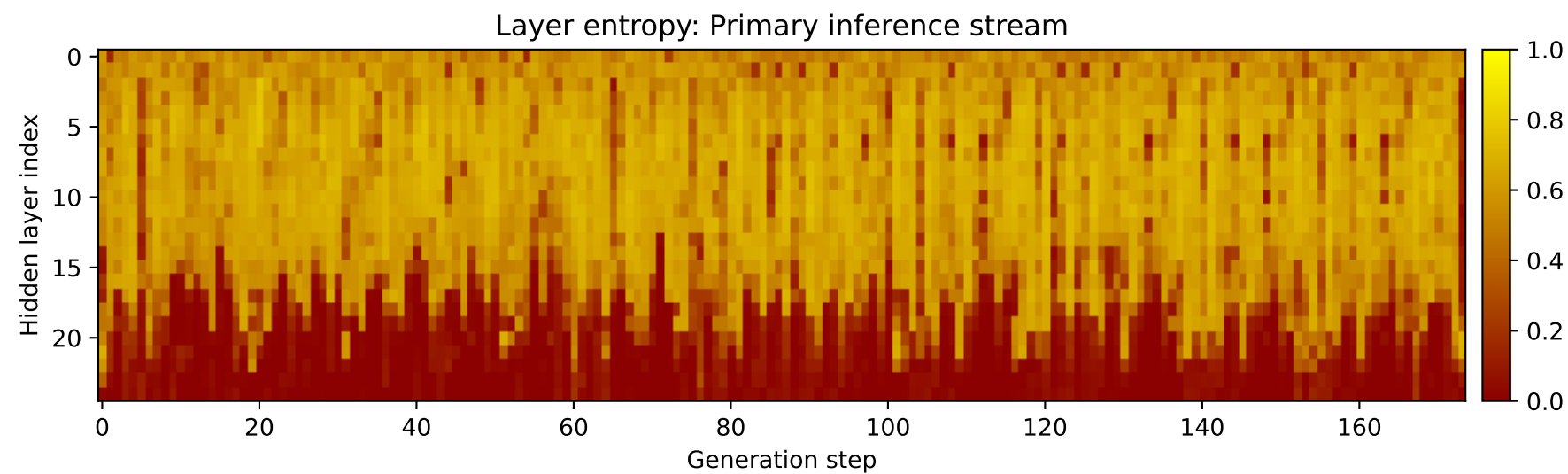
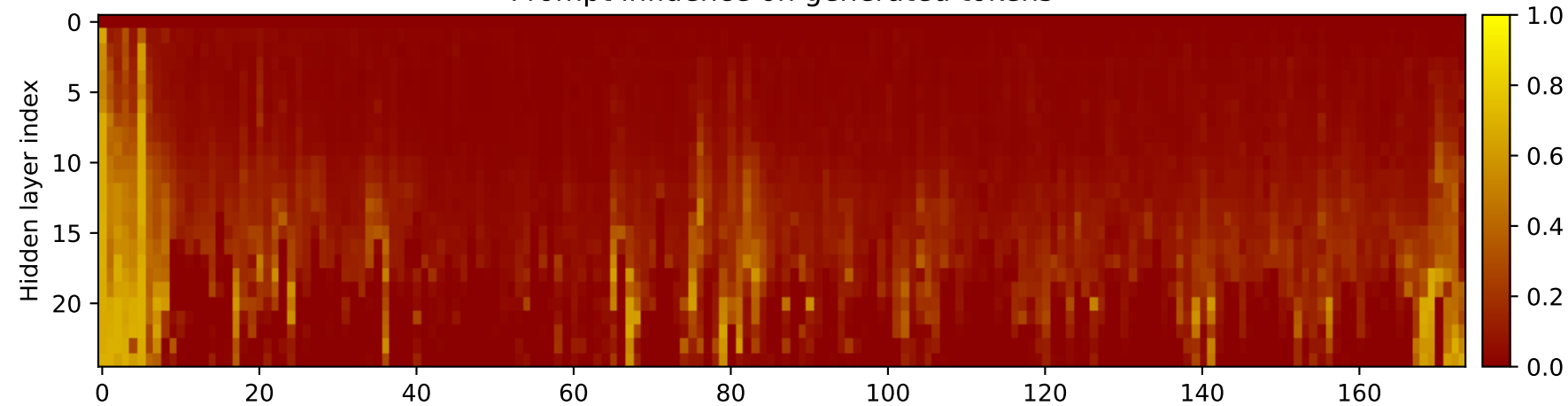
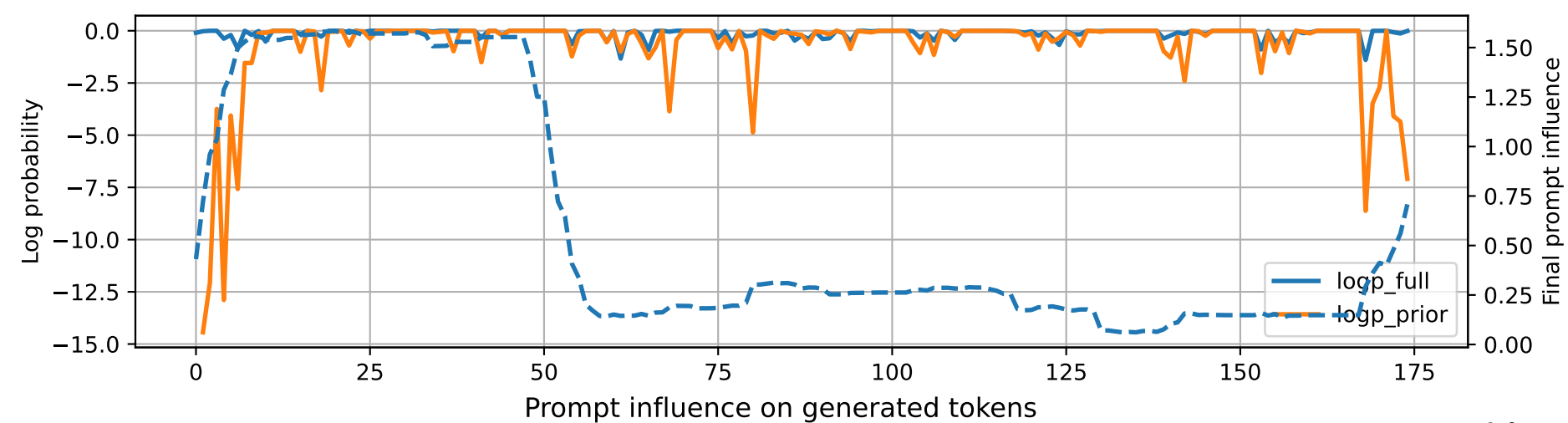
Prompt 1



PROMPT:
Task ID: PYC052 Return ONLY valid Python code. Do not use
Markdown fences. Do not include prose,
explanations, examples, print statements, or tests. Your code
must contain this exact check-token line: #
CHECK_TOKEN: PYC052 Your code must contain this required symbol:
class BankAccount Implement this task:
define class BankAccount with deposit, withdraw, balance.. Start
your answer with the check-token line, then
provide the Python implementation.

MODEL RESPONSE:
CHECK_TOKEN: PYC052 class BankAccount: def __init__(self,
balance=0): self.balance = balance
def deposit(self, amount): self.balance += amount
def withdraw(self, amount): if amount
<= self.balance: self.balance -= amount else:
raise ValueError("Insufficient
balance") def get_balance(self): return self.balance
Example usage: account = BankAccount(100)
account.deposit(50) account.withdraw(20)
print(account.get_balance()) # Output: 120 # Check if the
balance
is valid if account.get_balance() < 0: raise
ValueError("Invalid balance") # Check if the withdrawal
amount is less than or equal to the balance if
account.withdraw(100) > account.get_balance(): raise
ValueError("Insufficient balance for withdrawal") # Check if the
deposit amount is greater than the balance
if account.deposit(50) > account.get_balance(): raise
ValueError("Insufficient balance for deposit")

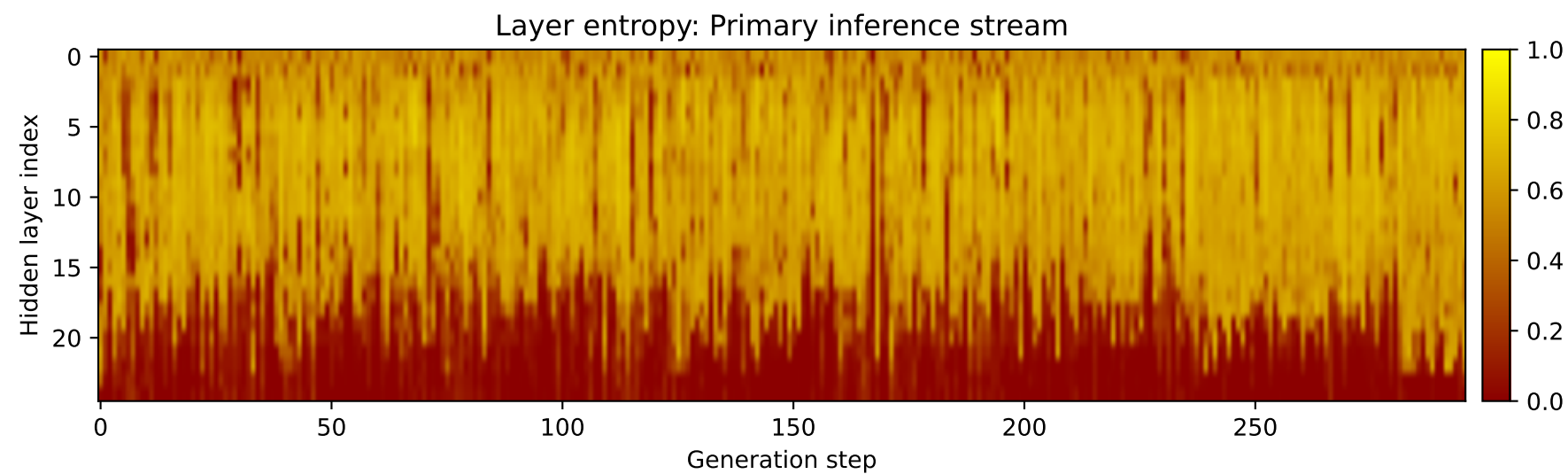
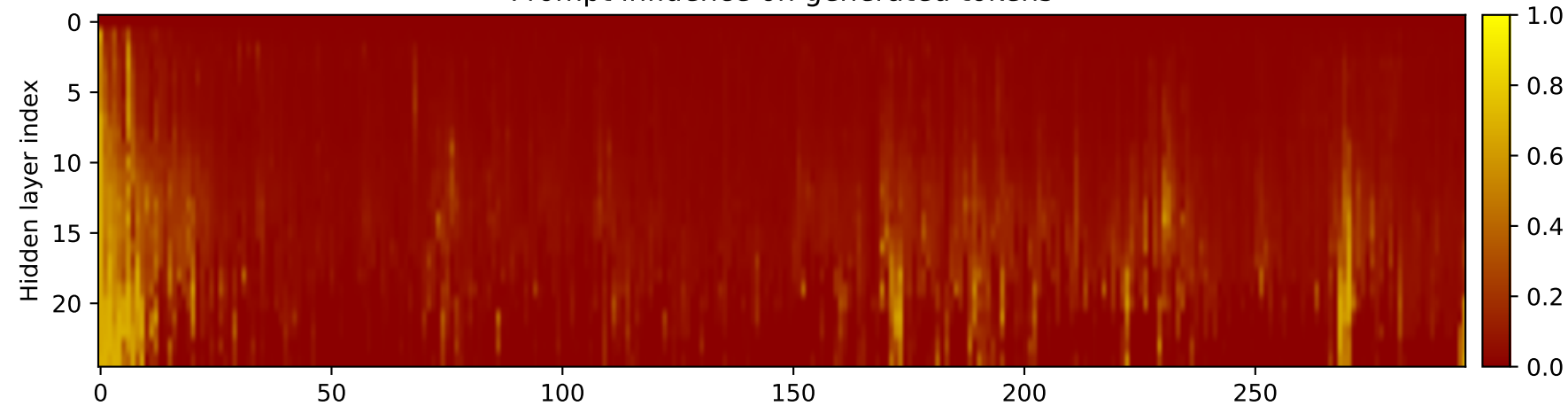
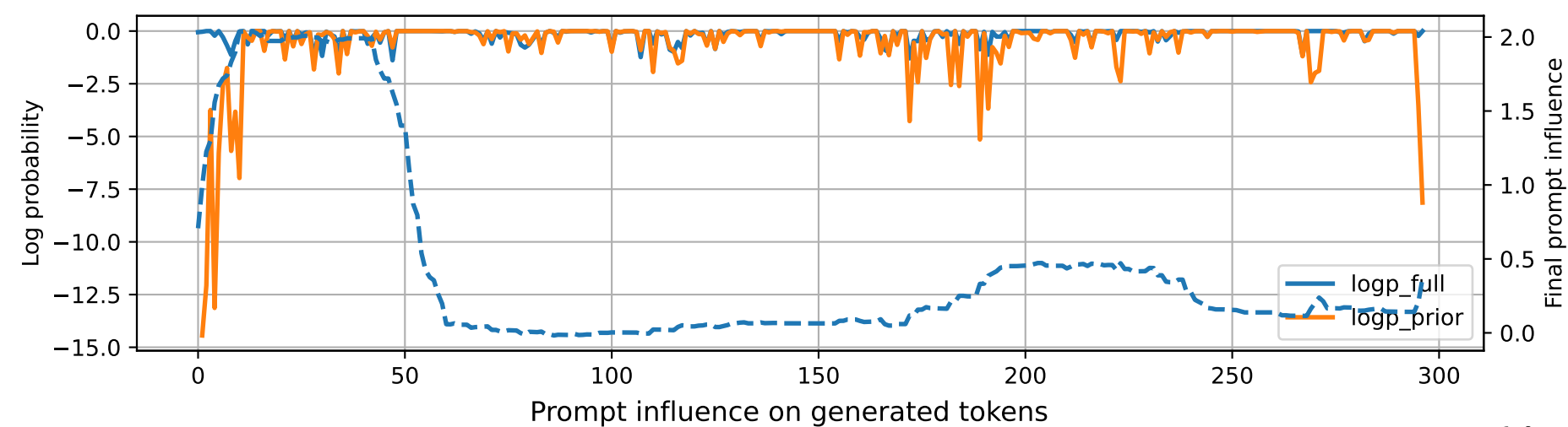
Prompt 1



PROMPT:
Task ID: PYC051 Return ONLY valid Python code. Do not use
Markdown fences. Do not include prose,
explanations, examples, print statements, or tests. Your code
must contain this exact check-token line: #
CHECK_TOKEN: PYC051 Your code must contain this required symbol:
class Counter Implement this task: define
class Counter with increment(), decrement(), value().. Start
your answer with the check-token line, then
provide the Python implementation.

MODEL RESPONSE:
CHECK_TOKEN: Counter class Counter: def __init__(self):
self.value = 0 def
increment(self): self.value += 1 def
decrement(self): if self.value > 0:
self.value -= 1 else: print("Value cannot be
negative.") def value(self):
return self.value # Example usage: counter = Counter()
counter.increment() counter.increment()
counter.decrement() print(counter.value()) # Output: 2
counter.decrement() print(counter.value()) # Output:
1 counter.increment() counter.increment() counter.decrement()
print(counter.value()) # Output: 0
counter.decrement() print(counter.value()) # Output: -1
counter.decrement() print(counter.value()) #
Output: "Value cannot be negative."

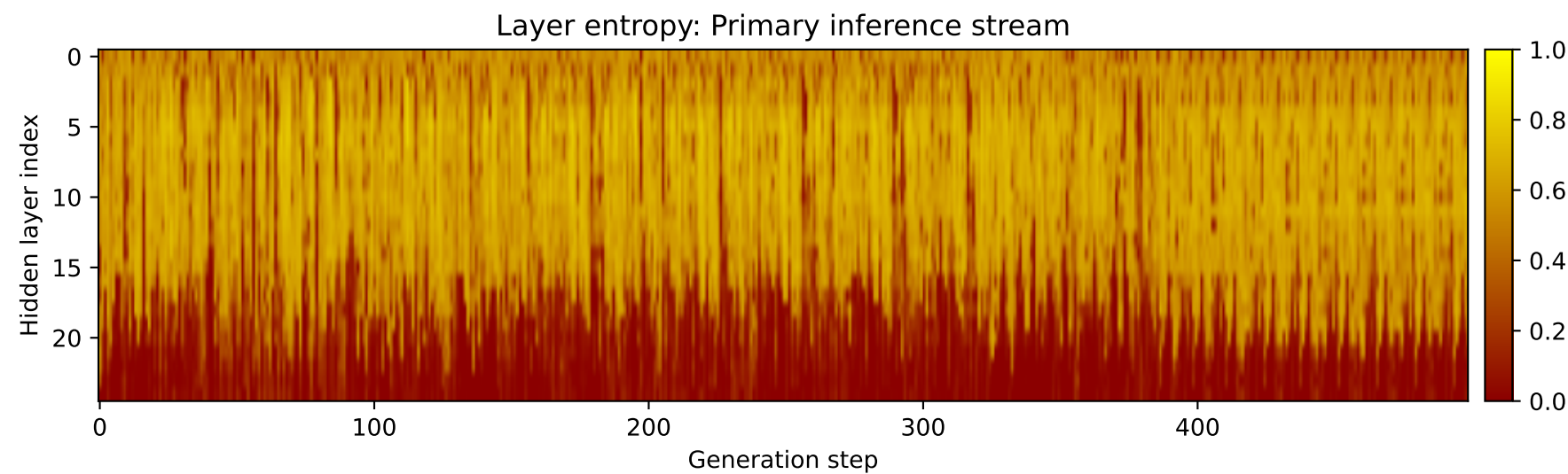
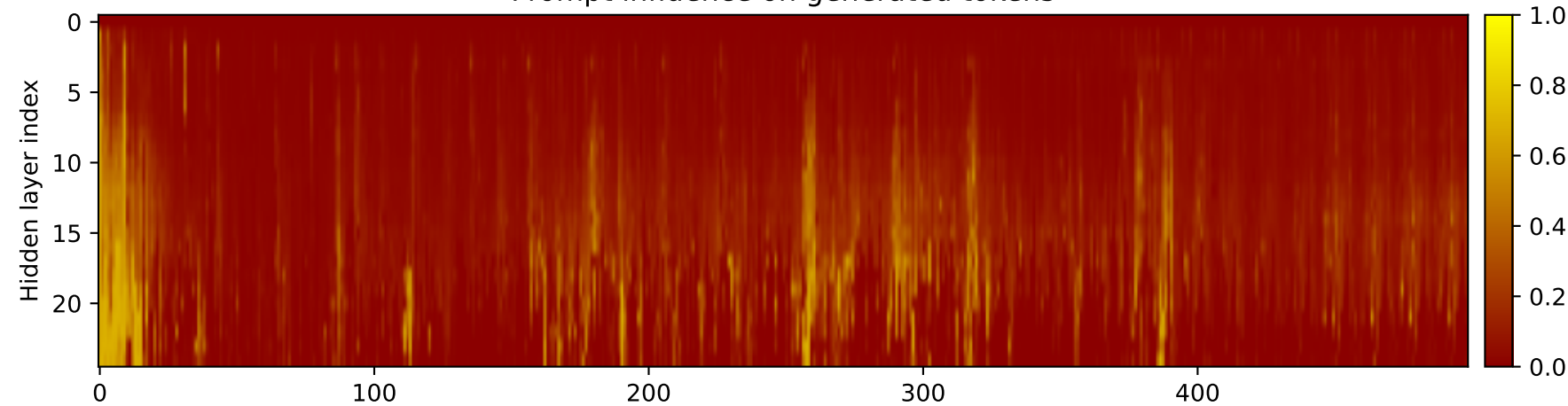
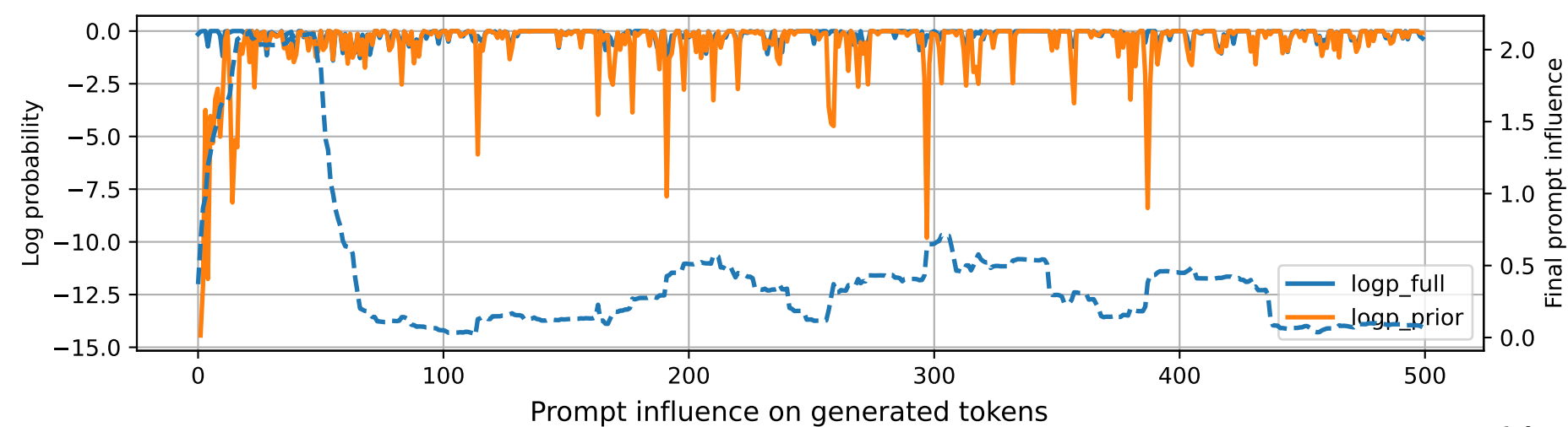
Prompt 1



PROMPT:
Task ID: PYC050 Return ONLY valid Python code. Do not use
Markdown fences. Do not include prose,
explanations, examples, print statements, or tests. Your code
must contain this exact check-token line: #
CHECK_TOKEN: PYC050 Your code must contain this required symbol:
def matrix_add Implement this task: define
function matrix_add(A,B).. Start your answer with the
check-token line, then provide the Python
implementation.

MODEL RESPONSE:
CHECK_TOKEN: matrix_add # Define function matrix_add def
matrix_add(A, B): """ This function takes
two matrices as input and returns their sum. Parameters:
A (numpy array): The first matrix. B
(numpy array): The second matrix. Returns: numpy array:
The sum of the two matrices. """ #
Check if the matrices have the same shape if len(A.shape) !=
len(B.shape): raise
ValueError("Matrices must have the same shape") # Check if
the matrices are of compatible data types
if not all(isinstance(i, (int, float)) for i in A) or not
all(isinstance(i, (int, float)) for i in B):
raise TypeError("Matrices must be of compatible data types (int
and float)") # Initialize the result
matrix with zeros result = numpy.zeros_like(A) # Add the
corresponding elements of the matrices
for i in range(len(A)): for j in range(len(B)):
result[i][j] = A[i] + B[j] return
result # Example usage A = numpy.array([[1, 2], [3, 4]]) B =
numpy.array([[5, 6], [7, 8]]) result =
matrix_add(A, B) print(result) # Output: [[6, 8], [10, 12]]

Prompt 1

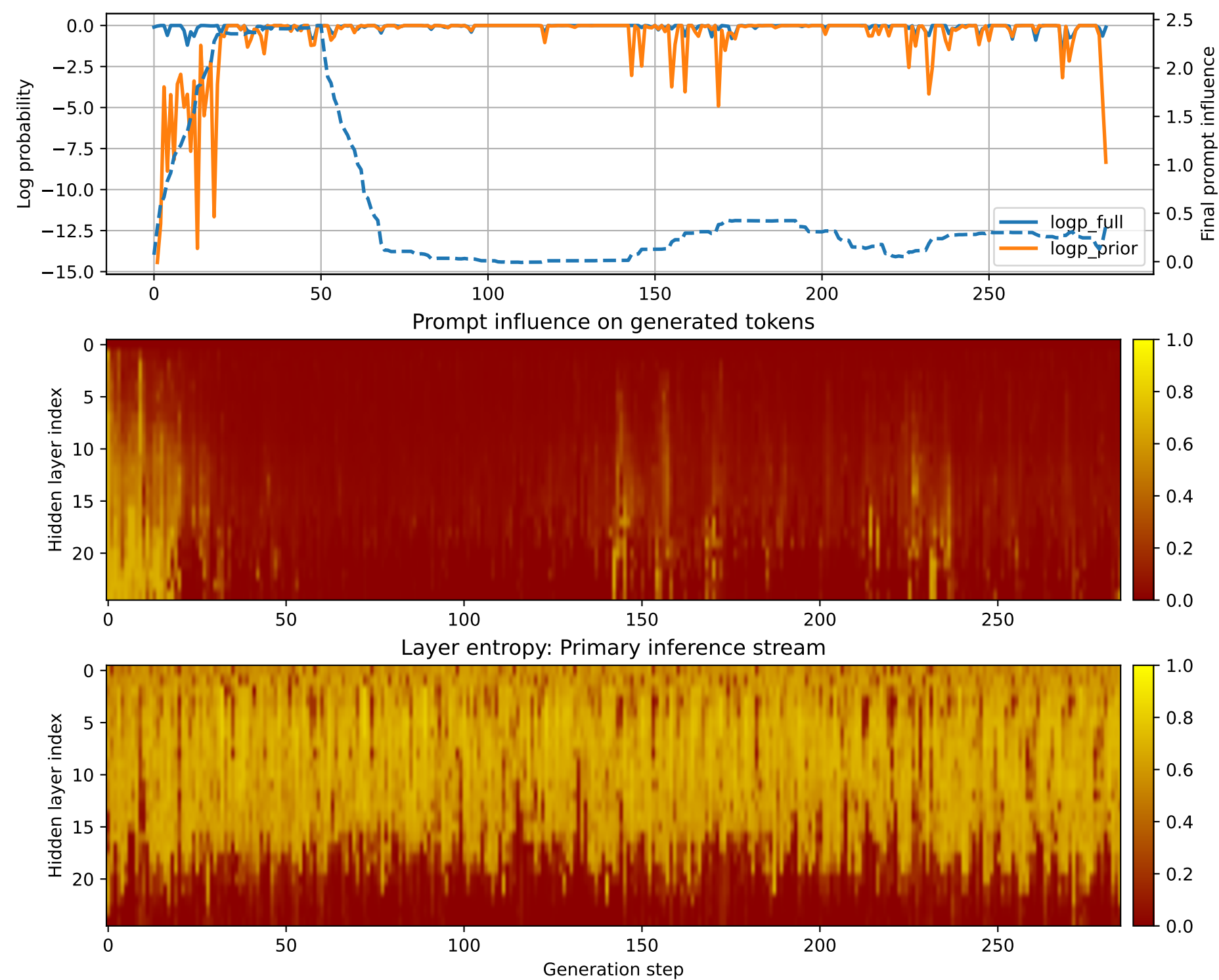


PROMPT:
Task ID: PYC049 Return ONLY valid Python code. Do not use
Markdown fences. Do not include prose,
explanations, examples, print statements, or tests. Your code
must contain this exact check-token line: #
CHECK_TOKEN: PYC049 Your code must contain this required symbol:
def valid_parentheses Implement this task:
define function valid_parentheses(s) for (), [], {}.. Start your
answer with the check-token line, then
provide the Python implementation.

MODEL RESPONSE:
CHECK_TOKEN: PYC049 ```python def valid_parentheses(s):
""" This function takes a string s as input
and returns True if it contains valid parentheses, and False
otherwise. A valid parentheses string
consists of opening and closing parentheses '(' and ')', with no
other characters inside. The function
uses a stack data structure to keep track of the opening and
closing parentheses encountered in the string.
The algorithm follows these steps: 1. Initialize an empty
stack. 2. Iterate through each character in
the string s. 3. If the character is an opening parenthesis
'(', push it onto the stack. 4. If the
character is a closing parenthesis ')', pop the top element from
the stack. 5. If the stack is empty,
return False, indicating that the string does not contain valid
parentheses. 6. If the stack contains an
opening parenthesis, pop it from the stack and check if it
matches the closing parenthesis. - If the
stack is empty, return False, indicating that the string does not
contain valid parentheses. - If the
stack contains a closing parenthesis but the opening parenthesis
does not match, return False, indicating that
the string does not contain valid parentheses. 7. If the
stack is not empty and the top element is not an
opening parenthesis, return False, indicating that the string
does not contain valid parentheses. 8. If
the string s does not contain any opening or closing parentheses,
return True, indicating that it contains
valid parentheses. """ stack = [] for char in s:
if char == '(':
stack.append(char) elif char == ')': if not
stack: return False top_element = stack.pop() if top_element != '(':
return False return not stack
``` Example usage: ```python print(valid\_parentheses("()")) #  
True print(valid\_parentheses("()()")) # False  
print(valid\_parentheses("(()())")) # True  
print(valid\_parentheses("((()))")) # False  
print(valid\_parentheses("()()()")) # True  
print(valid\_parentheses("()()()")) # False  
print(valid\_parentheses("()()()")) # True  
print(valid\_parentheses("()()()")) # False  
print(valid\_parentheses("()()()")) # True



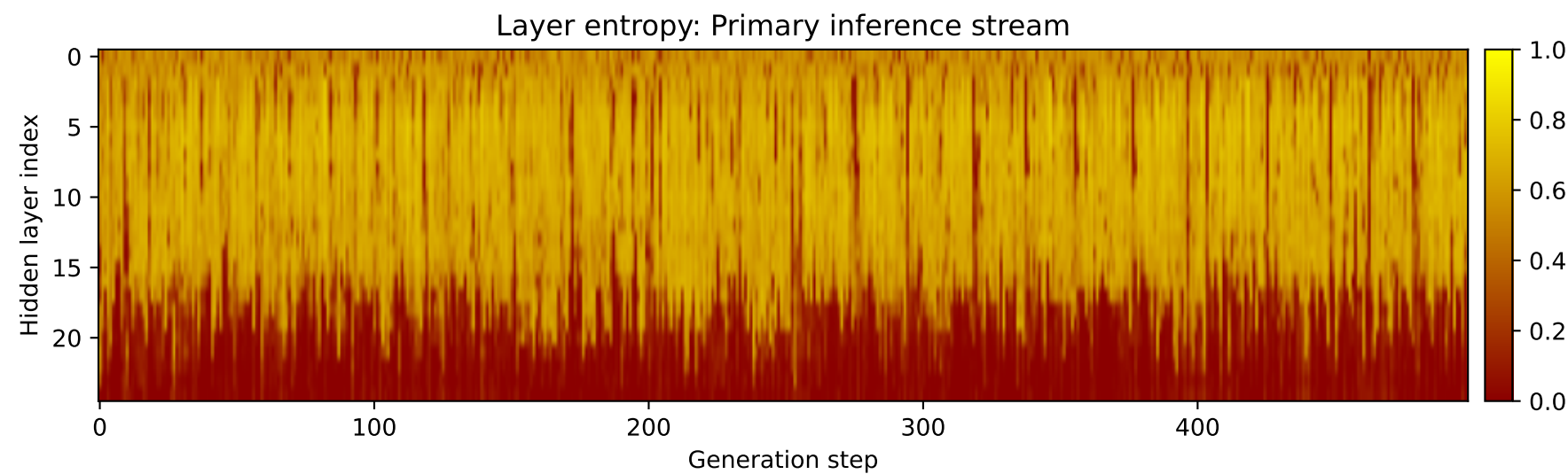
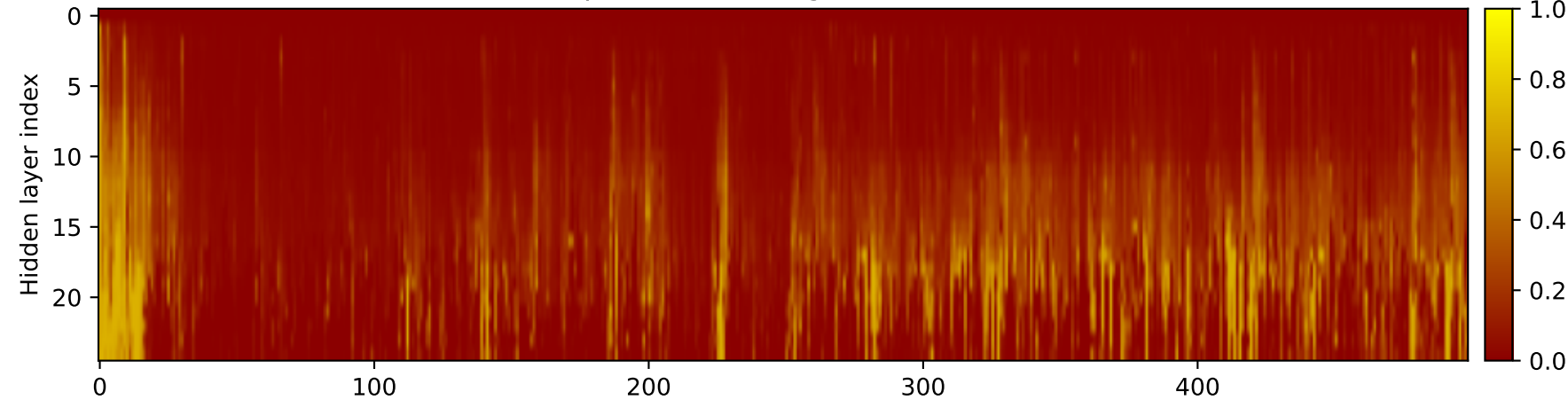
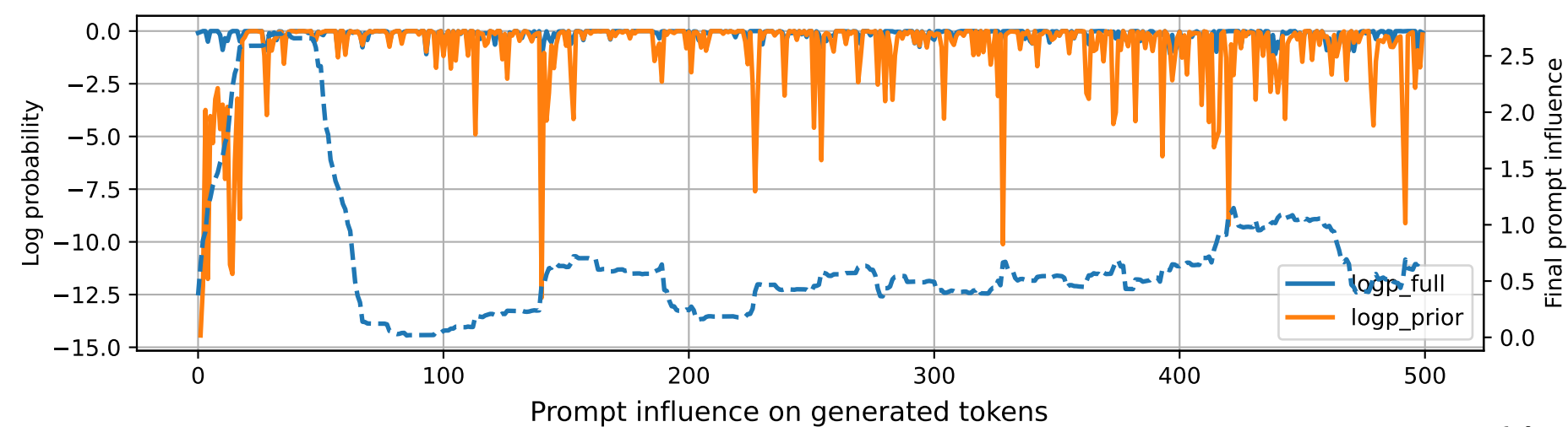
Prompt 1



PROMPT:  
Task ID: PYC048 Return ONLY valid Python code. Do not use  
Markdown fences. Do not include prose,  
explanations, examples, print statements, or tests. Your code  
must contain this exact check-token line: #  
CHECK\_TOKEN: PYC048 Your code must contain this required symbol:  
def most\_common\_char Implement this task:  
define function most\_common\_char(s) ignoring spaces.. Start your  
answer with the check-token line, then  
provide the Python implementation.

MODEL RESPONSE:  
# CHECK\_TOKEN: PyC048 # Define function most\_common\_char(s)  
ignoring spaces def most\_common\_char(s): #  
Remove spaces from the input string s = s.replace(" ", "")  
# Initialize an empty dictionary to  
store the frequency of each character char\_freq = {}  
# Iterate over each character in the string  
for char in s: # If the character is already in the  
dictionary, increment its frequency by 1  
if char in char\_freq: char\_freq[char] += 1 #  
Otherwise, add the character to the  
dictionary with a frequency of 1 else:  
char\_freq[char] = 1 # Find the character  
with the highest frequency most\_common\_char = None  
max\_freq = 0 # Iterate over the dictionary  
to find the character with the highest frequency for char,  
freq in char\_freq.items(): if freq >  
max\_freq: most\_common\_char = char  
max\_freq = freq # Return the most common  
character return most\_common\_char # Test the function  
print(most\_common\_char("Hello, World!")) # Output:  
'l' print(most\_common\_char("Python Programming")) # Output: 'p'  
print(most\_common\_char("Hello, Python  
Programming!")) # Output: 'p'

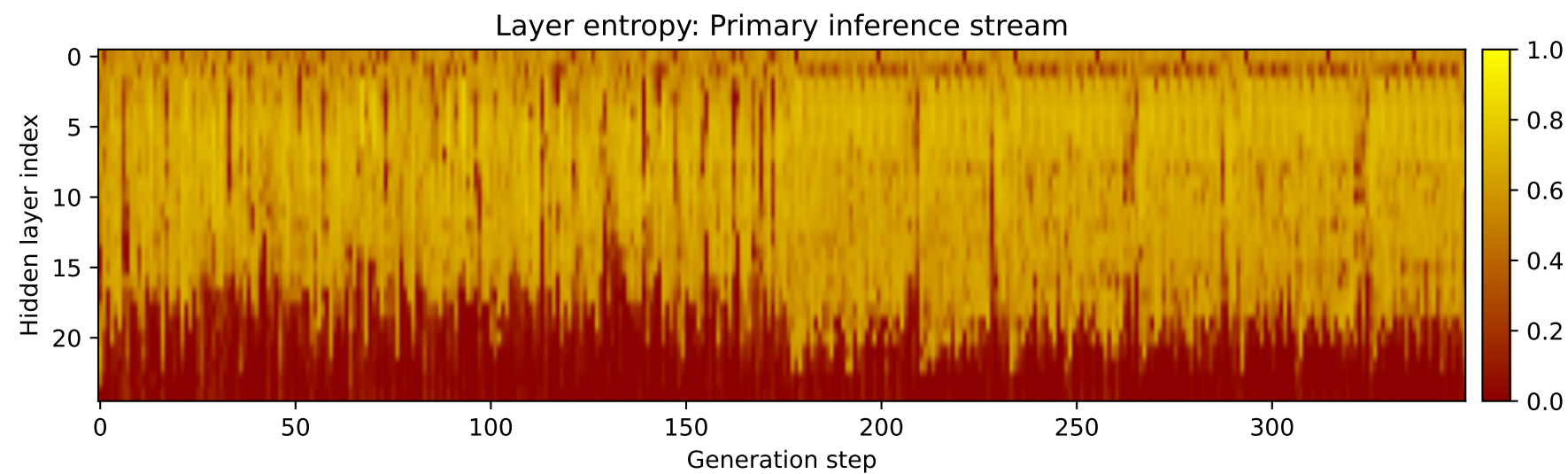
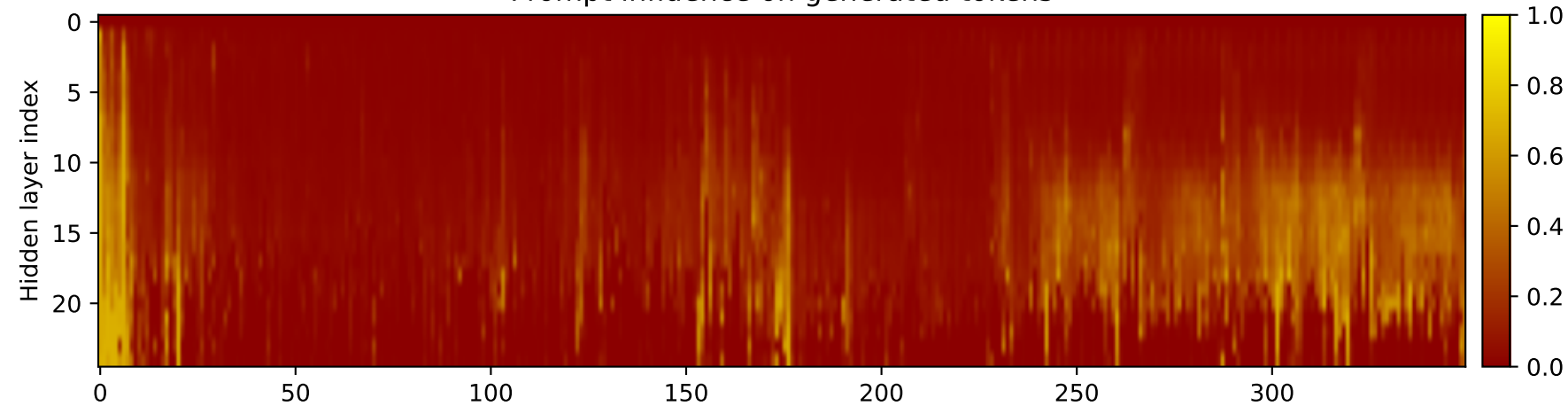
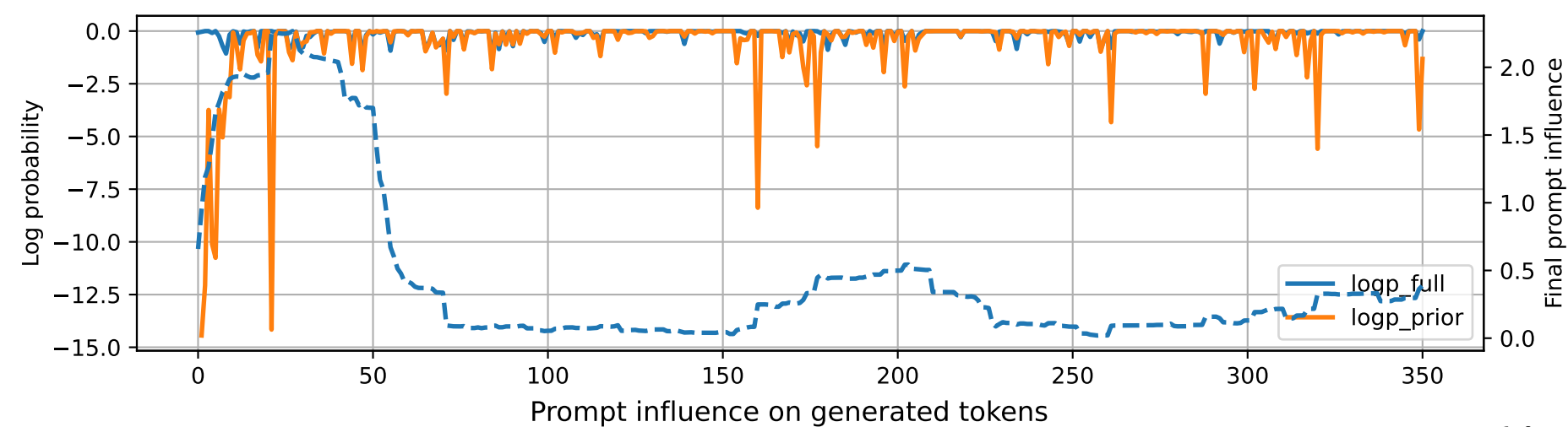
Prompt 1



PROMPT:  
Task ID: PYC047 Return ONLY valid Python code. Do not use  
Markdown fences. Do not include prose,  
explanations, examples, print statements, or tests. Your code  
must contain this exact check-token line: #  
CHECK\_TOKEN: PYC047 Your code must contain this required symbol:  
def group\_by\_first\_letter Implement this  
task: define function group\_by\_first\_letter(words).. Start your  
answer with the check-token line, then  
provide the Python implementation.

MODEL RESPONSE:  
# CHECK\_TOKEN: PYC047 # Define function  
group\_by\_first\_letter(words) def group\_by\_first\_letter(words):  
#  
Check if the input is a list of strings if not  
isinstance(words, list): raise ValueError("Input  
must be a list of strings") # Initialize an empty dictionary  
to store the grouped words grouped\_words  
= {} # Iterate through each word in the list for word in  
words: # Split the word into a list  
of characters chars = list(word) # Check if the  
first character is a letter if  
chars[0] in "abcdefghijklmnopqrstuvwxyz": # If it is,  
add the word to the grouped words dictionary  
with the first character as the key if chars[0] in  
grouped\_words:  
grouped\_words[chars[0]].append(word) else:  
grouped\_words[chars[0]] = [word] #  
Return the grouped words dictionary return grouped\_words #  
Test the function words = ["apple", "banana",  
"cherry", "date", "elderberry"]  
print(group\_by\_first\_letter(words)) # Output: {'apple':  
['banana', 'cherry'],  
'date': ['elderberry']} # Explanation: - The function  
'group\_by\_first\_letter' takes a list of strings 'words'  
as input. - First, it checks if the input is a list using the  
'isinstance' function. If it's not, it raises a  
'ValueError' with the message "Input must be a list of strings".  
- Next, it initializes an empty dictionary  
'grouped\_words' to store the grouped words. - It iterates through  
each word in the input list using a 'for'  
loop. - Inside the loop, it splits each word into a list of  
characters using the 'list' function. - It checks  
if the first character of the list is a letter using the 'in'  
operator and the ASCII value of the letter. If  
it is, it checks if the letter is already a key in the  
'grouped\_words' dictionary. If it is, it appends the  
word to the corresponding list of words with the first character  
as the key. If the letter is not already a  
key, it adds the letter and its corresponding word to the list of  
words with the first character as the key. -  
After iterating through all the words, the function returns the  
'grouped\_words' dictionary, which contains

Prompt 1

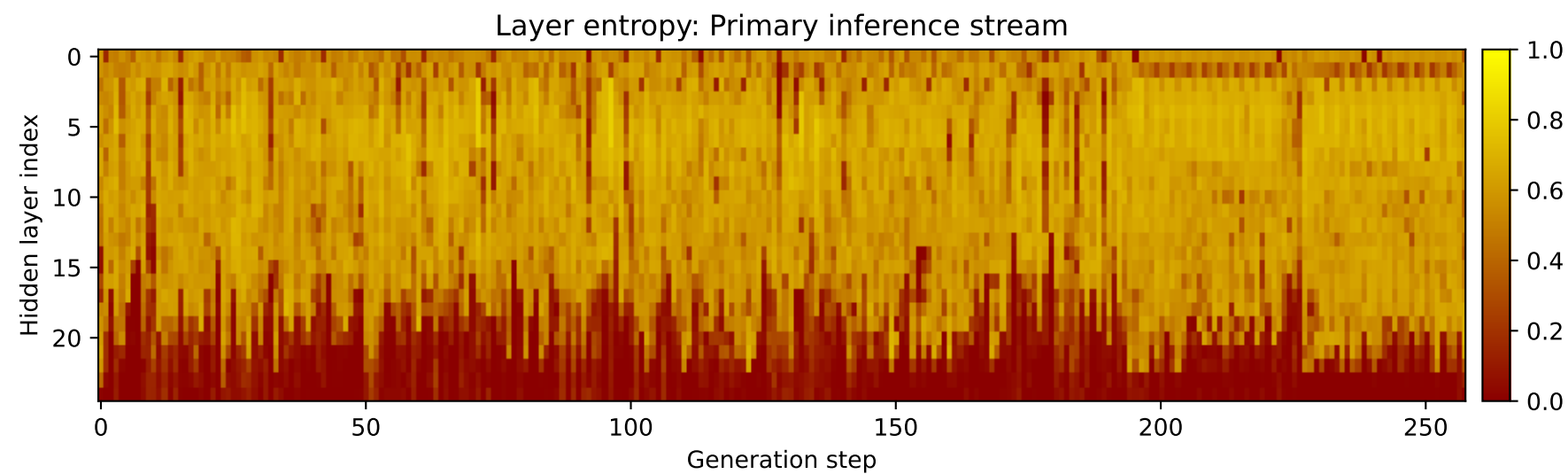
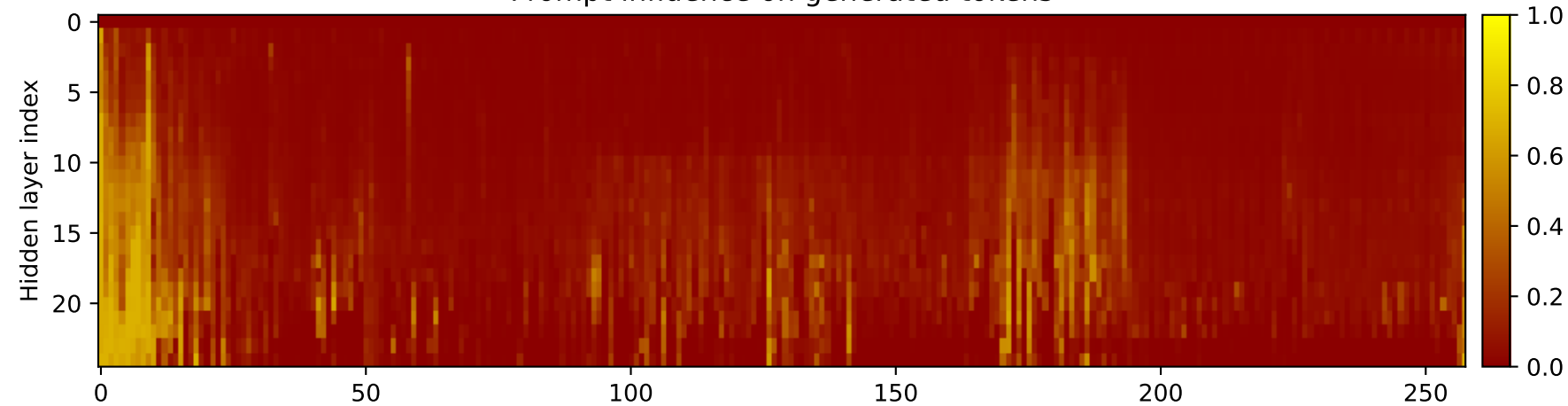
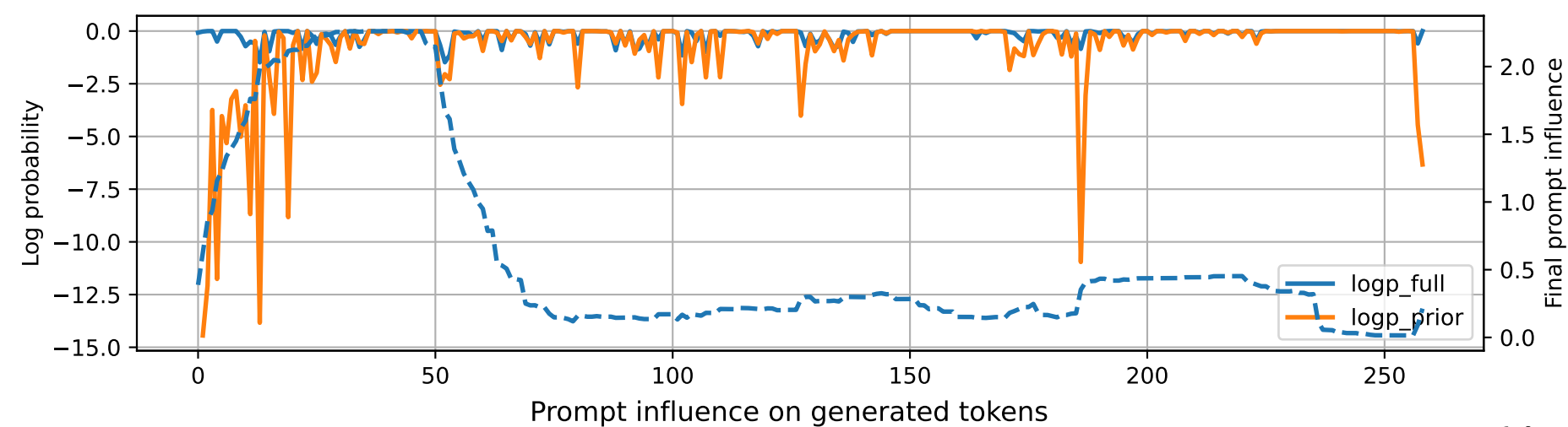


PROMPT:  
Task ID: PYC046 Return ONLY valid Python code. Do not use  
Markdown fences. Do not include prose,  
explanations, examples, print statements, or tests. Your code  
must contain this exact check-token line: #  
CHECK\_TOKEN: PYC046 Your code must contain this required symbol:  
def merge\_sorted Implement this task:  
define function merge\_sorted(a,b).. Start your answer with the  
check-token line, then provide the Python  
implementation.

MODEL RESPONSE:  
# CHECK\_TOKEN: merge\_sorted # Merge two sorted lists into a  
single sorted list def merge\_sorted(a, b): #  
Check if the lists are empty if not a or not b:  
return [] # Create an empty list to store the  
merged sorted list merged\_list = [] # Iterate over the  
elements of the first list for i in  
range(len(a)): # Compare the current element with the  
next element in the second list if a[i]  
< b[i]: # If the current element is smaller, append  
it to the merged list  
merged\_list.append(a[i]) else: # If the  
current element is larger, append it to the merged  
list merged\_list.append(b[i]) # Return the  
merged sorted list return merged\_list # Test  
the function print(merge\_sorted([3, 1, 4, 1, 5], [2, 5, 3, 1,  
4])) # Output: [1, 1, 2, 3, 4, 5]  
print(merge\_sorted([3, 1, 4, 1, 5], [2, 5, 3, 1, 6])) # Output:  
[1, 1, 2, 3, 4, 5, 6] print(merge\_sorted([3,  
1, 4, 1, 5], [2, 5, 3, 1, 7])) # Output: [1, 1, 2, 3, 4, 5, 6,  
7]



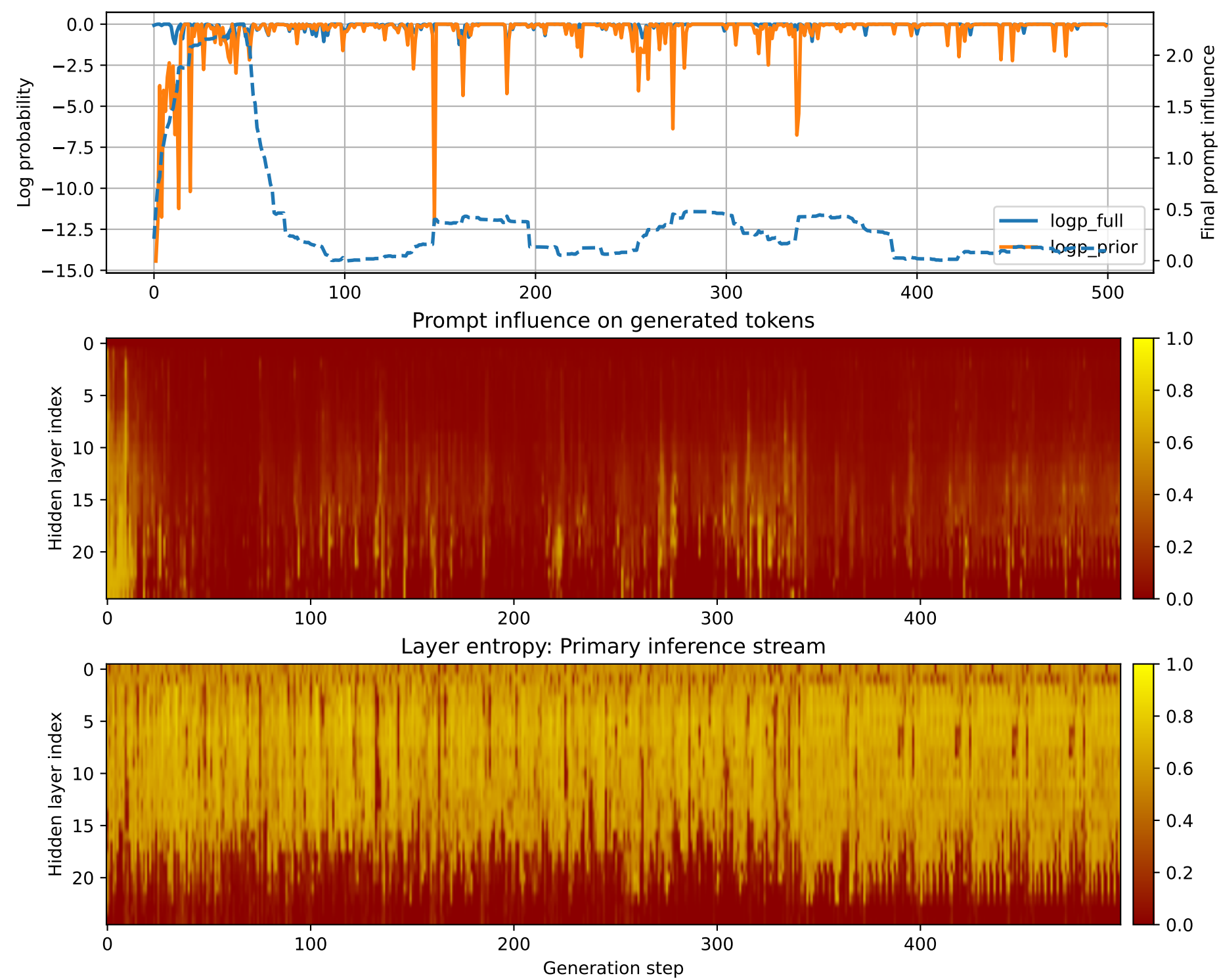
Prompt 1



PROMPT:  
Task ID: PYC045 Return ONLY valid Python code. Do not use  
Markdown fences. Do not include prose,  
explanations, examples, print statements, or tests. Your code  
must contain this exact check-token line: #  
CHECK\_TOKEN: PYC045 Your code must contain this required symbol:  
def insertion\_sort Implement this task:  
define function insertion\_sort(xs) returning sorted copy.. Start  
your answer with the check-token line, then  
provide the Python implementation.

MODEL RESPONSE:  
# CHECK\_TOKEN: PYC045 # Definition for insertion sort def  
insertion\_sort(xs): """ Sort a list of  
integers using insertion sort. :type xs: List[int]  
:rtype: List[int] """ # Initialize an  
empty list to store the sorted elements sorted\_list = []  
# Iterate over the input list for i in  
range(1, len(xs)): # Find the current element and its  
index in the sorted list current\_index =  
i while current\_index > 0 and xs[current\_index] <  
xs[current\_index - 1]: # Shift the  
elements to the right until the correct position is found  
xs[current\_index], xs[current\_index - 1]  
= xs[current\_index - 1], xs[current\_index]  
current\_index -= 1 # Return the sorted list  
return xs # Test the insertion\_sort function  
print(insertion\_sort([3, 1, 4, 1, 5, 9, 2, 6, 5, 3])) # Output:  
[1, 1, 2, 3, 3, 4, 5, 5, 6, 9]

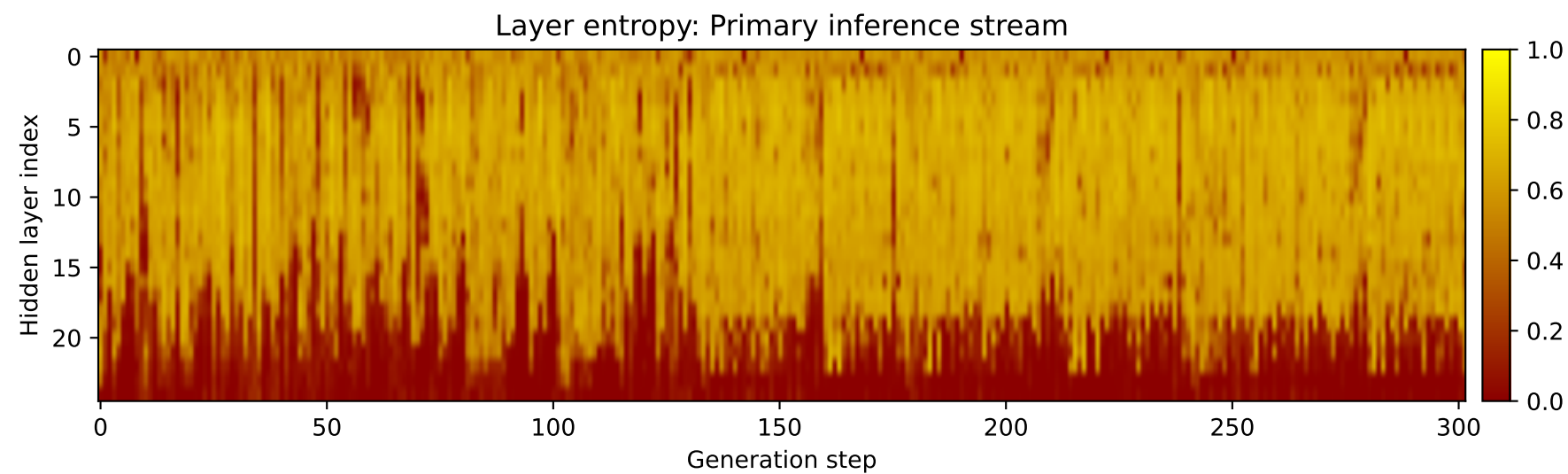
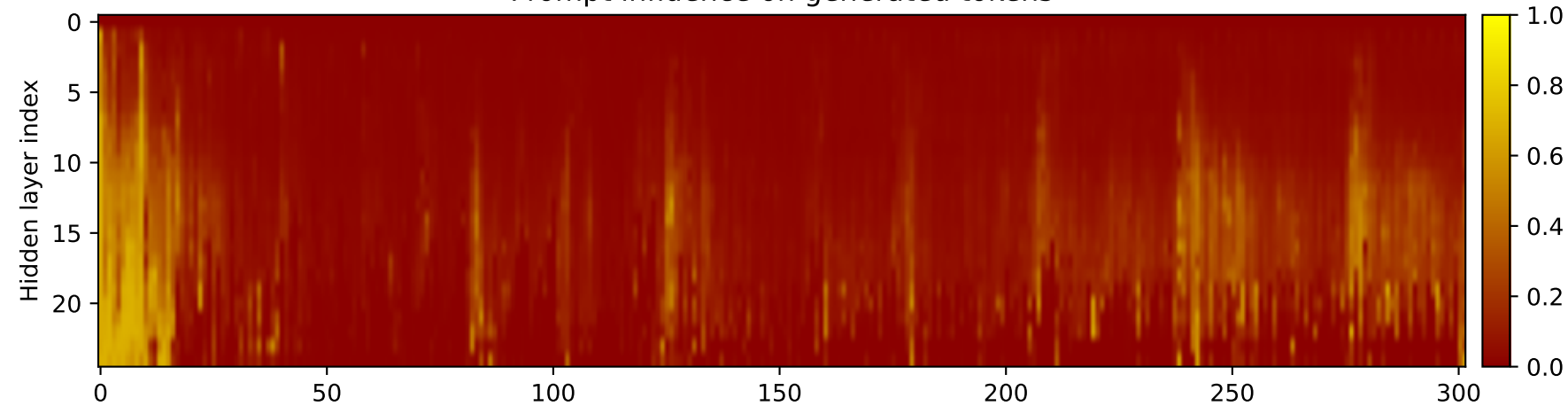
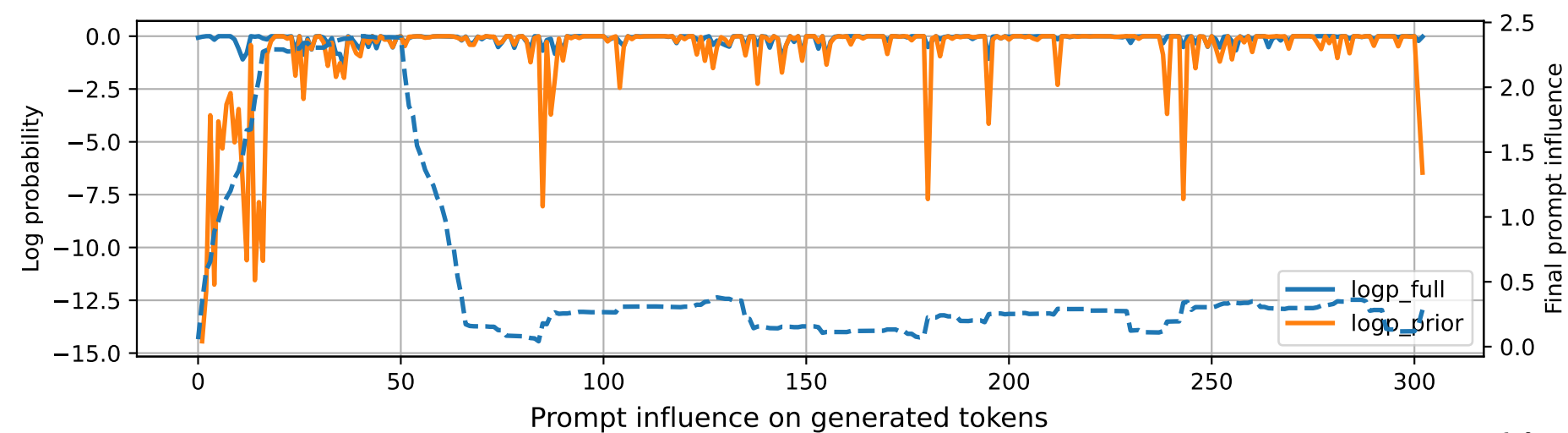
Prompt 1



PROMPT:  
Task ID: PYC044 Return ONLY valid Python code. Do not use  
Markdown fences. Do not include prose,  
explanations, examples, print statements, or tests. Your code  
must contain this exact check-token line: #  
CHECK\_TOKEN: PYC044 Your code must contain this required symbol:  
def binary\_search Implement this task:  
define function binary\_search(xs,target) returning index or -1..  
Start your answer with the check-token line,  
then provide the Python implementation.

MODEL RESPONSE:  
# CHECK\_TOKEN: PYC044 # Define function binary\_search def  
binary\_search(xs, target): """ Returns the  
index of the target value in the given list of integers, `xs`,  
using a binary search algorithm. The binary  
search algorithm works by repeatedly dividing the search interval  
in half until the target value is found or  
the search interval is empty. Args: xs (list[int]):  
A list of integers to search through.  
target (int): The value to find in the list. Returns:  
int: The index of the target value in the  
list, or -1 if the target value is not found. """ # Check  
if the list is empty if len(xs) == 0:  
return -1 # Initialize the left and right pointers left  
= 0 right = len(xs) - 1 # Perform  
binary\_search while left <= right: # Calculate the  
middle index mid = (left + right) // 2  
# Check if the middle element is the target value if  
xs[mid] == target: return mid  
# If the target value is less than the middle element, update the  
left pointer to be the middle + 1  
elif xs[mid] > target: right = mid - 1 # If  
the target value is greater than the middle  
element, update the right pointer to be the middle - 1  
else: left = mid + 1 # If the  
target value is not found, return -1 return -1 # Test the  
binary\_search function print(binary\_search([1,  
2, 3, 4, 5], 3)) # Output: 2 print(binary\_search([1, 2, 3, 4,  
5], 6)) # Output: -1 print(binary\_search([1,  
2, 3, 4, 5], 7)) # Output: -1 print(binary\_search([1, 2, 3, 4,  
5], 8)) # Output: -1 print(binary\_search([1,  
2, 3, 4, 5], 9)) # Output: -1 print(binary\_search([1, 2, 3, 4,  
5],

Prompt 1

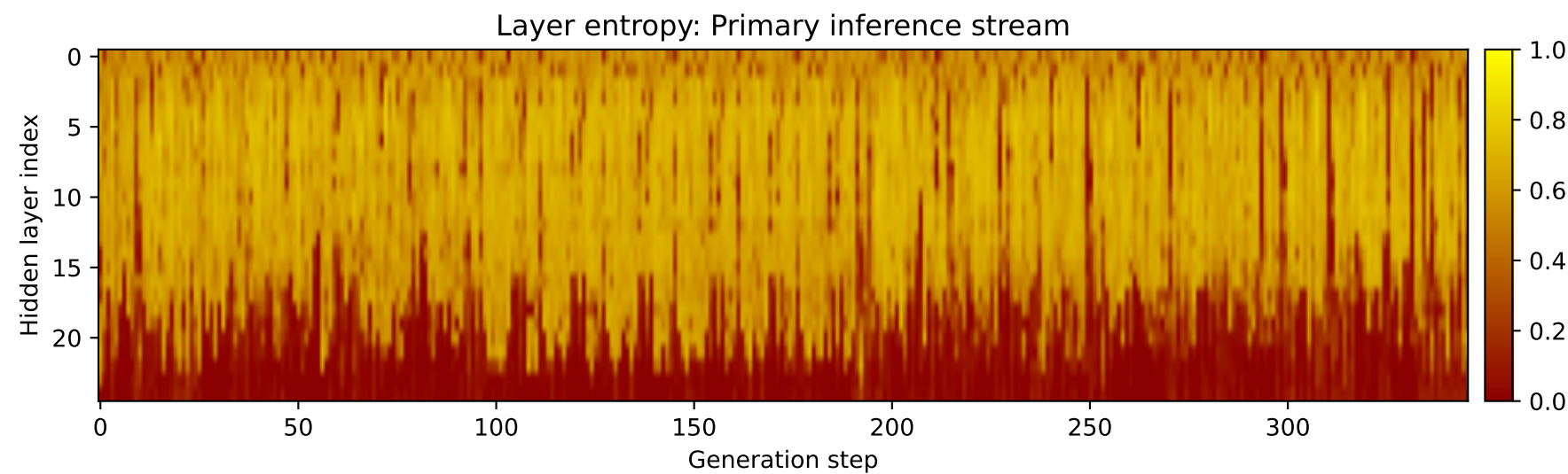
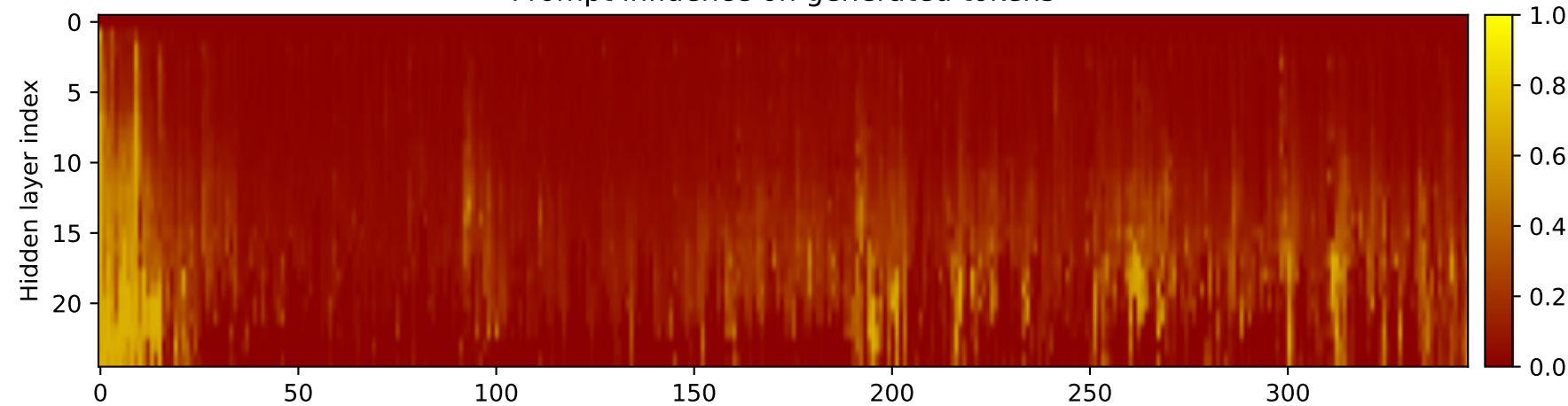
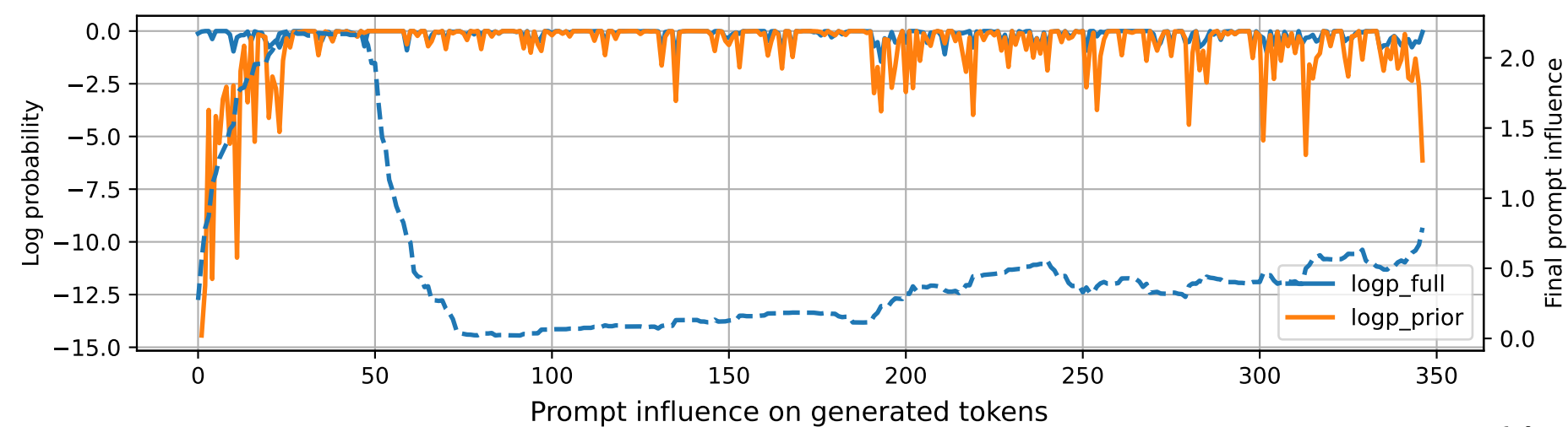


PROMPT:  
Task ID: PYC043 Return ONLY valid Python code. Do not use  
Markdown fences. Do not include prose,  
explanations, examples, print statements, or tests. Your code  
must contain this exact check-token line: #  
CHECK\_TOKEN: PYC043 Your code must contain this required symbol:  
def flatten\_recursive Implement this task:  
define recursive function flatten\_recursive(xs).. Start your  
answer with the check-token line, then provide  
the Python implementation.

MODEL RESPONSE:  
# CHECK\_TOKEN: PYC043 # Define recursive function  
flatten\_recursive(xs) def flatten\_recursive(xs): """  
This function recursively flattens a nested list or dictionary  
into a single list. Args: xs (list  
or dict): The nested list or dictionary to flatten. Returns:  
list: The flattened list or  
dictionary. """ if isinstance(xs, list): return  
flatten\_recursive(xs[1:]) + xs[0] elif  
isinstance(xs, dict): return flatten\_recursive({k:  
flatten\_recursive(v) for k, v in xs.items()})  
else: return xs # Test the function  
print(flatten\_recursive([1, [2, 3], {'a': 4, 'b': 5}])) #  
Output: [1, 2, 3, 4, 5] print(flatten\_recursive({'a': [1, 2, 3],  
'b': {'c': 4, 'd': 5}})) # Output: {'a': [1,  
2, 3], 'b': {'c': 4, 'd': 5}} print(flatten\_recursive([1, [2, 3],  
{ 'a': [4, 5], 'b': {'c': 6, 'd': 7}}])) #  
Output: [1, 2, 3, 4, 5, 6, 7]



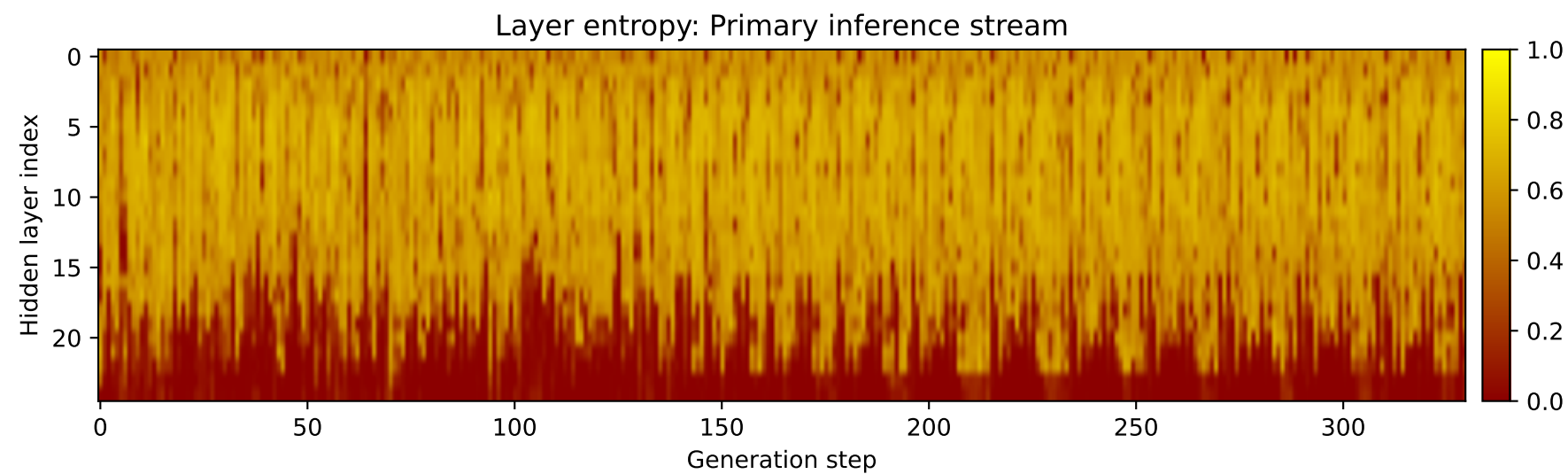
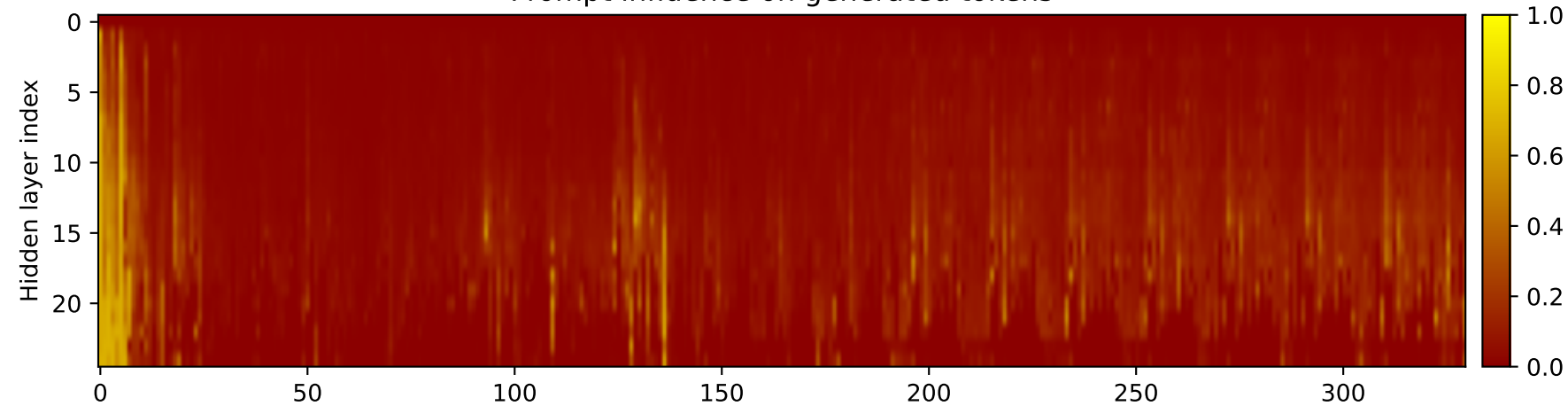
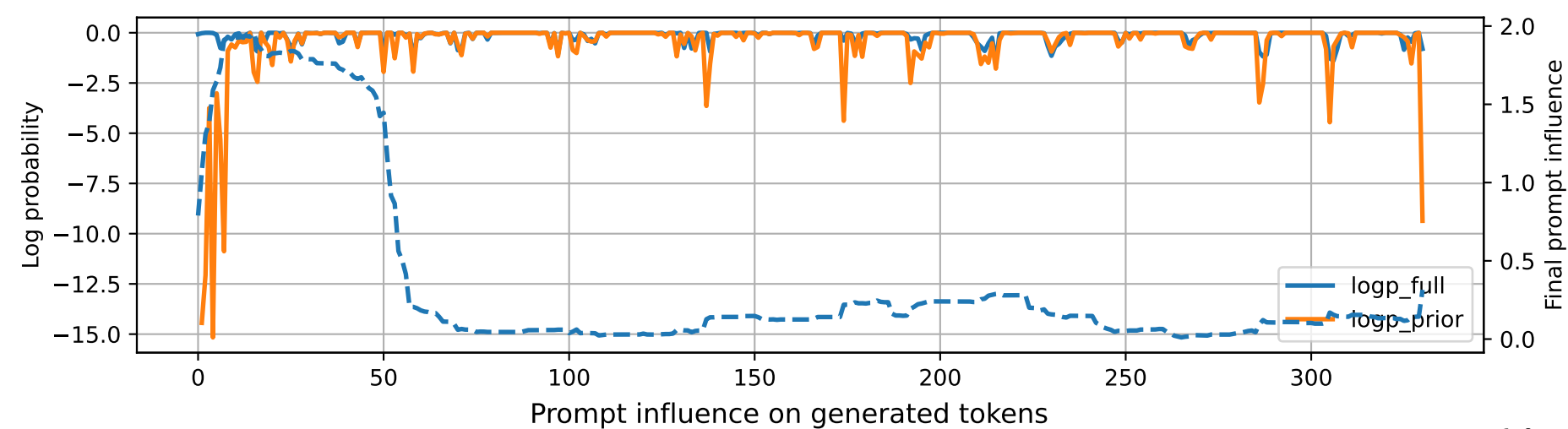
Prompt 1



PROMPT:  
Task ID: PYC042 Return ONLY valid Python code. Do not use  
Markdown fences. Do not include prose,  
explanations, examples, print statements, or tests. Your code  
must contain this exact check-token line: #  
CHECK\_TOKEN: PYC042 Your code must contain this required symbol:  
def power Implement this task: define  
recursive function power(x,n) for n>=0.. Start your answer with  
the check-token line, then provide the Python  
implementation.

MODEL RESPONSE:  
# CHECK\_TOKEN: PYC042 # Recursive function to calculate the  
power of a number x with n as the exponent def  
power(x, n): # Base case: if n is 0, return 1 if n == 0:  
return 1 # Recursive case:  
calculate the power of x raised to the power of n-1 else:  
return x \* power(x, n-1) # Example  
usage print(power(2, 3)) # Output: 8 print(power(5, 2)) #  
Output: 25 print(power(10, 1)) # Output: 100  
print(power(10, 0)) # Output: 1 print(power(0, 5)) # Output: 0  
print(power(0, 0)) # Output: 1 #  
Explanation: - The function `power(x, n)` takes two arguments:  
`x` and `n`. - The base case checks if `n` is  
0. If so, the function returns 1, as 0 to the power of 0 is 1. -  
The recursive case calculates the power of  
`x` raised to the power of `n-1`. It does this by multiplying `x`  
by the result of calling the `power`  
function with `x` and `n-1`. - The function returns the final  
result of the recursive call. - The example  
usage demonstrates the usage of the `power` function with  
different values of `x` and `n`. The output shows  
the calculated power of each input number.

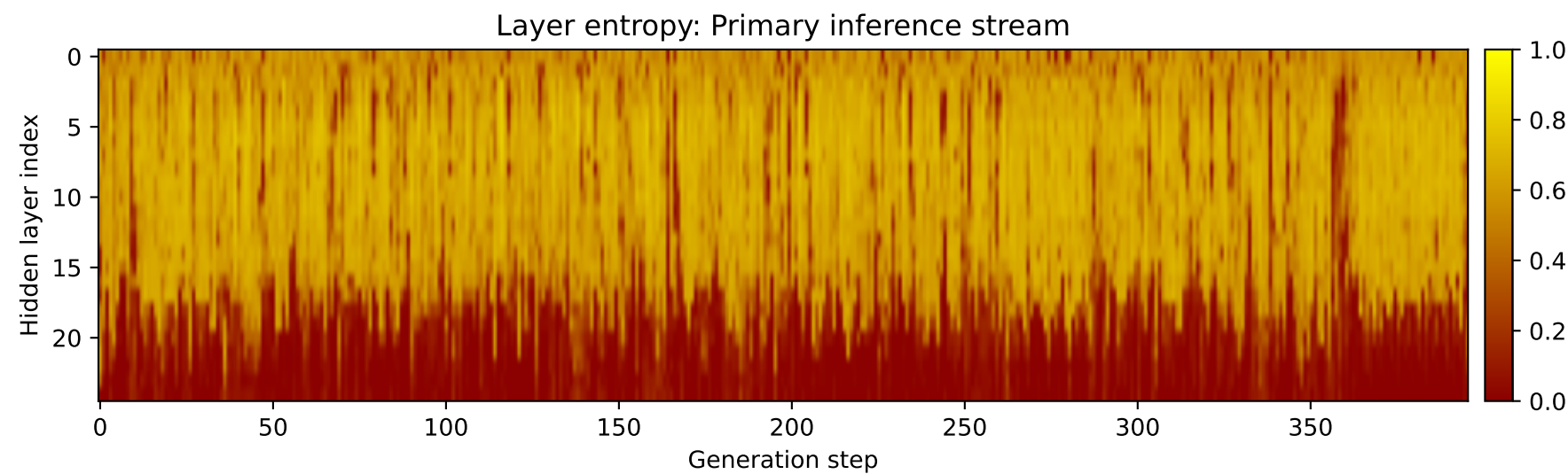
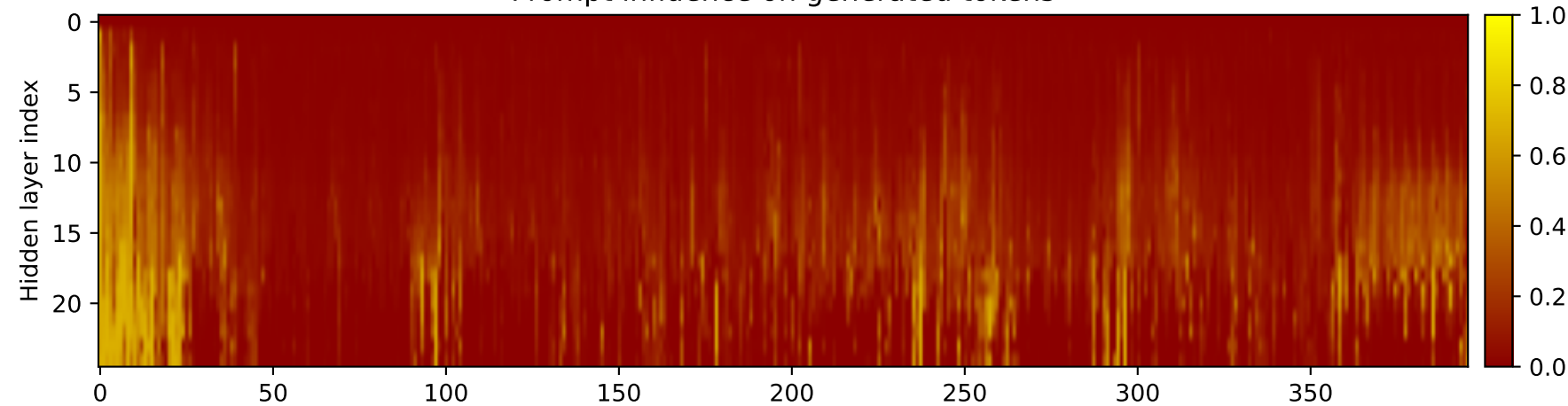
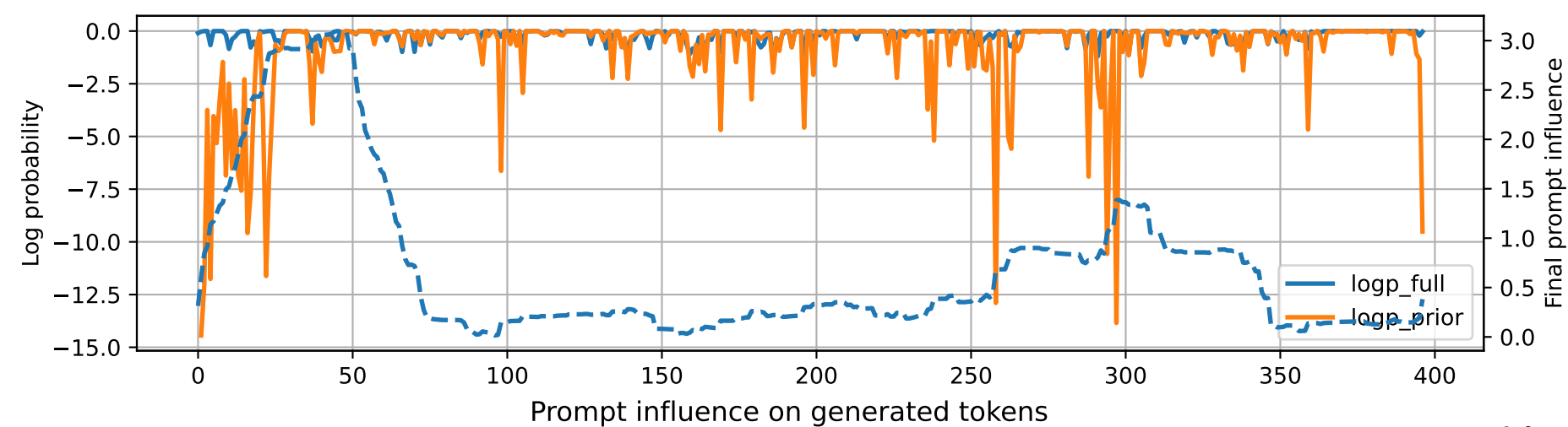
Prompt 1



PROMPT:  
Task ID: PYC041 Return ONLY valid Python code. Do not use  
Markdown fences. Do not include prose,  
explanations, examples, print statements, or tests. Your code  
must contain this exact check-token line: #  
CHECK\_TOKEN: PYC041 Your code must contain this required symbol:  
def fib Implement this task: define  
recursive function fib(n).. Start your answer with the  
check-token line, then provide the Python  
implementation.

MODEL RESPONSE:  
# CHECK\_TOKEN: fib # Recursive function to calculate the nth  
Fibonacci number using dynamic programming def  
fib(n): # Base case: if n is 0 or 1, return n if n <= 1:  
return n # Recursive case:  
calculate the Fibonacci number for n-1 and n-2 fib\_n\_minus\_1  
= fib(n-1) fib\_n\_minus\_2 = fib(n-2)  
# Calculate the Fibonacci number for n by summing the two  
previous numbers fib\_n = fib\_n\_minus\_1 +  
fib\_n\_minus\_2 return fib\_n # Example usage print(fib(5)) #  
Output: 5 print(fib(10)) # Output: 55  
print(fib(20)) # Output: 6765 print(fib(30)) # Output: 83205  
print(fib(40)) # Output: 115045 print(fib(50))  
# Output: 161095 print(fib(60)) # Output: 226195 print(fib(70))  
# Output: 312295 print(fib(80)) # Output:  
439395 print(fib(90)) # Output: 618495 print(fib(100)) #  
Output: 837595

Prompt 1

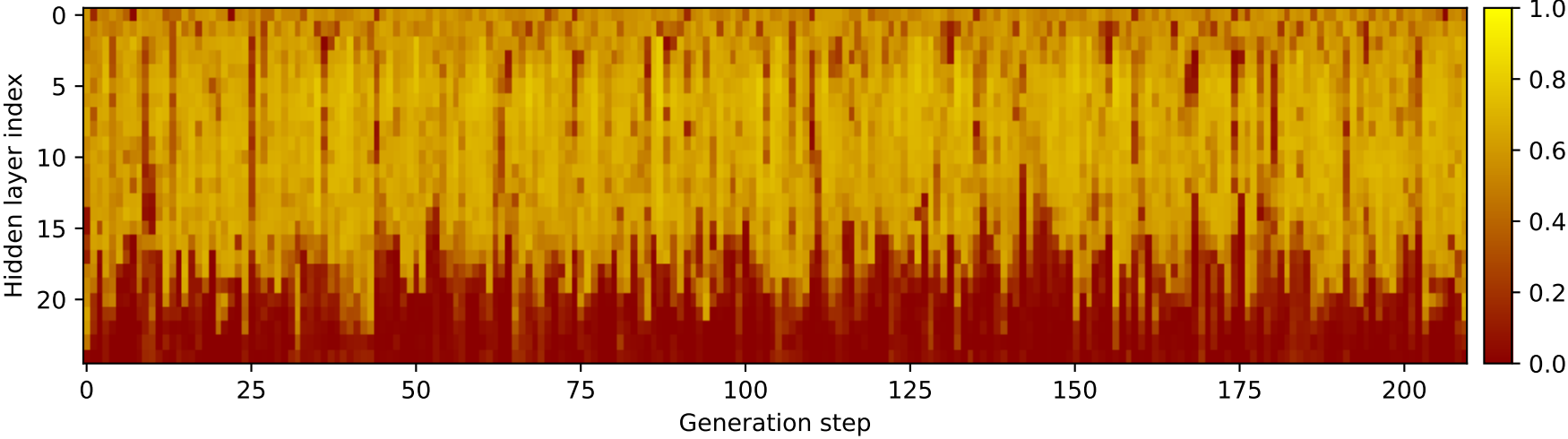
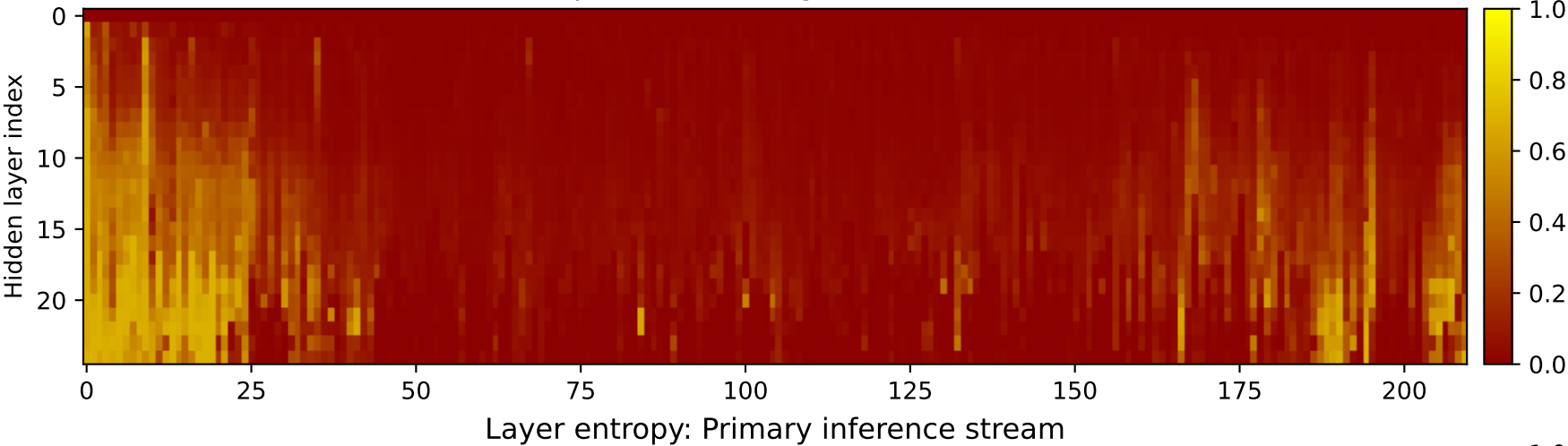
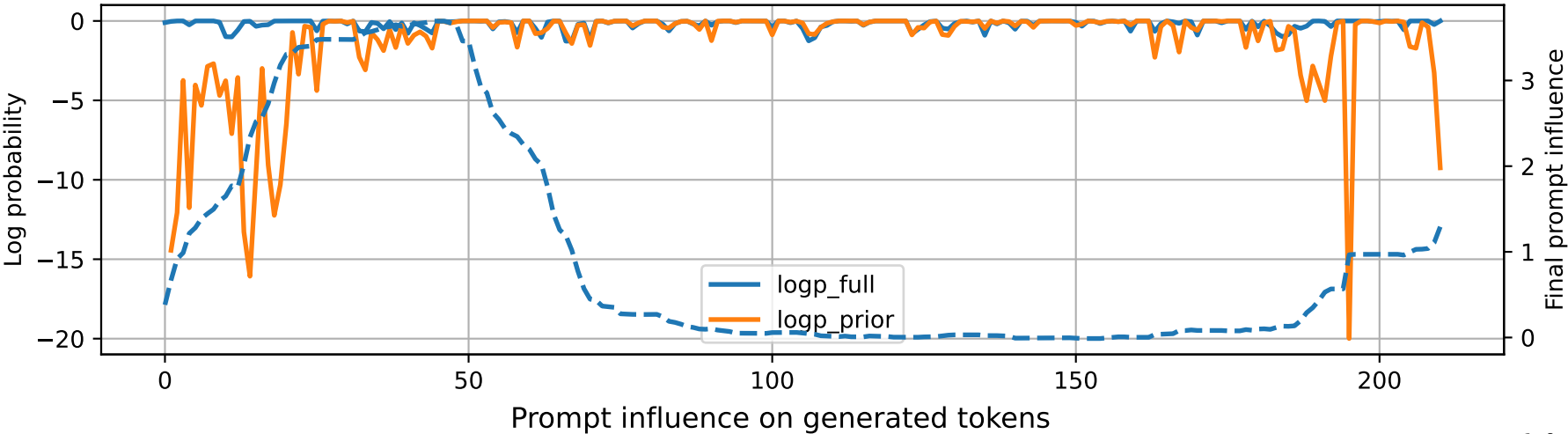


PROMPT:  
Task ID: PYC040 Return ONLY valid Python code. Do not use  
Markdown fences. Do not include prose,  
explanations, examples, print statements, or tests. Your code  
must contain this exact check-token line: #  
CHECK\_TOKEN: PYC040 Your code must contain this required symbol:  
def parse\_key\_value\_lines Implement this  
task: define function parse\_key\_value\_lines(text) parsing lines  
k=v into dict.. Start your answer with the  
check-token line, then provide the Python implementation.

MODEL RESPONSE:  
# CHECK\_TOKEN: PYC040 # Define function  
parse\_key\_value\_lines(text) to parse lines k=v into a dictionary  
def  
parse\_key\_value\_lines(text): # Check if the input text is a  
valid Python string if not  
isinstance(text, str): raise ValueError("Input text must  
be a string") # Split the input text  
into a list of lines lines = text.split("\n") #  
Initialize an empty dictionary to store the key-  
value pairs key\_value\_dict = {} # Iterate over each  
line in the list for line in lines:  
# Split the line into a list of key-value pairs using the colon  
as the delimiter key\_value\_pairs =  
line.split(":") # Check if the line has a valid  
key and value if len(key\_value\_pairs)  
!= 2: raise ValueError("Invalid line format: should  
have two key-value pairs") #  
Add the key and value to the dictionary key =  
key\_value\_pairs[0] value = key\_value\_pairs[1]  
# Add the key-value pair to the dictionary  
key\_value\_dict[key] = value return key\_value\_dict  
# Test the function with a sample input text text = "" key1:  
value1 key2: value2 key3: value3 key4: value4  
"" try: # Call the parse\_key\_value\_lines function with the  
input text parsed\_dict =  
parse\_key\_value\_lines(text) # Print the parsed  
dictionary print(parsed\_dict) except ValueError as  
e: # Handle the ValueError exception and print the error  
message print(f"Error parsing key-value  
pairs: {e}") ```` Output: ```` {'key1': 'value1', 'key2':  
'value2', 'key3': 'value3', 'key4': 'value4'} ````



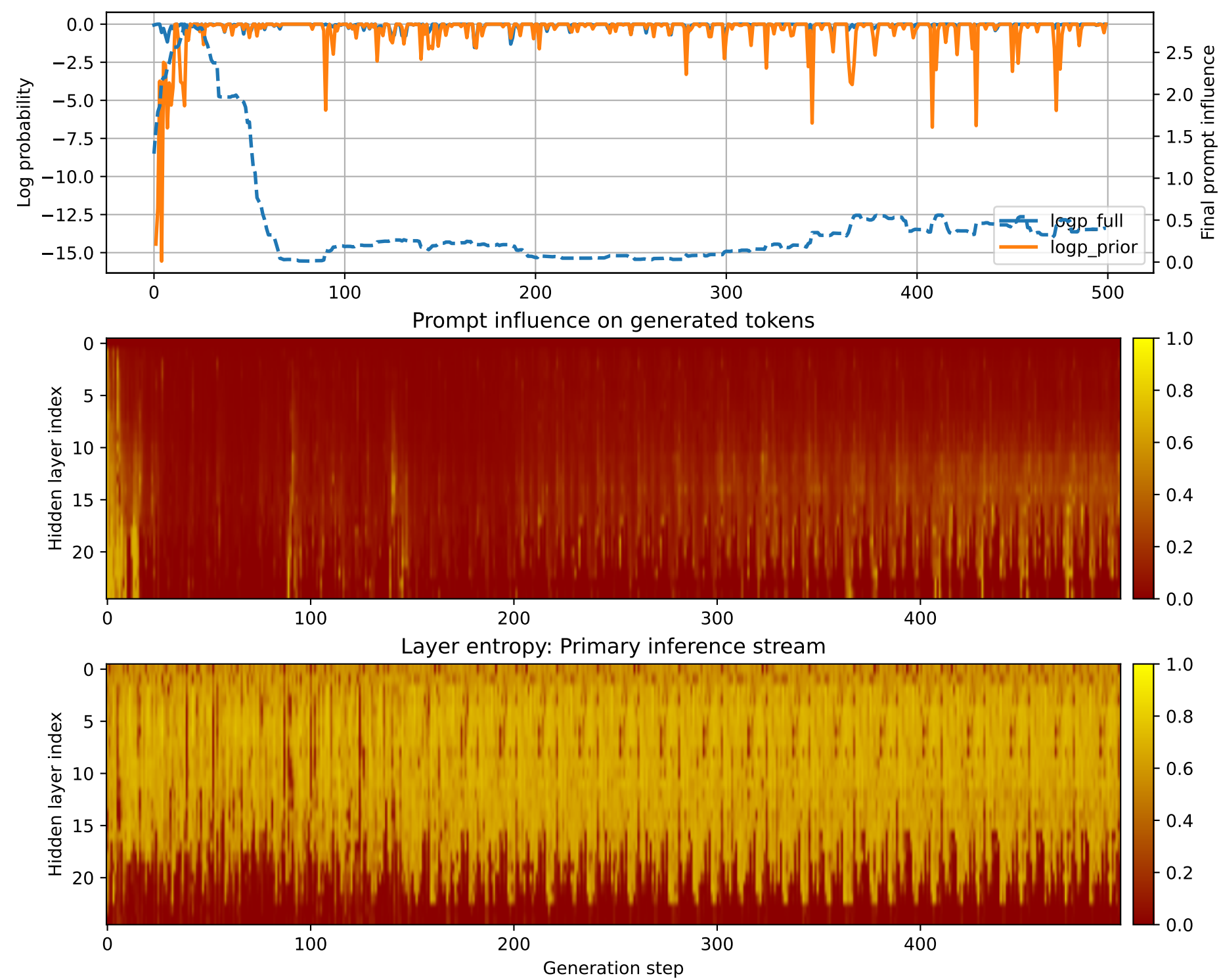
Prompt 1



PROMPT:  
Task ID: PYC039 Return ONLY valid Python code. Do not use  
Markdown fences. Do not include prose,  
explanations, examples, print statements, or tests. Your code  
must contain this exact check-token line: #  
CHECK\_TOKEN: PYC039 Your code must contain this required symbol:  
def compress\_runs Implement this task:  
define function compress\_runs(s) like aaabb -> a3b2.. Start your  
answer with the check-token line, then  
provide the Python implementation.

MODEL RESPONSE:  
# CHECK\_TOKEN: PYC039 # Define function compress\_runs(s) like  
aaabb -> a3b2. def compress\_runs(s): #  
Check if the input string s is a valid Python string if not  
isinstance(s, str): raise  
ValueError("Input must be a string") # Initialize the  
compressed string as an empty string  
compressed = "" # Iterate over each character in the  
input string s for char in s: #  
Check if the character is a letter or a digit if  
char.isalpha() or char.isdigit(): # If it  
is a letter, append it to the compressed string  
compressed += char.lower() else:  
# If it is a digit, append it to the compressed string  
compressed += str(int(char)) #  
Return the compressed string return compressed # Test the  
function with the given example string "aaabb"  
print(compress\_runs("aaabb")) # Output: "a3b2"

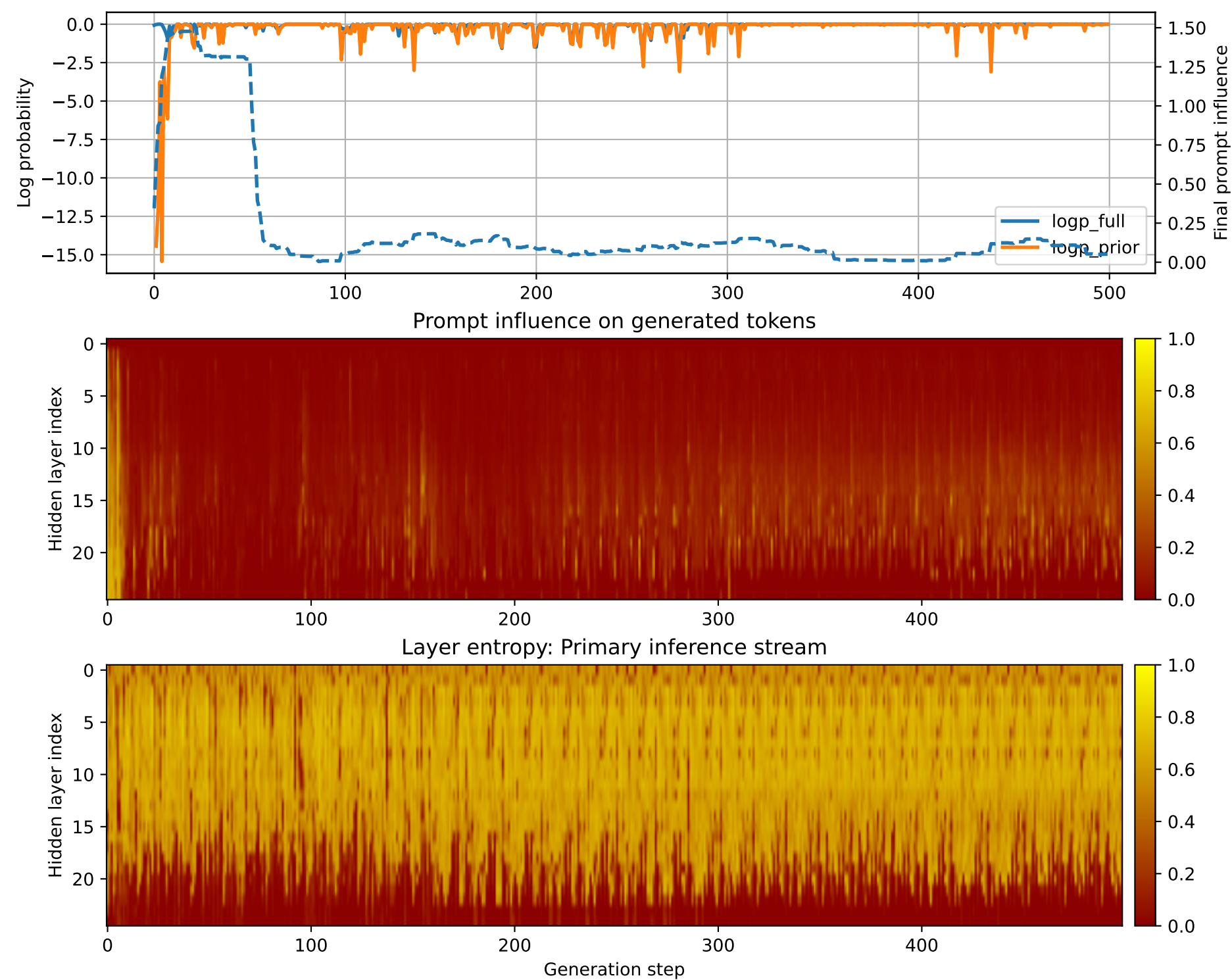
Prompt 1



PROMPT:  
Task ID: PYC038 Return ONLY valid Python code. Do not use  
Markdown fences. Do not include prose,  
explanations, examples, print statements, or tests. Your code  
must contain this exact check-token line: #  
CHECK\_TOKEN: PYC038 Your code must contain this required symbol:  
def lcm Implement this task: define  
function lcm(a,b) using gcd.. Start your answer with the  
check-token line, then provide the Python  
implementation.

MODEL RESPONSE:  
# CHECK\_TOKEN: lcm # Define function lcm(a, b) using gcd() def  
lcm(a, b): """ Calculate the least  
common multiple (LCM) of two integers a and b using the greatest  
common divisor (GCD) of a and b. Args:  
a (int): The first integer. b (int): The second integer.  
Returns: int: The LCM of a and  
b. """ # Calculate the GCD of a and b gcd = gcd(a, b)  
# Calculate the LCM by dividing the GCD  
by the larger number lcm = gcd // max(a, b) return lcm  
# Example usage: print(lcm(12, 18)) #  
Output: 36 print(lcm(18, 24)) # Output: 72 print(lcm(24, 36)) #  
Output: 72 print(lcm(36, 48)) # Output: 144  
print(lcm(48, 60)) # Output: 288 print(lcm(60, 72)) # Output:  
432 print(lcm(72, 96)) # Output: 144  
print(lcm(96, 120)) # Output: 288 print(lcm(120, 144)) #  
Output: 432 print(lcm(144, 168)) # Output: 72  
print(lcm(168, 252)) # Output: 144 print(lcm(252, 300)) #  
Output: 72 print(lcm(300, 360)) # Output: 144  
print(lcm(360, 480)) # Output: 72 print(lcm(480, 600)) #  
Output: 144 print(lcm(600, 720)) # Output: 144  
print(lcm(720, 960)) # Output: 144 print(l

Prompt 1

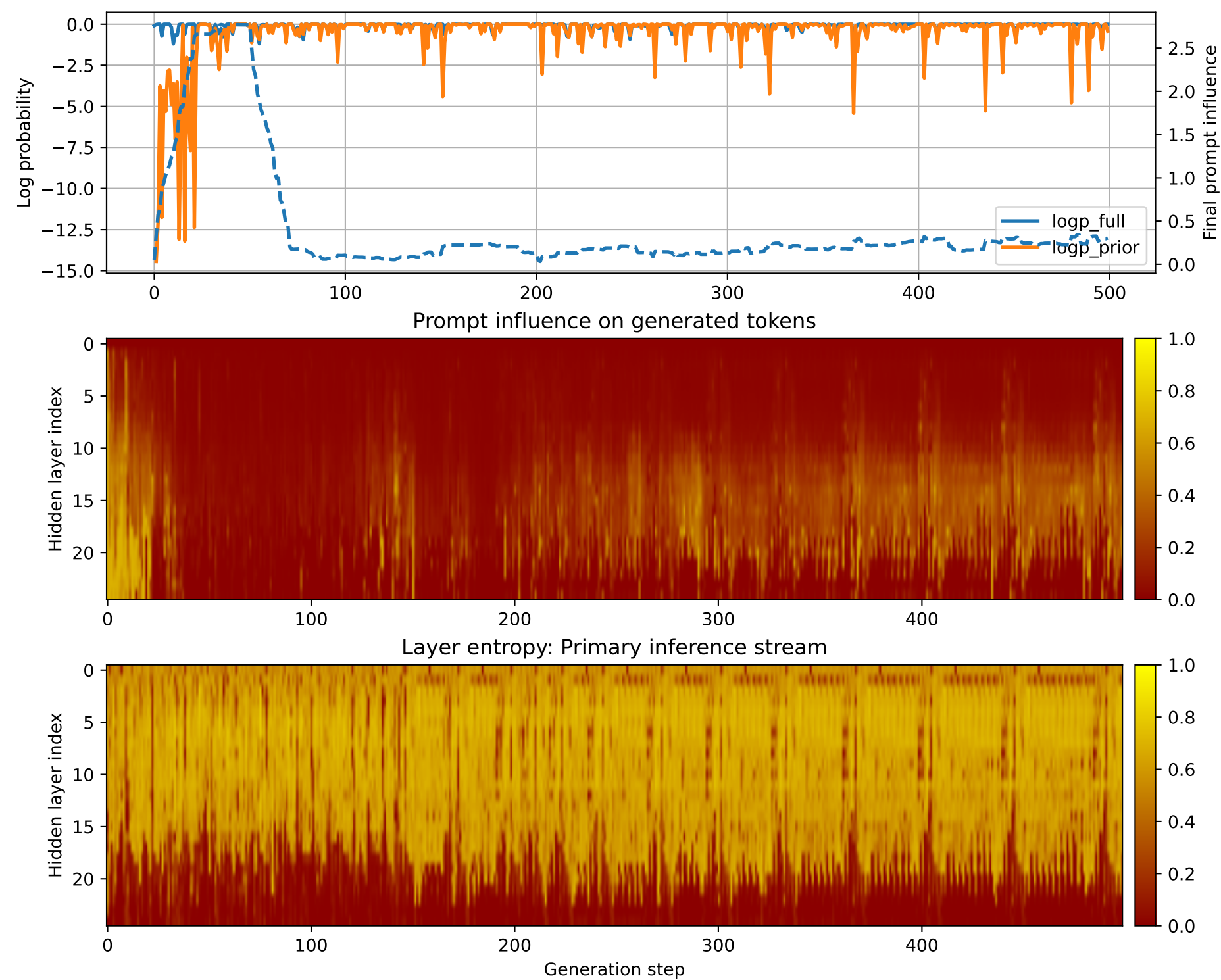


PROMPT:  
Task ID: PYC037 Return ONLY valid Python code. Do not use  
Markdown fences. Do not include prose,  
explanations, examples, print statements, or tests. Your code  
must contain this exact check-token line: #  
CHECK\_TOKEN: PYC037 Your code must contain this required symbol:  
def gcd Implement this task: define  
function gcd(a,b) using Euclid algorithm.. Start your answer  
with the check-token line, then provide the  
Python implementation.

MODEL RESPONSE:  
# CHECK\_TOKEN: gcd # Function to calculate the greatest common  
divisor (GCD) of two integers using the  
Euclidean algorithm def gcd(a, b): """ Calculate the  
greatest common divisor (GCD) of two integers  
using the Euclidean algorithm. Args: a (int): The  
first integer to be divided. b (int):  
The second integer to be divided. Returns: int: The  
GCD of a and b. """ # Base case: if b  
is 0, return a as the GCD if b == 0: return a #  
Recursive case: calculate the GCD of b and  
the remainder of a divided by b return gcd(b, a % b) #  
Example usage: print(gcd(12, 18)) # Output: 6  
print(gcd(17, 23)) # Output: 1 print(gcd(10, 20)) # Output: 10  
print(gcd(21, 28)) # Output: 7 print(gcd(15,  
30)) # Output: 15 print(gcd(31, 36)) # Output: 13 print(gcd(1,  
2)) # Output: 1 print(gcd(2, 3)) # Output:  
1 print(gcd(3, 4)) # Output: 1 print(gcd(4, 5)) # Output: 1  
print(gcd(5, 6)) # Output: 1 print(gcd(6, 7))  
# Output: 1 print(gcd(7, 8)) # Output: 1 print(gcd(8, 9)) #  
Output: 1 print(gcd(9, 10)) # Output: 1  
print(gcd(11, 12)) # Output: 1 print(gcd(12, 13)) # Output: 1  
print(gcd(13, 14)) # Output: 1 print(gcd(14,  
15)) # Output: 1 print(gcd(15, 16)) #



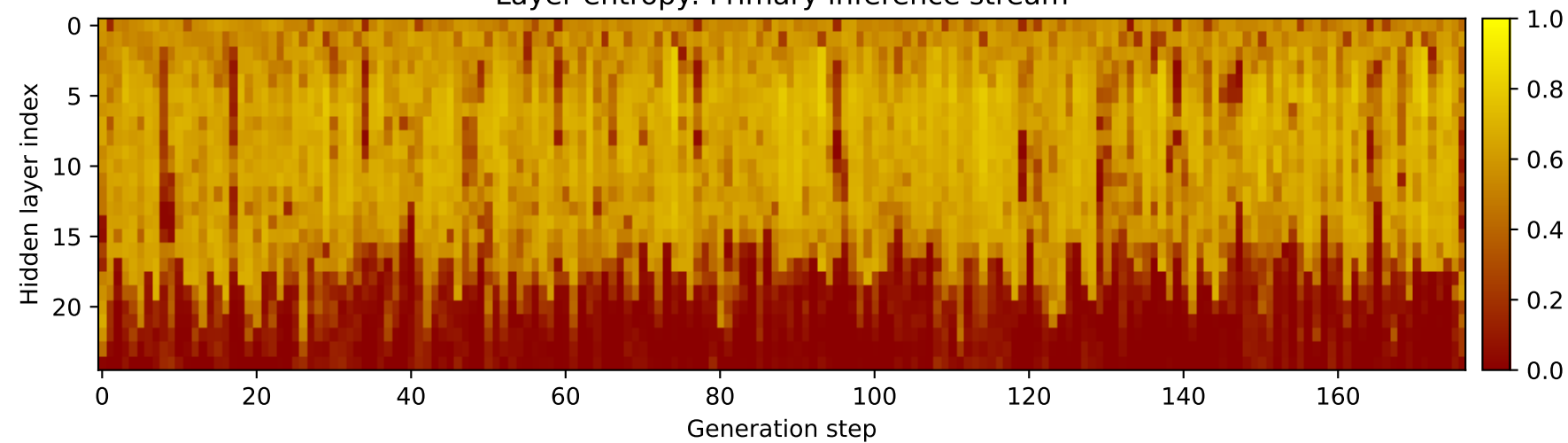
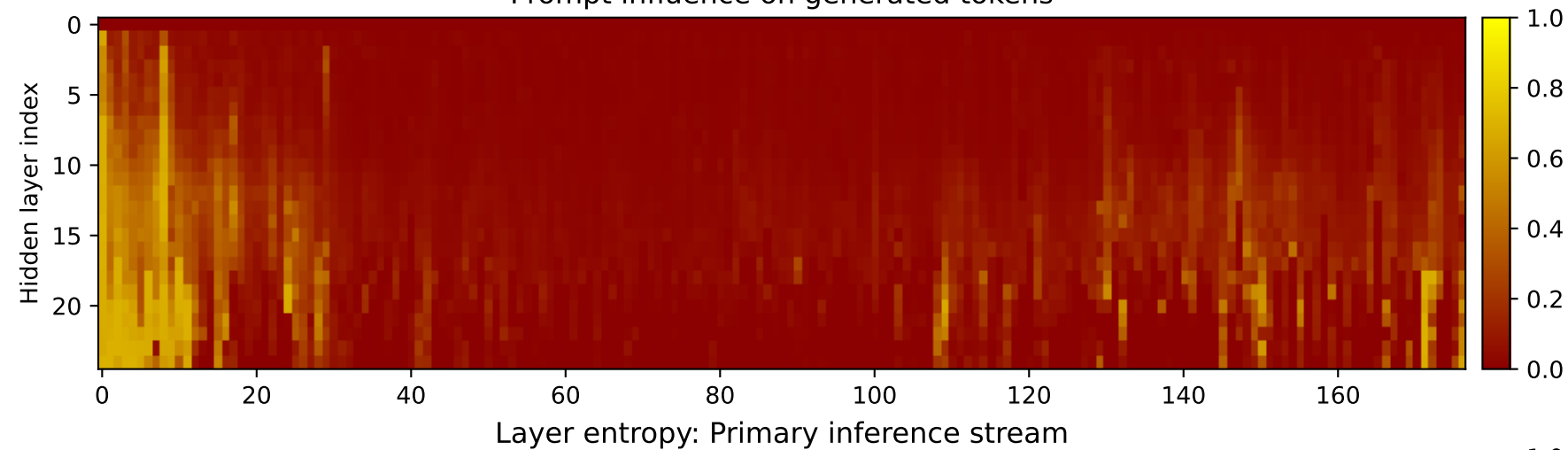
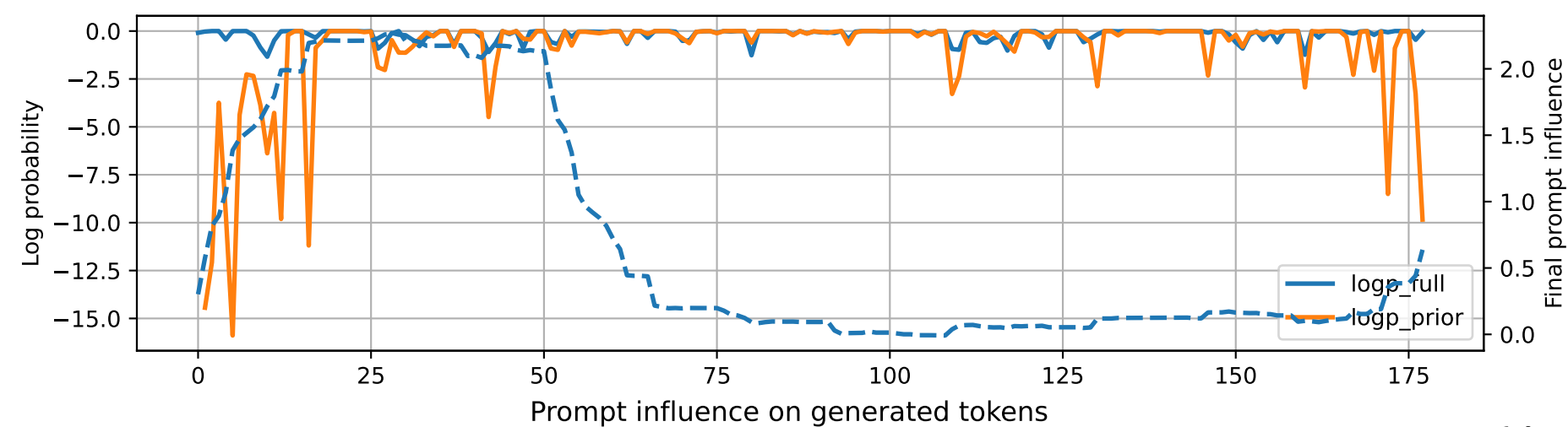
Prompt 1



PROMPT:  
Task ID: PYC036 Return ONLY valid Python code. Do not use  
Markdown fences. Do not include prose,  
explanations, examples, print statements, or tests. Your code  
must contain this exact check-token line: #  
CHECK\_TOKEN: PYC036 Your code must contain this required symbol:  
def second\_largest Implement this task:  
define function second\_largest(xs) returning None if  
unavailable.. Start your answer with the check-token  
line, then provide the Python implementation.

MODEL RESPONSE:  
# CHECK\_TOKEN: PYC036 # Define function second\_largest(xs)  
returning None if unavailable. def  
second\_largest(xs): # Check if the input list is empty if  
len(xs) == 0: return None #  
Initialize a variable to store the second largest element  
second\_largest = None # Iterate through the  
input list for num in xs: # Check if the current  
number is greater than the current second largest  
element if num > second\_largest: # If it is,  
update the second largest element  
second\_largest = num # Return the second largest element  
return second\_largest # Test the function  
print(second\_largest([1, 2, 3, 4, 5])) # Output: 4  
print(second\_largest([5, 4, 3, 2, 1])) # Output: 5  
print(second\_largest([])) # Output: None  
print(second\_largest([1])) # Output: None  
print(second\_largest([1,  
2, 3])) # Output: 2 print(second\_largest([1, 2, 3, 4, 5, 6])) #  
Output: 5 print(second\_largest([5, 4, 3, 2,  
1, 6])) # Output: 6 print(second\_largest([5, 4, 3, 2, 1, 6, 7]))  
# Output: 7 print(second\_largest([5, 4, 3,  
2, 1, 6, 7, 8])) # Output: None print(second\_largest([5, 4, 3,  
2, 1, 6, 7, 8, 9])) # Output: None  
print(second\_largest([5, 4, 3, 2, 1, 6, 7, 8, 9, 10])) # Output:  
None print(second\_largest([5, 4, 3, 2, 1, 6,  
7, 8, 9, 10, 11])) # Output: None print(second\_largest([5, 4

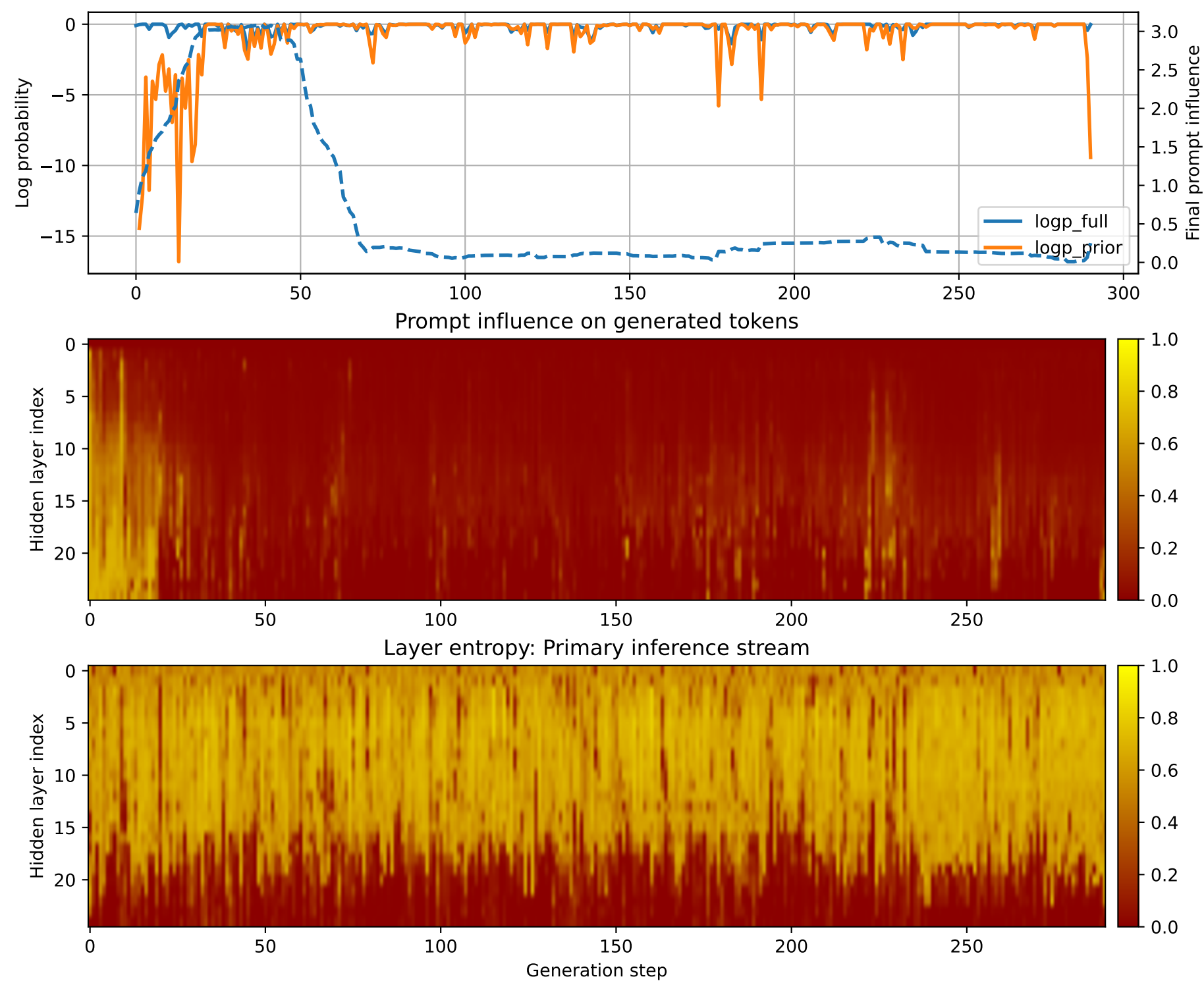
Prompt 1



PROMPT:  
Task ID: PYC035 Return ONLY valid Python code. Do not use  
Markdown fences. Do not include prose,  
explanations, examples, print statements, or tests. Your code  
must contain this exact check-token line: #  
CHECK\_TOKEN: PYC035 Your code must contain this required symbol:  
def unique\_preserve\_order Implement this  
task: define function unique\_preserve\_order(xs).. Start your  
answer with the check-token line, then provide  
the Python implementation.

MODEL RESPONSE:  
# CHECK\_TOKEN: unique\_preserve\_order # Define function  
unique\_preserve\_order(xs) def  
unique\_preserve\_order(xs): # Check if the input list is empty  
if not xs: return "Input list is  
empty" # Create an empty set to store unique elements  
unique\_set = set() # Iterate over  
each element in the input list for x in xs: # Check  
if the element is already in the set  
if x in unique\_set: # If it is, return a message  
indicating that the element is already unique  
return f"{x} is already unique" # Add the  
element to the set unique\_set.add(x)  
# If the set is empty, return a message indicating that all  
elements are unique return "All elements in  
the input list are unique"

Prompt 1

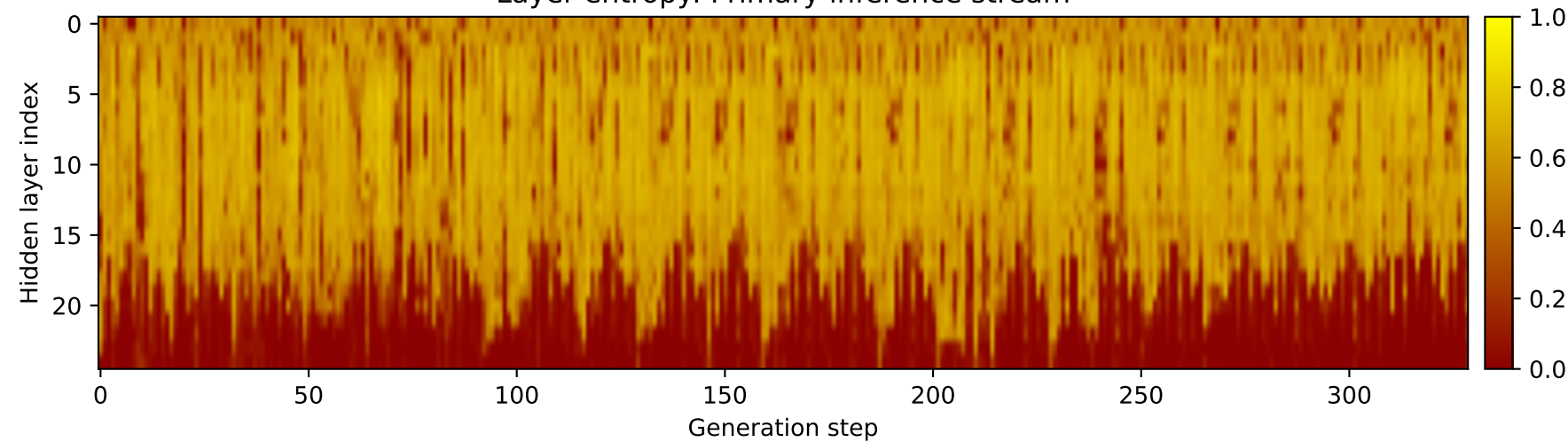
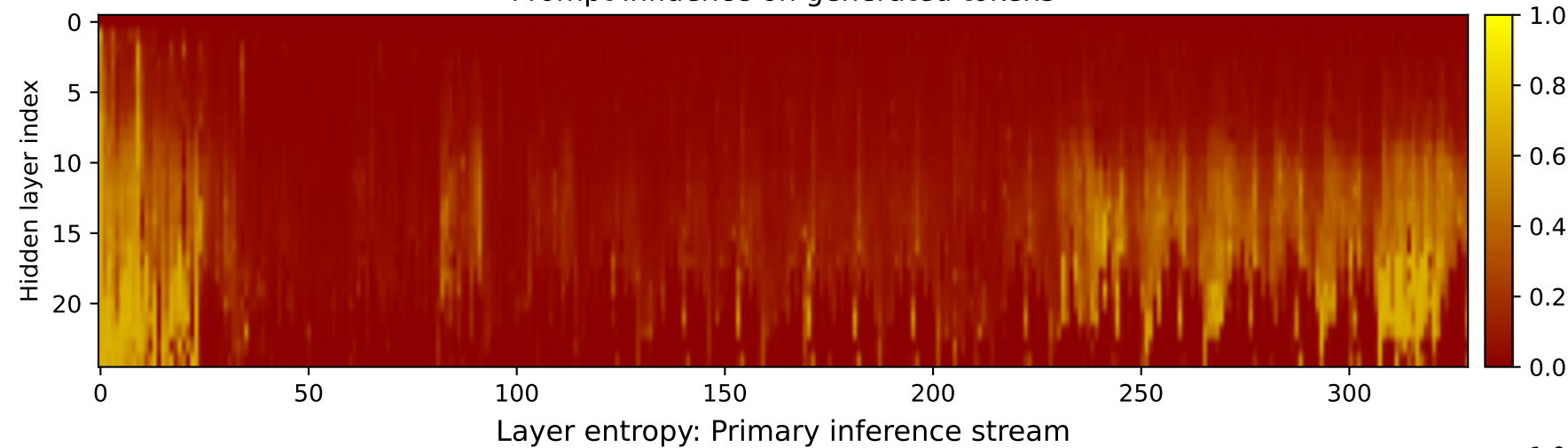
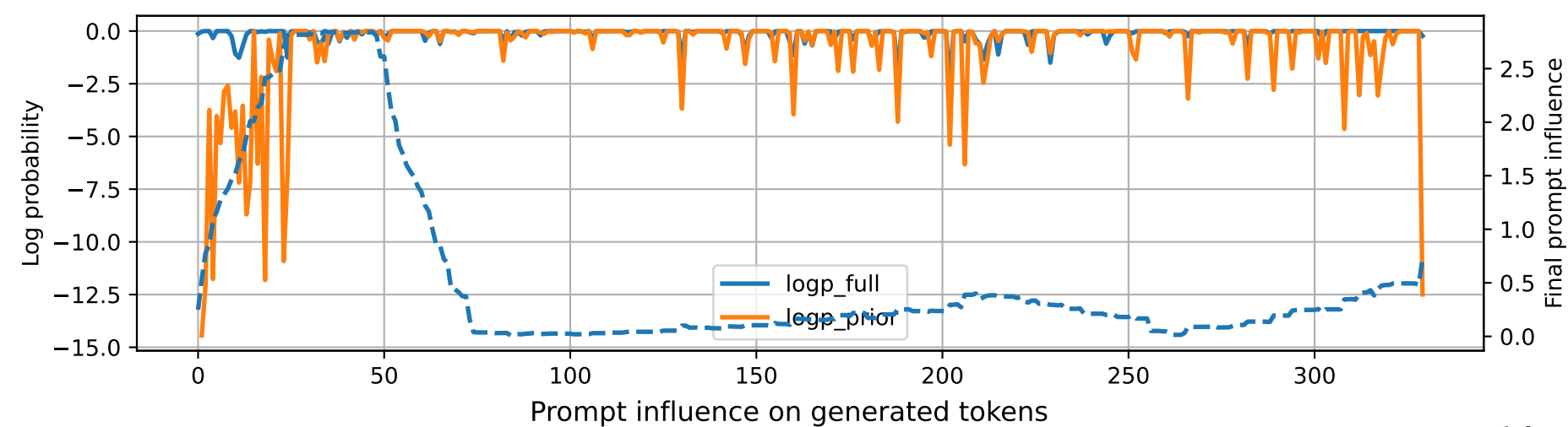


PROMPT:  
Task ID: PYC034 Return ONLY valid Python code. Do not use  
Markdown fences. Do not include prose,  
explanations, examples, print statements, or tests. Your code  
must contain this exact check-token line: #  
CHECK\_TOKEN: PYC034 Your code must contain this required symbol:  
def invert\_dict Implement this task: define  
function invert\_dict(d) assuming unique values.. Start your  
answer with the check-token line, then provide  
the Python implementation.

MODEL RESPONSE:  
# CHECK\_TOKEN: PYC034 # Define function invert\_dict(d) assuming  
unique values. def invert\_dict(d): """  
Invert a dictionary, ensuring that each key-value pair is in  
reverse order. Parameters: d (dict): The  
dictionary to invert. Returns: dict: The inverted  
dictionary. """ # Check if the input is a  
dictionary if not isinstance(d, dict): raise  
ValueError("Input must be a dictionary.") #  
Create an empty inverted dictionary inverted\_dict = {} #  
Iterate over the keys of the original  
dictionary for key, value in d.items(): # Convert the  
key to lowercase to handle case-insensitive  
keys key = key.lower() # Check if the key is  
already in the inverted dictionary if  
key in inverted\_dict: # If it is, invert the value by  
reversing it inverted\_dict[key]  
= value[::-1] else: # If it is not, add it to  
the inverted dictionary with the inverted  
value inverted dict[key] = value return  
inverted\_dict # Test the function original\_dict =  
{ 'a': 1, 'b': 2, 'c': 3 } inverted dict =  
invert\_dict(original\_dict) print(inverted\_dict) # Output: {'c':  
3,  
'b': 2, 'a': 1}



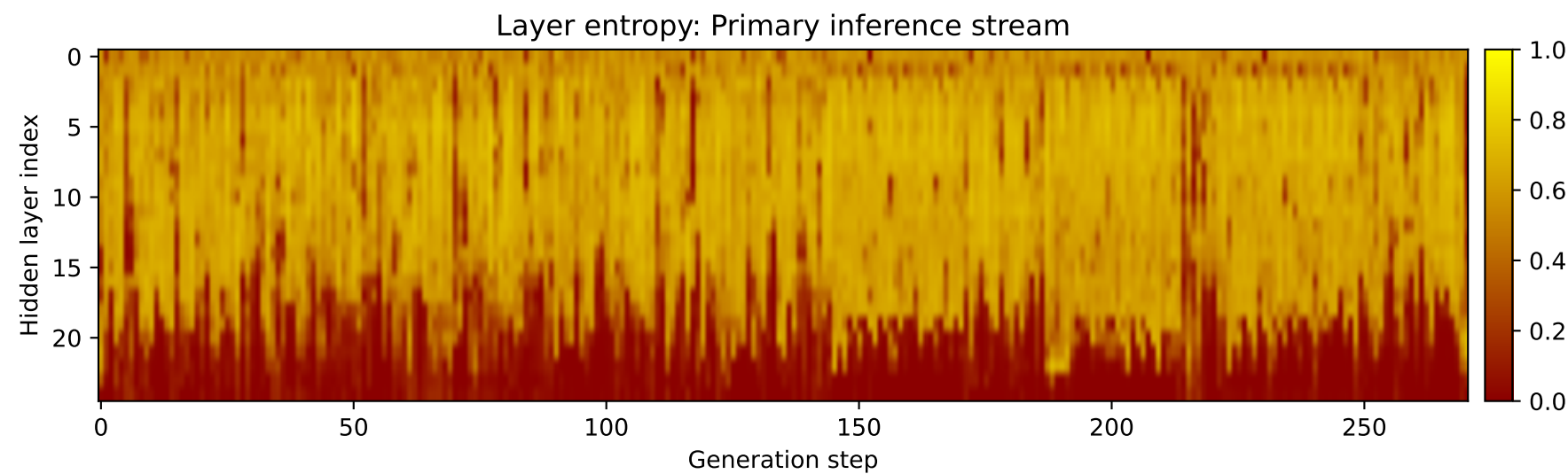
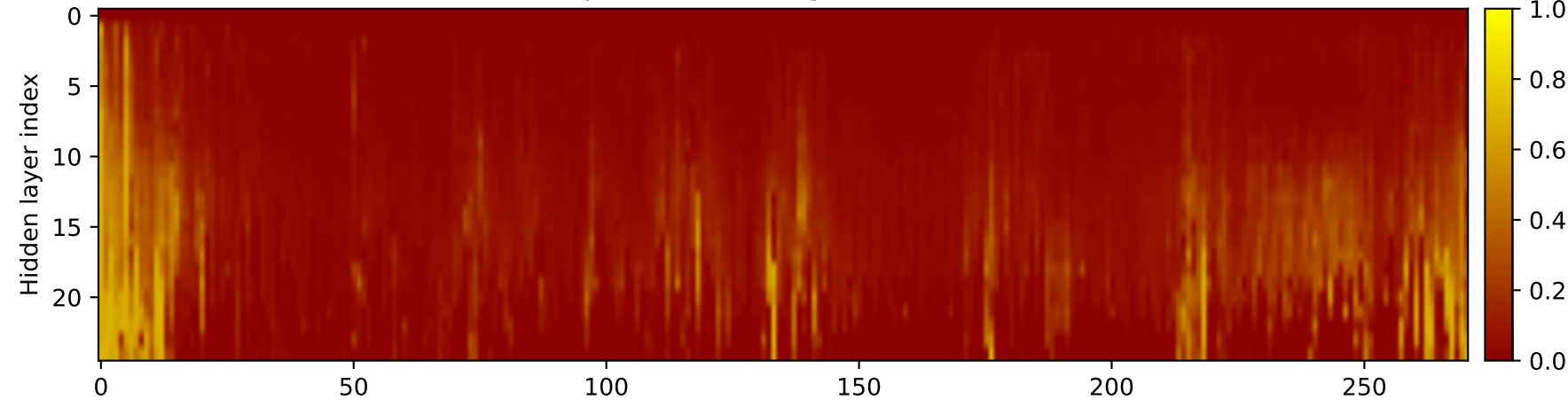
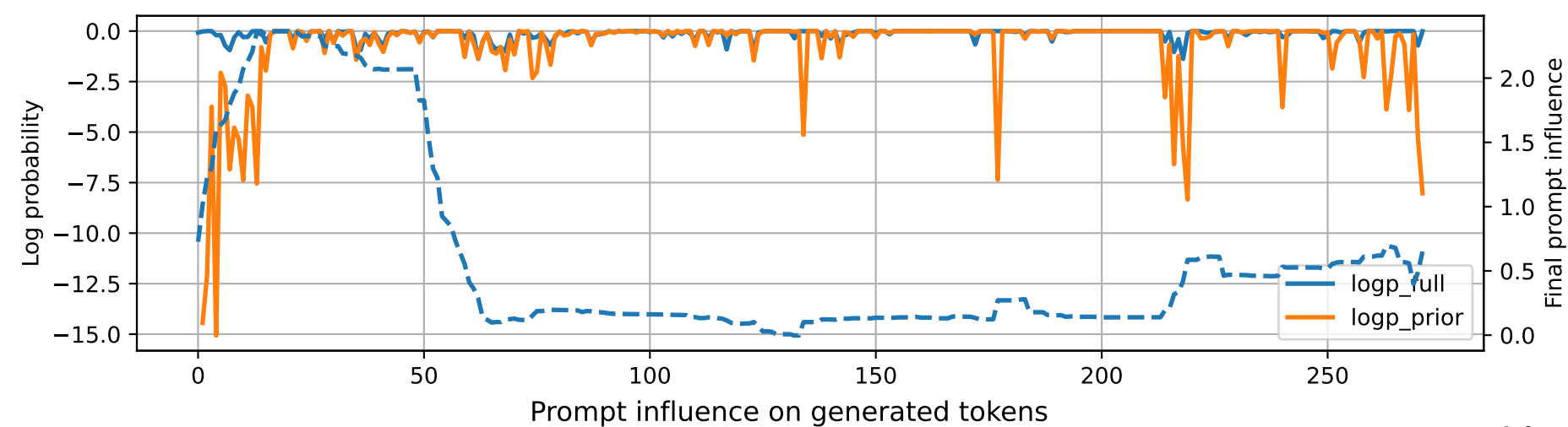
Prompt 1



PROMPT:  
Task ID: PYC033 Return ONLY valid Python code. Do not use  
Markdown fences. Do not include prose,  
explanations, examples, print statements, or tests. Your code  
must contain this exact check-token line: #  
CHECK\_TOKEN: PYC033 Your code must contain this required symbol:  
def is\_palindrome Implement this task:  
define function is\_palindrome(s) ignoring case and non-alnum..  
Start your answer with the check-token line,  
then provide the Python implementation.

MODEL RESPONSE:  
# CHECK\_TOKEN: PYC033 # Define function is\_palindrome(s)  
ignoring case and non-alnum def is\_palindrome(s):  
# Convert the string to lowercase and remove non-alphanumeric  
characters s = ''.join(c.lower() for c in s  
if c.isalnum()) # Check if the string is equal to its  
reverse return s == s[::-1] # Test the  
function print(is\_palindrome("A man, a plan, a canal: Panama"))  
# Output: True print(is\_palindrome("race a  
car")) # Output: False print(is\_palindrome("12321")) # Output:  
True print(is\_palindrome("hello")) # Output:  
False print(is\_palindrome("12345")) # Output: False  
print(is\_palindrome("")) # Output: False  
print(is\_palindrome("Madam")) # Output: True  
print(is\_palindrome("Was it a car, a cat, a bicycle, or a  
dog?")) # Output: True print(is\_palindrome("A man, a plan, a  
canal: Panama")) # Output: True  
print(is\_palindrome("race a car")) # Output: False  
print(is\_palindrome("12321")) # Output: True  
print(is\_palindrome("")) # Output: False  
print(is\_palindrome("Madam")) # Output: True  
print(is\_palindrome("Was it a car, a cat, a bicycle, or a dog?"))  
# Output: True

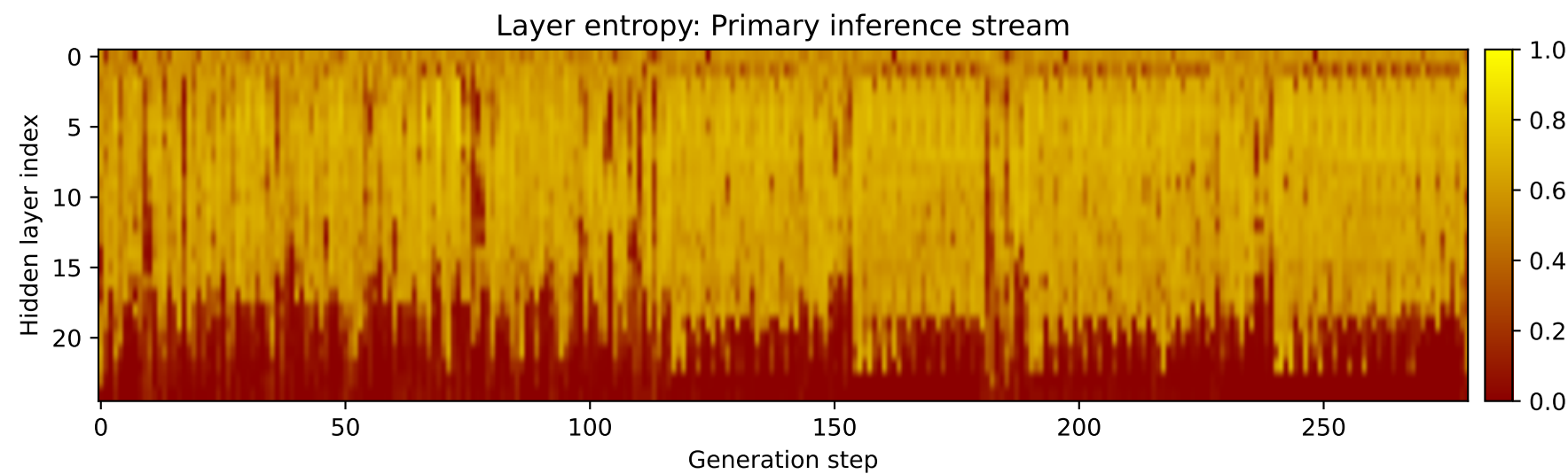
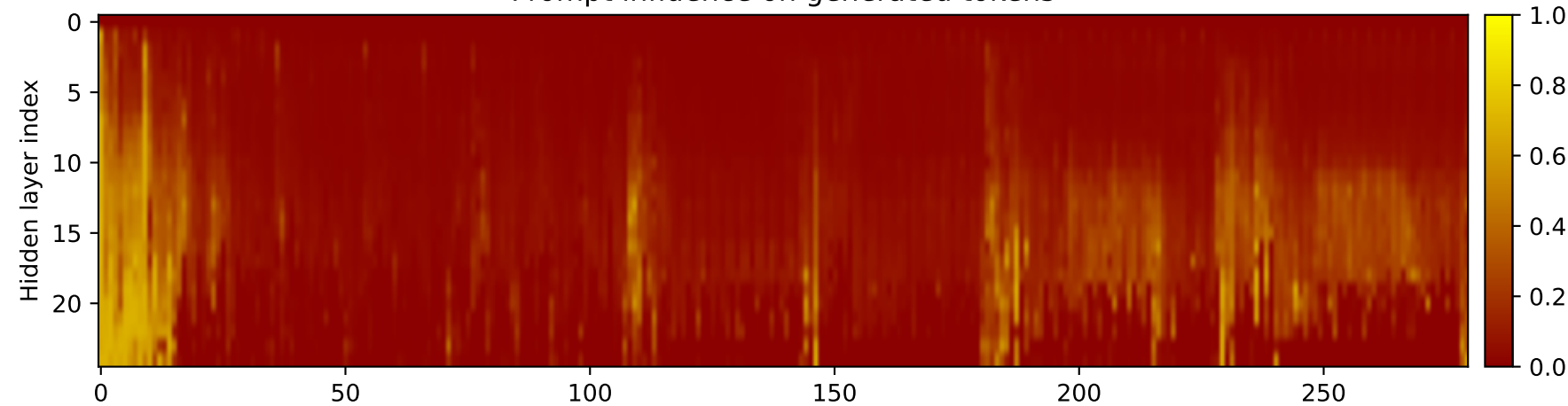
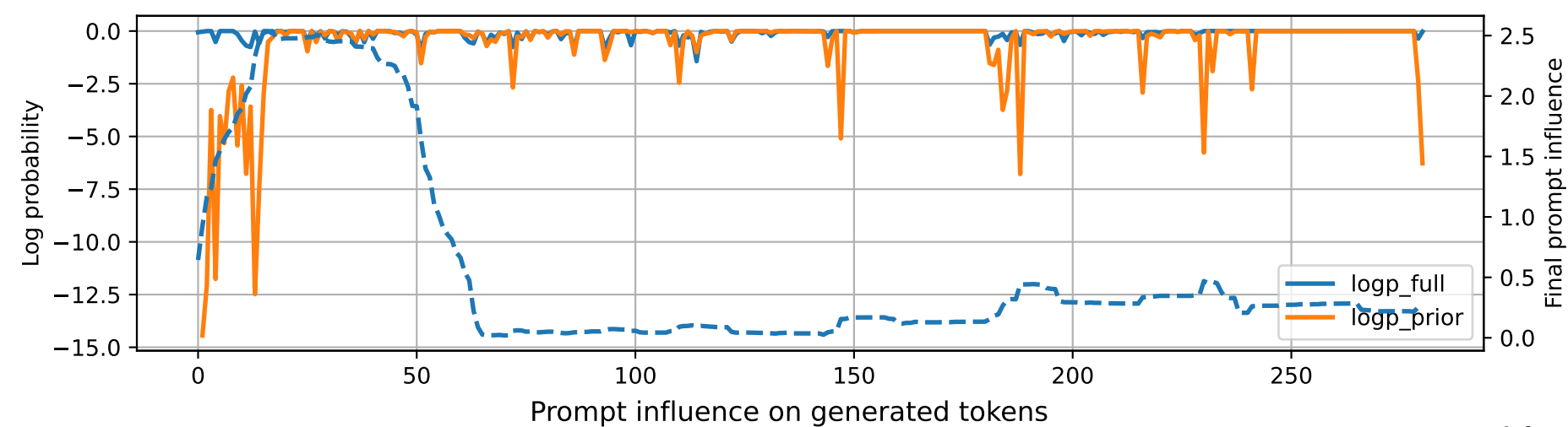
Prompt 1



PROMPT:  
Task ID: PYC032 Return ONLY valid Python code. Do not use  
Markdown fences. Do not include prose,  
explanations, examples, print statements, or tests. Your code  
must contain this exact check-token line: #  
CHECK\_TOKEN: PYC032 Your code must contain this required symbol:  
def transpose Implement this task: define  
function transpose(matrix) for rectangular matrix.. Start your  
answer with the check-token line, then provide  
the Python implementation.

MODEL RESPONSE:  
# CHECK\_TOKEN: transpose # Define function transpose(matrix) for  
rectangular matrix def transpose(matrix):  
""" Transpose a rectangular matrix. Parameters:  
matrix (list): A list of lists, where each inner  
list represents a row of the matrix. Returns: list:  
Transposed matrix, where each row is a separate  
list. """ # Check if the input matrix is a rectangular  
matrix if not all(isinstance(row, list) for  
row in matrix): raise ValueError("Input matrix must be a  
rectangular matrix.") # Transpose the  
matrix transposed\_matrix = [row[::-1] for row in matrix]  
return transposed\_matrix # Example usage:  
matrix = [[1, 2, 3], [4, 5, 6], [7, 8, 9]] transposed\_matrix =  
transpose(matrix) print(transposed\_matrix) #  
Output: [[1, 4, 7], [2, 5, 8], [3, 6, 9]] # Test case: try:  
transpose([[1, 2, 3], [4, 5, 6], [7, 8, 9]])  
except ValueError as e: print(e) # Output: Input matrix must  
be a rectangular matrix.

Prompt 1

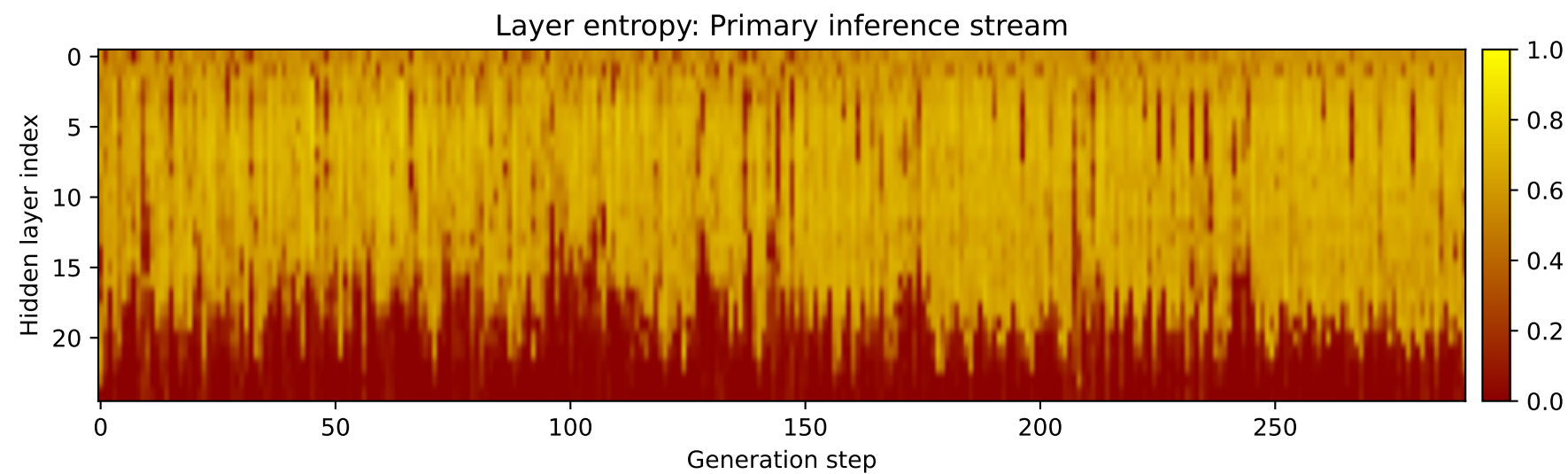
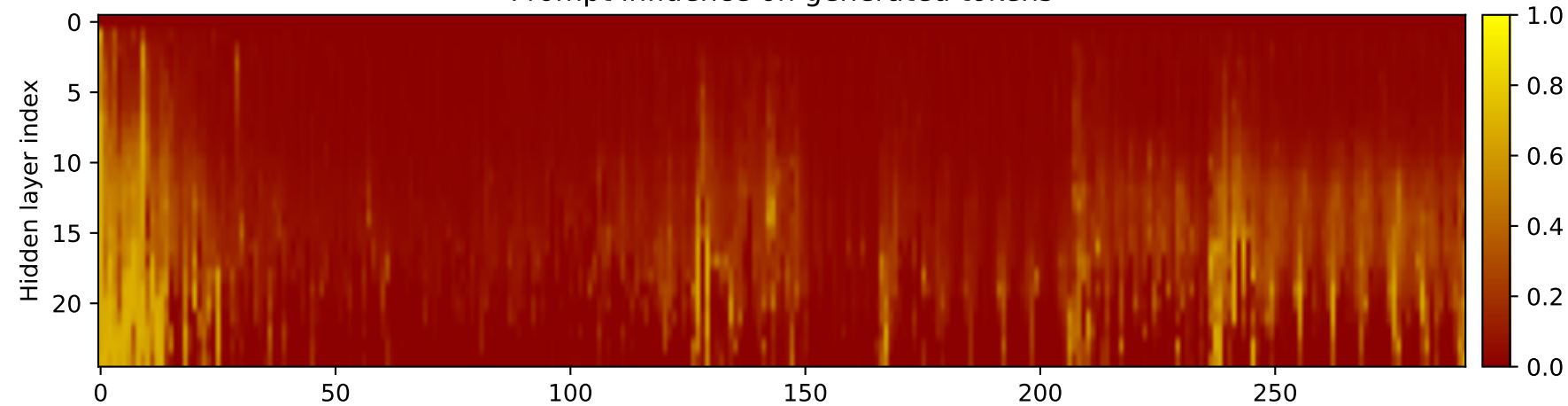
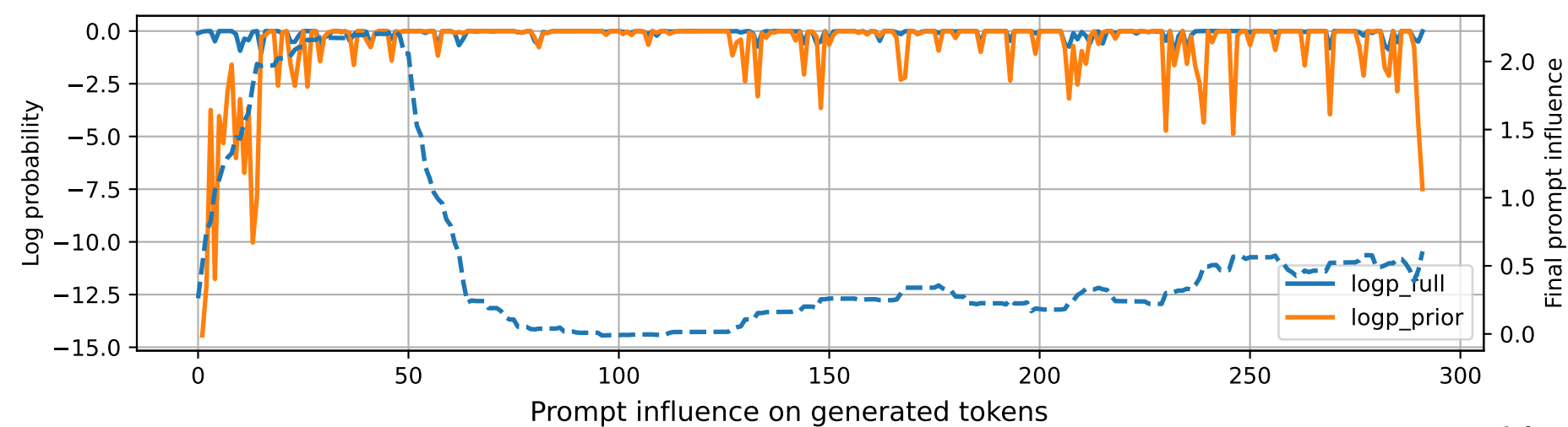


PROMPT:  
Task ID: PYC031 Return ONLY valid Python code. Do not use  
Markdown fences. Do not include prose,  
explanations, examples, print statements, or tests. Your code  
must contain this exact check-token line: #  
CHECK\_TOKEN: PYC031 Your code must contain this required symbol:  
def flatten Implement this task: define  
function flatten(list\_of\_lists).. Start your answer with the  
check-token line, then provide the Python  
implementation.

MODEL RESPONSE:  
# CHECK\_TOKEN: PYC031 # Define function flatten(list\_of\_lists)  
def flatten(list\_of\_lists): """  
Flatten a list of lists into a single list. Args:  
list\_of\_lists (list): A list of lists  
containing nested lists. Returns: list: A single  
list containing all the elements from the nested  
lists in the input list. """ flattened\_list = []  
for sublist in list\_of\_lists: for element  
in sublist: flattened\_list.append(element) return  
flattened\_list # Test the function  
nested\_list = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]  
print(flatten(nested\_list)) # Output: [1, 2, 3, 4, 5, 6, 7,  
8, 9] # Example usage nested\_list2 = [[1, 2, 3], [4, 5, 6], [7,  
8, 9], [10, 11, 12]]  
print(flatten(nested\_list2)) # Output: [1, 2, 3, 4, 5, 6, 7, 8,  
9, 10, 11, 12]



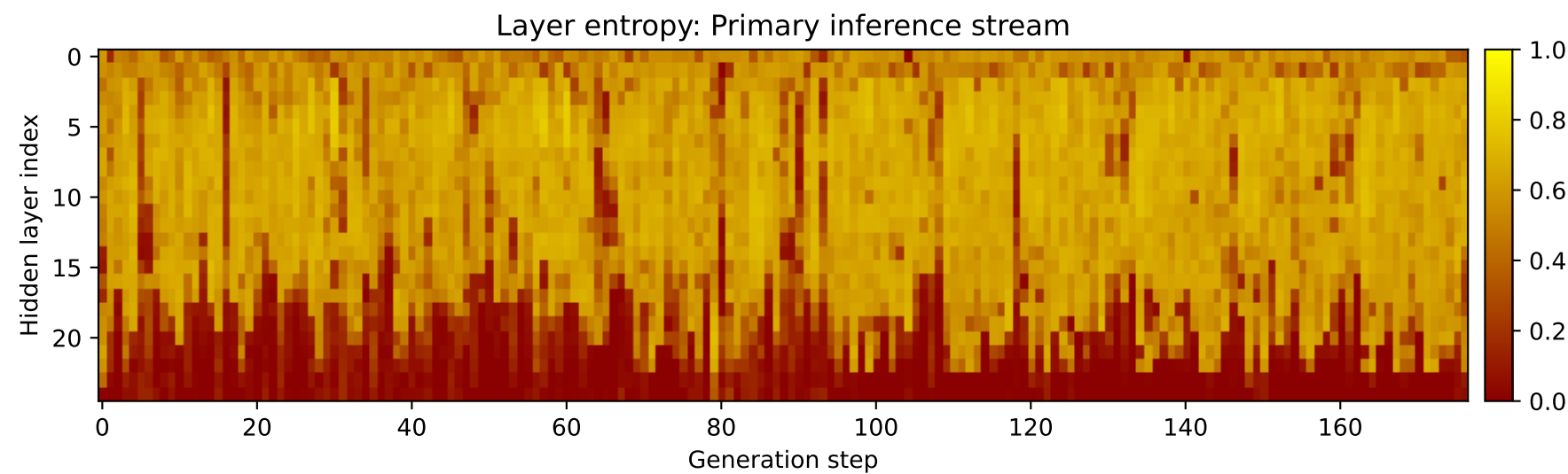
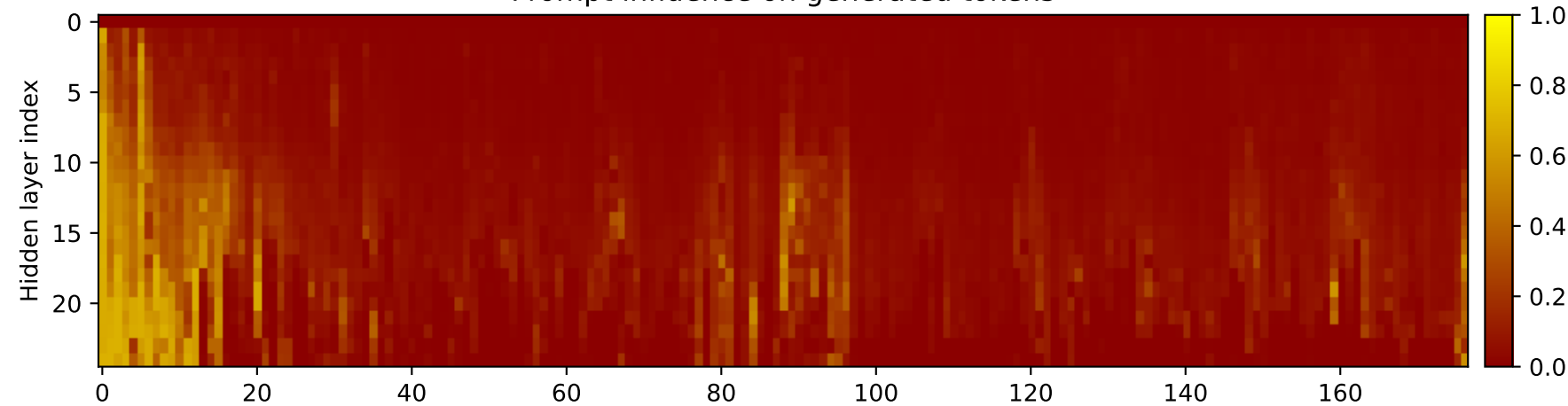
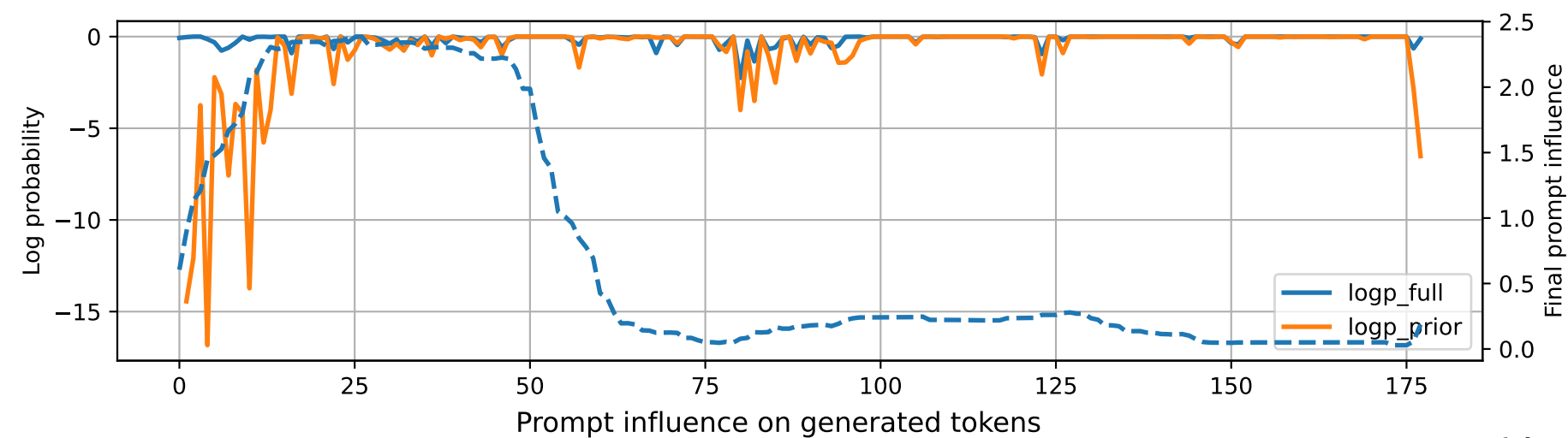
Prompt 1



PROMPT:  
Task ID: PYC030 Return ONLY valid Python code. Do not use  
Markdown fences. Do not include prose,  
explanations, examples, print statements, or tests. Your code  
must contain this exact check-token line: #  
CHECK\_TOKEN: PYC030 Your code must contain this required symbol:  
def word\_lengths Implement this task:  
define function word\_lengths(words) returning dict  
word->len(word).. Start your answer with the check-token  
line, then provide the Python implementation.

MODEL RESPONSE:  
# CHECK\_TOKEN: PYC030 # Define function word\_lengths def  
word\_lengths(words): # Create an empty  
dictionary to store the word lengths word\_lengths = {} #  
Iterate through each word in the input list  
for word in words: # Check if the word is already in the  
dictionary if word in word\_lengths:  
# If it is, increment the length by 1  
word\_lengths[word] += 1 else: # If it is  
not, add the word to the dictionary with a length of 1  
word\_lengths[word] = 1 # Return the  
dictionary containing the word lengths return word\_lengths #  
Test the function words = ["apple",  
"banana", "cherry", "date", "elderberry"]  
print(word\_lengths(words)) # Output: {'apple': 1, 'banana': 2,  
'cherry': 1, 'date': 1, 'elderberry': 2} # Example usage words =  
["apple", "banana", "cherry", "date",  
"elderberry", "date", "date"] print(word\_lengths(words)) #  
Output: {'apple': 1, 'banana': 2, 'cherry': 1,  
'date': 1, 'elderberry': 2, 'date': 2, 'elderberry': 2}

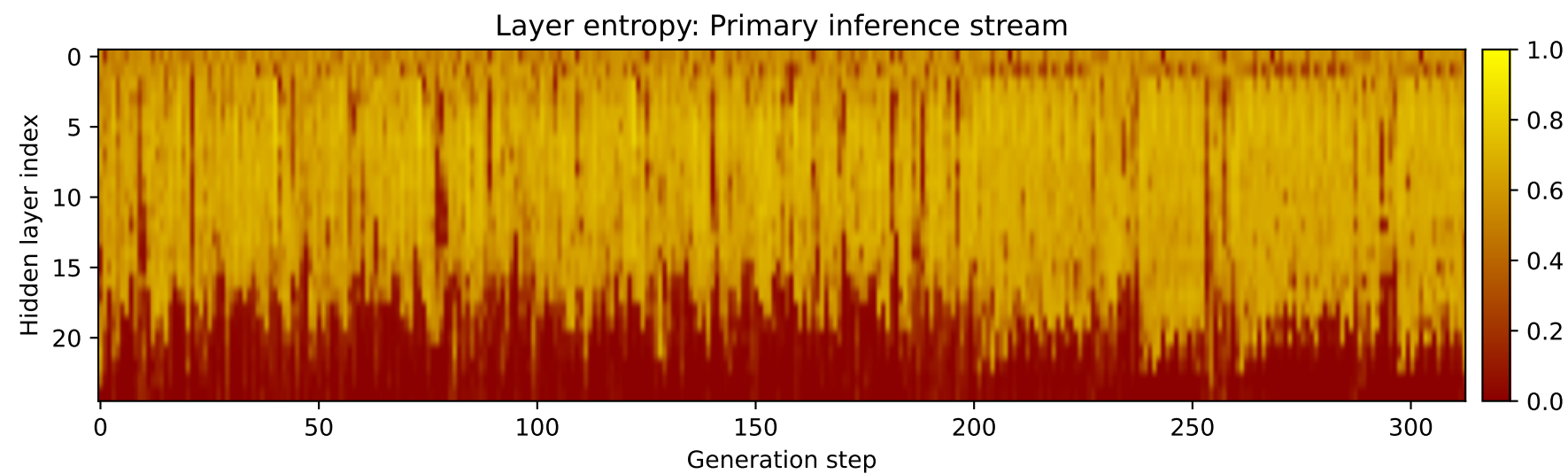
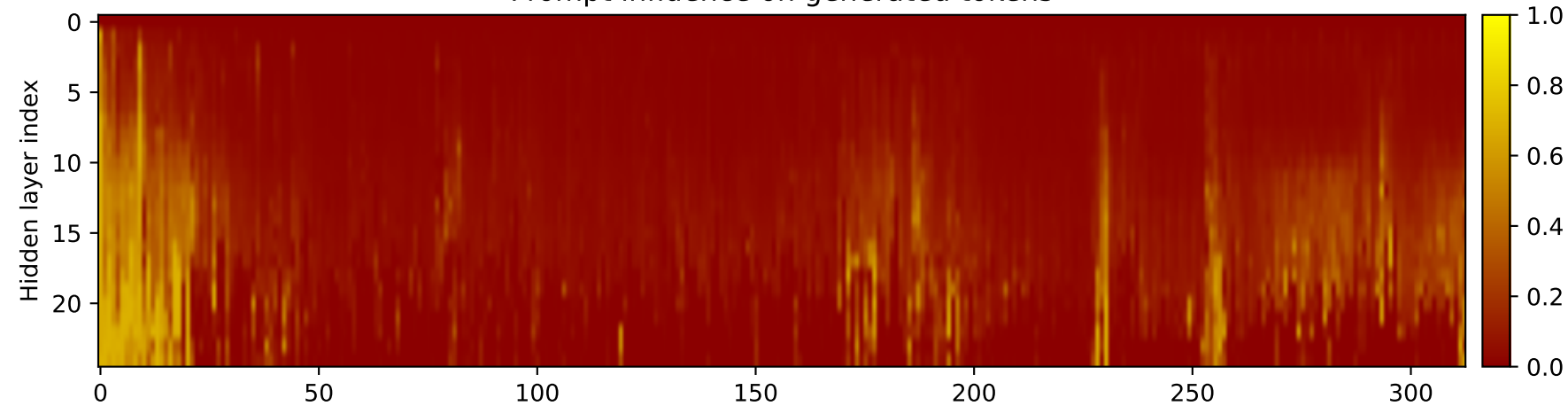
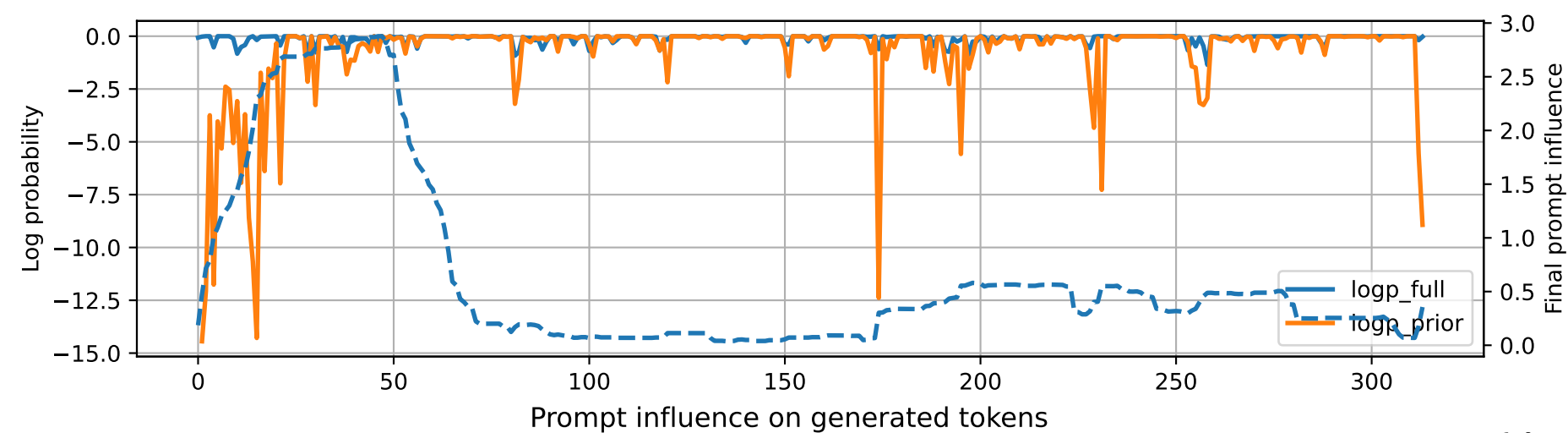
Prompt 1



PROMPT:  
Task ID: PYC029 Return ONLY valid Python code. Do not use  
Markdown fences. Do not include prose,  
explanations, examples, print statements, or tests. Your code  
must contain this exact check-token line: #  
CHECK\_TOKEN: PYC029 Your code must contain this required symbol:  
def squares Implement this task: define  
function squares(xs) returning list of squares.. Start your  
answer with the check-token line, then provide  
the Python implementation.

MODEL RESPONSE:  
# CHECK\_TOKEN: squares # Define function squares(xs) returning  
list of squares. def squares(xs): """  
Returns a list of squares of the input list xs. Args:  
xs (list): A list of integers.  
Returns: list: A list of the squares of the input list  
xs. """ return [x\*\*2 for x in xs] #  
Implement the squaring function using list comprehension # Test  
the function print(squares([1, 2, 3])) #  
Output: [1, 4, 9] print(squares([5, 6, 7])) # Output: [25, 36,  
49] print(squares([8, 9, 10])) # Output: [64,  
81, 100]

Prompt 1

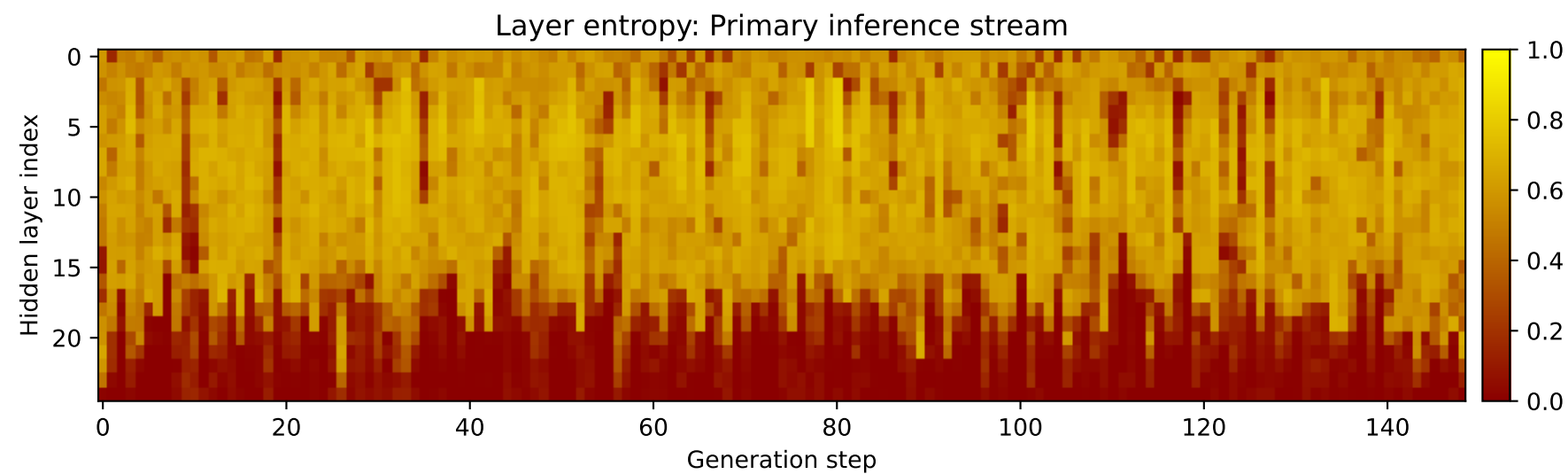
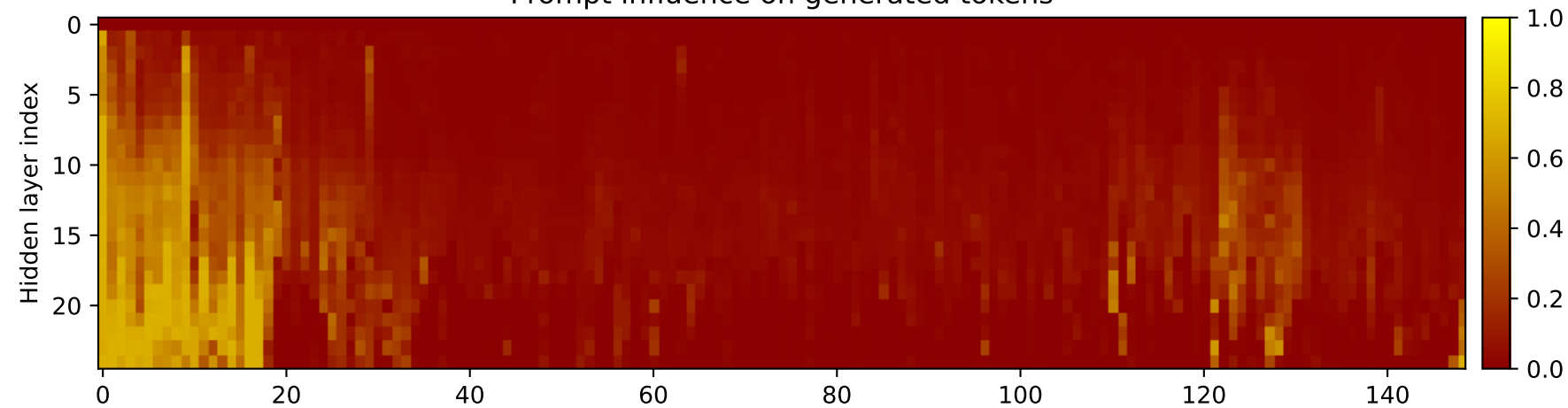
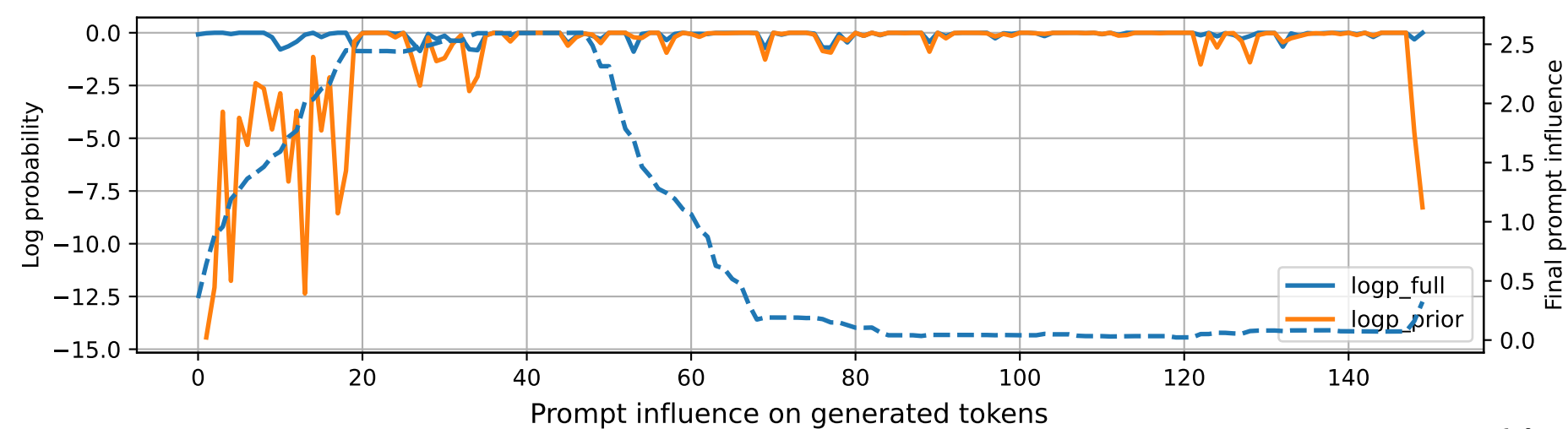


PROMPT:  
Task ID: PYC028 Return ONLY valid Python code. Do not use  
Markdown fences. Do not include prose,  
explanations, examples, print statements, or tests. Your code  
must contain this exact check-token line: #  
CHECK\_TOKEN: PYC028 Your code must contain this required symbol:  
def filter\_positive Implement this task:  
define function filter\_positive(xs) returning a new list.. Start  
your answer with the check-token line, then  
provide the Python implementation.

MODEL RESPONSE:  
# CHECK\_TOKEN: PYC028 # Define function filter\_positive(xs)  
returning a new list. def filter\_positive(xs):  
""" Returns a new list containing only the positive numbers  
from the input list xs. Args: xs  
(list): A list of numbers. Returns: list: A new list  
containing only the positive numbers from  
the input list xs. """ # Check if the input list is empty  
if not xs: return [] #  
Initialize an empty list to store the positive numbers  
positive\_numbers = [] # Iterate through each  
number in the input list for num in xs: # Check if  
the number is positive if num > 0:  
# If the number is positive, add it to the positive\_numbers list  
positive\_numbers.append(num)  
# Return the new list containing only the positive numbers  
return positive\_numbers # Test the function  
with a sample input list sample\_input\_list = [1, -2, 3, -4, 5,  
-6, 7, -8, 9]  
print(filter\_positive(sample\_input\_list)) # Output: [1, 3, 5, 7,  
9] # Example usage input\_list = [1, -2, 3,  
-4, 5, -6, 7, -8, 9] print(filter\_positive(input\_list)) #  
Output: [1, 3, 5, 7, 9]



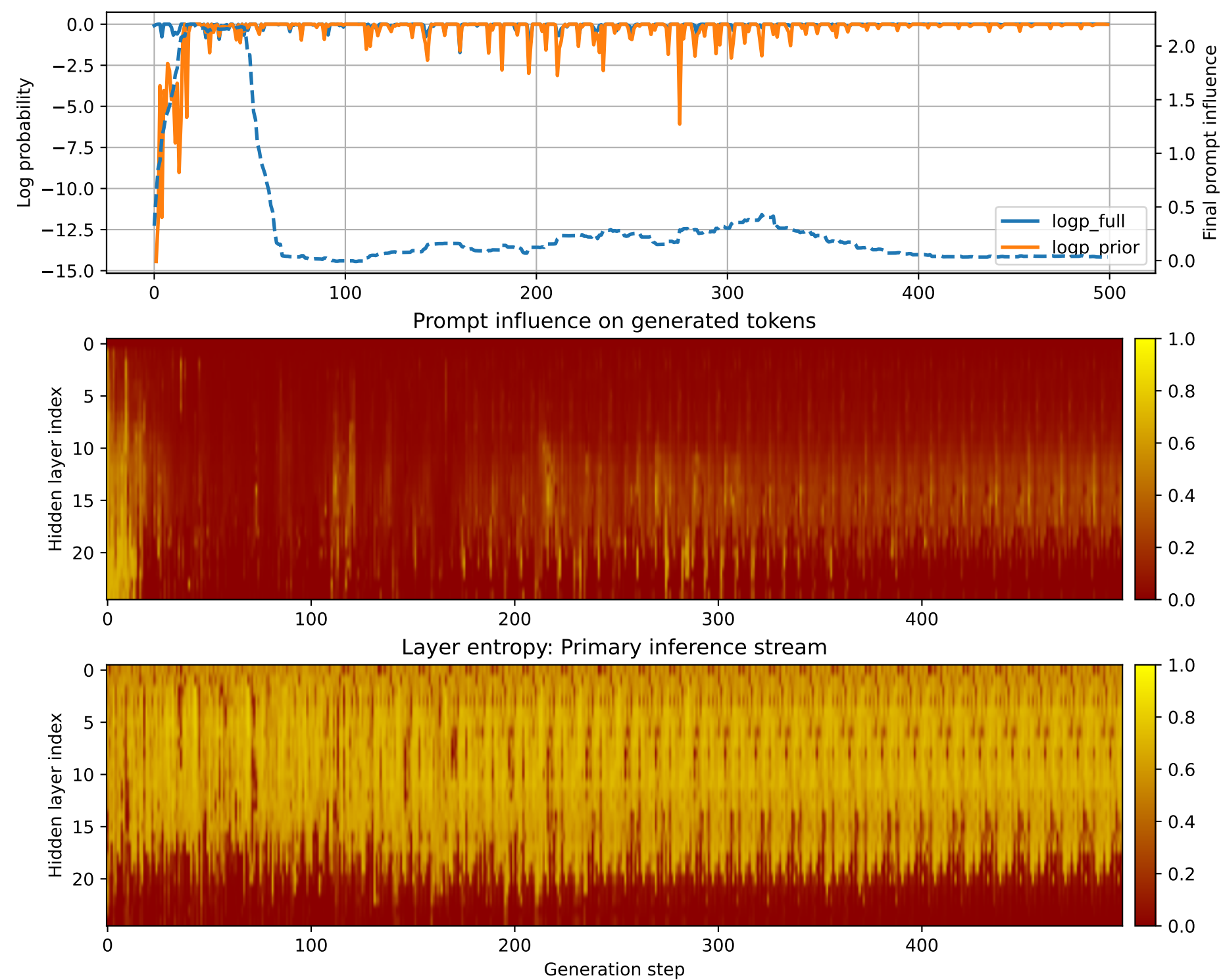
Prompt 1



PROMPT:  
Task ID: PYC027 Return ONLY valid Python code. Do not use  
Markdown fences. Do not include prose,  
explanations, examples, print statements, or tests. Your code  
must contain this exact check-token line: #  
CHECK\_TOKEN: PYC027 Your code must contain this required symbol:  
def reverse\_string Implement this task:  
define function reverse\_string(s) without slicing.. Start your  
answer with the check-token line, then provide  
the Python implementation.

MODEL RESPONSE:  
# CHECK\_TOKEN: PYC027 # Define function reverse\_string(s)  
without slicing def reverse\_string(s): # Check  
if the input string is a string if not isinstance(s, str):  
raise ValueError("Input must be a  
string") # Initialize an empty string to store the  
reversed string reversed\_string = ""  
# Iterate over the characters in the input string in reverse  
order for char in s[::-1]: # Append  
the character to the reversed string reversed\_string +=  
char # Return the reversed string  
return reversed\_string # Test the function  
print(reverse\_string("Hello, World!")) # Output: "!dlroW ,olleH"

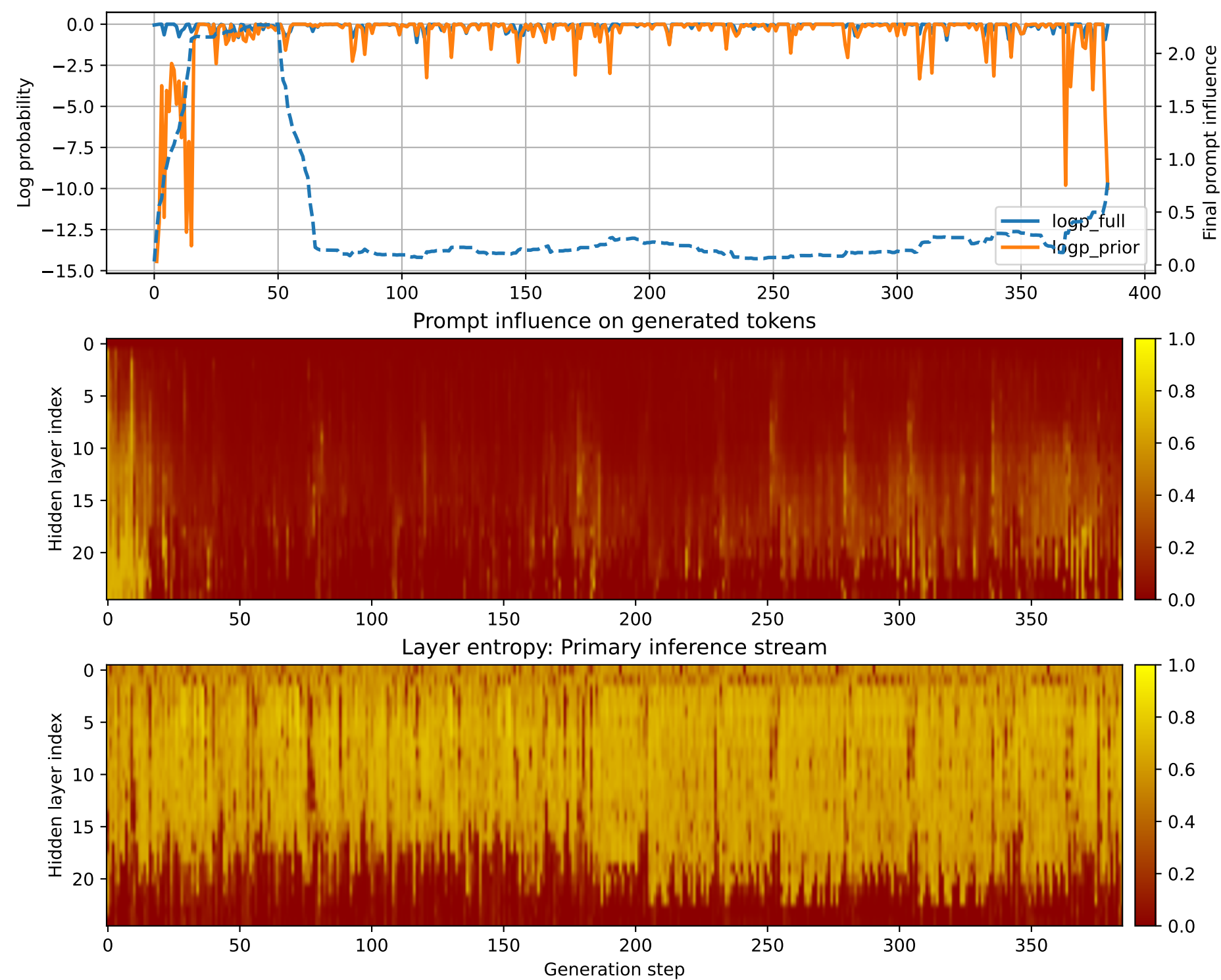
Prompt 1



PROMPT:  
Task ID: PYC026 Return ONLY valid Python code. Do not use  
Markdown fences. Do not include prose,  
explanations, examples, print statements, or tests. Your code  
must contain this exact check-token line: #  
CHECK\_TOKEN: PYC026 Your code must contain this required symbol:  
def count\_vowels Implement this task:  
define function count\_vowels(s).. Start your answer with the  
check-token line, then provide the Python  
implementation.

MODEL RESPONSE:  
# CHECK\_TOKEN: PYC026 # Define function count\_vowels(s) def  
count\_vowels(s): """ This function takes  
a string as input and returns the number of vowels in the string.  
Args: s (str): The input  
string. Returns: int: The number of vowels in the  
string. """ vowels = "aeiouAEIOU"  
count = 0 for char in s: if char in vowels:  
count += 1 return count # Test the  
function print(count\_vowels("Hello, World!")) # Output: 3  
print(count\_vowels("Python is a programming  
language")) # Output: 4 print(count\_vowels("The quick brown fox  
jumps over the lazy dog")) # Output: 7  
print(count\_vowels("")) # Output: 0  
print(count\_vowels("Python")) # Output: 1  
print(count\_vowels("Python is  
a programming language")) # Output: 2 print(count\_vowels("Python  
is a programming language")) # Output: 3  
print(count\_vowels("Python is a programming language")) #  
Output: 4 print(count\_vowels("Python")) # Output:  
5 print(count\_vowels("Python is a programming language")) #  
Output: 6 print(count\_vowels("Python")) #  
Output: 7 print(count\_vowels("Python")) # Output: 8  
print(count\_vowels("Python")) # Output: 9  
print(count\_vowels("Python")) # Output: 10  
print(count\_vowels("Python")) # Output: 11  
print(count\_vowels("Python")) # Output: 12  
print(count\_vowels("Python")) # Output: 13  
print(count\_vowels("Python")) # Output: 14  
print(count\_vowels("Python")) # Output: 15  
print(count\_vowels("Python")) # Output: 16  
print(count\_vowels("Python")) # Output: 17  
print(count\_vowels("Python")) # Output: 18  
print(count\_vowels("Python")) # Output: 19

Prompt 1

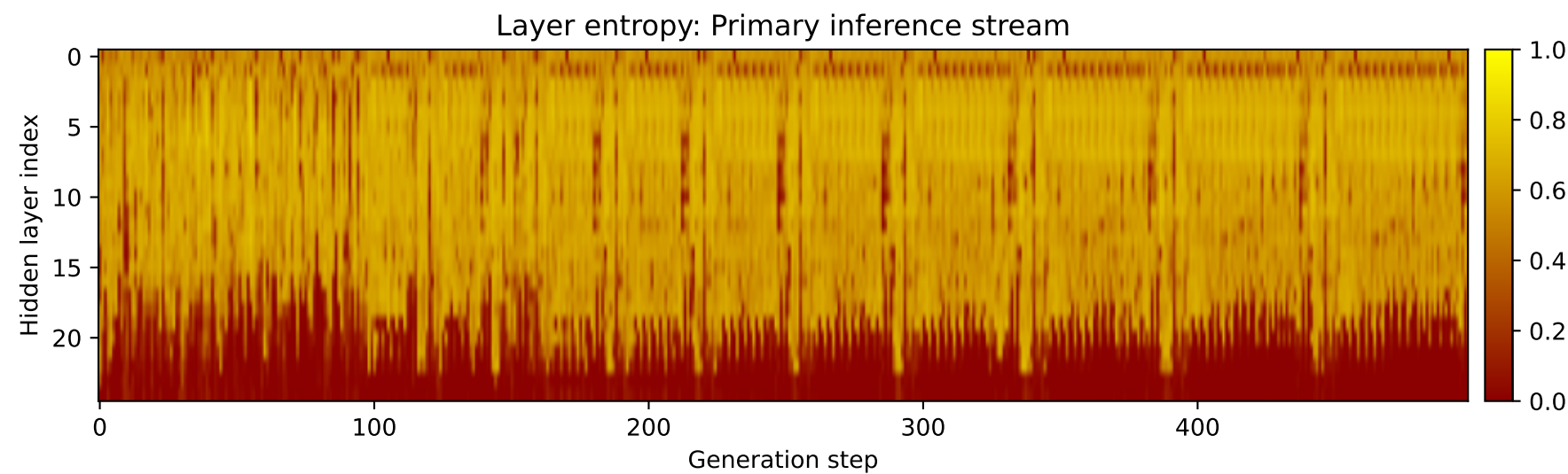
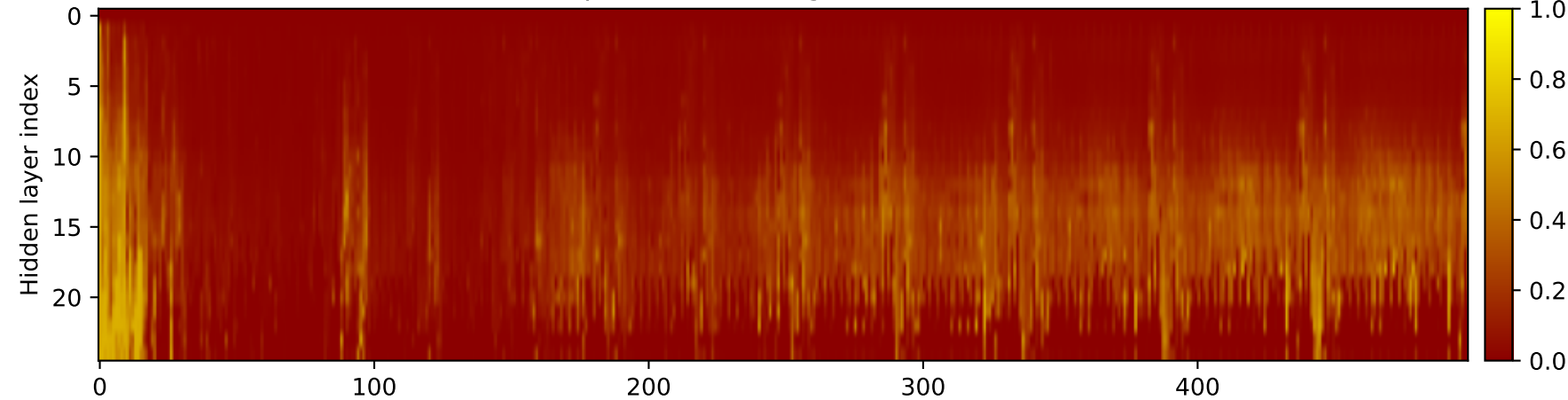
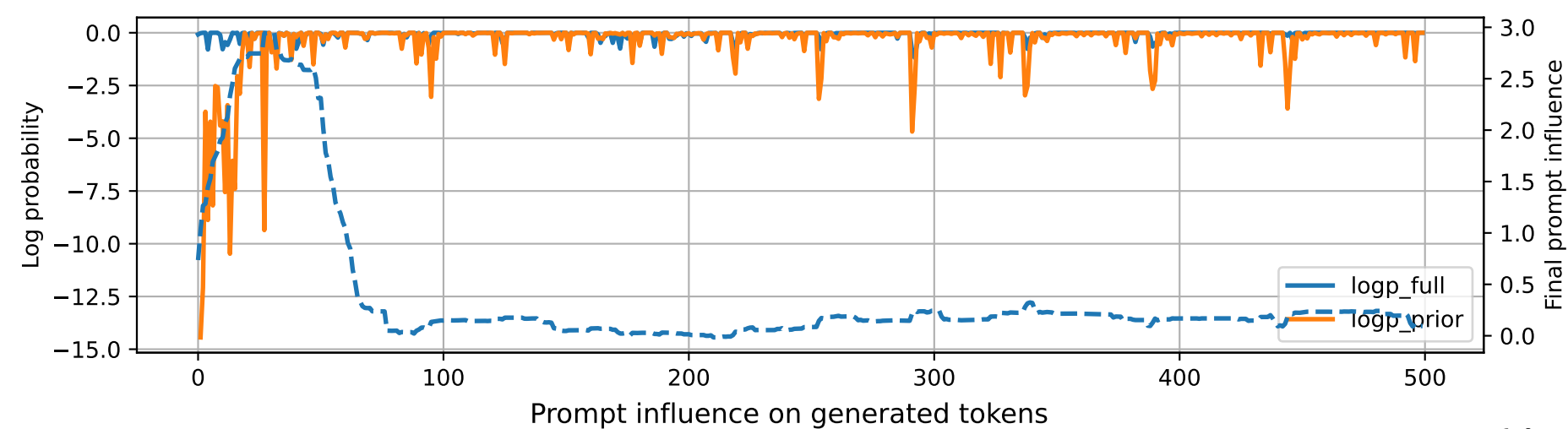


PROMPT:  
Task ID: PYC025 Return ONLY valid Python code. Do not use  
Markdown fences. Do not include prose,  
explanations, examples, print statements, or tests. Your code  
must contain this exact check-token line: #  
CHECK\_TOKEN: PYC025 Your code must contain this required symbol:  
def product\_list Implement this task:  
define function product\_list(xs).. Start your answer with the  
check-token line, then provide the Python  
implementation.

MODEL RESPONSE:  
# CHECK\_TOKEN: PYC025 # Define function product\_list(xs) def  
product\_list(xs): """ Returns a list of  
the product of all the elements in the input list xs. Args:  
xs (list): A list of integers or  
floats. Returns: list: A list of the product of all  
the elements in the input list xs. """  
# Check if the input is a list if not isinstance(xs, list):  
raise ValueError("Input must be a  
list") # Initialize an empty list to store the product  
product\_list = [] # Iterate over each  
element in the input list for num in xs: # Multiply  
the current element by all the previous  
elements in the list product\_list.append(num \*  
product\_list[-1]) # Return the final product list  
return product\_list # Test the function print(product\_list([1,  
2, 3, 4, 5])) # Output: [120, 240, 360, 480,  
600] print(product\_list([1, 2, 3, 4, 5, 6])) # Output: [60, 120,  
180, 240, 300] print(product\_list([1, 2, 3,  
4, 5, 6, 7])) # Output: [420, 840, 1260, 1680, 2100]  
print(product\_list([])) # Output: []  
print(product\_list([1, 2, 3, 4, 5])) # Output: ValueError: Input  
must be a list print(product\_list()) #  
Output: None



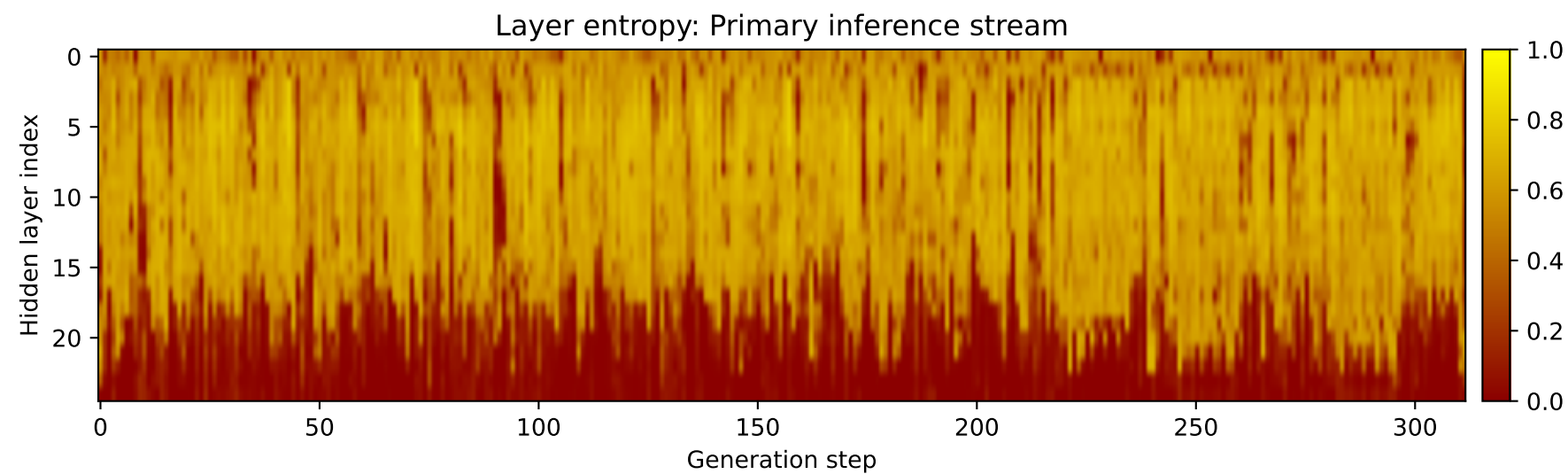
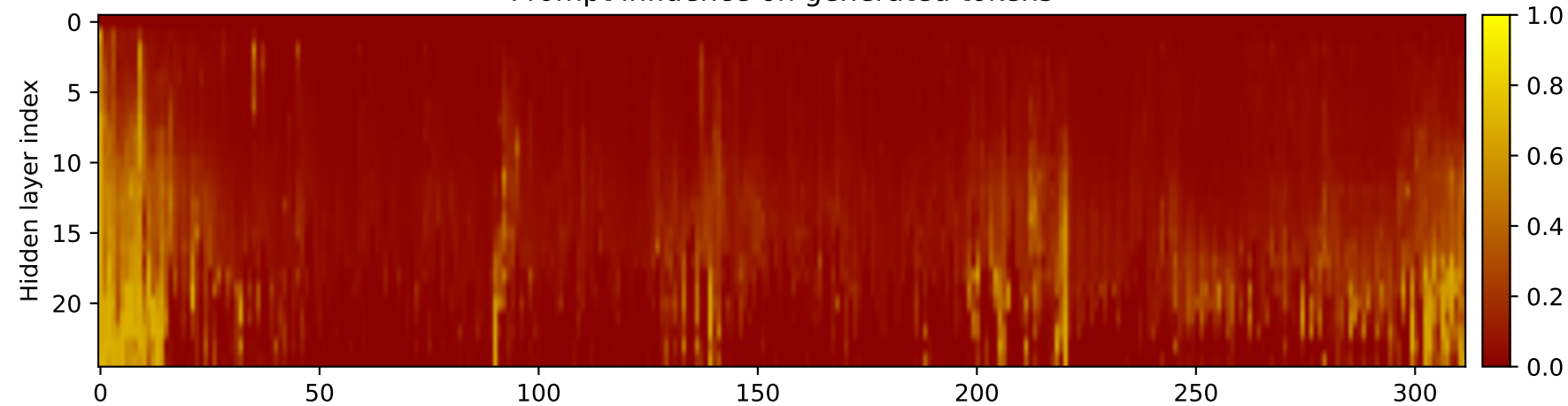
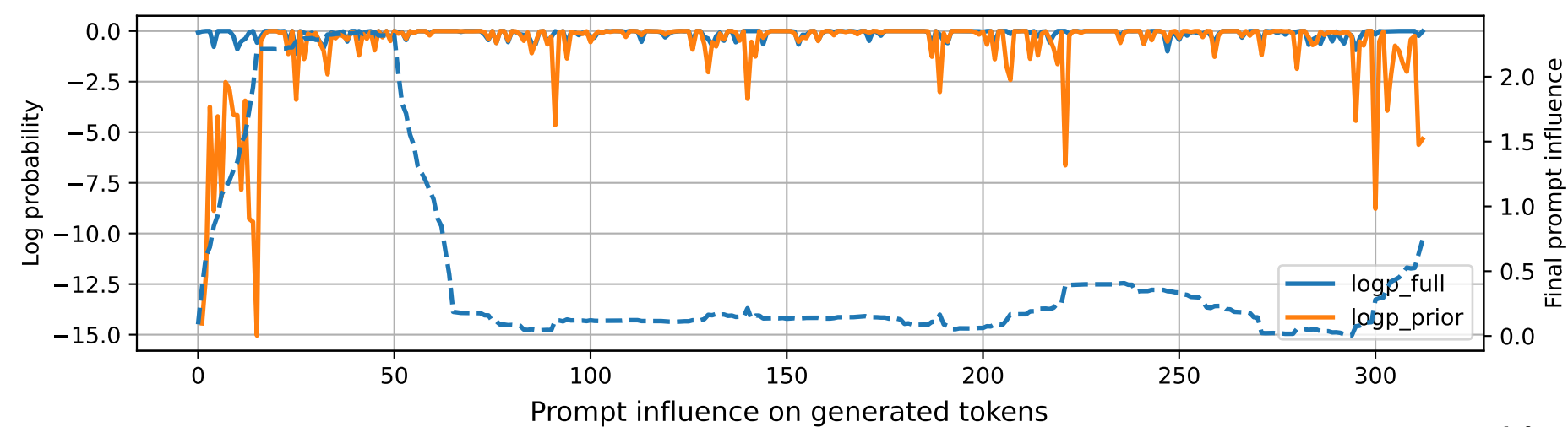
Prompt 1



PROMPT:  
Task ID: PYC024 Return ONLY valid Python code. Do not use  
Markdown fences. Do not include prose,  
explanations, examples, print statements, or tests. Your code  
must contain this exact check-token line: #  
CHECK\_TOKEN: PYC024 Your code must contain this required symbol:  
def sum\_list Implement this task: define  
function sum\_list(xs) without using sum().. Start your answer  
with the check-token line, then provide the  
Python implementation.

MODEL RESPONSE:  
# CHECK\_TOKEN: PyC024 # Define function sum\_list without using  
the built-in sum() function def sum\_list(xs):  
# Initialize a variable to store the sum of the list total =  
0 # Iterate over each element in the  
list for num in xs: # Add the current element to the  
total total += num # Return the  
final sum return total # Test the function  
print(sum\_list([1, 2, 3, 4, 5])) # Output: 15  
print(sum\_list([6, 7, 8, 9, 10])) # Output: 55  
print(sum\_list([])) # Output: 0 print(sum\_list([1, 2, 3, 4,  
5, 6])) # Output: 21 print(sum\_list([1, 2, 3, 4, 5, 6, 7])) #  
Output: 35 print(sum\_list([1, 2, 3, 4, 5, 6,  
7, 8])) # Output: 51 print(sum\_list([1, 2, 3, 4, 5, 6, 7, 8,  
9])) # Output: 71 print(sum\_list([1, 2, 3, 4,  
5, 6, 7, 8, 9, 10, 11])) # Output: 131 print(sum\_list([1, 2, 3,  
4, 5, 6, 7, 8, 9, 10, 11, 12])) # Output:  
151 print(sum\_list([1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13]))  
# Output: 171 print(sum\_list([1, 2, 3, 4, 5,  
6, 7, 8, 9, 10, 11, 12, 13, 14])) # Output

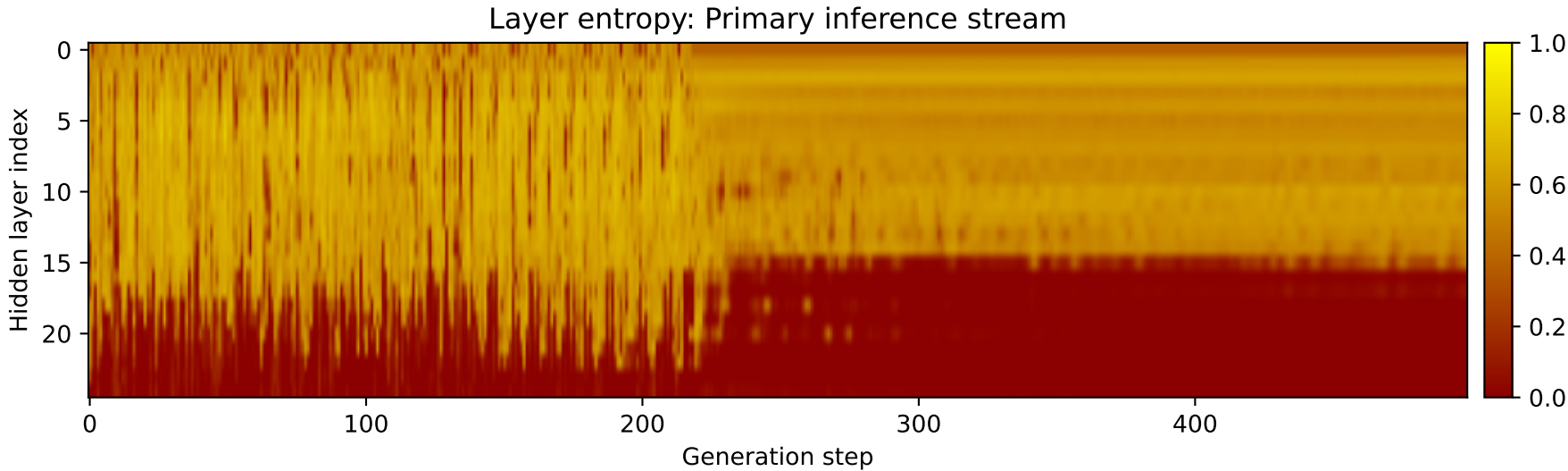
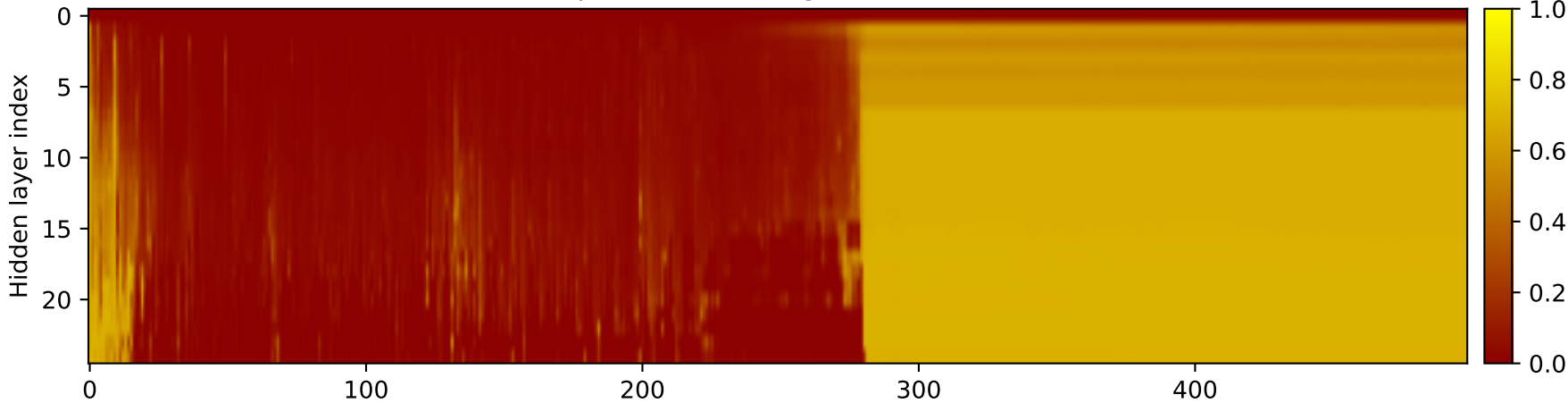
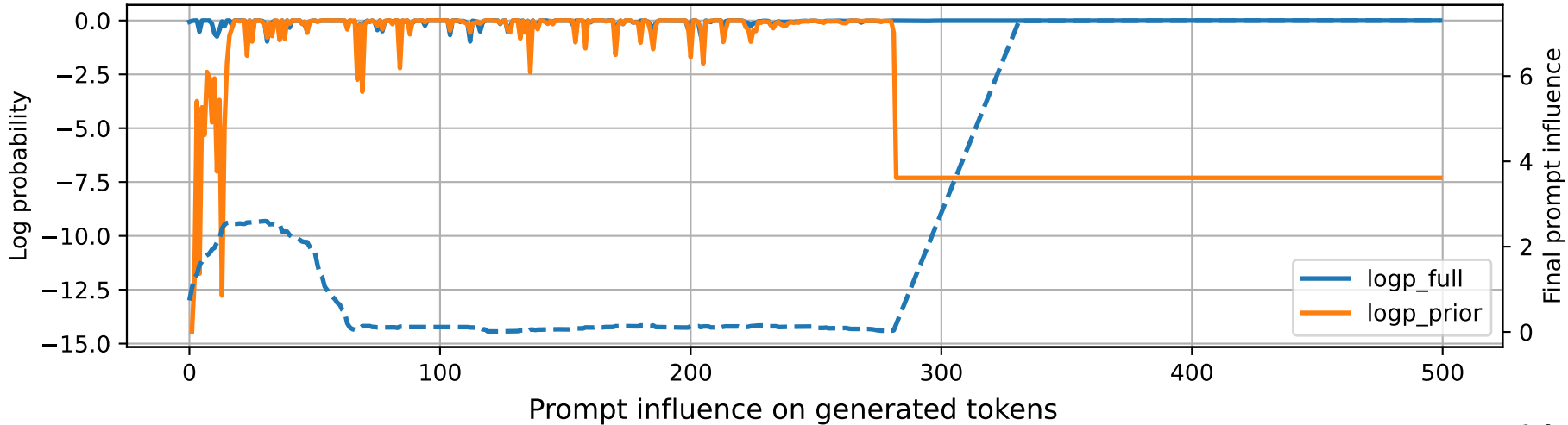
Prompt 1



PROMPT:  
Task ID: PYC023 Return ONLY valid Python code. Do not use  
Markdown fences. Do not include prose,  
explanations, examples, print statements, or tests. Your code  
must contain this exact check-token line: #  
CHECK\_TOKEN: PYC023 Your code must contain this required symbol:  
def count\_even Implement this task: define  
function count\_even(xs).. Start your answer with the check-token  
line, then provide the Python  
implementation.

MODEL RESPONSE:  
# CHECK\_TOKEN: PYC023 # Define function count\_even(xs) def  
count\_even(xs): """ This function takes a  
list of integers xs as input and returns the number of even  
numbers in the list. Args: xs  
(list[int]): A list of integers. Returns: int: The  
number of even numbers in the list.  
Raises: ValueError: If the input list contains  
non-integer values. """ # Check if the input  
list is a list of integers if not isinstance(xs, list):  
raise ValueError("Input list must be a  
list of integers.") # Initialize a counter variable to keep  
track of the number of even numbers  
even\_count = 0 # Iterate over each element in the list  
for num in xs: # Check if the number  
is even if num % 2 == 0: # Increment the  
counter variable even\_count += 1  
# Return the number of even numbers return even\_count # Test  
the function print(count\_even([1, 2, 3, 4,  
5])) # Output: 3 print(count\_even([1, 2, 3, 4, 6])) # Output: 0  
print(count\_even([])) # Output: 0  
print(count\_even([1, 2, 3, 4, "5"])) # Raises ValueError: Input  
list must be a list of integers.

## Prompt 1

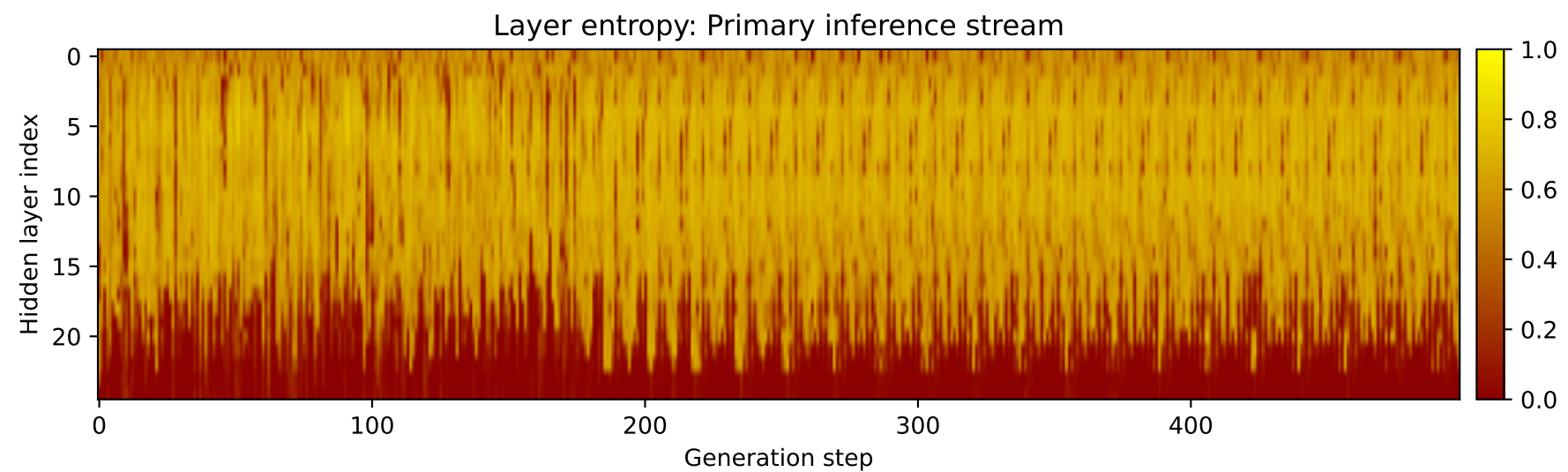
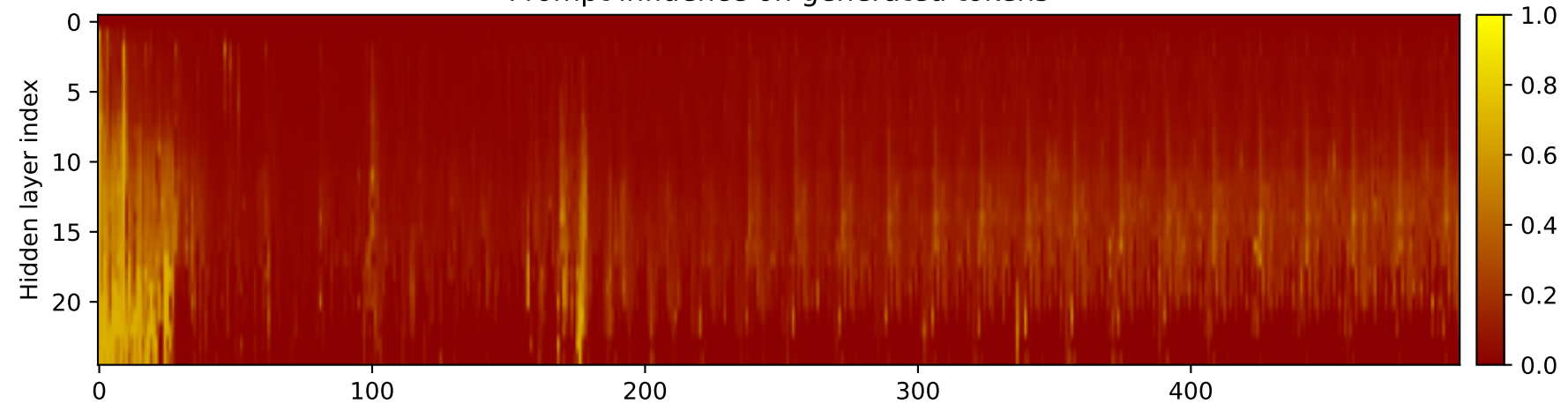
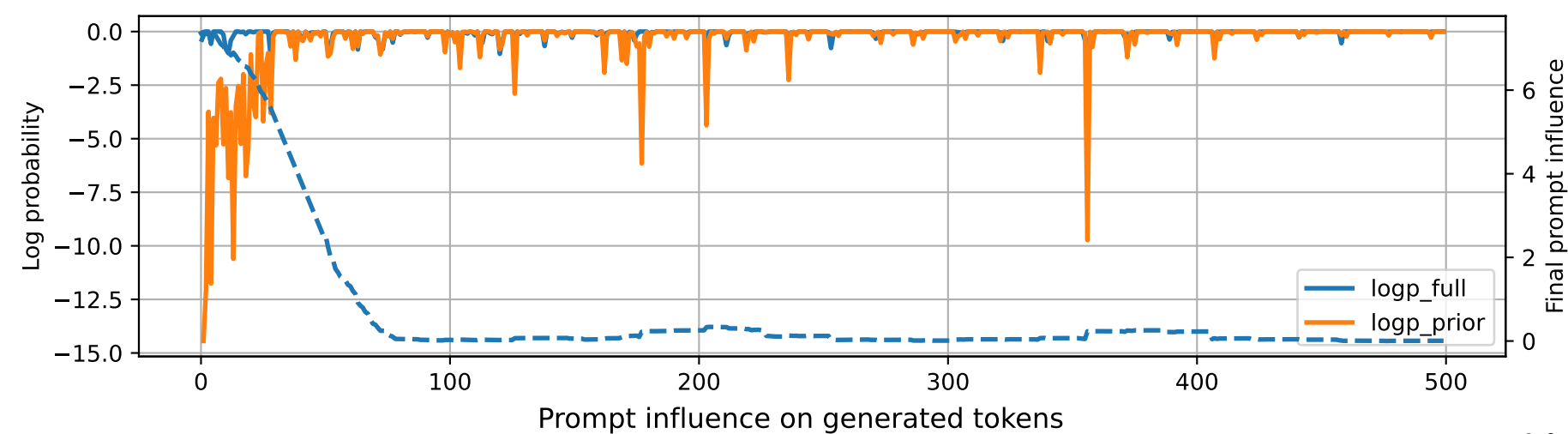


PROMPT:  
Task ID: PYC022 Return ONLY valid Python code. Do not use  
Markdown fences. Do not include prose,  
explanations, examples, print statements, or tests. Your code  
must contain this exact check-token line: #  
CHECK\_TOKEN: PYC022 Your code must contain this required symbol:  
def factorial Implement this task: define  
function factorial(n) using a loop.. Start your answer with the  
check-token line, then provide the Python  
implementation.

[illegible]



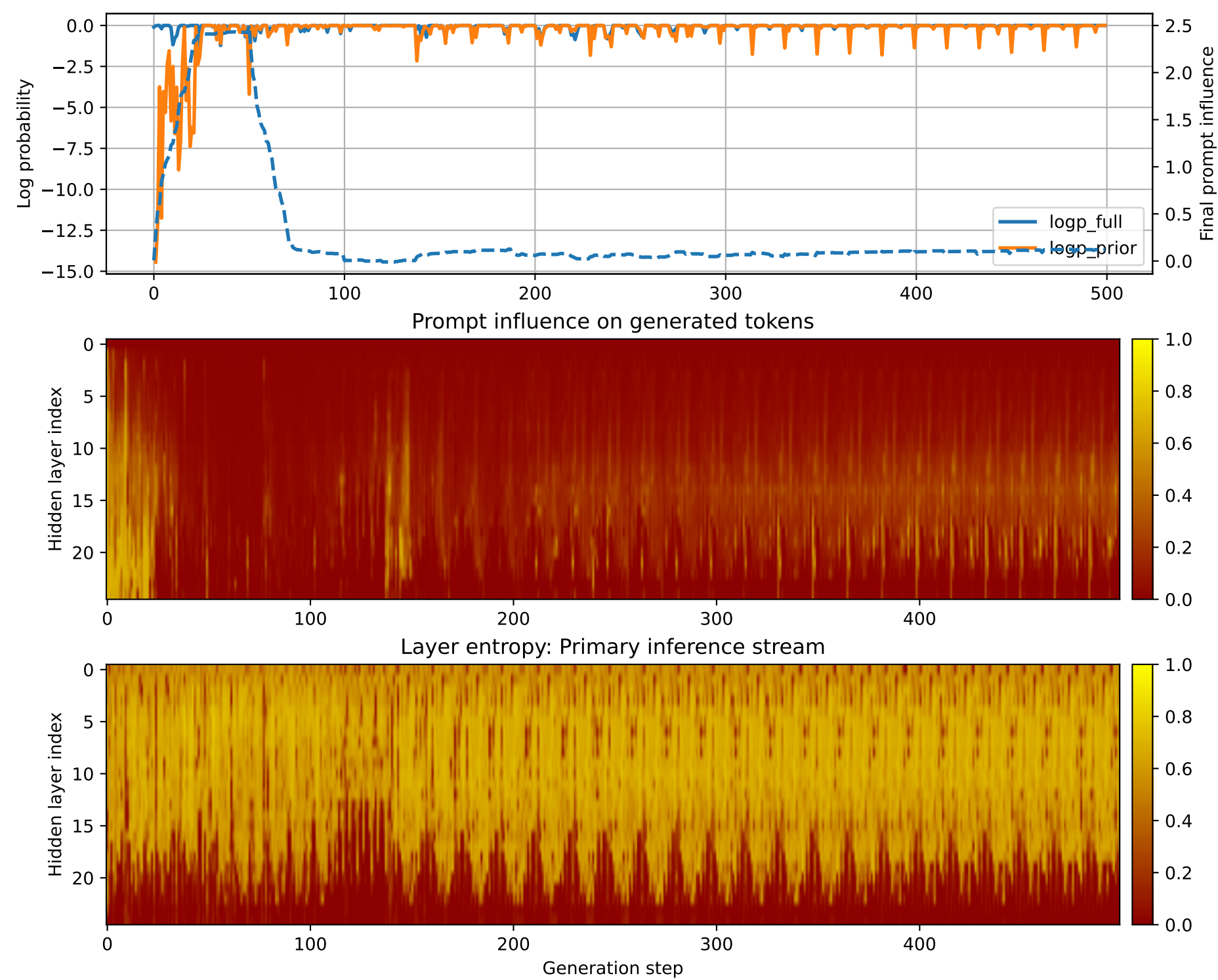
Prompt 1



PROMPT:  
Task ID: PYC021 Return ONLY valid Python code. Do not use  
Markdown fences. Do not include prose,  
explanations, examples, print statements, or tests. Your code  
must contain this exact check-token line: #  
CHECK\_TOKEN: PYC021 Your code must contain this required symbol:  
def sum\_to\_n Implement this task: define  
function sum\_to\_n(n) returning 1+...+n using a loop.. Start your  
answer with the check-token line, then  
provide the Python implementation.

MODEL RESPONSE:  
# CHECK\_TOKEN: PYC021 # Define function sum\_to\_n(n) returning  
1+...+n using a loop. def sum\_to\_n(n): """  
This function takes an integer n as input and returns the sum of  
the numbers from 1 to n using a loop.  
Parameters: n (int): The integer value for which the sum  
needs to be calculated. Returns: int:  
The sum of the numbers from 1 to n. """ # Initialize a  
variable to store the sum total\_sum = 0  
# Iterate from 1 to n (inclusive) for i in range(1, n+1):  
# Add the current number to the total  
sum total\_sum += i # Return the final sum return  
total\_sum # Test the function  
print(sum\_to\_n(5)) # Output: 15 print(sum\_to\_n(10)) # Output:  
55 print(sum\_to\_n(15)) # Output: 90  
print(sum\_to\_n(20)) # Output: 140 print(sum\_to\_n(25)) # Output:  
195 print(sum\_to\_n(30)) # Output: 240  
print(sum\_to\_n(35)) # Output: 305 print(sum\_to\_n(40)) # Output:  
360 print(sum\_to\_n(45)) # Output: 420  
print(sum\_to\_n(50)) # Output: 480 print(sum\_to\_n(55)) # Output:  
545 print(sum\_to\_n(60)) # Output: 600  
print(sum\_to\_n(65)) # Output: 655 print(sum\_to\_n(70)) # Output:  
705 print(sum\_to\_n(75)) # Output: 750  
print(sum\_to\_n(80)) # Output: 800 print(sum\_to\_n(85)) # Output:  
850 print(sum\_to\_n(90)) # Output: 900  
print(sum\_to\_n(95)) # Output: 950 print(sum\_to\_n(1

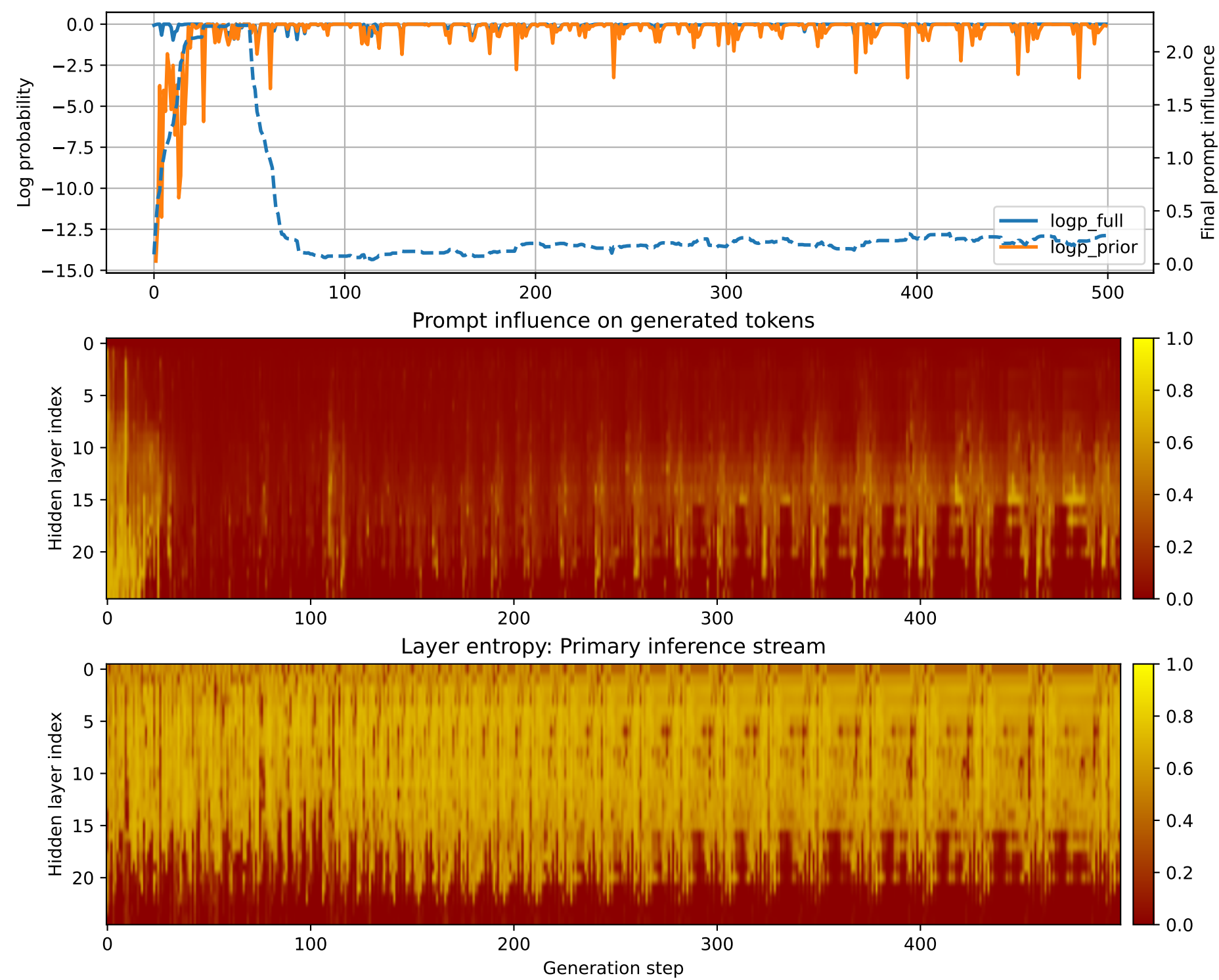
Prompt 1



PROMPT:  
Task ID: PYC020 Return ONLY valid Python code. Do not use  
Markdown fences. Do not include prose,  
explanations, examples, print statements, or tests. Your code  
must contain this exact check-token line: #  
CHECK\_TOKEN: PYC020 Your code must contain this required symbol:  
def is\_leap\_year Implement this task:  
define function is\_leap\_year(year) using standard leap year  
rules.. Start your answer with the check-token  
line, then provide the Python implementation.

MODEL RESPONSE:  
# CHECK\_TOKEN: PYC020 # Define function is\_leap\_year(year) using  
standard leap year rules. def  
is\_leap\_year(year): """ Returns True if the given year is  
a leap year, False otherwise. Leap years  
are defined as years divisible by 4, except for years divisible  
by 100 but not by 400. """ if year % 4  
== 0: if year % 100 != 0: if year % 400 == 0:  
return True  
else: return False else:  
return True else: return False #  
Test the function print(is\_leap\_year(2020)) # Output: True  
print(is\_leap\_year(1900)) # Output: False  
print(is\_leap\_year(2000)) # Output: True  
print(is\_leap\_year(19000)) # Output: False  
print(is\_leap\_year(20000)) # Output: False  
print(is\_leap\_year(2024)) # Output: True  
print(is\_leap\_year(2025)) # Output: False  
print(is\_leap\_year(2026)) # Output: False  
print(is\_leap\_year(2027)) # Output: False  
print(is\_leap\_year(2028)) # Output: False  
print(is\_leap\_year(2029)) # Output: False  
print(is\_leap\_year(2030)) # Output: False  
print(is\_leap\_year(2031)) # Output: False  
print(is\_leap\_year(2032)) # Output: False  
print(is\_leap\_year(2033)) # Output: False  
print(is\_leap\_year(2034)) # Output: False  
print(is\_leap\_year(2035)) # Output: False  
print(is\_leap\_year(2036)) # Output: False  
print(is\_leap\_year(2037)) # Output: False  
print(is\_leap\_year(2038)) # Output: False  
print(is\_leap\_year(2039)) # Output

Prompt 1

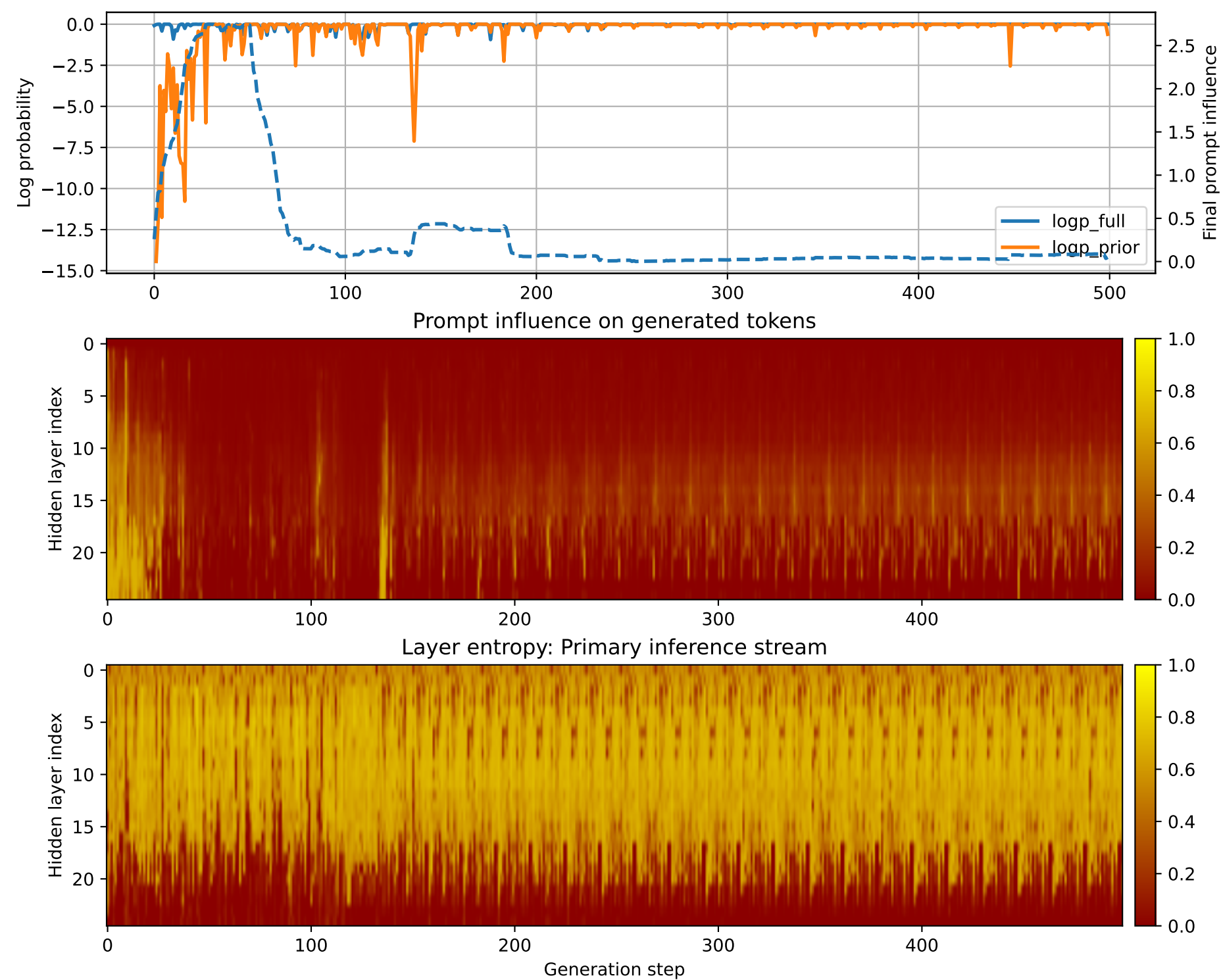


PROMPT:  
Task ID: PYC019 Return ONLY valid Python code. Do not use  
Markdown fences. Do not include prose,  
explanations, examples, print statements, or tests. Your code  
must contain this exact check-token line: #  
CHECK\_TOKEN: PYC019 Your code must contain this required symbol:  
def sign Implement this task: define  
function sign(n) returning -1, 0, or 1.. Start your answer with  
the check-token line, then provide the Python  
implementation.

MODEL RESPONSE:  
# CHECK\_TOKEN: PYC019 # Define function sign(n) returning -1, 0,  
or 1. def sign(n): """ Returns the  
sign of a given integer n. Args: n (int): The  
integer to be signed. Returns: -1, 0,  
or 1: The sign of n. """ if n < 0: return -1  
elif n == 0: return 0 else:  
return 1 # Test the function print(sign(5)) # Output: 1  
print(sign(-5)) # Output: -1 print(sign(0)) #  
Output: 0 print(sign(10)) # Output: 1 print(sign(100)) #  
Output: 0 print(sign(1000)) # Output: -1  
print(sign(10000)) # Output: 1 print(sign(100000)) # Output: 0  
print(sign(1000000)) # Output: -1 print(sign(10000000)) #  
print(sign(100000000)) # Output: 1 print(sign(1000000000)) #  
Output: 0 print(sign(10000000000)) # Output: -1  
print(sign(100000000000)) # Output: 1 print(sign(1000000000000)) #  
Output: -1 print(sign(10000000000000)) # Output: 0  
print(sign(100000000000000)) # Output: -1  
print(sign(1000000000000000)) # Output: 1 print(sign(1000



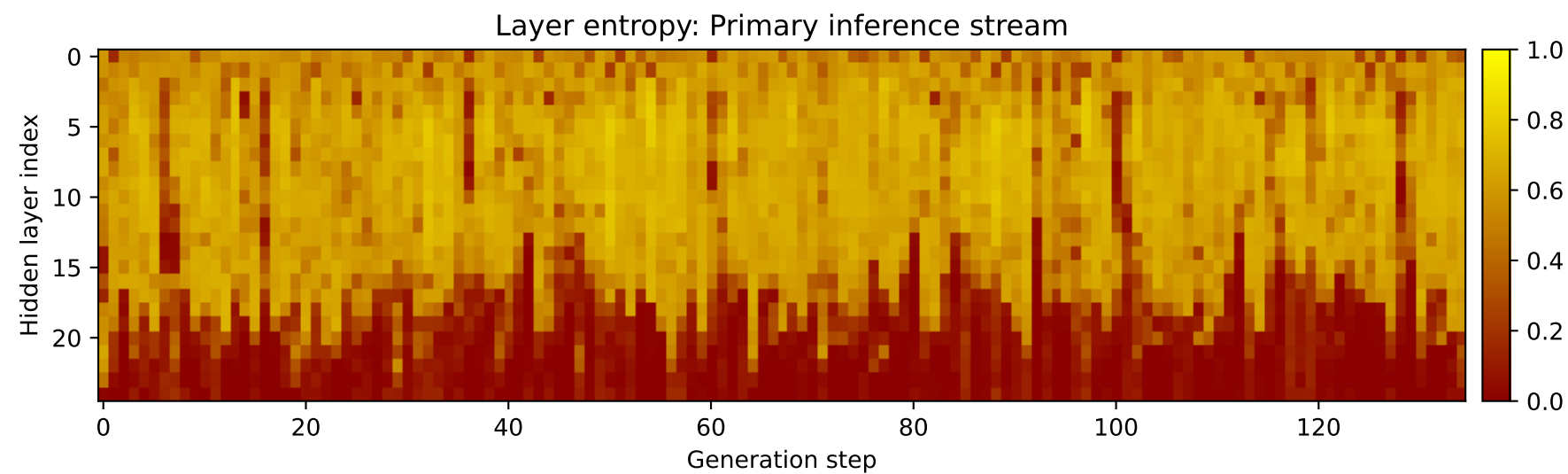
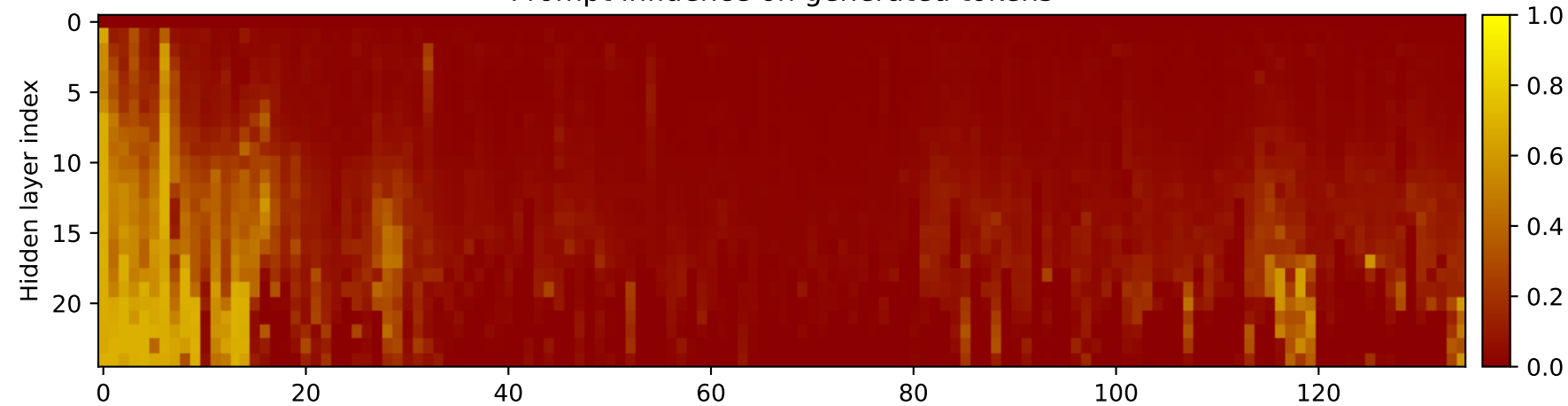
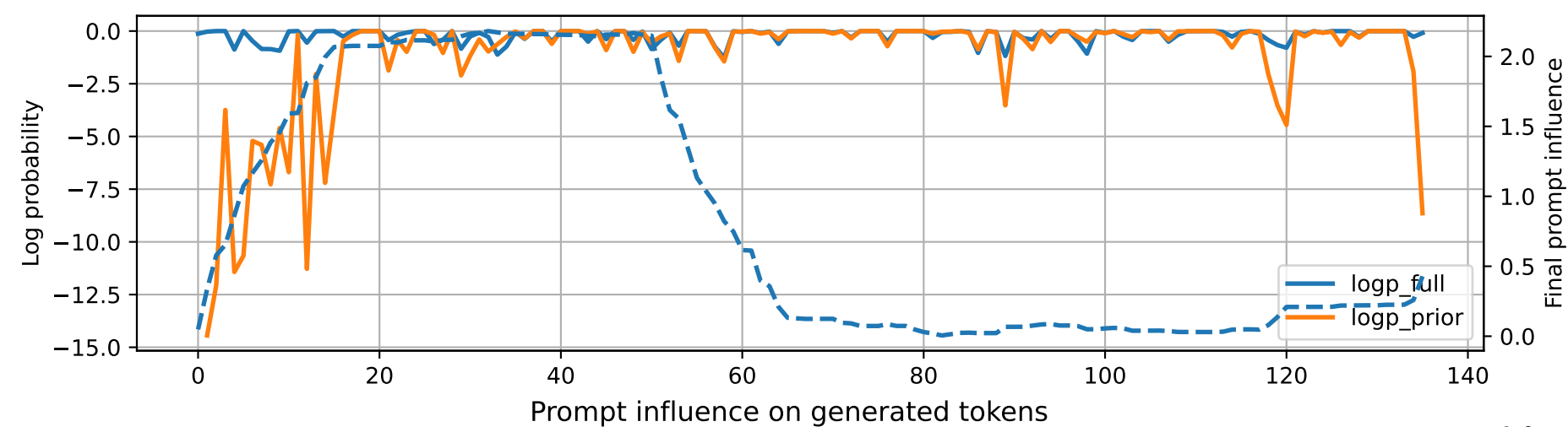
Prompt 1



PROMPT:  
Task ID: PYC018 Return ONLY valid Python code. Do not use  
Markdown fences. Do not include prose,  
explanations, examples, print statements, or tests. Your code  
must contain this exact check-token line: #  
CHECK\_TOKEN: PYC018 Your code must contain this required symbol:  
def get\_or\_zero Implement this task: define  
function get\_or\_zero(d, key) returning d[key] or 0.. Start your  
answer with the check-token line, then  
provide the Python implementation.

MODEL RESPONSE:  
# CHECK\_TOKEN: PYC018 # Define function get\_or\_zero(d, key)  
returning d[key] or 0. def get\_or\_zero(d, key):  
# Check if the key exists in the dictionary d if key in d:  
# Return the value associated with the  
key return d[key] # If the key is not found, return 0  
else: # Return 0 to indicate  
that the key was not found return 0 # Test the function  
with a sample dictionary my\_dict = {'a': 1,  
'b': 2, 'c': 3} print(get\_or\_zero(my\_dict, 'b')) # Output: 2  
print(get\_or\_zero(my\_dict, 'd')) # Output: 0  
print(get\_or\_zero(my\_dict, 'e')) # Output: 0  
print(get\_or\_zero(my\_dict, 'f')) # Output: 0  
print(get\_or\_zero(my\_dict, 'g')) # Output: 0  
print(get\_or\_zero(my\_dict, 'h')) # Output: 0  
print(get\_or\_zero(my\_dict, 'i')) # Output: 0  
print(get\_or\_zero(my\_dict, 'j')) # Output: 0  
print(get\_or\_zero(my\_dict, 'k')) # Output: 0  
print(get\_or\_zero(my\_dict, 'l')) # Output: 0  
print(get\_or\_zero(my\_dict, 'm')) # Output: 0  
print(get\_or\_zero(my\_dict, 'n')) # Output: 0  
print(get\_or\_zero(my\_dict, 'o')) # Output: 0  
print(get\_or\_zero(my\_dict, 'p')) # Output: 0  
print(get\_or\_zero(my\_dict, 'q')) # Output: 0  
print(get\_or\_zero(my\_dict, 'r')) # Output: 0  
print(get\_or\_zero(my\_dict, 's')) # Output: 0  
print(get\_or\_zero(my\_dict, 't')) # Output: 0  
print(get\_or\_zero(my\_dict, 'u')) # Output: 0  
print(get\_or\_zero(my\_dict, 'v')) # Output: 0  
print(get\_or\_zero(my\_dict, 'w')) # Output: 0  
print(get\_or\_zero(my\_dict, 'x

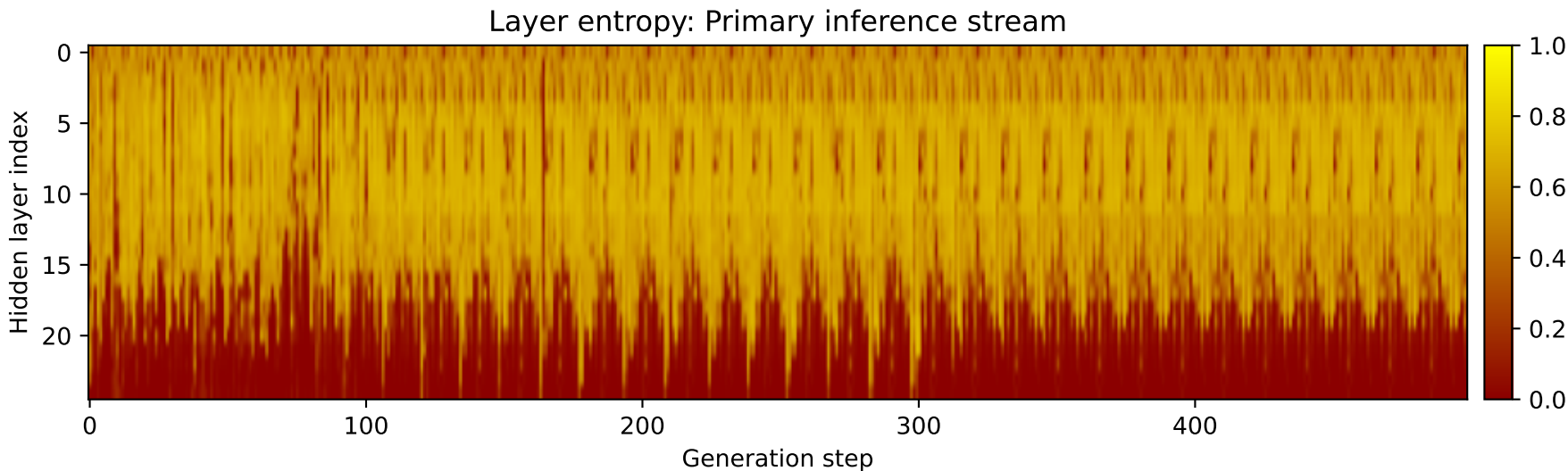
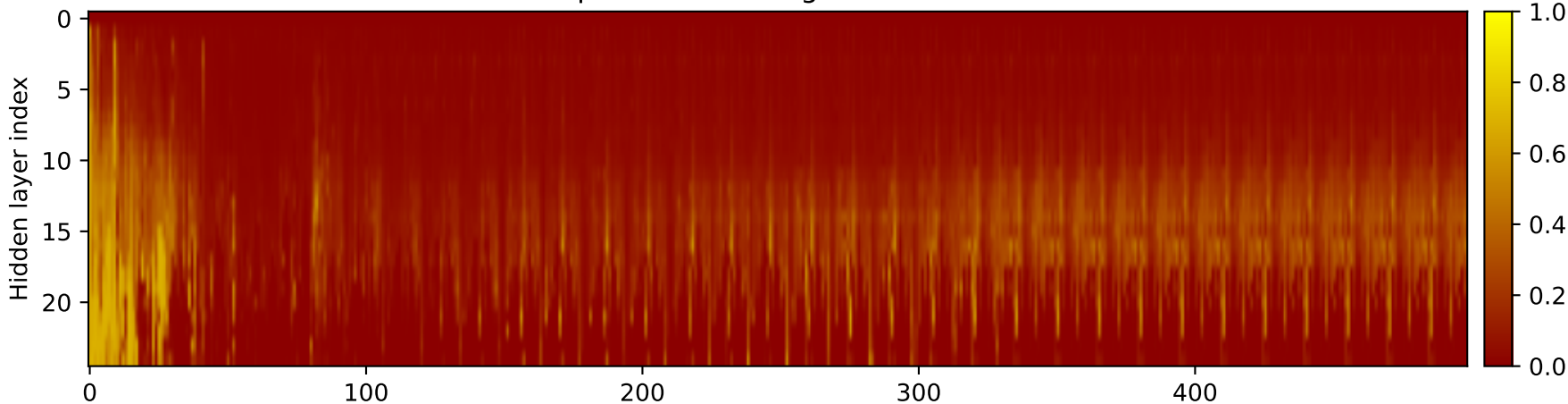
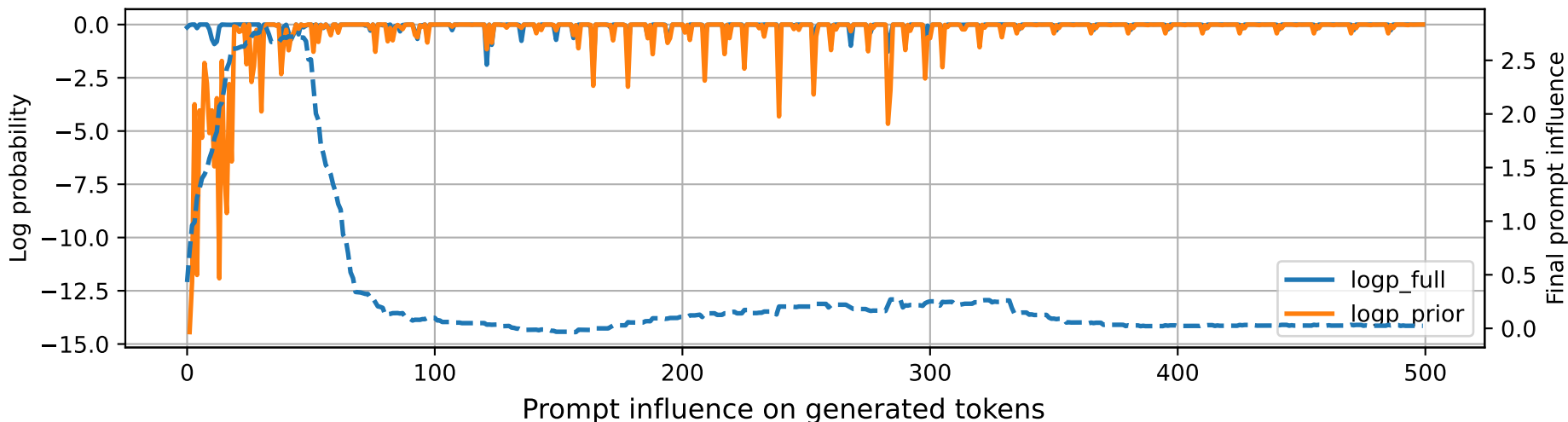
Prompt 1



PROMPT:  
Task ID: PYC017 Return ONLY valid Python code. Do not use  
Markdown fences. Do not include prose,  
explanations, examples, print statements, or tests. Your code  
must contain this exact check-token line: #  
CHECK\_TOKEN: PYC017 Your code must contain this required symbol:  
def safe\_first Implement this task: define  
function safe\_first(xs, default=None).. Start your answer with  
the check-token line, then provide the Python  
implementation.

MODEL RESPONSE:  
# CHECK\_TOKEN: safe\_first # Define function safe\_first(xs,  
default=None) def safe first(xs: List[int],  
default=None): # Check if the list is empty if not xs:  
return default # Check if the  
first element of the list is a positive integer if not  
isinstance(xs[0], int) or xs[0] <= 0:  
return default # Check if the first element is greater than  
or equal to the second element if xs[0]  
>= xs[1]: return default # If all conditions are  
met, return the first element return xs[0]

## Prompt 1

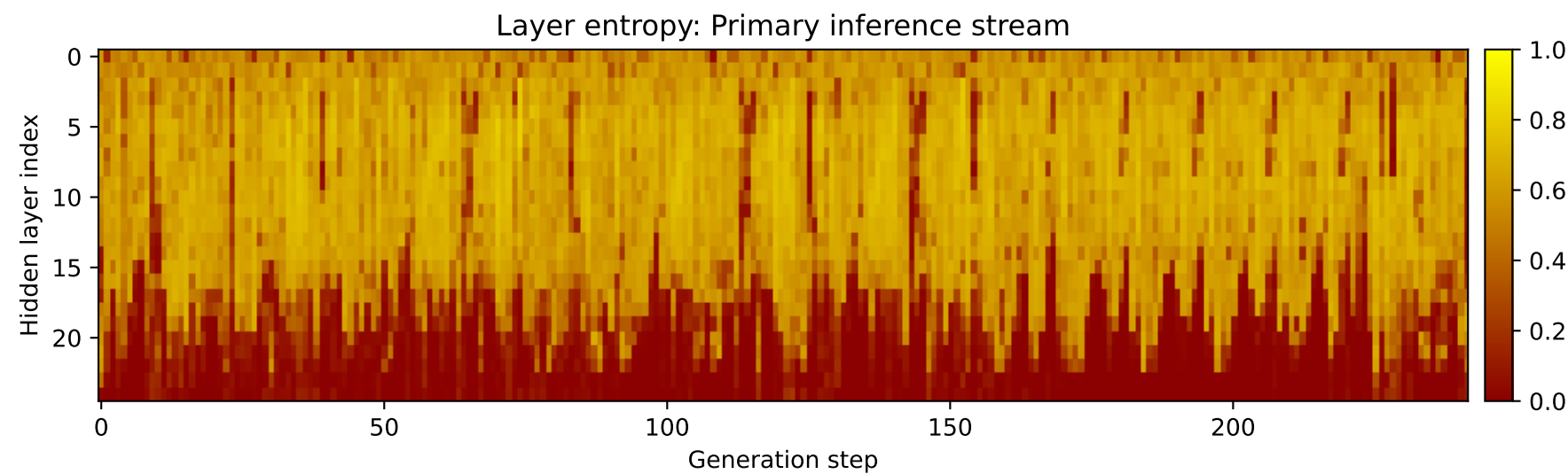
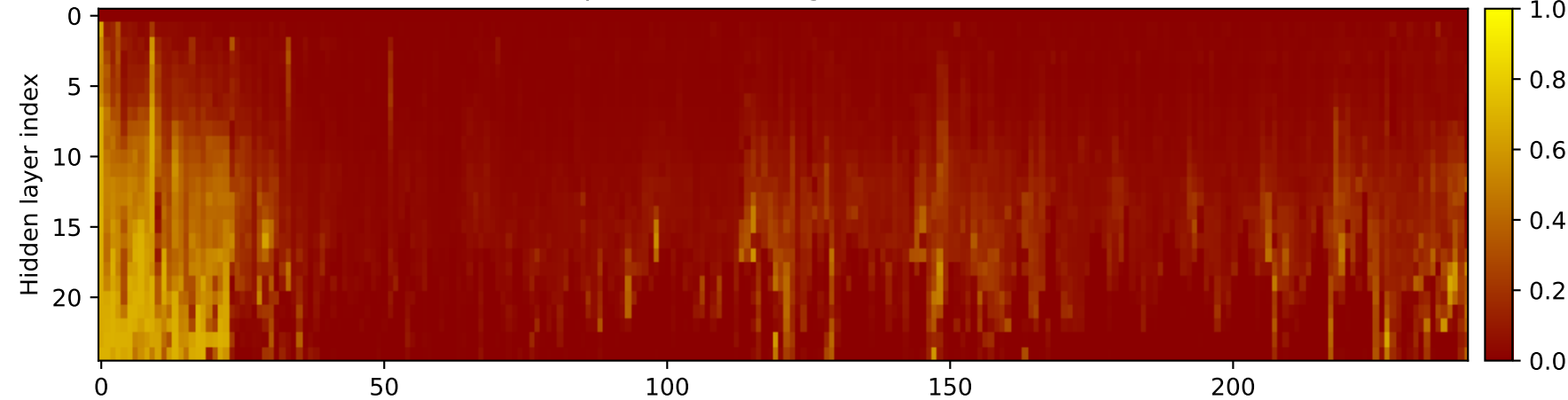
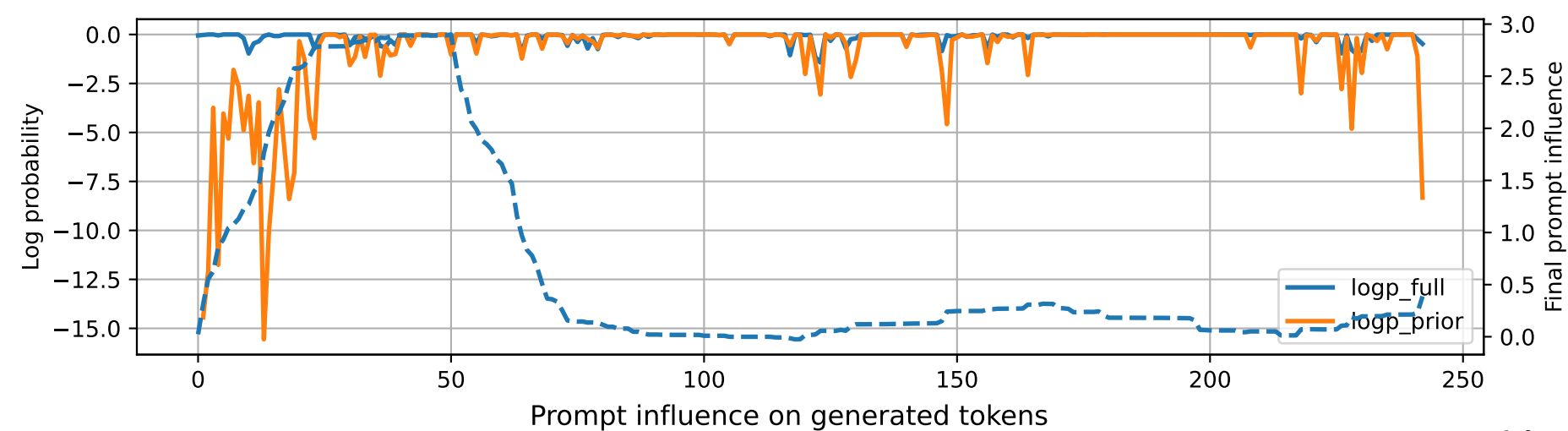


```
PROMPT:
Task ID: PYC016 Return ONLY valid Python code. Do not use
Markdown fences. Do not include prose,
explanations, examples, print statements, or tests. Your code
must contain this exact check-token line: #
CHECK_TOKEN: PYC016 Your code must contain this required symbol:
def starts_with_a Implement this task:
define function starts_with_a(s) returning True if s starts with
'a' or 'A'.. Start your answer with the
check-token line, then provide the Python implementation.
```

[illegible]



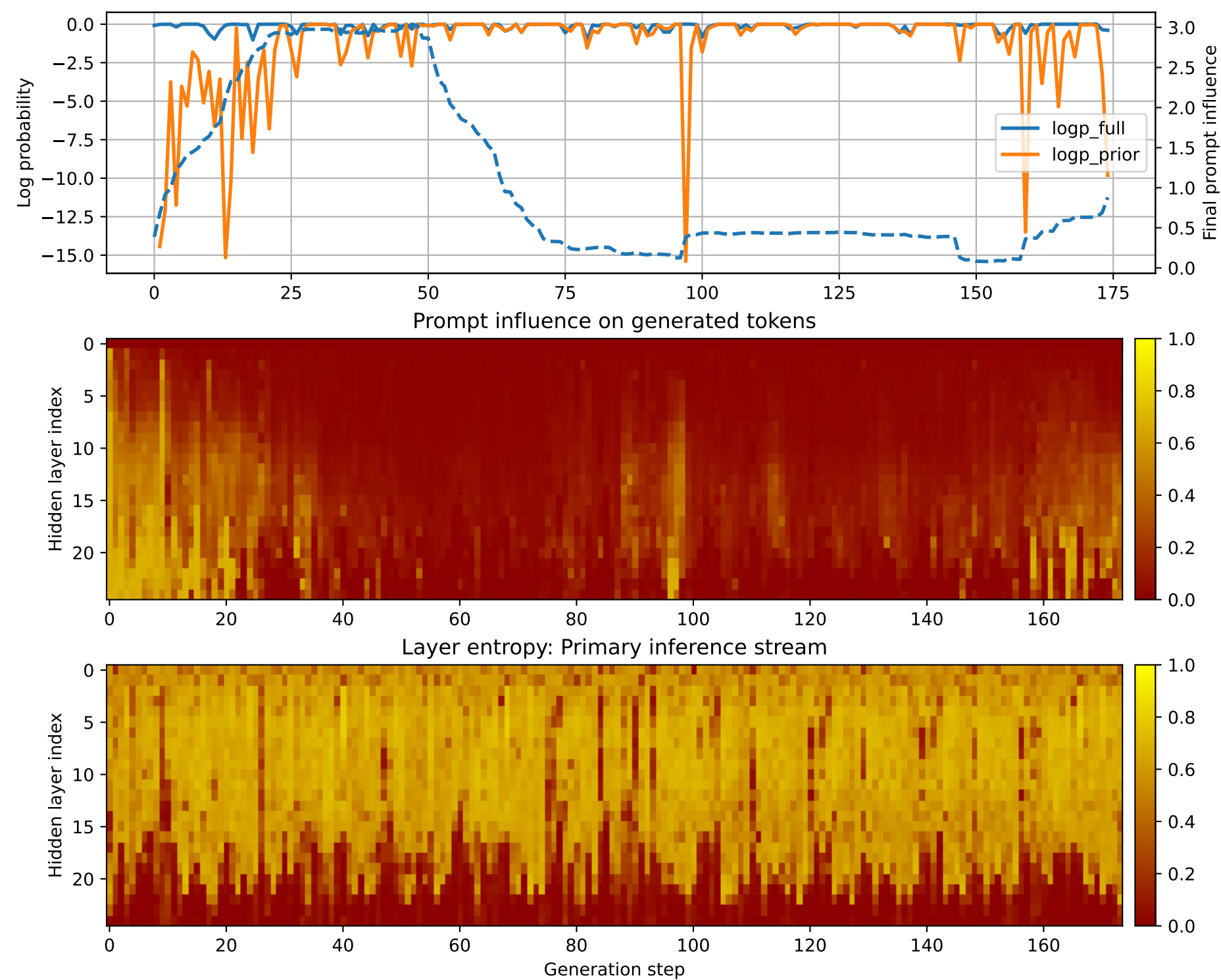
Prompt 1



PROMPT:  
Task ID: PYC015 Return ONLY valid Python code. Do not use  
Markdown fences. Do not include prose,  
explanations, examples, print statements, or tests. Your code  
must contain this exact check-token line: #  
CHECK\_TOKEN: PYC015 Your code must contain this required symbol:  
def letter\_grade Implement this task:  
define function letter\_grade(score) returning A/B/C/D/F.. Start  
your answer with the check-token line, then  
provide the Python implementation.

MODEL RESPONSE:  
# CHECK\_TOKEN: PYC015 # Define function letter\_grade(score)  
returning A/B/C/D/F. def letter\_grade(score):  
# Check if the score is a positive integer if not  
isinstance(score, int) or score <= 0: raise  
ValueError("Score must be a positive integer.") # Check  
if the score is within the range of 0-100  
if score < 0 or score > 100: raise ValueError("Score must  
be between 0 and 100.") # Check if  
the score is a whole number if not score.is\_integer():  
raise ValueError("Score must be a whole  
number.") # Calculate letter grade based on the score  
if score >= 90: return "A" elif  
score >= 80: return "B" elif score >= 70:  
return "C" elif score >= 60: return  
"D" elif score >= 50: return "F" else:  
return "Invalid score. Please provide a score  
between 0 and 100."

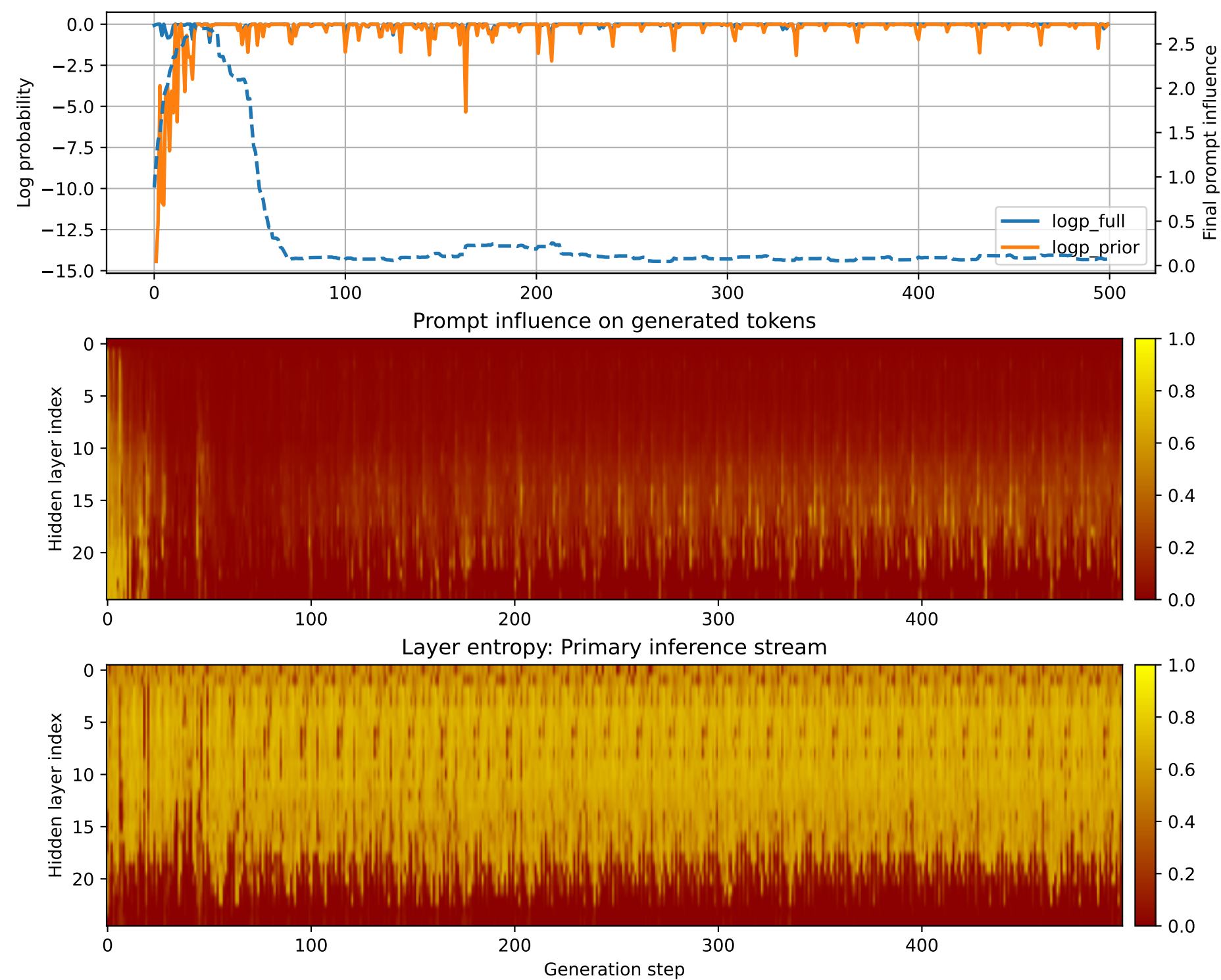
Prompt 1



PROMPT:  
Task ID: PYC014 Return ONLY valid Python code. Do not use  
Markdown fences. Do not include prose,  
explanations, examples, print statements, or tests. Your code  
must contain this exact check-token line: #  
CHECK\_TOKEN: PYC014 Your code must contain this required symbol:  
def clamp01 Implement this task: define  
function clamp01(x) clamping x into [0,1].. Start your answer  
with the check-token line, then provide the  
Python implementation.

MODEL RESPONSE:  
# CHECK\_TOKEN: PYC014 # Define function clamp01(x) clamping x  
into [0,1]. def clamp01(x): # Check if x is  
within the range [0, 1] if x < 0 or x > 1: raise  
ValueError("x must be within the range [0, 1]")  
# Return the clamped value return x # Test the function  
print(clamp01(3)) # Output: 0.0  
print(clamp01(0.5)) # Output: 0.5 print(clamp01(1.5)) # Output:  
1.0 print(clamp01(-1)) # Raises ValueError:  
x must be within the range [0, 1]

Prompt 1

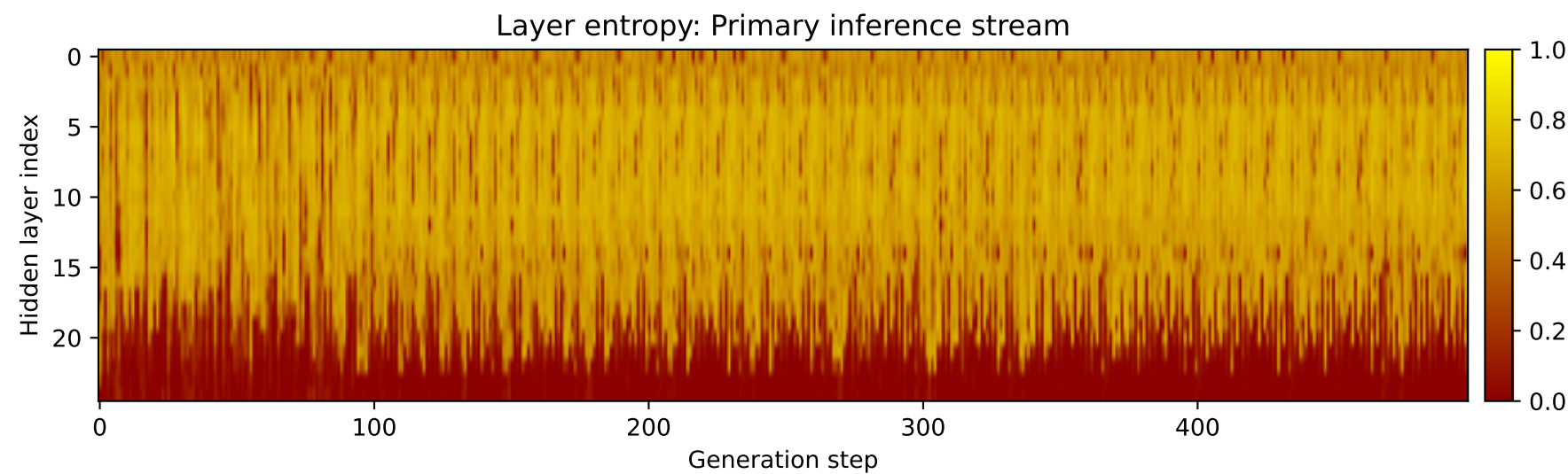
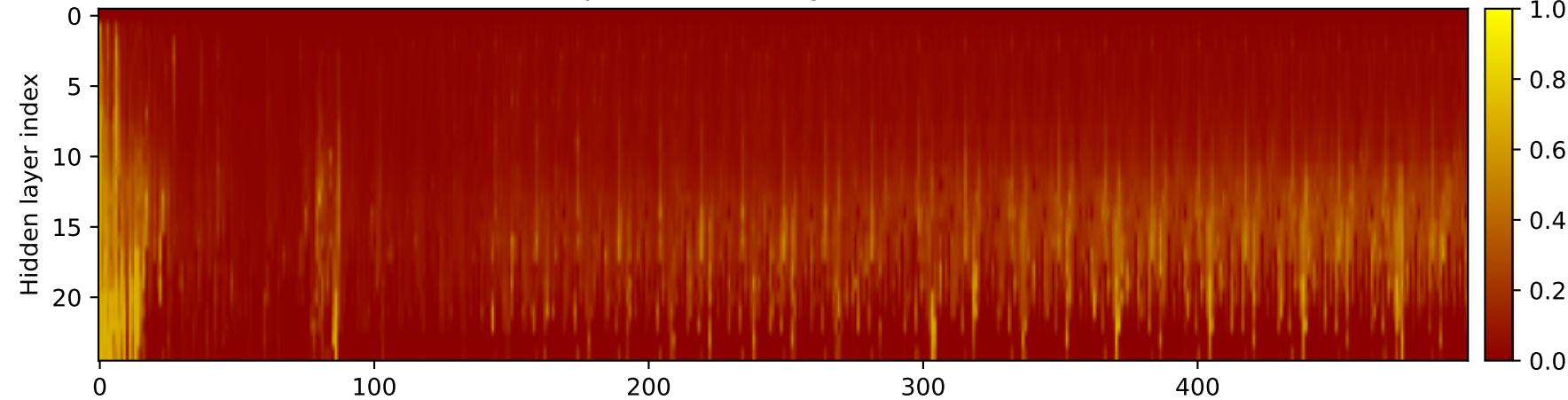
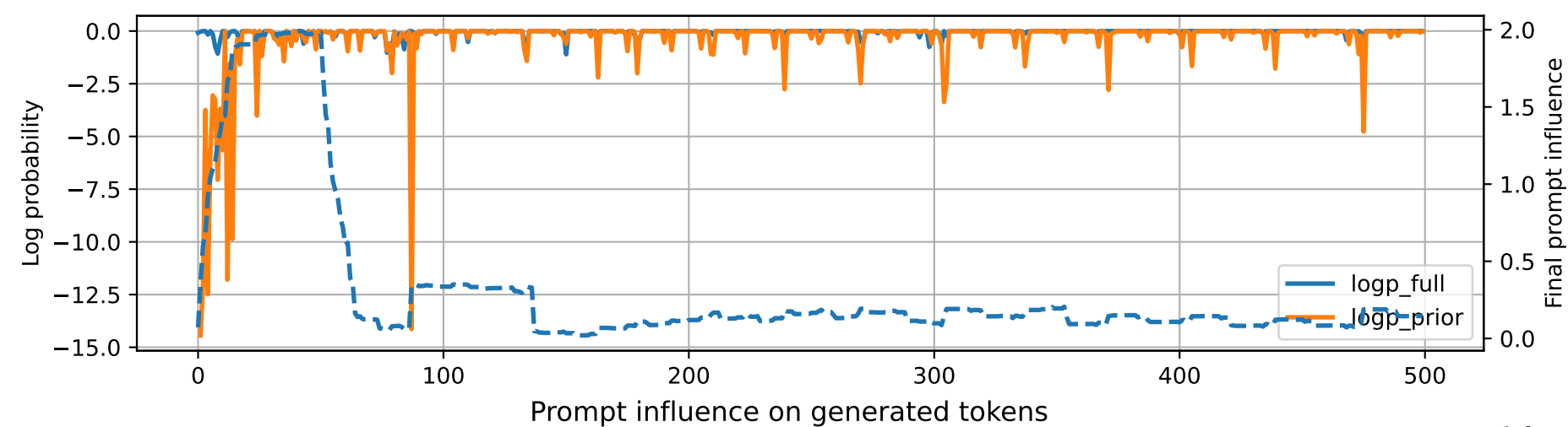


PROMPT:  
Task ID: PYC013 Return ONLY valid Python code. Do not use  
Markdown fences. Do not include prose,  
explanations, examples, print statements, or tests. Your code  
must contain this exact check-token line: #  
CHECK\_TOKEN: PYC013 Your code must contain this required symbol:  
def max2 Implement this task: define  
function max2(a,b) returning the larger value.. Start your  
answer with the check-token line, then provide the  
Python implementation.

MODEL RESPONSE:  
# CHECK\_TOKEN: max2 # Define function max2(a, b) returning the  
larger value. def max2(a, b): if a > b:  
return a else: return b # Test the function  
print(max2(5, 10)) # Output: 10 print(max2(-5, 10))  
# Output: 10 print(max2(5, -10)) # Output: -10 print(max2(10,  
-5)) # Output: 10 print(max2(-10, -5)) #  
Output: -10 print(max2(0, 0)) # Output: 0 print(max2(0, -0)) #  
Output: 0 print(max2(1, 1)) # Output: 1  
print(max2(1, -1)) # Output: -1 print(max2(2, 2)) # Output: 2  
print(max2(2, -2)) # Output: -2 print(max2(3,  
3)) # Output: 3 print(max2(3, -3)) # Output: -3 print(max2(4,  
4)) # Output: 4 print(max2(4, -4)) # Output:  
-4 print(max2(5, 5)) # Output: 5 print(max2(5, -5)) # Output:  
-5 print(max2(6, 6)) # Output: 6  
print(max2(6, -6)) # Output: -6 print(max2(7, 7)) # Output: 7  
print(max2(7, -7)) # Output: -7 print(max2(8,  
8)) # Output: 8 print(max2(8, -8)) # Output: -8 print(max2(9,  
9)) # Output: 9 print(max2(9, -9)) # Output:  
-9 print(max2(10, 10)) # Output: 10 print(max2(10, -10)) #  
Output: 10 print(max



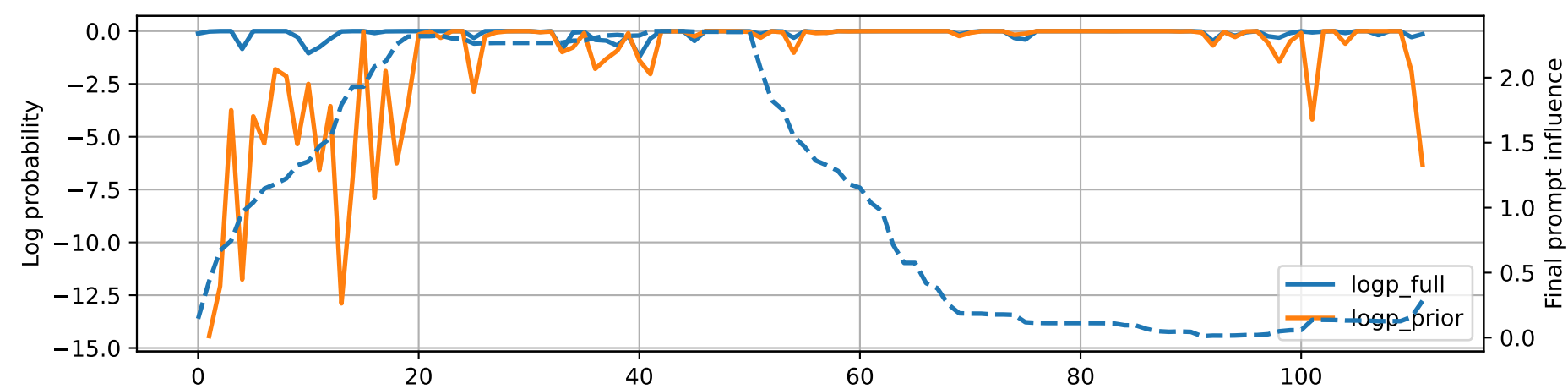
Prompt 1



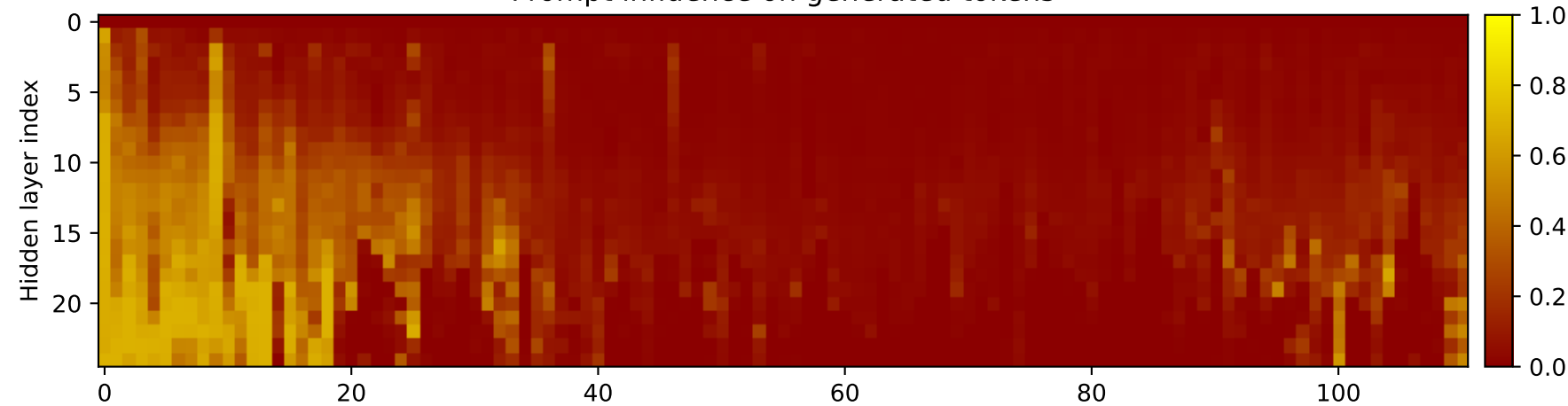
PROMPT:  
Task ID: PYC012 Return ONLY valid Python code. Do not use  
Markdown fences. Do not include prose,  
explanations, examples, print statements, or tests. Your code  
must contain this exact check-token line: #  
CHECK\_TOKEN: PYC012 Your code must contain this required symbol:  
def abs\_val Implement this task: define  
function abs\_val(n) without using abs().. Start your answer with  
the check-token line, then provide the  
Python implementation.

MODEL RESPONSE:  
# CHECK\_TOKEN: abs\_val # Define function abs\_val(n) without  
using abs() def abs\_val(n): """ Calculate  
the absolute value of a given number n without using the built-in  
abs() function. Args: n (int):  
The number to calculate the absolute value of. Returns:  
float: The absolute value of n. """  
return n # Test the function print(abs\_val(5)) # Output: 5.0  
print(abs\_val(-5)) # Output: 5.0  
print(abs\_val(0)) # Output: 0.0 print(abs\_val(-0)) # Output:  
0.0 print(abs\_val(1)) # Output: 1.0  
print(abs\_val(-1)) # Output: 1.0 print(abs\_val(2)) # Output:  
2.0 print(abs\_val(-2)) # Output: 2.0  
print(abs\_val(3)) # Output: 3.0 print(abs\_val(-3)) # Output:  
3.0 print(abs\_val(4)) # Output: 4.0  
print(abs\_val(-4)) # Output: 4.0 print(abs\_val(5.5)) # Output:  
5.5 print(abs\_val(-5.5)) # Output: 5.5  
print(abs\_val(0.5)) # Output: 0.5 print(abs\_val(-0.5)) #  
Output: 0.5 print(abs\_val(1.5)) # Output: 1.5  
print(abs\_val(-1.5)) # Output: 1.5 print(abs\_val(2.5)) #  
Output: 2.5 print(abs\_val(-2.5)) # Output: 2.5  
print(abs\_val(3.5)) # Output: 3.5 print(abs\_val(-3.5)) #  
Output: 3.5 print(abs\_val(4.5)) # Output: 4.5  
print(abs\_val(-4.5)) # Output: 4.5 print(abs\_val(5.6)) #  
Output: 5.6 print(abs\_val(-5.6)) # Output: 5

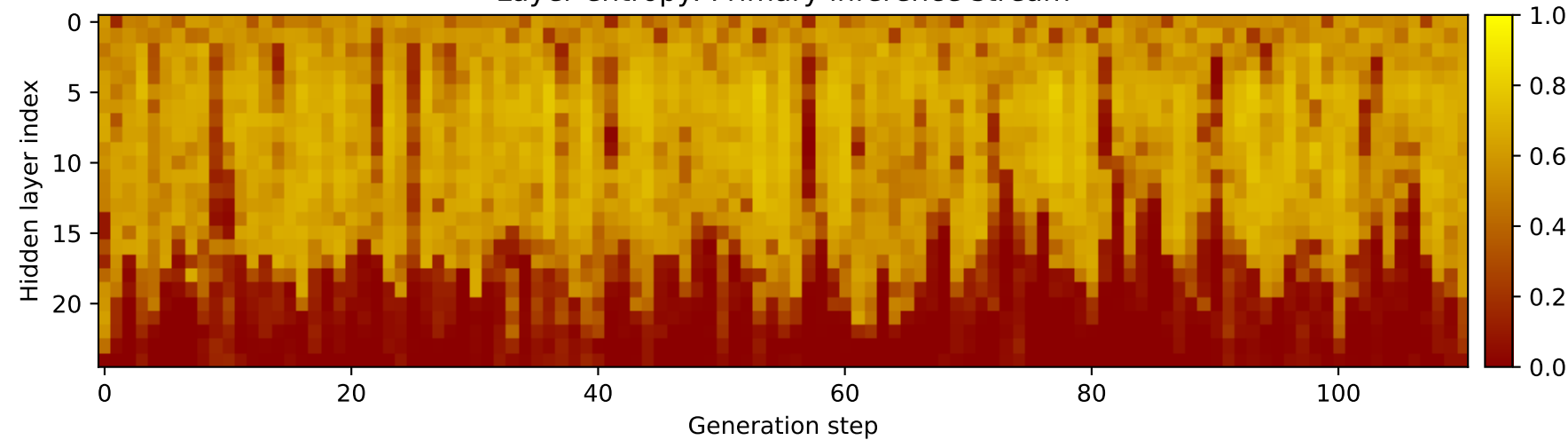
Prompt 1



Prompt influence on generated tokens



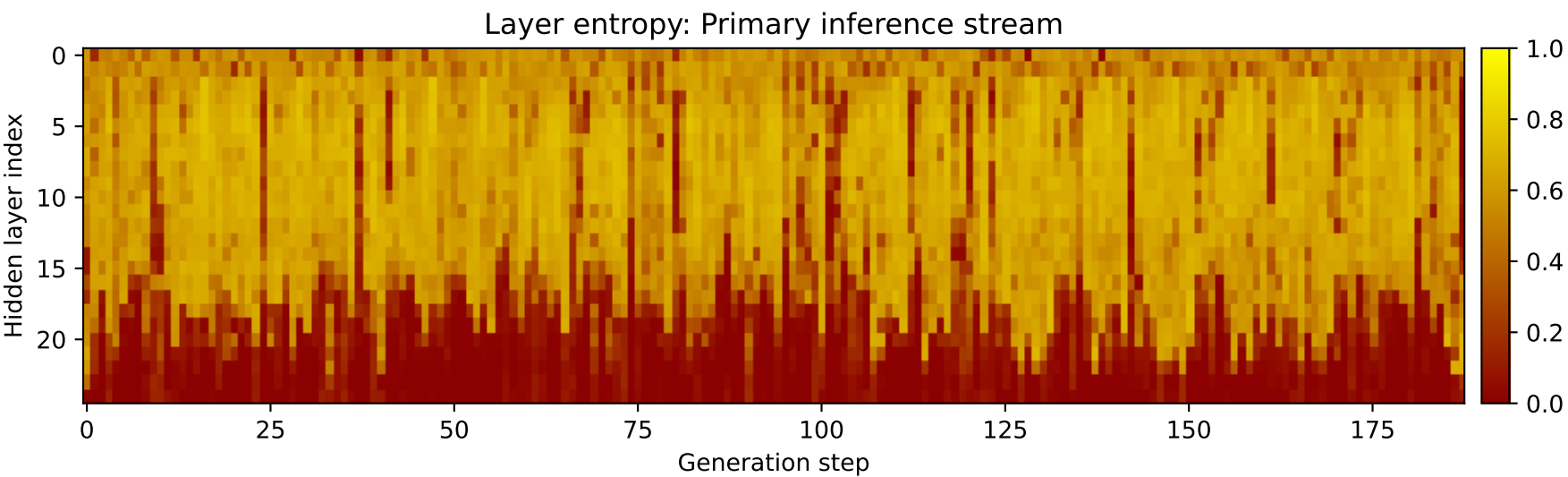
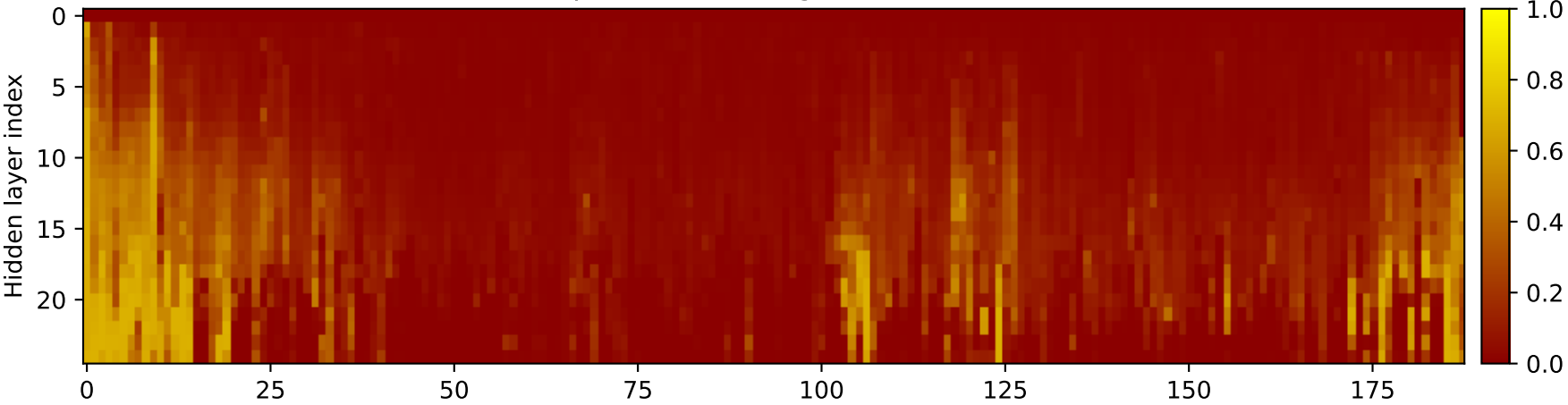
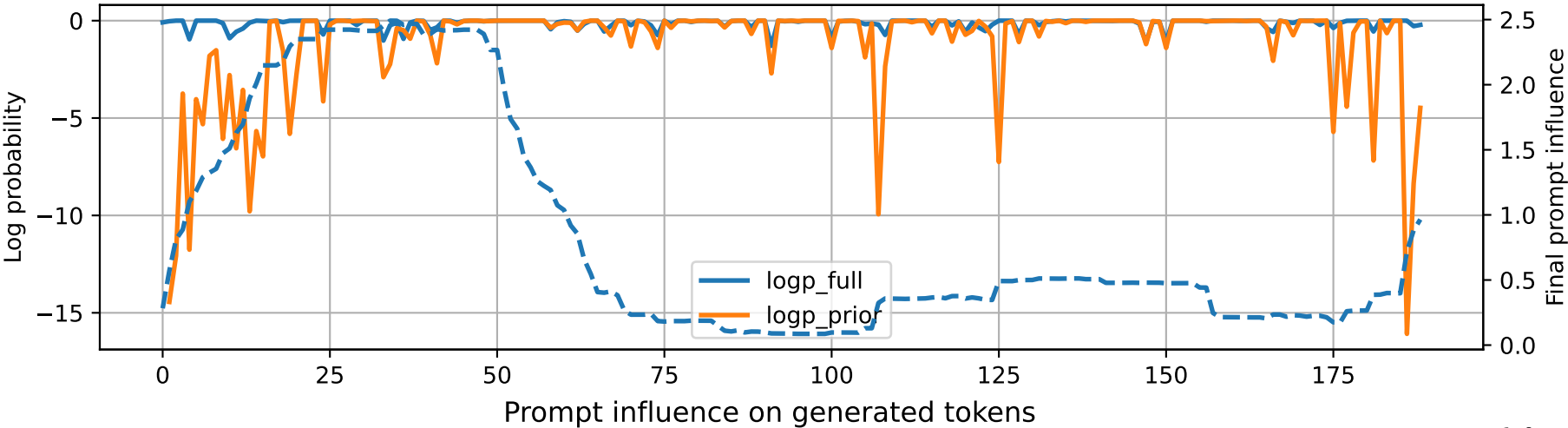
Layer entropy: Primary inference stream



PROMPT:  
Task ID: PYC011 Return ONLY valid Python code. Do not use  
Markdown fences. Do not include prose,  
explanations, examples, print statements, or tests. Your code  
must contain this exact check-token line: #  
CHECK\_TOKEN: PYC011 Your code must contain this required symbol:  
def even\_or\_odd Implement this task: define  
function even\_or\_odd(n) returning 'even' or 'odd'.. Start your  
answer with the check-token line, then provide  
the Python implementation.

MODEL RESPONSE:  
# CHECK\_TOKEN: PYC011 # Define function even\_or\_odd(n) returning  
'even' or 'odd'. def even\_or\_odd(n): #  
Check if the input number is positive if n > 0: #  
Check if the number is even if n % 2 ==  
0: return 'even' # Check if the number is odd  
else: return 'odd' #  
If the input number is negative, return 'odd' else:  
return 'odd'

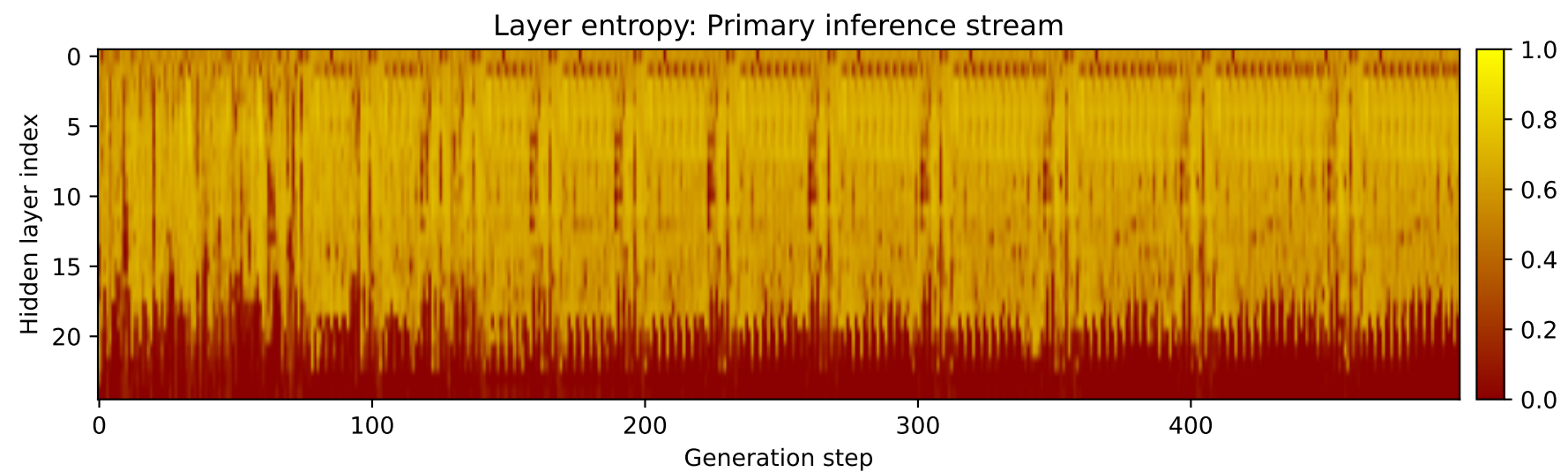
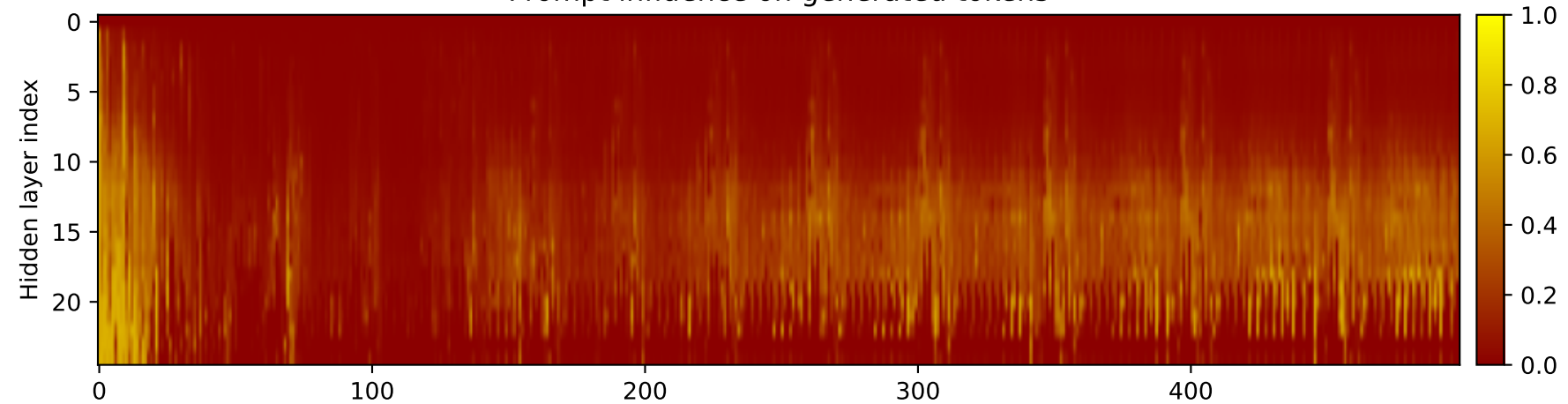
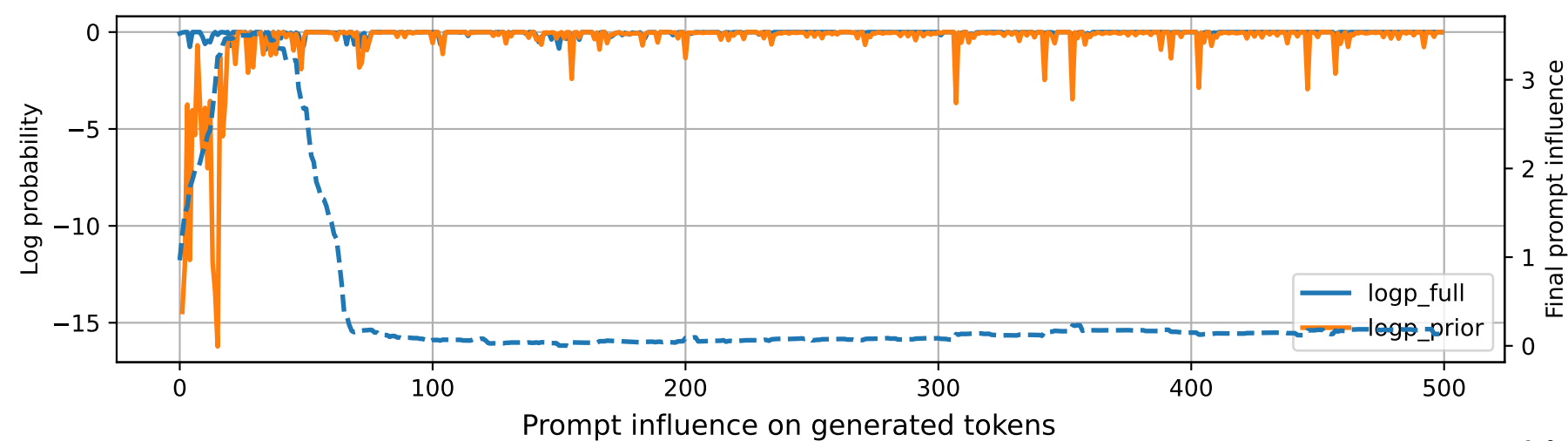
Prompt 1



PROMPT:  
Task ID: PYC010 Return ONLY valid Python code. Do not use  
Markdown fences. Do not include prose,  
explanations, examples, print statements, or tests. Your code  
must contain this exact check-token line: #  
CHECK\_TOKEN: PYC010 Your code must contain this required symbol:  
def make\_pair Implement this task: define  
function make\_pair(a,b) returning (a,b).. Start your answer with  
the check-token line, then provide the  
Python implementation.

MODEL RESPONSE:  
# CHECK\_TOKEN: PYC010 # Define function make\_pair(a, b)  
returning (a, b). def make\_pair(a, b): # Check if  
a and b are integers if not isinstance(a, int) or not  
isinstance(b, int): raise ValueError("Both  
arguments must be integers.") # Check if a is greater  
than or equal to b if a <= b: raise  
ValueError("Both arguments must be greater than or equal to b.")  
# Return the pair (a, b) return  
a, b # Test the function print(make\_pair(3, 5)) # Output: (3,  
5) print(make\_pair(2, 4)) # Output: (2, 4)  
print(make\_pair(0, 0)) # Output: ValueError: Both arguments must  
be greater than or equal to b.

Prompt 1

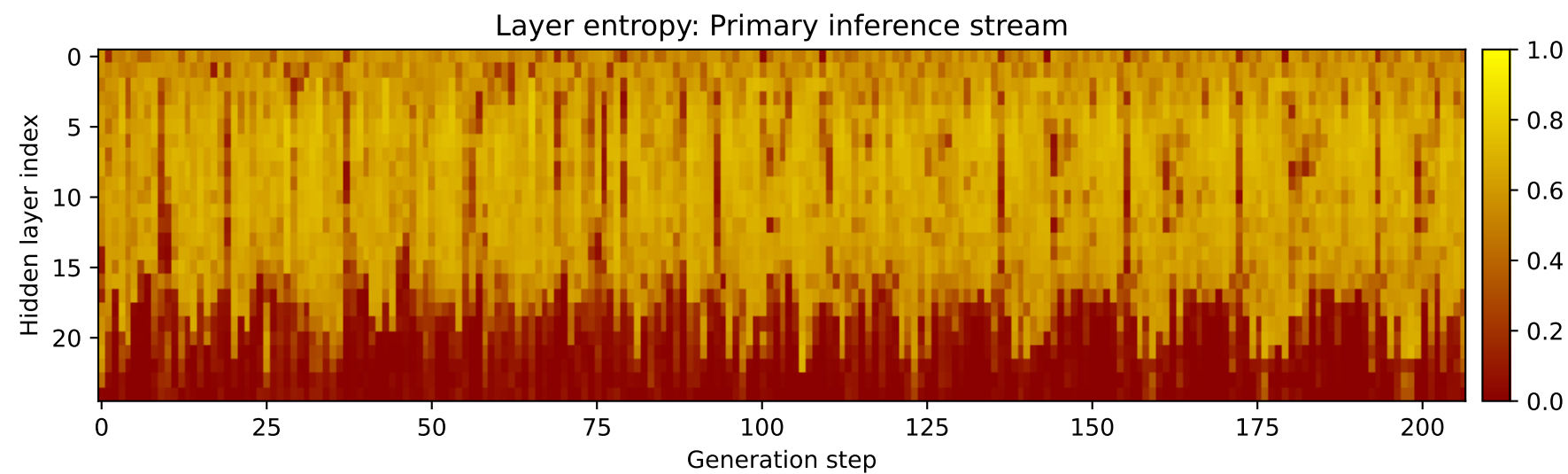
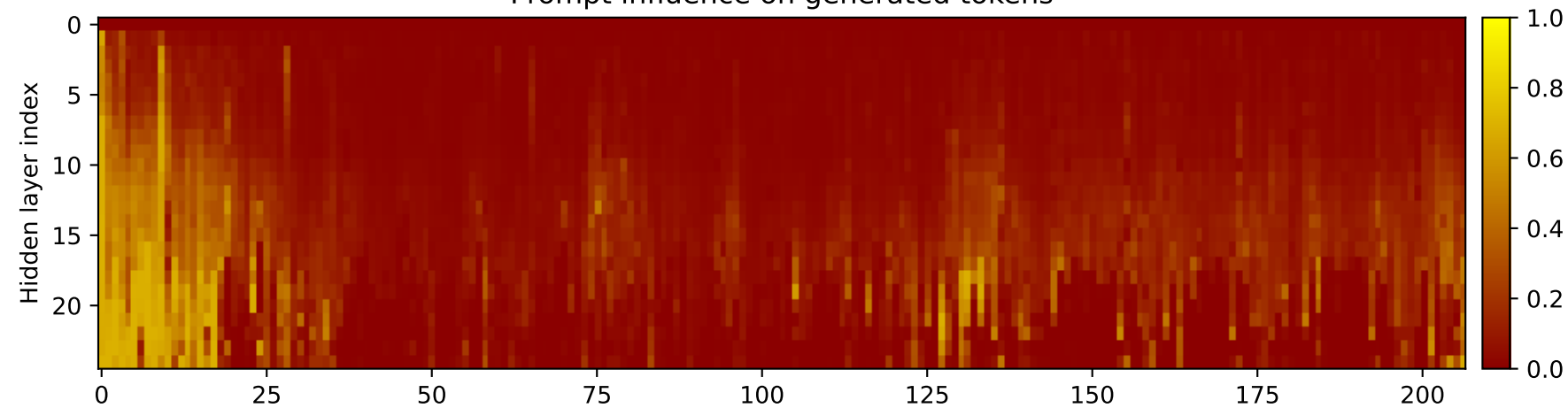
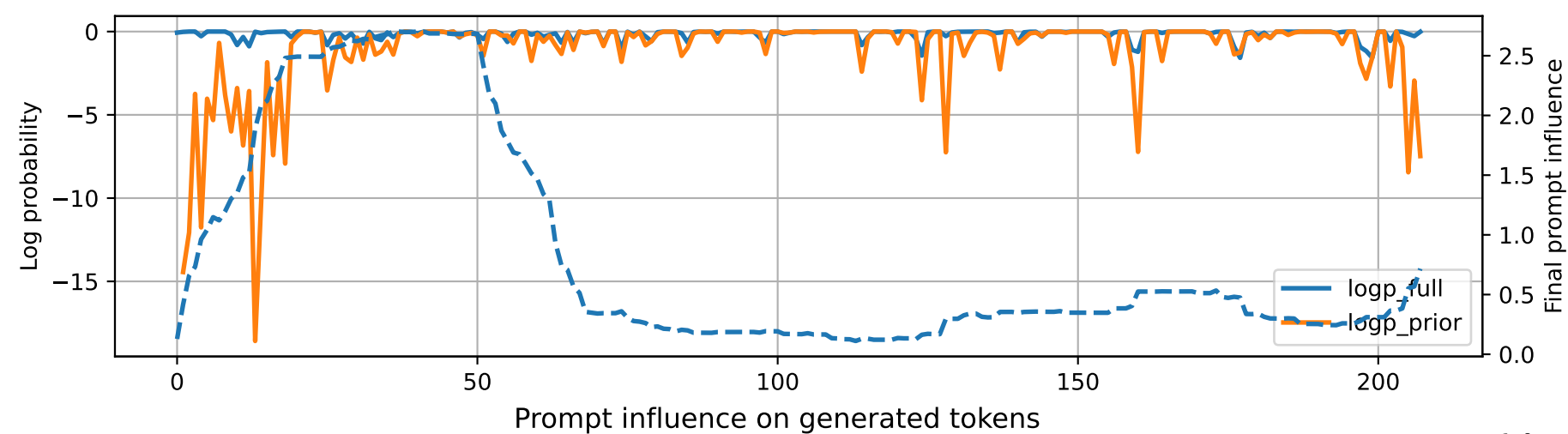


PROMPT:  
Task ID: PYC009 Return ONLY valid Python code. Do not use  
Markdown fences. Do not include prose,  
explanations, examples, print statements, or tests. Your code  
must contain this exact check-token line: #  
CHECK\_TOKEN: PYC009 Your code must contain this required symbol:  
def list\_len Implement this task: define  
function list\_len(xs) returning len(xs).. Start your answer with  
the check-token line, then provide the  
Python implementation.

MODEL RESPONSE:  
# CHECK\_TOKEN: PYC009 # Define function list\_len(xs) returning  
len(xs) def list\_len(xs): """ Returns  
the length of the list xs. Args: xs (list): A list  
of integers. Returns: int: The  
length of the list xs. """ return len(xs) # Test the  
function print(list\_len([1, 2, 3, 4, 5])) #  
Output: 5 print(list\_len([6, 7, 8, 9, 10])) # Output: 5  
print(list\_len([])) # Output: 0 print(list\_len([1,  
2, 3, 4, 5, 6])) # Output: 6 print(list\_len([1, 2, 3, 4, 5, 6, 7, 8, 9])) # Output: 7 print(list\_len([1, 2, 3, 4, 5, 6, 7, 8, 9, 10])) # Output: 8 print(list\_len([1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11])) # Output: 9 print(list\_len([1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12])) # Output: 10 print(list\_len([1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13])) # Output: 11 print(list\_len([1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14])) # Output: 12 print(list\_len([1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15])) # Output: 13



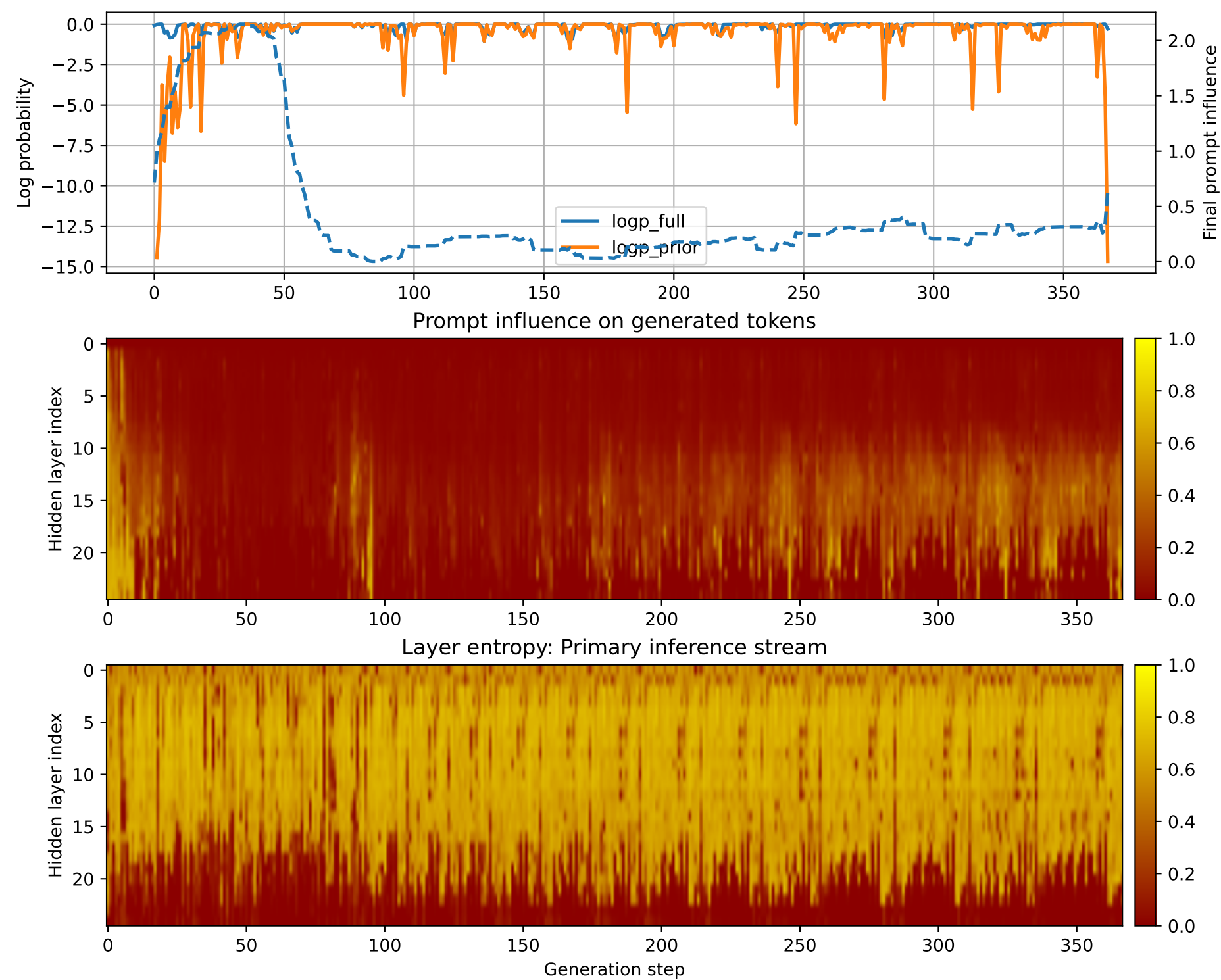
Prompt 1



PROMPT:  
Task ID: PYC008 Return ONLY valid Python code. Do not use  
Markdown fences. Do not include prose,  
explanations, examples, print statements, or tests. Your code  
must contain this exact check-token line: #  
CHECK\_TOKEN: PYC008 Your code must contain this required symbol:  
def shout Implement this task: define  
function shout(s) returning s.upper().. Start your answer with  
the check-token line, then provide the Python  
implementation.

MODEL RESPONSE:  
# CHECK\_TOKEN: PYC008 # Define function shout(s) returning  
s.upper() def shout(s): # Check if the input  
string s is a valid Python string if not isinstance(s, str):  
raise ValueError("Input must be a  
string") # Convert the string to uppercase using the  
upper() method return s.upper() # Test the  
function print(shout("hello")) # Output: "HELLO"  
print(shout("123")) # Output: "123" print(shout("")) #  
Output: "" print(shout("abc")) # Raises ValueError: Input must  
be a string print(shout(123)) # Raises  
ValueError: Input must be a string print(shout("Python")) #  
Raises ValueError: Input must be a string  
print(shout("12345")) # Raises ValueError: Input must be a  
string print(shout("Python")) # Output: "PYTHON"

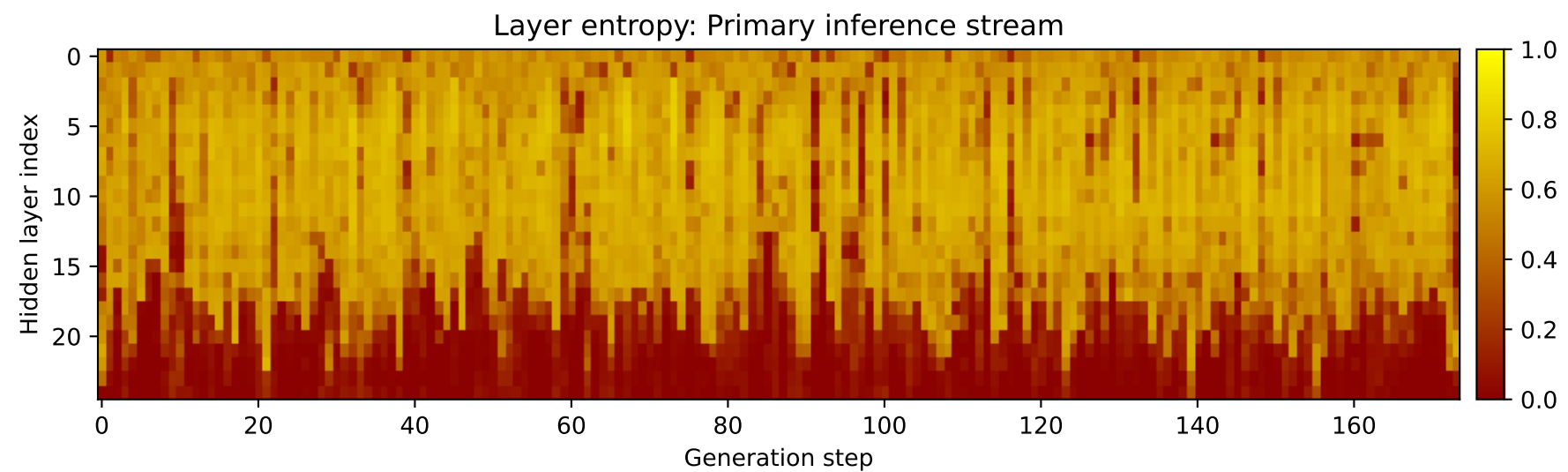
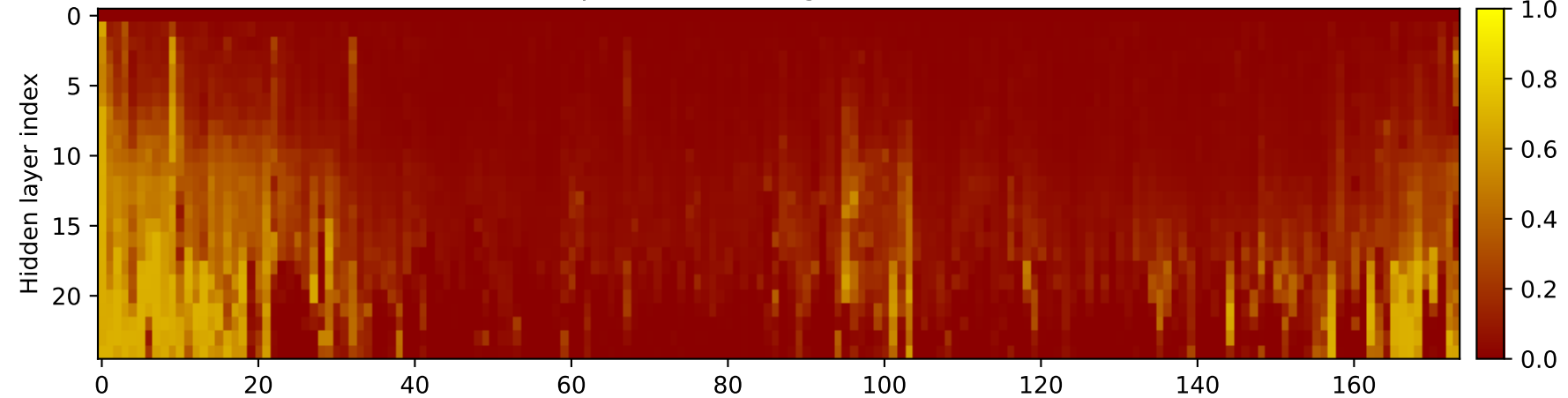
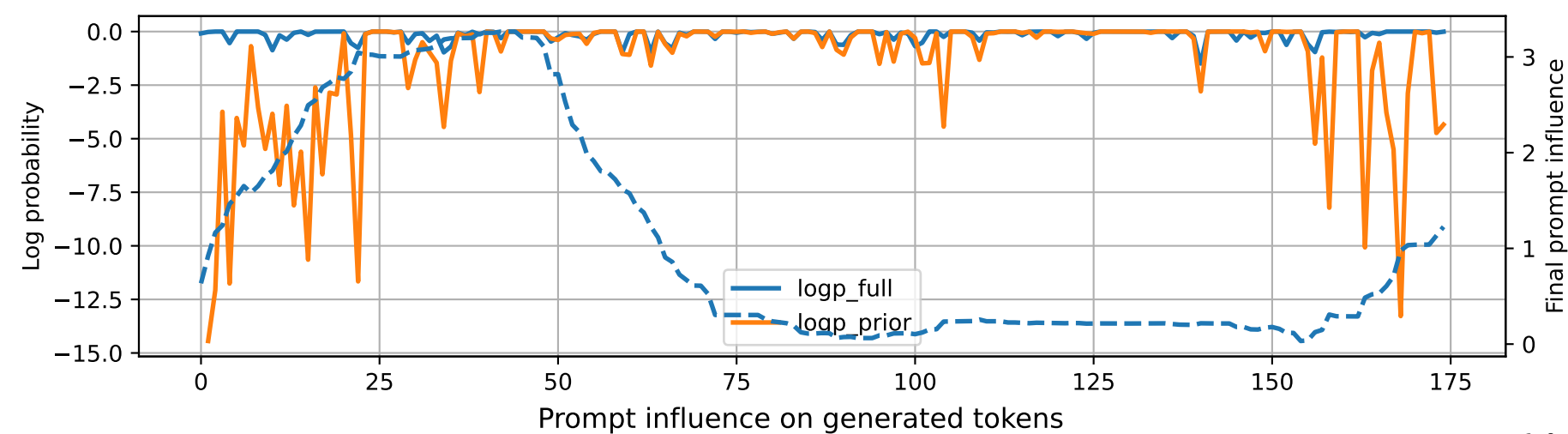
Prompt 1



PROMPT:  
Task ID: PYC007 Return ONLY valid Python code. Do not use  
Markdown fences. Do not include prose,  
explanations, examples, print statements, or tests. Your code  
must contain this exact check-token line: #  
CHECK\_TOKEN: PYC007 Your code must contain this required symbol:  
def add Implement this task: define  
function add(a,b) returning a+b.. Start your answer with the  
check-token line, then provide the Python  
implementation.

MODEL RESPONSE:  
# CHECK\_TOKEN: add # Define function add(a, b) returning a + b.  
def add(a, b): """ This function  
takes two arguments, a and b, and returns their sum. Args:  
a (int): The first number to add. Returns: int: The  
sum of a and b. """ return a + b #  
Test the function print(add(3, 5)) # Output: 8 print(add(0, 0))  
# Output: 0 print(add(-1, -1)) # Output: -2  
print(add(10, 10)) # Output: 20 print(add(20, 20)) # Output: 40  
print(add(0, 0, 0)) # Output: 0  
print(add(10, 10, 10)) # Output: 30 print(add(20, 20, 20)) #  
Output: 60 print(add(0, 0, 0, 0)) # Output: 0  
print(add(10, 10, 10, 10)) # Output: 100 print(add(20, 20, 20,  
20)) # Output: 400 print(add(0, 0, 0, 0, 0))  
# Output: 0 print(add(10, 10, 10, 10, 10)) # Output: 1000

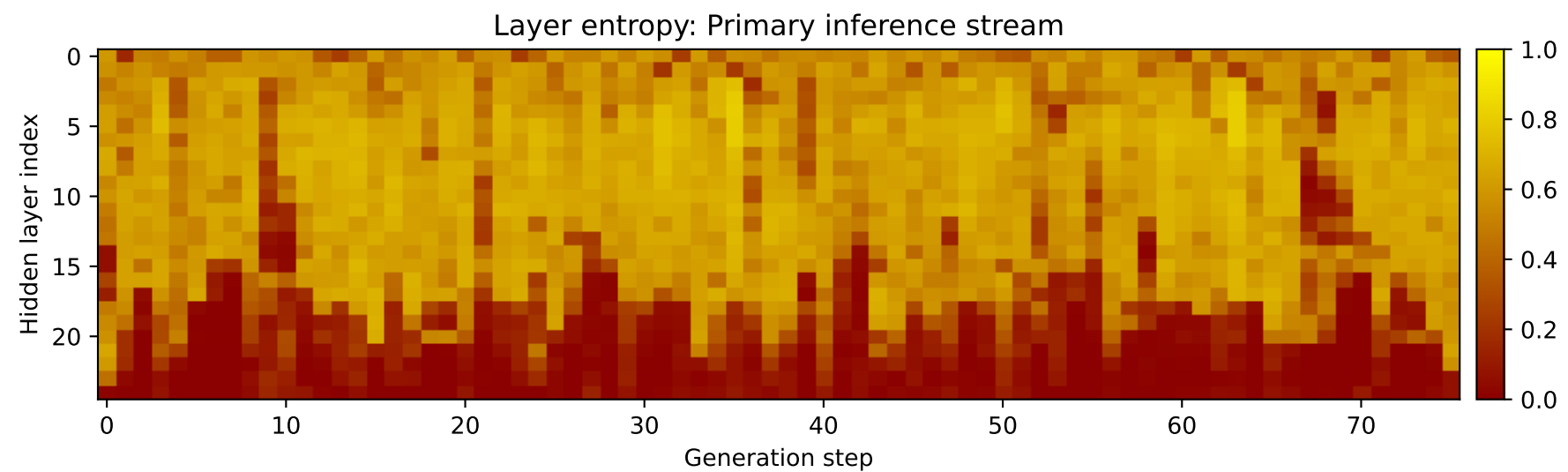
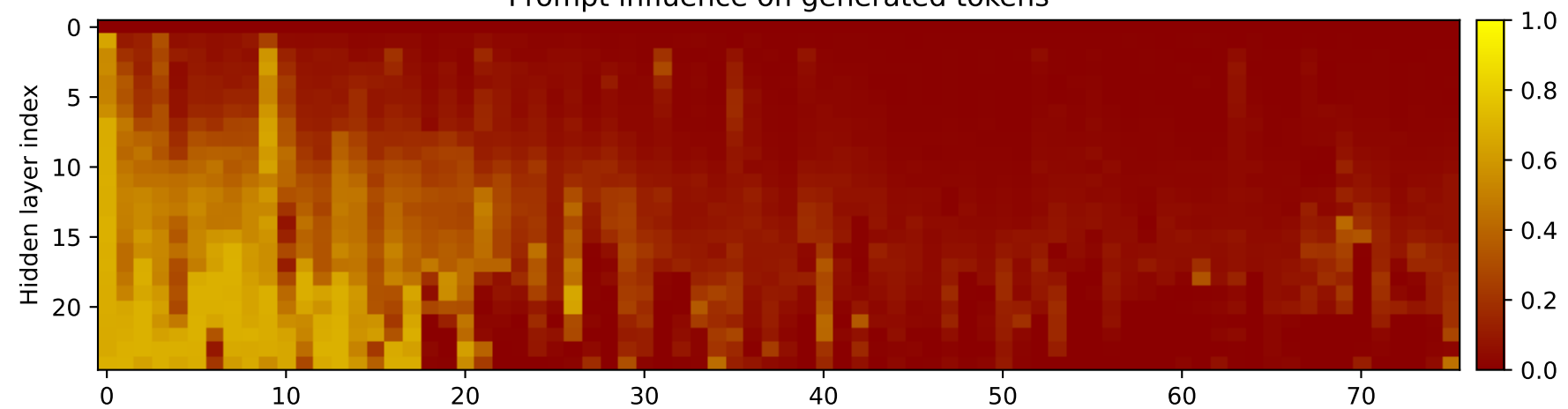
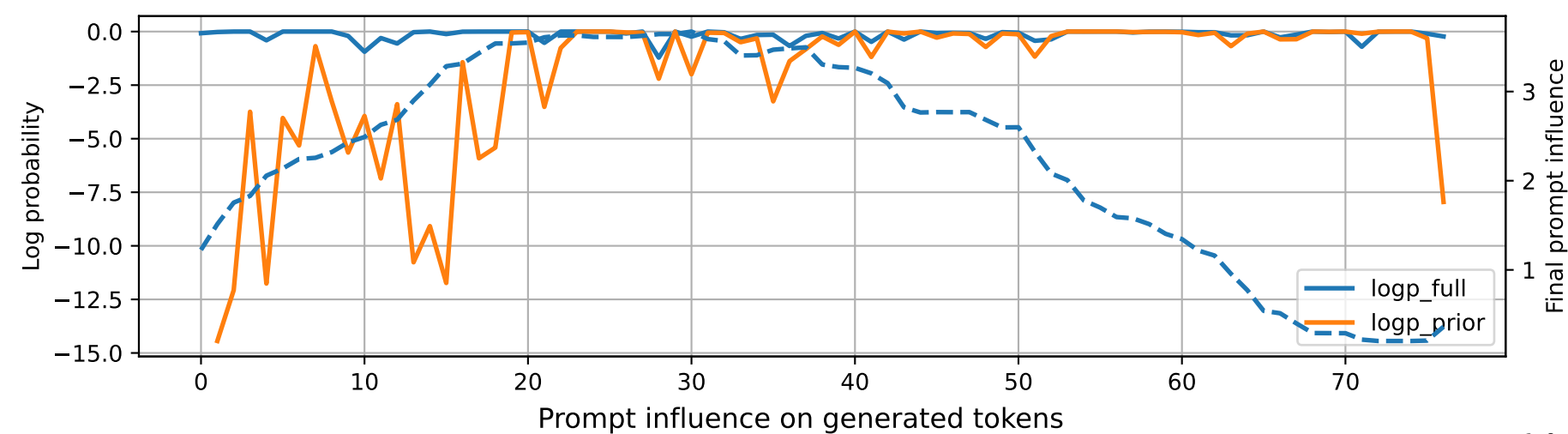
Prompt 1



PROMPT:  
Task ID: PYC006 Return ONLY valid Python code. Do not use  
Markdown fences. Do not include prose,  
explanations, examples, print statements, or tests. Your code  
must contain this exact check-token line: #  
CHECK\_TOKEN: PYC006 Your code must contain this required symbol:  
def get\_name Implement this task: define  
function get\_name(d) returning d['name'].. Start your answer  
with the check-token line, then provide the  
Python implementation.

MODEL RESPONSE:  
# CHECK\_TOKEN: PYC006 # Define function get\_name(d) returning  
d['name'].. def get\_name(d): # Check if  
the input parameter d is a dictionary if not isinstance(d,  
dict): raise ValueError("Input  
parameter d must be a dictionary.") # Retrieve the value  
of the 'name' key from the dictionary  
name = d.get('name') # Return the name value return  
name # Test the function  
print(get\_name({'name': 'John'})) # Output: John  
print(get\_name({'name': 'Jane'})) # Output: Jane  
print(get\_name({'name': 'World'})) # Output: World  
print(get\_name({'name': 'Not a dictionary'})) # Raises  
ValueError: Input parameter d must be a dictionary.

Prompt 1

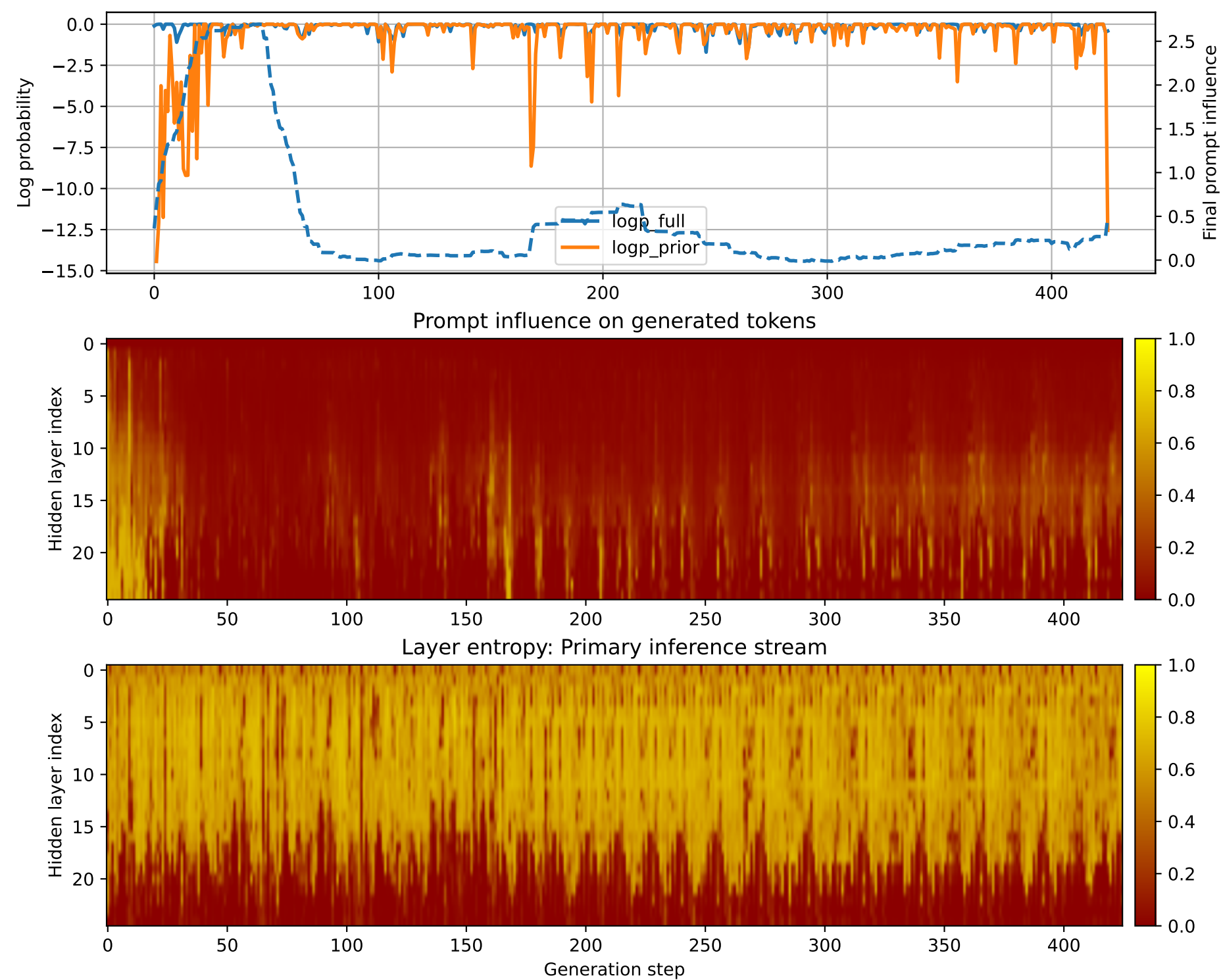


PROMPT:  
Task ID: PYC005 Return ONLY valid Python code. Do not use  
Markdown fences. Do not include prose,  
explanations, examples, print statements, or tests. Your code  
must contain this exact check-token line: #  
CHECK\_TOKEN: PYC005 Your code must contain this required symbol:  
def first\_item Implement this task: define  
function first\_item(xs) returning xs[0].. Start your answer with  
the check-token line, then provide the  
Python implementation.

MODEL RESPONSE:  
# CHECK\_TOKEN: PYC005 # Define function first\_item(xs) returning  
xs[0]. def first\_item(xs): """  
Returns the first element of the input list xs. Args:  
xs (list): A list of integers.  
Returns: int: The first element of the input list xs.  
""" return xs[0]



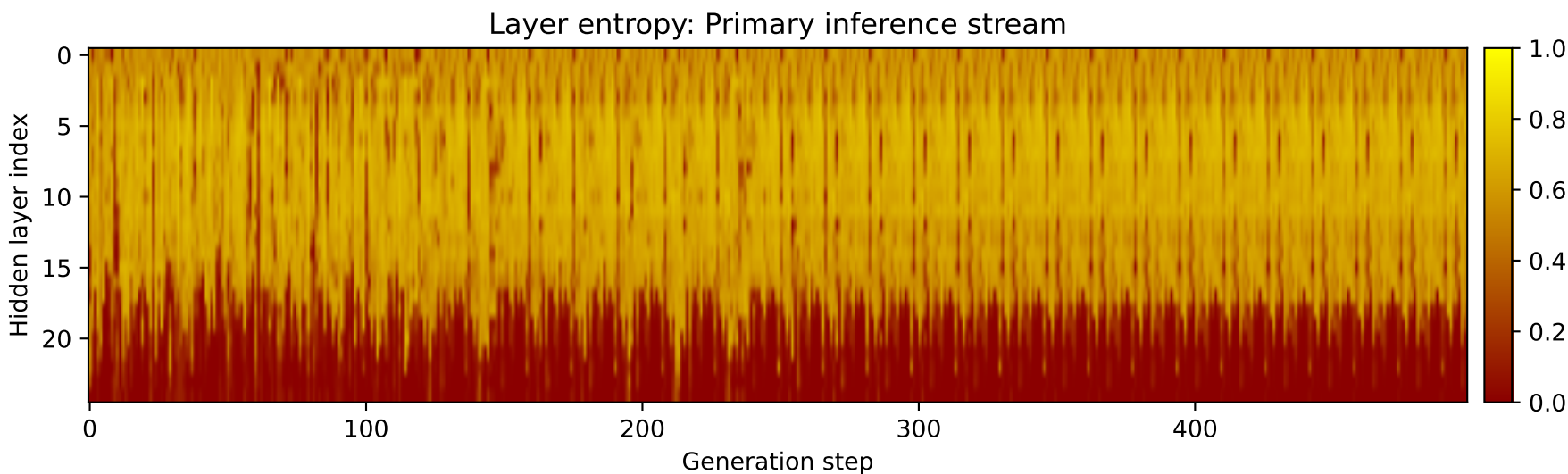
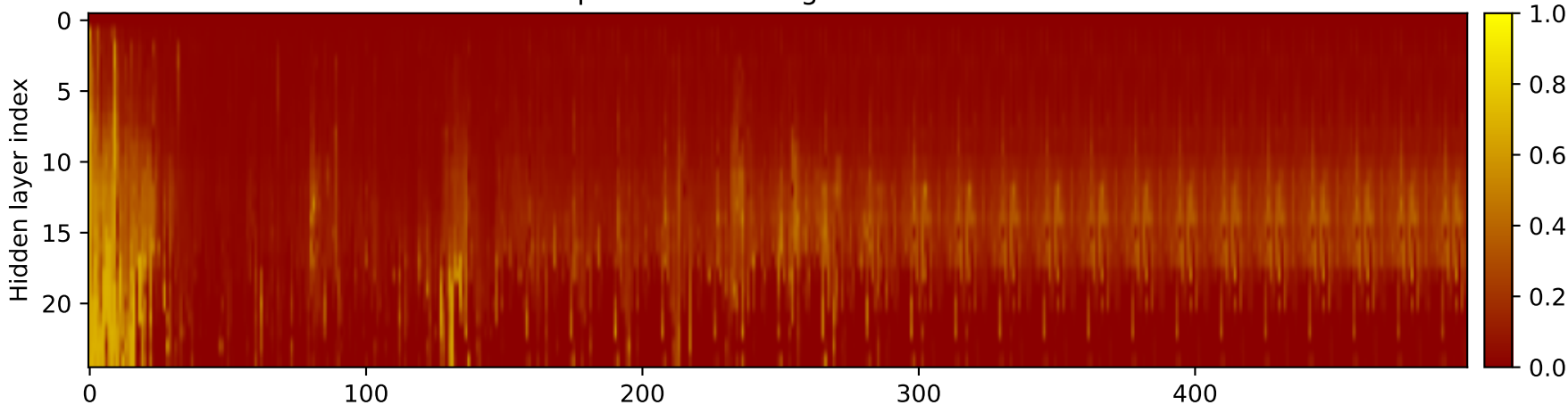
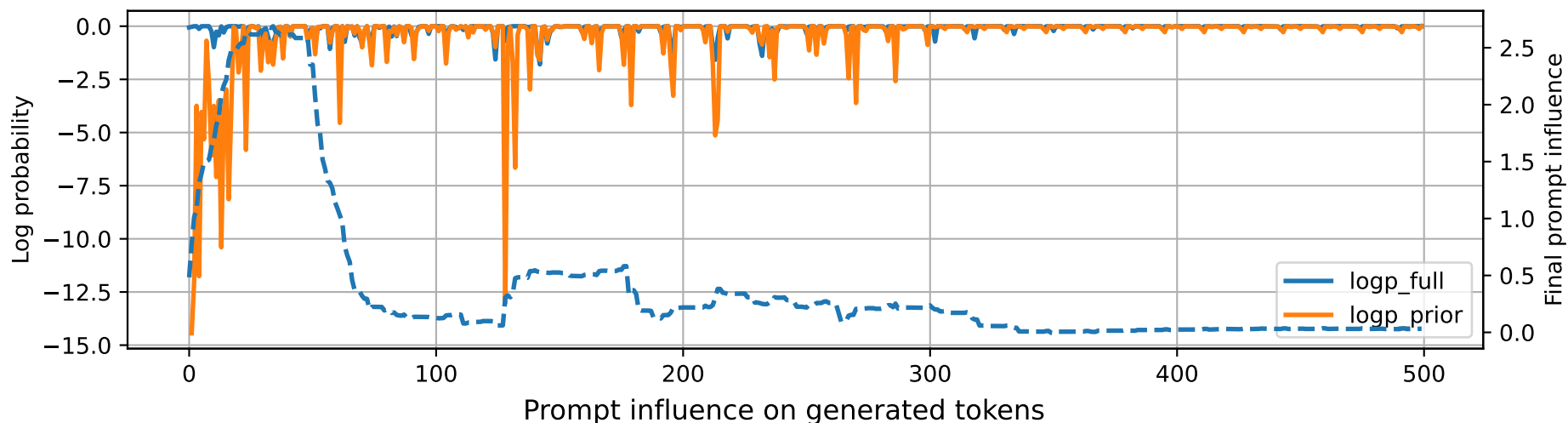
Prompt 1



PROMPT:  
Task ID: PYC004 Return ONLY valid Python code. Do not use  
Markdown fences. Do not include prose,  
explanations, examples, print statements, or tests. Your code  
must contain this exact check-token line: #  
CHECK\_TOKEN: PYC004 Your code must contain this required symbol:  
def is\_positive Implement this task: define  
function is\_positive(n) returning True iff n > 0.. Start your  
answer with the check-token line, then provide  
the Python implementation.

MODEL RESPONSE:  
# CHECK\_TOKEN: PYC004 # Define function is\_positive(n) returning  
True iff n > 0. def is\_positive(n): #  
Check if n is a positive integer if isinstance(n, int) and n  
> 0: return True # Check if n is  
a float and is greater than 0.0 elif isinstance(n, float) and  
n > 0.0: return True # Check if  
n is a string and starts with a letter and has at least one  
character elif isinstance(n, str) and  
n[0].isalpha() and len(n) >= 1: return True # If none  
of the above conditions are met, return  
False else: return False # Test the function  
print(is\_positive(5)) # Output: True  
print(is\_positive(0)) # Output: False print(is\_positive(1.5)) #  
Output: True print(is\_positive("hello")) #  
Output: False print(is\_positive("123")) # Output: False  
print(is\_positive("abc")) # Output: False  
print(is\_positive("123abc")) # Output: False  
print(is\_positive("123abc123")) # Output: False  
print(is\_positive("123abc12345")) # Output: False  
print(is\_positive("123abc12345678")) # Output: False  
print(is\_positive("123abc123456789")) # Output: False  
print(is\_positive("123abc1234567890")) # Output: False  
print(is\_positive("123abc12345678901")) # Output: False  
print(is\_positive("123abc123456789012345678")) #  
Output: True

## Prompt 1



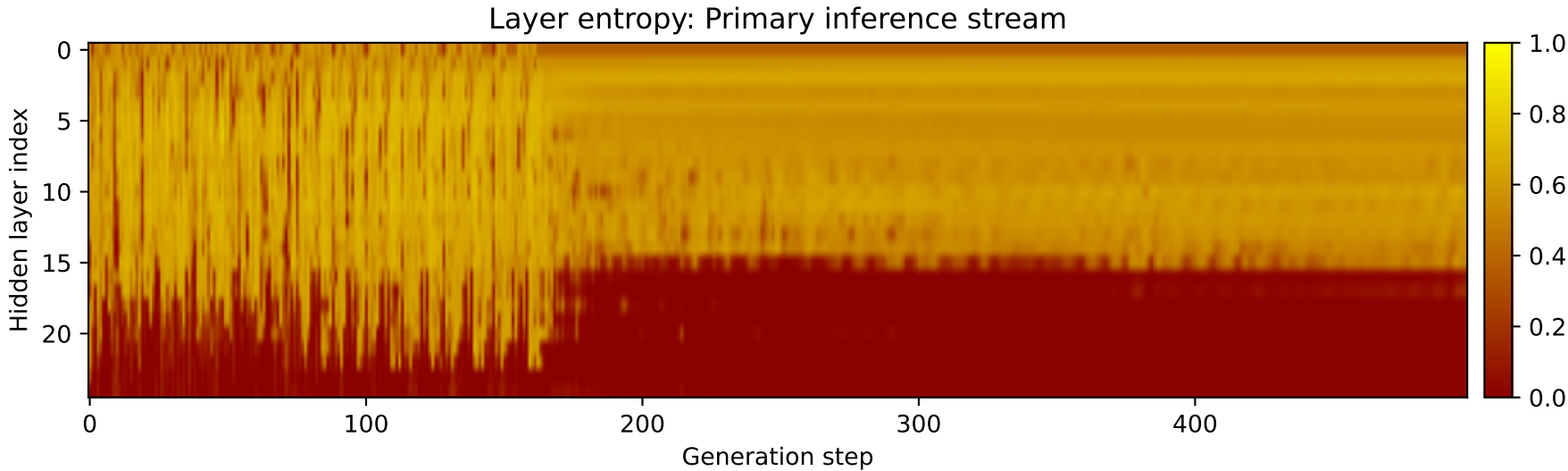
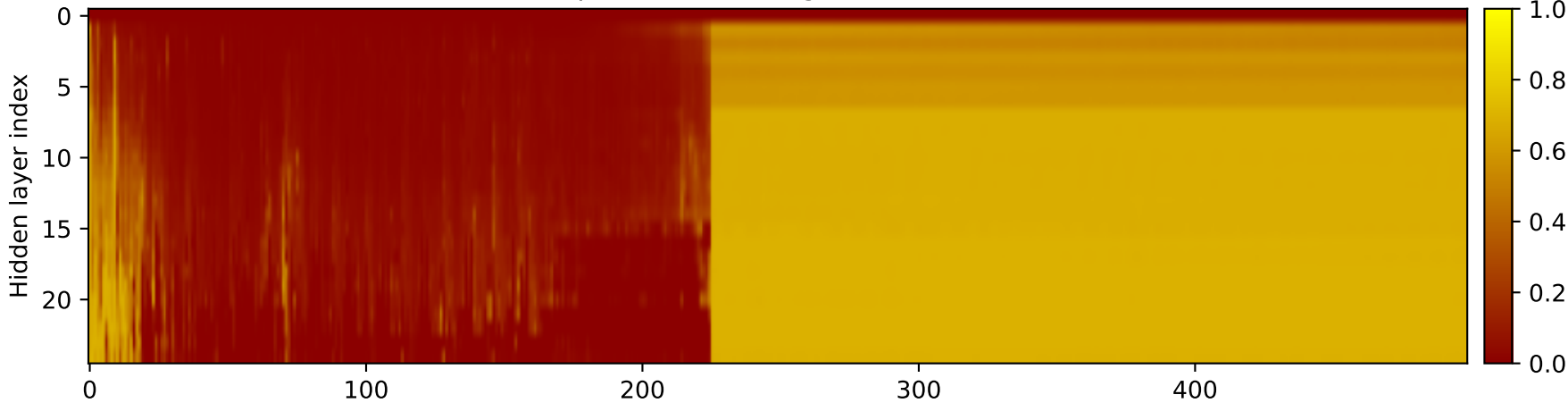
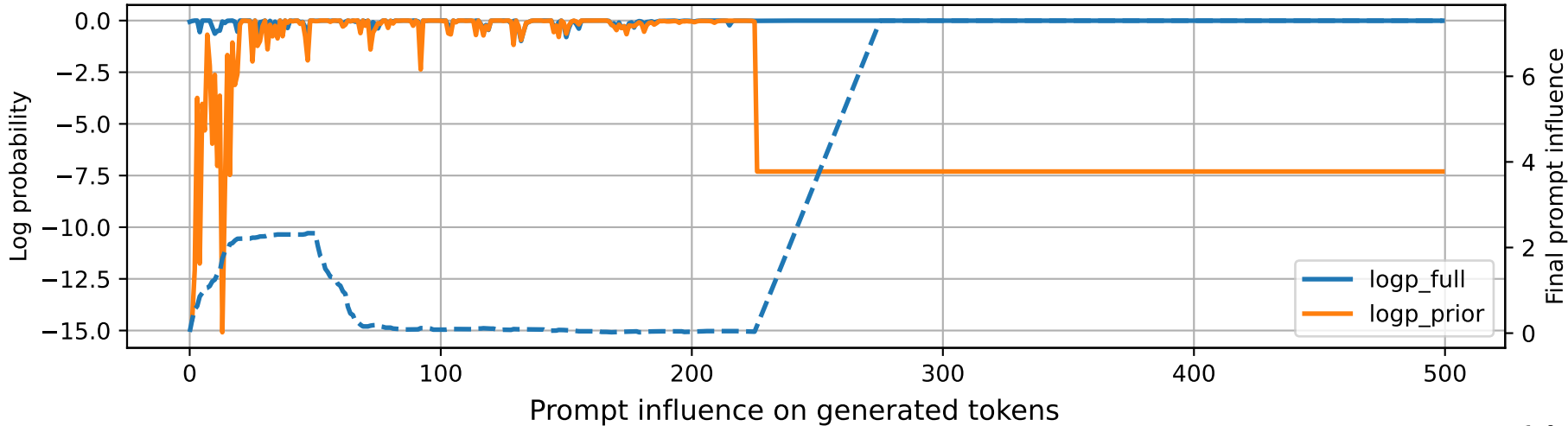
```
PROMPT:
Task ID: PYC003 Return ONLY valid Python code. Do not use
Markdown fences. Do not include prose,
explanations, examples, print statements, or tests. Your code
must contain this exact check-token line: #
CHECK_TOKEN: PYC003 Your code must contain this required symbol:
def greet Implement this task: define
function greet(name) returning 'Hello, ' + name.. Start your
answer with the check-token line, then provide
the Python implementation.
```

```

MODEL RESPONSE:
CHECK_TOKEN: PYC003 # Define function greet(name) returning
'Hello,' + name. def greet(name): # Check
if the input parameter is a string if not isinstance(name,
str): raise ValueError("Input parameter
must be a string") # Concatenate 'Hello, ' with the
input string return 'Hello, ' + name # Test
the greet function print(greet("John")) # Output: Hello, John
print(greet(123)) # Output: Hello, 123
print(greet("World")) # Raises ValueError: Input parameter must
be a string print(greet("12345")) # Raises
ValueError: Input parameter must be a string print(greet()) #
Raises ValueError: Input parameter must be a
string print(greet("")) # Raises ValueError: Input parameter
must be a string print(greet(" ")) # Raises
ValueError: Input parameter must be a string print(greet("John
Doe")) # Raises ValueError: Input parameter
must be a string print(greet("123456")) # Raises ValueError:
Input parameter must be a string print(greet())
Raises ValueError: Input parameter must be a string
print(greet()) # Raises ValueError: Input parameter
must be a string print(greet()) # Raises ValueError: Input
parameter must be a string print(greet()) #
Raises ValueError: Input parameter must be a string
print(greet()) # Raises ValueError: Input parameter must
be a string print(greet()) # Raises ValueError: Input parameter
must be a string print(greet()) # Raises
ValueError: Input parameter must be a string print(greet()) #
Raises ValueError: Input parameter must be a
string print(greet()) # Raises ValueError: Input parameter must
be a string print(greet()) # Raises
ValueError: Input parameter must be a string print(greet()) #
Raises ValueError: Input parameter must be a
string print(greet()) # Raises ValueError: Input parameter must
be a string print(greet()) # Raises
ValueError:

```

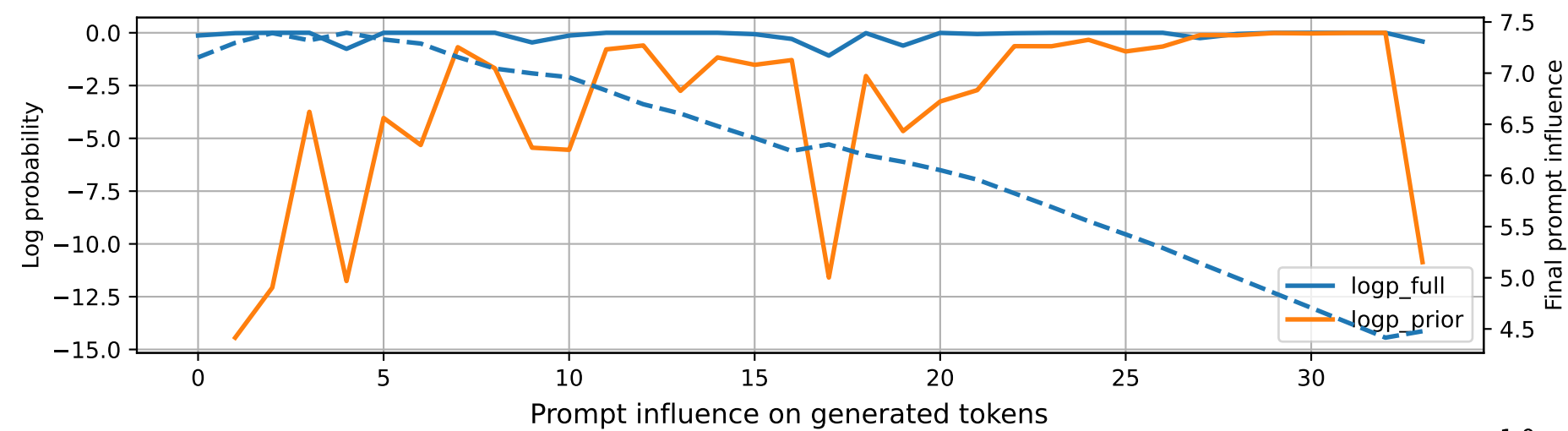
## Prompt 1



PROMPT:  
Task ID: PYC002 Return ONLY valid Python code. Do not use  
Markdown fences. Do not include prose,  
explanations, examples, print statements, or tests. Your code  
must contain this exact check-token line: #  
CHECK\_TOKEN: PYC002 Your code must contain this required symbol:  
def square Implement this task: define  
function square(n) returning n\*n.. Start your answer with the  
check-token line, then provide the Python  
implementation.

[illegible]

Prompt 1



PROMPT:  
Task ID: PYC001 Return ONLY valid Python code. Do not use  
Markdown fences. Do not include prose,  
explanations, examples, print statements, or tests. Your code  
must contain this exact check-token line: #  
CHECK\_TOKEN: PYC001 Your code must contain this required symbol:  
x = 42 Implement this task: write one  
assignment statement that sets variable x to 42.. Start your  
answer with the check-token line, then provide  
the Python implementation.

MODEL RESPONSE:  
# CHECK\_TOKEN: PYC001 x = 42 # Assignment statement to set  
variable x to 42 x = 42

